WM-PLN:
The World-Model Calculus

(DRAFT --- October 11, 2026)

Zar and Oruži[1]

[1]Oruži denotes AI-assisted collaboration using ChatGPT, Codex, and Claude Code.

Abstract

Probabilistic Logic Networks (PLN) couples logical structure with uncertain inference. In practice, however, many PLN presentations mix three distinct layers: (i) a probabilistic world-model posterior, (ii) an evidence algebra for compositional aggregation, and (iii) operational truth-value views (strength, confidence, interval bounds). This conflation produces *semantic drift*: algebraic operators are treated as if they were the logic itself, and link-level inference attempts to propagate correlation-sensitive quantities without carrying the information required to do so correctly.

We present the *world-model calculus*, the mathematical foundation underlying WM-PLN[1], designed to be (a) correct by construction and (b) extensible to tractable sublayers. The core move is to make the complete layer a *posterior-state calculus*: the proof object is a revisable world-model posterior state, and all truth values are obtained by querying that state. We give a reference complete instance as a Dirichlet posterior over complete worlds (`JointEvidence`), prove that evidence extraction commutes with revision, and formalize a no-go theorem showing that complete link evidence cannot, in general, be computed from local per-link evidence alone. We then show how Bayesian-network-style sublayers, higher-order reductions, quantifier semantics, intensional inheritance, and inference-control theory arise as compiled tactics with explicit assumptions within the world-model framework.

All results are formalized in Lean 4 with Mathlib, totaling over 500 theorem-backed modules across evidence algebra, world-model calculus, compiled BN inference, higher-order chaining, and applied demonstrations.

[1]Informally also $\boldsymbol{\Omega}$-PLN, after the world space $\boldsymbol{\Omega}$ that anchors every definition in the calculus.

Contents

I Orientation **9**

1 Introduction **10**
1.1 Where Semantic Drift Came From 10
1.2 Running Example: Maple Court Community Commons . . . 12
1.3 What the Refoundation Changes 14
1.4 What This Document Claims 15
1.5 Claim Taxonomy and Reading Guide 16
1.6 Concept Map: PLN to WM-PLN 16
1.7 How to Read This Document 17

2 Mathematical Desiderata for a Sound PLN **19**

II The Semantic Kernel **21**

3 Queries, Judgments, and World Models **22**
3.1 The Query Language 22
3.2 World Models 23
3.3 World Models with Side Conditions 25
3.4 The Solomonoff Connection 26
3.5 The Universal Property of the World-Model Interface 28

4 Reference Complete Semantics: Posterior-State Calculus **31**
4.1 The Reference Instance: Dirichlet over Worlds 31
4.2 The Sequent-Calculus View 32
4.2.1 Judgments 33
4.2.2 Core Rules 33
4.3 Probability Views 34
4.4 Derived Link Calculus: Classic PLN Rules as Admissible Rewrites 34

4.4.1 The Deduction Formula 34
4.4.2 Induction and Abduction 35
4.5 The Kernel as a Typed Rewrite Calculus 35

5 The Local-Completeness No-Go Theorem 38
5.1 Interpretation 39

III Evidence, State, and Views 41

6 Evidence Algebra 42
6.1 The Generic Evidence Pattern 42
6.2 The Atomic Event Space Comes First 43
6.2.1 A general identifiability boundary 45
6.2.2 Evidence availability is not addressability 45
6.2.3 Relational novelty: facts are not objects 46
6.2.4 Selected representatives are views, not ledgers 47
6.2.5 Registered binary trials 47
6.2.6 Unit tests and coverage 48
6.2.7 ProbLog explanations and MLN groundings 48
6.2.8 Gaussian learning and provenance-aware sufficient statistics 49
6.2.9 Clustered uncertainty and capture–recapture 49
6.2.10 Case study: complete relational accounting of two search rounds 50
6.2.11 Architecture is an effect modifier, but the Round 2 gain is hub-sensitive 51
6.3 Information Ordering 55
6.4 Compositional Tensor (Quantale Structure) 55
6.5 Logical Operations (Heyting Algebra) 56
6.6 What Evidence Algebra Is Not 57

7 Evidence Carriers: Distributional and Interval Semantics 59
7.1 Instance 1: Beta-Binomial (Binary Domains) 60
7.2 Instance 2: Dirichlet-Multinomial (Multi-Valued Domains) . . 61
7.3 Instance 3: Normal-Gamma (Continuous Domains) 62
7.4 Applied Carrier Validation and Benchmark Demos 65
7.4.1 Applied Example: Hybrid Sensor Monitoring 66
7.4.2 Applied Example: Widget Factory Source-Conditional Mixtures 68

7.4.3 Applied Example: Kalman Filter Wake/Sleep Temporal Filtering . . . 69
7.4.4 Applied Example: Gas Sensor Array Drift Classification 70
7.4.5 Applied Example: Steel Plates Fault Classification . . 77
7.4.6 Applied Example: NAB Machine Temperature Anomaly Detection . . . 80
7.4.7 Applied Example: Good Judgment Project Forecast Aggregation . . . 85
7.4.8 Summary of Numerical Results . . . 88
7.5 Walley Predictive vs. Bayesian Credible Intervals . . . 88
7.6 The De Finetti Bridge . . . 90
7.7 Knuth–Skilling Foundations . . . 91

8 Truth-Value Views: Strength, Confidence, Intervals 92
8.1 Two Notions of Improvement . . . 92
8.2 Views as Projections . . . 93
8.3 Simple Truth Values . . . 94
8.4 Confidence as a View . . . 95
8.5 Indefinite Truth Values . . . 96
8.6 Why Views Are Not Proof Objects . . . 97

9 Forgetting, Provenance, and State Management 98
9.1 Why Forgetting Matters . . . 99
9.2 The Forgetting Layer . . . 99
9.3 Conservation Theorems . . . 100
9.4 Provenance and Explanation . . . 100
9.4.1 PLN Evidence Stamps as Provenance . . . 101

IV Compiled Inference and Tractable Fragments 103

10 Σ-Guarded Rewrites 104
10.1 The Rewrite Pattern . . . 104
10.2 The Xi Rule Registry . . . 105
10.3 Compiled Tactics vs. Foundational Semantics . . . 106

11 Bayesian-Network World Models and Compiled BN Inference 107
11.1 BN World-Model Class . . . 108
11.2 Exact BN Query Bridge . . . 108

11.3 Chain, Fork, and Collider Exactness . . . 108
11.4 Class-Packaged BN Discharge . . . 110
11.5 Worked Proof-Carrying Contraction . . . 110
11.6 Bridge Theorems: Naive Bayes and k-NN . . . 115
11.7 Factor Graphs: Semantic vs. Operational . . . 115
11.8 PLN v0.9 Examples as WM Calculus Instances . . . 116
11.8.1 Stepped Walkthrough: DeductionRevision with Provenance . . . 117
11.9 Applied BN Validation and Benchmark Lanes . . . 119
11.9.1 Probabilistically Guarded Admissibility on Real BN Benchmarks . . . 119
11.9.2 Hetionet / Rephetio: the next flagship benchmark . . . 130
11.9.3 GWAS locus-to-gene: replication-first benchmark lane 132

12 Error, Thresholds, and Transport Discipline 150
12.1 View Transport Under Query Equivalence . . . 150
12.2 Threshold Semantics and OSLF Bridge . . . 151
12.3 Threshold Transport Boundaries . . . 151
12.4 Categorical Transport Layer . . . 152
12.5 Certified Chaining: Fundamental Limitations . . . 152
12.6 Distributional Inference: The Constructive Resolution . . . 156

V Main Extensions 171

13 [WIP] Higher-Order Reduction 172
13.1 Current HOL Core and Classical Henkin Route . . . 173
13.2 FOL Embedding into HOL . . . 175
13.3 HOL as a World-Model Lens . . . 175
13.4 Direct Set-Semantics Grounding of HOL . . . 176
13.5 Higher-Order PLN over HOL and WM . . . 177
13.6 Where Satisfying-Set Reduction Still Fits . . . 178
13.7 Higher-Order Probability: Now Semantically Grounded . . . 178
13.7.1 Hierarchical and Infinite-Order Probabilistic HOL . . . 179
13.7.2 Certified Higher-Order Chaining from Estimated Variables . . . 181
13.8 Logical-Induction-Ready Dynamic Belief Infrastructure . . . 184
13.9 Current HOL Completeness Status . . . 187

14 Quantifiers and the Fuzzy/QFM Layer **188**
14.1 Finite-Model Quantifier Semantics . . . 189
14.2 Fuzzy Quantifier Semantics . . . 189
14.3 Scope Note: Stable Finite-Domain Bundle vs. Infinitary Semantics . . . 191

15 Intensional Inheritance **192**
15.1 What Intensional Inheritance Captures . . . 192
15.2 Typed WM Grounding . . . 193
15.3 Solomonoff Bridge . . . 194
15.4 Mixed Semantic Packaging . . . 194

16 Temporal and Causal Inference **196**
16.1 Temporal Operators . . . 196
16.2 Causal Profile . . . 197
16.3 Temporal as Another WM Instance . . . 198

VI Large-Scale Inference, Control, and Boundaries **199**

17 Large-Scale Inference at Theorem Level **200**
17.1 Multideduction and Identifiability . . . 200
17.2 Trail-Free Protocol Dynamics . . . 201
17.3 Pooling and Fusion . . . 201
17.4 Order-Cost Governance . . . 202
17.5 BRGI and Feasible Sets . . . 203
17.6 Stochastic Experiment Layer . . . 204

18 Inference Control **205**
18.1 Selector Specifications . . . 205
18.2 Coverage and Submodularity . . . 206
18.3 Coverage Surrogate Boundaries . . . 206
18.4 Composed Core Endpoints . . . 207

VII Typed Realization of the Kernel **208**

19 The Kernel as a Typed Language Family **209**
19.1 The Algebraic Core . . . 210
19.1.1 Pattern Vocabulary . . . 210

19.1.2 Core Rules from the AddCommMonoid Universal Property . . . 211
19.1.3 The 2×2 Calculus Square . . . 211
19.2 Intrinsic Encoding and Adequacy . . . 213
19.2.1 Biconditional and Star Adequacy . . . 213
19.2.2 Backward Asymmetry and Image-Restricted Box . . . 214
19.3 The 13-Axis Vertex Family . . . 214
19.3.1 Extension Families . . . 215
19.3.2 Vertex Preorder and Forward Fiber . . . 216
19.4 OSLF Layer: Modal Operators and Semantic Bridge . . . 217
19.4.1 Diamond Theorems and Galois Corollaries . . . 218
19.4.2 Relational Encoding and Diamond Chaining . . . 218
19.4.3 Evidence Barbs (Process-Algebraic Observables) . . . 218
19.4.4 Neighborhood Semantics Fiber . . . 219
19.4.5 First-Order Logic Fiber . . . 219
19.5 Native Type Synthesis: What the Type Pass Sees . . . 219
19.6 Compiled BN Inference Bridge . . . 220
19.7 Formalization Inventory and Future Work . . . 221
19.7.1 Remaining Future Work . . . 221
19.7.2 The Deeper Direction: WM as Universal Evidence for GSLTs . . . 222

VIII Bridges, Scope, and Relation to PLN 224

20 Bridges to Neighboring Frameworks 225
20.1 MLN Import and Subsumption . . . 225
20.1.1 Clause-Native Core (Ground MLN Semantics) . . . 226
20.1.2 First-Order Template Grounding . . . 227
20.1.3 The Semantic Subsumption Chain . . . 228
20.1.4 UW-CSE Benchmark . . . 231
20.1.5 Beyond MLN: Compatibility-Gated Evidence Roles . . . 232
20.1.6 Abstract Lane (Supporting Infrastructure) . . . 235
20.1.7 MLN Theory Combination vs. WM Revision . . . 235
20.1.8 Scope Limitations . . . 235
20.2 ProbLog Bridge . . . 236
20.3 Naive Bayes and k-NN Bridges . . . 237
20.4 Noisy-OR Bridge . . . 238
20.5 PLN v0.9 Subsumption . . . 240
20.6 PLN-NARS Bridge . . . 240

20.7 FOL and Set-Semantics Bridges . . . 241
20.7.1 Model theory: Tarskian FOL, Henkin HOL . . . 242
20.7.2 Set representation: the `SetStructure` abstraction . . . 242
20.7.3 Singleton adequacy . . . 243
20.7.4 WM strength and model-theoretic consequence . . . 243
20.7.5 Four routes and their agreement . . . 244
20.7.6 HOL-level limitations . . . 244
20.7.7 Infinite domains . . . 245
20.8 PPLBench: Probabilistic Programming Interoperability . . . 246
20.9 DeFi Hybrid Risk Monitoring . . . 246

21 What WM-PLN Covers **247**
21.1 The Kernel . . . 247
21.2 Views and Evidence . . . 247
21.3 Compiled Inference . . . 248
21.4 Extensions . . . 248
21.5 Large-Scale Inference and Control . . . 248
21.6 WM-OSLF Bridge . . . 249
21.7 Cross-Cutting Bridges . . . 249
21.8 Artifact and Reproducibility . . . 249

22 Current Scope and Limitations **251**
22.1 Operational Runtimes . . . 251
22.2 Infinite-Model Quantifiers . . . 251
22.3 Chapter 12 Settlement . . . 251
22.4 Universal Prediction . . . 252
22.5 Global PLN Reorganization . . . 252

23 Conclusion **253**

IX Applications and Benchmark Validation **256**

24 [WIP] Advanced-AI Outcomes (WM Scaffold) **257**
24.1 WM Query Form . . . 257
24.2 What the Current Data Actually Supports . . . 258
24.3 Derived Outcome Queries . . . 259
24.4 Current Status . . . 259
24.5 What Can and Cannot Be Said . . . 260
24.6 Interpretation Rule . . . 262

25 PPLBench: Nine-Lane Benchmark Validation **263**
25.1 What We Tested . . . 264
25.2 Archive Snapshot . . . 267
25.3 Scripts and Model Lane . . . 267
25.4 Easy Refresh Protocol . . . 267
25.5 Refresh Checks . . . 268
25.6 Held-Out Normal-Normal Demo . . . 269
25.7 Interpretation . . . 269
25.8 Proper Stan vs WM-PLN on Normal-Normal . . . 271
25.9 Exact Hook-B Adaptive Normal-Normal . . . 272
25.10 Semantic Noisy-Or Upgrade . . . 274
25.11 Crowd Annotation via Multiclass Message State . . . 278
25.12 Exact Dirichlet-Categorical Lane . . . 280
25.13 Hierarchical `n_schools` via Carried Evidence State . . . 283
25.14 Logistic Regression: Bernoulli-Logit Site Carrier . . . 286
25.15 Robust Regression: Continuous Carrier Under Heavy-Tailed Noise . . . 289
25.16 Cross-Backend NumPyro Check . . . 292
25.17 Scheduler Promotion and Certification . . . 293

X Reference Appendices **296**

A Notation and Symbol Glossary **297**

B Compact Theorem Map **298**

C Code Navigation Guide **300**
C.1 Entry Points . . . 300
C.2 Applied Gaussian Demo Suite . . . 300
C.3 Regression Targets . . . 301
C.4 Build . . . 302

D PLN Book Coverage Map **303**

E Advanced WM Wrappers **305**

Part I

Orientation

Chapter 1

Introduction

Goal: Diagnose semantic drift in PLN and motivate the world-model refoundation.
Semantic object: Three-layer architecture (world model, evidence algebra, views).
Proved: Nothing (orientation chapter).
Layer: Orientation.
Assumed: Familiarity with basic probability; PLN book familiarity helpful but not required.

Probabilistic Logic Networks, as described by Ben Goertzel, Matthew Iklé, Izabela Freire Goertzel, and Ari Heljakka [15], represent uncertain judgments using truth values: a *strength* (probability-like estimate) and a *confidence* (reliability-like quantity derived from sample size). The ambition was generous—to build a general-purpose uncertain reasoner capable of deduction, induction, abduction, and revision, all within a unified algebraic framework.

That ambition produced real insights. The evidence-count representation, the separation of strength from confidence, the recognition that revision must be compositional, and the aspiration to connect logical structure with probabilistic semantics—these were the right instincts. But in the passage from intuition to formalization, something went wrong.

1.1 Where Semantic Drift Came From

The difficulty is not that PLN's rules are wrong. Many of them are correct—under assumptions that were left implicit. The difficulty is that the original presentation mixes three conceptually distinct layers without always marking where one ends and another begins:

1. **The world-model layer.** A probabilistic model—a joint distribution, a Bayesian network, or some structured posterior—that can answer any query by conditioning. This is the *complete* semantics.

2. **The evidence-algebra layer.** An algebraic structure on observation counts (n^+, n^-) with revision as addition, information ordering, and lattice operations. This is a *compositional bookkeeping* layer.

3. **The operational-view layer.** Strength, confidence, weight, interval bounds—these are *extracted summaries* of evidence, useful for thresholding, display, and approximate computation, but not themselves the semantics.

The calculus additionally has a *typed rewrite presentation*: the core rules encode as a typed language definition whose native types classify WM terms by their rewrite behavior (Section 4.5). This typed realization is not a fourth semantic layer but a computational presentation of the kernel—it is how the calculus becomes a typed, mechanically checkable object.

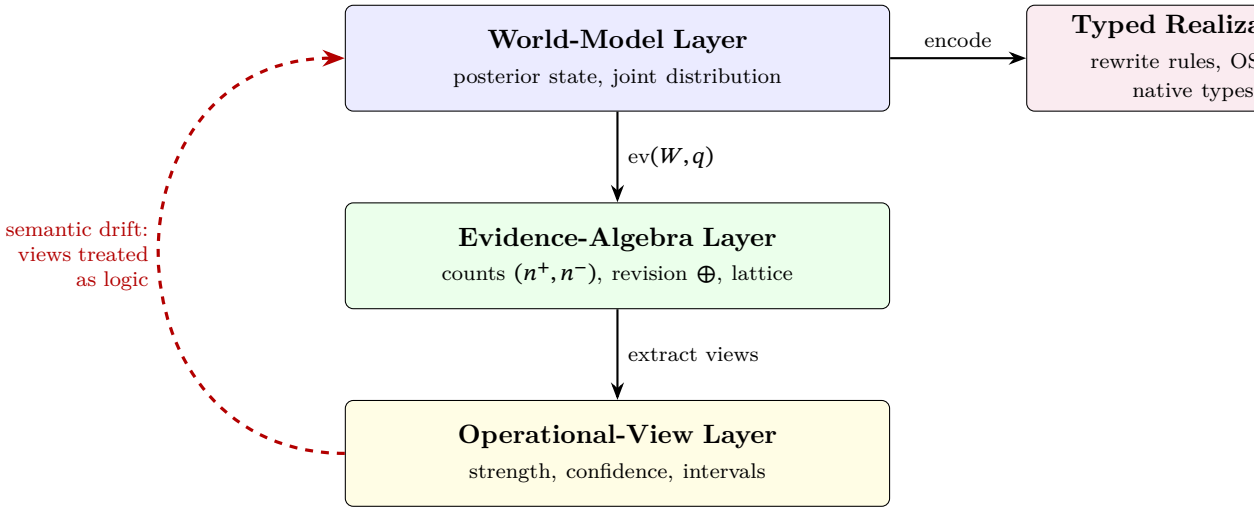


Figure 1.1: The architecture of WM-PLN: three semantic layers (left) plus the typed realization (right). Solid arrows show the correct information flow: query the world model for evidence, then extract operational views. The typed realization encodes the kernel as a rewrite system with native types (Section 4.5). The dashed red arrow marks the conflation that produces semantic drift.

1.2 Running Example: Maple Court Community Commons

To make these layers concrete, we introduce a running example that recurs throughout the book. *Maple Court* is a small housing community—a co-op with private apartments and shared spaces: a hallway, laundry room, elevator, and a rooftop community garden. Two autonomous agents serve the commons: a hallway cleaning robot and a garden-tending robot. Residents, building stewards, and the robots cooperate through layered world models.

- **World-model layer:** a small Bayesian network with latent states RoomOccupied, ShowerRunning, PipeLeak linked to observable sensors. The BN contains all three canonical motifs: a *collider* (ShowerRunning → BathroomHumidity ← PipeLeak), a *fork* (RoomOccupied → MotionDetected and RoomOccupied → PowerUseHigh), and a *chain* (PipeLeak → WallHumidity → MoldRisk).
- **Evidence layer:** motion detections and leak alarms are Beta evidence (binary); bathroom and hallway humidity are Normal-Gamma evidence (continuous); laundry room status (free/busy/full) and elevator health (normal/slow/faulty) are Dirichlet evidence (multi-valued).
- **View layer:** "Is a pipe leaking?" extracts a strength and confidence from the apartment posterior. "Should the cleaning robot mop the hallway?" uses the exceedance query $P(\text{Humidity} > 70\%)$.

Maple Court operates at two scales: each apartment maintains a *private* WM (fast-changing, high-resolution), while the building maintains a *shared* WM for common spaces (medium timescale). Apartments export only evidence summaries—not raw sensor traces—to the shared model, preserving privacy by design. A third, *community-level* WM (aggregate, slow timescale) appears in Chapter 17 when we consider federation across multiple buildings.

This scenario exercises evidence aggregation (Ch. 6), distributional semantics (Ch. 7), BN compiled inference (Ch. 11), temporal filtering (Ch. 16), and the full spectrum of PLN inference patterns—all developed in subsequent chapters. Semantic drift would occur if, for example, the cleaning robot chained strength values through the leak→humidity→mold path without tracking the variance lost at each step. The Lean formalization (`Mettapedia/Examples/PLN/MapleCourtDemo.lean`, 743 lines, 0 sorry) provides a kernel-checked reference instance.

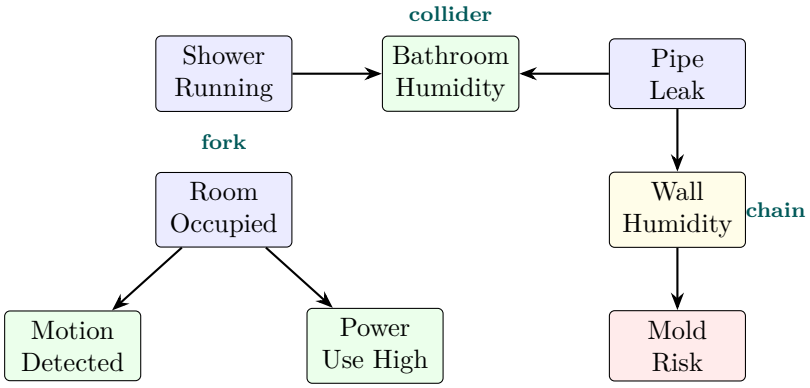


Figure 1.2: Maple Court apartment BN: the three canonical motifs. Blue nodes are latent, green are observed, yellow is intermediate, red is derived.

When these layers are conflated, several things go wrong. Operational views get treated as if they *were* the logic: interval bounds are composed with rules designed for point probabilities, confidence heuristics are chained without tracking what information they lost. Link-level inference attempts to compute complete posteriors from local link truth values, ignoring the correlations that live only in the joint state. The result is semantic drift: the system *almost* works, but its correctness depends on assumptions that are neither stated nor checked.

The answer-profile idea. The simplest imaginable world model is a giant table that assigns an evidence value to every possible query—an *answer profile*. Richer models store structured state (a Bayesian network, a posterior distribution, a sufficient-statistic product) from which the table can be computed. The mathematical core of a world model is then: *every state determines an answer profile, and combining states combines their profiles additively.* Section 3.5 makes this precise: a world model is exactly an additive monoid homomorphism from states to profiles. This does not trivialize the project—the hard work is in choosing expressive state spaces, proving no-go theorems, and certifying fast inference paths—but it makes the essence visible.

1.3 What the Refoundation Changes

WM-PLN is a response to semantic drift. Its core move is to replace free-floating truth-value propagation with a *revisable state of knowledge*. The system stores what it knows in a posterior state, not in isolated truth values. When a question is asked, it queries that state and returns evidence. Strength, confidence, and intervals are summaries extracted from evidence for display, thresholding, and action—but they are not themselves the thing being stored or proved.

Concretely, the refoundation separates five layers:

1. **Kernel semantics.** A world-model posterior state is the primary proof object. The interface is one arrow: an additive monoid homomorphism from states to answer profiles (Section 3.5). Four verbs operate on states: *revise*, *query*, *forget*, *explain*.

2. **Evidence algebra.** A quantale and Heyting frame on observation counts, serving compositional aggregation. The algebra is not the complete semantics—it is the currency that the kernel extracts and that views summarize.

3. **Truth-value views.** Strength, confidence, intervals, distributions: typed projections from evidence. Useful but lossy.

4. **Compiled shards.** Classic PLN rules (deduction, induction, abduction) as derived rewrites, correct under explicit side conditions within declared model classes. A shard is a certified shortcut: it commutes with the full extractor on the queries it claims to handle.

5. **Inference control.** Premise selection, coverage bounds, order-cost governance, scheduling. Control decides what to compute next; it is not the semantics.

The result is not a "second PLN." It is the reference semantics that operational PLN rules approximate, under explicit assumptions, within declared model classes. Where the original book's rules are correct, WM-PLN proves them correct and states the conditions. Where they are not, WM-PLN provides counterexamples and corrected, side-conditioned replacements.

1.4 What This Document Claims

This document is a self-contained mathematical presentation of the PLN refoundation. It does not require the reader to have the PLN book open, though familiarity with the PLN tradition will enrich the reading. It is *not* a review of the PLN book—that is a separate document [43].

Every theorem stated here has been formalized in Lean 4 with Mathlib. Where a result is conditional on explicit assumptions, those assumptions are stated. Where a result establishes a boundary (a counterexample or no-go theorem), that boundary is stated plainly. Where work remains operational or future, that status is marked.

Trust Base & Scope.

- *Formalized* means that the stated theorem or counterexample is checked by the Lean 4 kernel against imported Mathlib facts.
- *Lean trust base*: the Lean kernel, Mathlib, and the standard elaboration/runtime path needed to check the proof terms.
- *Proved in Lean*: the world-model/evidence calculi, compiled-rule theorems, and counterexamples stated here as mathematical claims.
- *Numerically instantiated in Python*: full-dataset protocol runs (GJP, Gas, Steel, NAB) using the same closed-form update/query laws; these are auditable experiment instantiations, not kernel-checked proofs.
- *Domain scope*: the reviewed quantifier, intensional, and inference-control theorems use finite domains ($|U| < \infty$). An arbitrary-domain infinitary layer exists in the codebase (`PLNFirstOrder/Infinite.lean`) but is not yet integrated into this manuscript. The world-model kernel (Parts II–III) and compiled BN inference (Part IV) are domain-agnostic. Theorems requiring finite domains are marked [FIN] in their titles.
- *Deferred/open*: some runtime infrastructure remains operational rather than theoremic; the active HOL foundation is classical Henkin, while intuitionistic-strengthening and fuller belief-process fusion remain open (Section 13.9).

The mantra throughout:

Probability is what you query; evidence is what you carry.

1.5 Claim Taxonomy and Reading Guide

Before presenting any mathematics, we fix the language of claims. Every result in this document falls into one of four categories:

Core An exact theorem, proved unconditionally (or with only standard mathematical hypotheses like non-negativity or finiteness). These are the bedrock.

Conditional A theorem that holds under explicitly stated structural assumptions—a declared model class, a side condition like d-separation or screening-off, a positivity hypothesis. These are correct *relative to* the stated assumptions. The assumptions are part of the theorem, not an afterthought.

Boundary A counterexample or no-go theorem establishing that some desirable property *fails* in general. These are as important as the positive results: they tell us exactly where fast inference cannot claim completeness, and they motivate the structural assumptions in conditional theorems.

Operational A definition, interface, or specification that exists in the formalization but whose full operational realization depends on runtime infrastructure (e.g., a MeTTa execution engine) not yet available. These are honest placeholders: the mathematical specification is precise, but the implementation story is future work.

Every chapter will mark its results with these labels. The reader should always know whether a claim is proved, conditioned, bounded, or operational.

1.6 Concept Map: PLN to WM-PLN

Table 1.1 shows how each major PLN concept maps to its WM-PLN formalization and where it appears in this document. For the Normal-Gamma-specific specialization of the evidence and view layers, see Table 7.1 in Section 7.3.

PLN Concept	PLN Ch.	WM-PLN Form	Here
Evidence counts	3	Evidence algebra ($\oplus$, quantale, Heyting)	Ch. 6
Strength, confidence	3	Truth-value views (lossy projections)	Ch. 8
Revision	3	Evidence addition $e_1 \oplus e_2$	Ch. 6
Deduction, induction, abd.	5	Σ-guarded compiled tactics	Chs. 10–11
Indefinite truth values	4	Interval semantics (Walley/Kyburg)	Ch. 8
Distributional TVs	7	Beta/Dirichlet/Normal-Gamma posteriors	Ch. 7
Higher-order extensional	10	HOL reduction + ProbHOL	Ch. 13
Quantifiers + fuzzy	11	Finite-domain QFM bundle	Ch. 14
Intensional inheritance	12	Typed WM grounding + Solomonoff bridge	Ch. 15
Temporal / causal	14	Event calculus + causal profile	Ch. 16
Inference control	13	Selector specifications + coverage bounds	Ch. 18
—	—	Native type synthesis via OSLF/GSLT	Ch. 19

Table 1.1: PLN concept → WM-PLN formalization (overview).

1.7 How to Read This Document

- **Part I** (this chapter plus Desiderata) orients the reader: what went wrong with truth-value-only PLN, what the refoundation changes, and the eight mathematical requirements.
- **Part II** (Chapters 3–5) presents the semantic kernel: queries, world models, the reference Dirichlet posterior-state calculus, and the local-completeness no-go theorem.
- **Part III** (Chapters 6–9) develops the evidence object and its operations: the shared algebra, the full conjugate carrier family (Beta, Dirichlet, Normal-Gamma), truth-value views as projections, and state management (forgetting, provenance, conservation).
- **Part IV** (Chapters 10–12) shows how fast PLN inference arises from the kernel via compiled rewrites over Bayesian-network world models.
- **Part V** (Chapters 13–16) presents the main extensions: higher-order reduction, quantifiers, intensional inheritance, and temporal/causal inference. Each chapter follows a uniform extension template.
- **Part VI** (Chapters 17–18) addresses large-scale inference theory and inference control.

- **Part VII** (Chapter 19) packages the entire WM calculus as a 13-axis family of typed language definitions, derives OSLF modal operators, and extracts native types per vertex.
- **Parts VIII–IX** (Chapters 20–23) bridge to neighboring frameworks, take stock of what is proved, and conclude.
- **Part X** (PPLBench and benchmark validation) demonstrates that the WM-PLN stack operates as a probabilistic programming backend with nine benchmark lanes and explicit carried-state/queried-view discipline.

Chapter 2

Mathematical Desiderata for a Sound PLN

Goal: State eight requirements any PLN foundation must satisfy.
Semantic object: Desiderata as formal properties of a hypothetical calculus.
Proved: Nothing (specification chapter); satisfaction shown in later chapters.
Layer: Kernel.
Assumed: Nothing.

What must any foundation for PLN preserve? We state eight desiderata. Not all of them can be satisfied simultaneously in full generality—the no-go theorem in Chapter 5 shows that local completeness conflicts with compositionality for arbitrary world models. But any proposed foundation should address each desideratum explicitly, either satisfying it or stating clearly why and where it fails.

1. **Posterior-state semantics.** The foundational proof object is a posterior state—a structured distribution over possible worlds, or a sufficient statistic thereof. Truth values for specific queries are obtained by extraction from this state, not by direct algebraic manipulation of link-level numbers.

2. **Revisability.** New evidence can always be incorporated. Revision is the fundamental operation: it combines two posterior states into a single revised state. It must be associative and commutative (evidence order should not matter for the final state).

3. **Queryability.** Any well-formed query—a proposition, a link, a conditional—can be answered by projecting from the posterior state. The query language is separate from the evidence representation.

4. **Evidence compositionality.** Evidence extraction commutes with revision:

$$\mathrm{ev}(W_1 + W_2, q) = \mathrm{ev}(W_1, q) \oplus \mathrm{ev}(W_2, q), \qquad \mathrm{ev}(0, q) = 0.$$

   Equivalently: the curried extraction map $W \mapsto (\lambda q.\ \mathrm{ev}(W, q))$ is an additive monoid homomorphism from states to answer profiles. This single algebraic condition is the entire semantic interface (Section 3.5).

5. **Tractable compiled fragments.** For restricted world-model classes (e.g., Bayesian networks with bounded treewidth), query answering is tractable. Classic PLN rules arise as compiled rewrites within such classes.

6. **Explicit side conditions.** Every compiled rewrite carries its assumptions as explicit hypotheses: d-separation, screening-off, positivity, finiteness. There are no "silent assumptions."

7. **Extracted truth-value views.** Strength, confidence, interval bounds, and distributional summaries are derived views, not primitive objects. They are indispensable for operational use, but they are not the semantics.

8. **Theorem-level boundaries.** Where completeness or compositionality fails, there is a formal counterexample or no-go theorem stating what cannot be done. These boundaries guide the design of tractable fragments.

WM-PLN satisfies all eight desiderata, with the caveat that tractability (desideratum 5) always comes at the cost of restricting the model class, and the no-go theorem (Chapter 5) establishes that local completeness is inherently impossible for unrestricted world models.

Part II

The Semantic Kernel

Chapter 3

Queries, Judgments, and World Models

Problem: How does a system ask questions of its uncertain state? We need a query language, a state type, and an extraction law that guarantees combining states combines their answers. The entire semantic interface reduces to one requirement: extraction is a monoid homomorphism.

Layer: Kernel.
Semantic object: `PLNQuery`, `WorldModel`, `WorldModelSigma`.
Proved: Well-formedness of the query–world-model interface; universal property.
Assumed: Nothing.

The first technical move in WM-PLN is to separate what we ask (*queries*) from what we know (*posterior states*). This separation is the foundation of everything that follows.

3.1 The Query Language

In PLN, one may ask: "What is the probability that A is true?" or "What is the probability that B is true given A?" These are queries against an uncertain knowledge base. We represent them as a simple inductive type:

```
inductive PLNQuery (Atom : Type*) where
  | prop  : Atom -> PLNQuery Atom
  | link  : Atom -> Atom -> PLNQuery Atom
  | linkCond : List Atom -> Atom -> PLNQuery Atom
```

A `prop` query asks about a marginal event. A `link` query asks about the conditional relationship between two atoms ($P(B \mid A)$, roughly). A `linkCond` query asks about a conditional given multiple conditioning atoms. This is the minimal query language needed to express the core PLN inference patterns.

The query language is deliberately simple. It is not a full predicate logic; it is the interface through which the world model is interrogated. Richer query languages (first-order, higher-order, temporal) are built on top of this core in later chapters.

Maple Court: In the Maple Court scenario: "Is there a pipe leak?" is a `prop` query on the atom *PipeLeak*. "Does high humidity indicate a leak?" is a `link` query from *HighHumidity* to *PipeLeak*. "Does humidity indicate a leak, given that the shower is running?" is a `linkCond` query conditioning on *ShowerRunning*. Each query, when applied to the apartment's posterior state, returns an evidence pair (n^+, n^-)—for instance, the pipe-leak query might return $(12, 88)$, meaning 12 positive and 88 negative sensor readings.

3.2 World Models

A *world model* is anything that can answer queries. The simplest imaginable world model is a table that assigns an evidence value to every query—an *answer profile* `Query → Evidence`. Richer models store structured state from which the answer profile can be computed. The mathematical content of a world model is a single statement:

> *Every state determines an answer profile, and combining states combines their profiles additively.*

In Lean, we capture this as a typeclass:

```
class WorldModel (State : Type*) (Query : Type*)
    [EvidenceType State] where
  evidence      : State -> Query -> Evidence
  evidence_add  : forall W1 W2 q,
    evidence (W1 + W2) q = evidence W1 q + evidence W2 q
  evidence_zero : forall q, evidence 0 q = 0
```

Here `State` is the type of posterior states (an additive commutative monoid, so revision is `+`), and `Query` is the query type. The method

`evidence` extracts an evidence value for any query from any state. In the simplest instance, evidence is a binary pair (n^+, n^-), but the interface is generic: any `AddCommMonoid` can serve as evidence carrier (Beta counts for binary queries, Dirichlet vectors for multi-valued queries, Normal-Gamma statistics for continuous queries—see Chapter 6). The two laws say that extraction commutes with revision and that the empty state has zero evidence. Together, these make the curried map $W \mapsto (\lambda q.\ \mathrm{ev}(W, q))$ an additive monoid homomorphism from states to answer profiles:

$$\widehat{\mathrm{ev}} : \texttt{State} \xrightarrow{+} (\texttt{Query} \to \texttt{Evidence})$$

This is the entire semantic interface. Everything else—views, compiled rules, inference control—is built on top of this one arrow. Section 3.5 develops the categorical consequences.

For the implementer: to build a new world model, implement `evidence` and prove the two laws. That is all. The rest of the WM-PLN stack—views, compiled BN inference, forgetting, provenance—works automatically for any conforming instance.

Notation. We write $\mathrm{ev}(W, q)$ for $\texttt{evidence}(W, q)$, and $W_1 + W_2$ for the revised state. The evidence algebra on the extracted values uses $\oplus$ for pointwise addition of counts.[1]

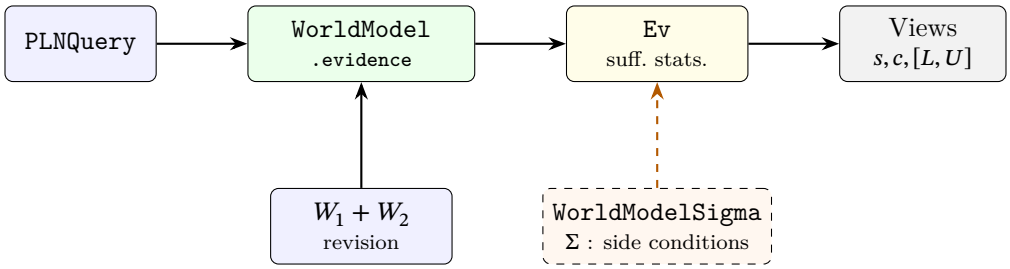


Figure 3.1: The world-model query interface. A `PLNQuery` is answered by extracting evidence (sufficient statistics from any conjugate carrier) from the world-model state; views are lossy projections. Revision (+) combines states. A `WorldModelSigma` bundles structural side conditions for compiled rewrites.

Four verbs. The world-model interface supports four operations, two in the kernel and two in the state-management layer:

[1] Formalized in `Mettapedia/Logic/PLNWorldModel.lean`.

Revise (kernel): combine two states additively ($W_1 + W_2$). This is the primary operation; it corresponds to Bayesian conjugate updating in the reference instance.

Query (kernel): extract evidence from a state for a given question ($\text{ev}(W, q)$). Together with revision, this is the entire semantics.

Forget (state management): remove evidence under explicit scope constraints, preserving answers outside the scope. Forgetting is the dual of revision: it shows that states are not just accumulators but can be *partially retracted*. See Section 9.2.

Explain (state management): trace which observations support an answer, via provenance-enriched evidence. Explanation shows that states are not just number sources but can be *audited*. See Section 9.4.

Revision and querying are the minimal interface; forgetting and explaining show *why* the kernel stores states rather than bare truth values. A bare truth value can be queried but cannot be partially retracted or explained.

The same kernel admits a *typed realization*: the world-model interface can be expressed as a family of typed rewrite rules, where each rule family corresponds to a vertex in a language-definition graph. Native types do not introduce a second semantics—they expose, at the rewrite level, the structural behavior already present in the semantic kernel. Part VII develops this fully.

3.3 World Models with Side Conditions

The kernel interface makes no structural assumptions: any additive state that extracts evidence qualifies. But most *fast* inference depends on structure. When Maple Court's building model uses a Bayesian network to compute humidity-leak correlations in one step instead of marginalizing over all possible worlds, it relies on the DAG factorization and d-separation properties of that network. When the deduction rule computes $P(C \mid A)$ from $P(B \mid A)$ and $P(C \mid B)$, it relies on the screening-off assumption $A \perp C \mid B$.

These are not optional niceties—they are the structural assumptions that make tractable inference *exact* rather than approximate. We capture them with a *sigma-bundled world model*:

```
structure WorldModelSigma (State : Type*) (Query : Type*)
    [EvidenceType State] where
  wm : WorldModel State Query
```

```
  sigma : Prop  -- side-condition context
```

The Σ (side-condition context) is a proposition that the world model satisfies additional structural properties. Every compiled rewrite carries its Σ explicitly—there are no hidden assumptions.

For the implementer: when a compiled rule fires, it checks (or requires proof of) its Σ. If the side condition is not met, the rule does not apply. This is how WM-PLN avoids the silent misapplication that causes semantic drift in traditional PLN.

3.4 The Solomonoff Connection

The νPLN framework of Nil Geisweiller [12] grounds PLN in Solomonoff Universal Induction. The key ideas fold naturally into the world-model framework:

The *global probability space* $(\Omega, \mathcal{F}, \nu)$ corresponds to a world-model state where Ω is the set of possible worlds, each containing a probabilistic predicate $\hat{\mu} : \mathcal{D} \to \Omega \to \text{Bool}$ and its complete evaluation history. The *universal distribution* ν then defines a prior over possible generators $\hat{\mu} \in \mathcal{L}$, where $\mathcal{L}$ is a probabilistic programming language; Bayesian updating with observed evidence produces the posterior. The *observable predicate* μ—the true generator—lives in the true world ω, and an observer has access only to a finite subset of evaluations, which is precisely the evidence. Finally, concepts and predicates are isomorphic via satisfying sets and indicator functions: the inheritance relationship $A \Rightarrow B$ on the concept side corresponds to implication on the predicate side. We work on the predicate side throughout.

The crucial insight from νPLN is that de Finetti's representation theorem [8] justifies evidence counts as sufficient statistics. For an infinite exchangeable sequence of binary observations, the counts (n^+, n^-) contain all the information needed for Bayesian updating. When we assume a Beta prior $\text{Beta}(\alpha, \beta)$, the posterior after observing n^+ positive and n^- negative outcomes is $\text{Beta}(\alpha + n^+, \beta + n^-)$.

This is why PLN's evidence representation (n^+, n^-) is not merely a convenient bookkeeping device—it is the *sufficient statistic* for the posterior under the exchangeability assumption. Chapter 6 develops this algebraic structure in detail.

For the implementer: evidence counts are not an arbitrary design choice. De Finetti's theorem says they are the mathematically forced representation

for exchangeable binary data. Any system that stores less than the counts is provably losing information.

Giry monad and Markov kernels. The categorical underpinning of this story is the *Giry monad* [14]: the functor sending a measurable space to its space of probability measures, with monadic multiplication given by integration ("flattening"). The formalization proves that Kyburg's flattening operation *is* the Giry monad bind: flatten(pd) = pd.mixingMeasure.bind pd.kernel, together with all three monad laws (left identity, right identity, associativity). The de Finetti connection then shows that a `BernoulliMixture` is a specific Kyburg flattening, completing the chain: exchangeability → de Finetti mixture → Kyburg flattening → Giry bind → evidence sufficiency.

This infrastructure delivers the *categorical* justification for evidence revision: $\oplus$ is the sufficient-statistic shadow of the Giry monad's associative bind. What it does *not* yet deliver is a direct Markov-kernel presentation of the world-model interface—where the WM state itself is a Markov kernel $\Omega \to \mathcal{P}(\text{Query} \to \texttt{Evidence})$—that would bypass the multiset-of-structures representation entirely.[2]

See `Mettapedia/ProbabilityTheory/HigherOrderProbability/GiryMonad.lean` (288 lines, 0 sorry), `Mettapedia/ProbabilityTheory/HigherOrderProbability/DeFinettiConnection.lean` (489 lines, 0 sorry), and `Mettapedia/ProbabilityTheory/HigherOrderProbability/KyburgFlattening.lean`. A substantial body of archived formalization pushes this program toward Dirichlet and general Borel-space cases.[3]

[2]Open question: can the Giry monad infrastructure provide a direct measure-theoretic grounding of the world model, without passing through the multiset-of-pointed-structures representation? The ingredients are in place (Giry monad, Markov kernels, de Finetti connection), but assembling them into a `WorldModel (Measure Ω) Query` instance is future work. This would give a direct measure-theoretic↔WM bridge (cf. Section 20.7).

[3]The `external/exchangeability/` archive (117 files, ∼43,700 lines) formalizes de Finetti's theorem via three independent proof routes from Kallenberg (2005): Koopman/mean ergodic theorem, L^2 contractability bounds, and Aldous's reverse martingale argument. The directing-measure construction produces a Markov kernel $\nu : \Omega \to \mathcal{P}(\alpha)$ (proved `IsMarkovKernel`). On the categorical side, `CategoryTheory/DeFinetti*.lean` (15 files, ∼9,100 lines, 0 sorry) builds the Kleisli(Giry) global diagram, a Markov-category bridge, and a kernel-level universal-mediator theorem (`KernelLatentThetaUniversalMediatorInMarkovCore`). Together these provide the infrastructure for extending the exchangeability→sufficiency story from the binary/Beta case (proved above) to Dirichlet-Multinomial and continuous Normal-Gamma cases—but the terminal `WorldModel (Measure Ω) Query` instance assembling these pieces remains future work. **Markov extension (April 2026):** The Diaconis–Freedman (1980) theorem—the Markov-exchangeable analogue where transition counts replace binary

3.5 The Universal Property of the World-Model Interface

Section 3.2 introduced the world-model interface as one arrow $\widehat{\text{ev}} : \texttt{State} \to^{+} (\texttt{Query} \to \texttt{Evidence})$. This section develops the categorical consequences.

Theorem 3.1 (Universal property (CORE))**.** *A world model is exactly an additive monoid homomorphism from states to evidence profiles:*

$$\mathit{WorldModel}(\mathit{State}, \mathit{Query}) \simeq \text{AddMonoidHom}(\mathit{State}, \mathit{Query} \to \texttt{Evidence})$$

The forward direction (`evidenceProfileHom`) bundles evidence extraction into a single arrow; the reverse (`ofProfileHom`) unbundles a profile homomorphism into the typeclass fields. The per-query projection $\text{ev}(-, q) : \mathit{State} \to \texttt{Evidence}$ *is then just evaluation-at-*q *composed with the profile arrow.*

See `Mettapedia/Logic/PLNWorldModel.lean`.

The terminal extensional world model. The answer-profile object Prof(`Query`) := `Query` → `Evidence` is itself a world model: extraction is function evaluation, so both laws hold by `rfl`. Every world model maps into it via evidence extraction, and the map is unique up to the answer profile it determines. This makes Prof(`Query`) the *terminal* extensional world model—the "giant table" that every structured state projects onto.

In categorical terms, world models over `Query` form a slice category **AddCommMon**/Prof(`Query`), and Prof(`Query`) is its terminal object. A morphism between world models is a commuting triangle:

```
structure WorldModelHom (State1 State2 Query) where
  hom  : State1 ->+ State2
  comm : forall W q, evidence (hom W) q = evidence W q
```

The canonical morphism to the terminal model *is* evidence extraction.[4]

counts—is now fully formalized in `Mettapedia/ProbabilityTheory/Exchangeability/MarkovDeFinetti*.lean` (28 files, ∼23,900 lines, 0 sorry). The crown theorem `startRestrictedRowKernelData_directingRowKernel` establishes that the canonical directing kernel satisfies start-restricted row-kernel data, completing the representation theorem for Markov-exchangeable recurrent prefix measures. This extends the evidence-sufficiency story: under Markov exchangeability, the transition-count matrix $N(i, j)$ is sufficient for Bayesian updating over finite-state Markov chains.

[4] Open question: is this slice category enriched in a useful way? The observational preorder (pointwise evidence ordering) gives a 2-categorical structure; the Giry monad work suggests a measure-theoretic enrichment.

Why this matters. The universal property means that the *entire* world-model interface is one algebraic condition. The five rules sometimes listed separately—evidence additivity, revision commutativity and associativity, combine-commutativity, and zero identity—*all derive from the single requirement that extraction is an* `AddMonoidHom`. Commutativity and associativity of revision follow from commutativity and associativity of addition in the evidence monoid. This does not trivialize the project: the hard work is in choosing expressive state spaces, proving no-go theorems, and certifying fast inference paths. But it makes the *interface* minimal and the *achievements* sharper.

See `Mettapedia/Logic/PLNWorldModelProfiles.lean`.

Two orders on evidence. The `Evidence` type carries two natural preorders:

1. The **information order** (the current $\leq$): more positive *and* more negative evidence both count as "more information." This is the right order for the frame, quantale, and subobject-classifier story.

2. The implicit **truth order**: more positive evidence helps, more negative evidence hurts. This is the order that strength respects.

These are *not* the same order, and strength is *not* monotone for the information order:

Theorem 3.2 (Strength non-monotonicity (BOUNDARY))**.** *There exist* $e_1 \leq e_2$ *(in the information order) with* $\mathsf{strength}(e_1) > \mathsf{strength}(e_2)$*. Concretely:* $(1,0) \leq (1,1)$ *but* $\mathsf{strength}(1,0) = 1 > \frac{1}{2} = \mathsf{strength}(1,1)$.

See `toStrength_not_monotone_info_order` in `Mettapedia/Logic/PLNWorldModelProfiles.lean`.

This theorem has far-reaching consequences. It means that the strength-enriched category (Chapter 13) is *not* the image of the evidence quantale under a monotone map. The enriched composition (PLN deduction) and the quantale tensor ($\otimes$) are two related but distinct composition operations:

- **Direct path** (tensor): $\mathsf{strength}(e_1 \otimes e_2) \geq \mathsf{strength}(e_1) \cdot \mathsf{strength}(e_2)$ — a lower bound.

- **Full deduction** (with priors): $\mathsf{strength}(\mathrm{ded}(e_{AB}, e_{BC}, p_B, p_C)) = \mathrm{dedStrength}(s_{AB}, s_{BC}, p_B, p_C)$ — exact under side conditions.

The correct categorical statement is therefore not "the Lawvere enrichment is the quantale enrichment," but rather: *the Lawvere-enriched strength semantics is the scalar shadow of an evidence-level deduction semantics, where tensor gives the direct path and* `deductionEvidence` *adds the residuation correction.*[5]

For the implementer: the universal property means you can swap state representations without changing the semantics. Any two world models that produce the same answer profiles are extensionally equivalent. A Bayesian-network implementation, a factor-graph implementation, and a flat-table implementation are all "the same world model" if they agree on every query. The interface is what matters; the internal state structure is an implementation choice.

Part III develops the evidence object itself—the algebra that gives `Evidence` its structure, the full conjugate carrier family, views as lossy projections, and state management (forgetting, provenance, conservation).

[5]Open question: is there a natural "truth preorder" on `Evidence` for which `strength` IS monotone? The candidate $e_1 \preceq e_2 \iff e_1.\text{pos} \leq e_2.\text{pos} \wedge e_1.\text{neg} \geq e_2.\text{neg}$ is monotone for strength but loses the lattice structure. Whether the evidence algebra admits a clean two-order presentation (information order + truth order) as a single enriched structure is an open categorical question.

Chapter 4

Reference Complete Semantics: Posterior-State Calculus

Goal: Define the core calculus: posterior revision + query interface.
Layer: Kernel.
Semantic object: Dirichlet posterior state over worlds; revision and conditioning.
Proved: Revision commutation; deduction formula admissibility; sequent rules.
Assumed: Finite propositional scope (atoms indexed by $\mathrm{Fin}(n)$).

The complete core calculus is intentionally simple. It is the calculus of revising posterior states plus a query interface. Everything else in WM-PLN—evidence algebras, truth-value views, compiled rules, tractable fragments—is built on top of this core.

4.1 The Reference Instance: Dirichlet over Worlds

For finite propositional scope (atoms indexed by $\mathrm{Fin}(n)$), we instantiate the `WorldModel` interface with a Dirichlet world-table:

$$\texttt{JointEvidence}(n) := \mathrm{Fin}(2^n) \to \mathbb{R}^{\infty}_{\geq 0}$$

interpreted as Dirichlet pseudo-counts over the 2^n complete worlds. Each coordinate $E(w)$ records how much evidence supports world w.

Evidence extraction works by summation over the worlds compatible with each query type. For a propositional query $\texttt{prop}(A)$, the extractor

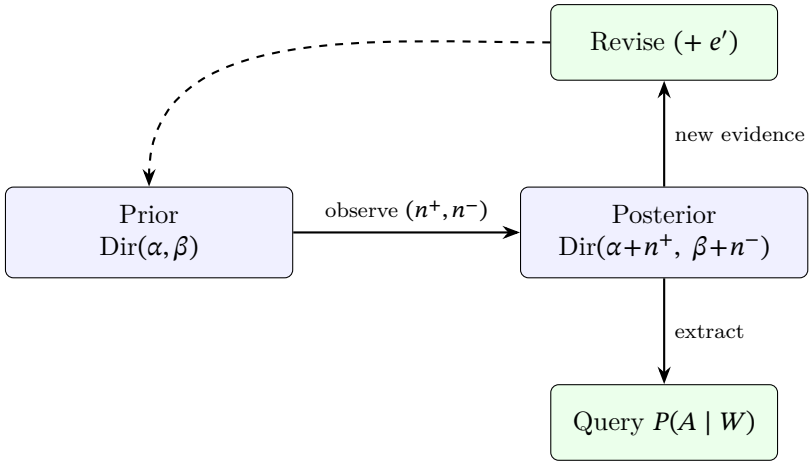


Figure 4.1: The posterior-state update cycle. A Dirichlet prior is updated by observing evidence counts, producing a posterior. The posterior answers queries (downward) and accepts further revision (upward, cycling back).

sums counts where atom A is true (positive evidence) versus false (negative evidence). For a link query $\texttt{link}(A, B)$, it sums counts where A is true and B is true against those where A is true and B is false—conditioning on the antecedent. For a conditional-link query $\texttt{linkCond}([A_1, \ldots, A_k], B)$, it sums counts where all A_i hold and B holds versus all A_i hold and B fails, giving the multi-parent conditional evidence that Bayesian-network CPTs require.

Revision is pointwise addition of pseudo-counts. The Lean prototype proves that this satisfies the world-model interface, including the critical compositionality law:

$$\text{ev}(E_1 + E_2, q) = \text{ev}(E_1, q) \oplus \text{ev}(E_2, q)$$

for all queries q.[1]

4.2 The Sequent-Calculus View

We present WM-PLN as a sequent-style system with two contexts:

- An *evidence context* Γ, a finite multiset of posterior fragments (elements of the state type).

[1]Formalized in `Mettapedia/Logic/PLNJointEvidence.lean`.

- A *side-condition context* Σ, containing structural assumptions about the chosen world-model class (e.g., a BN DAG, positivity, d-separation facts).

4.2.1 Judgments

Two judgment forms suffice:

1. **World-model judgment:** $\Sigma; \Gamma \vdash_{\text{wm}} W$—from the evidence pieces in Γ, we construct the revised posterior state W.

2. **Query judgment:** $\Sigma; \Gamma \vdash q \Downarrow e$—querying the revised posterior derived from Γ yields evidence e for query q.

When `State` is an additive commutative monoid, the world-model judgment is deterministic: $W := \sum \Gamma$. The query judgment then extracts via the world-model interface: $e := \text{ev}(W, q)$.

4.2.2 Core Rules

The complete core calculus consists of four rules:

$$\frac{}{\Sigma; \varnothing \vdash_{\text{wm}} 0} \text{ (WM-Unit)} \qquad \frac{}{\Sigma; \{W\} \vdash_{\text{wm}} W} \text{ (WM-Ev)}$$

$$\frac{\Sigma; \Gamma \vdash_{\text{wm}} W \quad \Sigma; \Delta \vdash_{\text{wm}} W'}{\Sigma; \Gamma \uplus \Delta \vdash_{\text{wm}} (W + W')} \text{ (WM-Rev)}$$

$$\frac{\Sigma; \Gamma \vdash_{\text{wm}} W}{\Sigma; \Gamma \vdash q \Downarrow \text{ev}(W, q)} \text{ (Q-Extract)}$$

The key algebraic law—evidence compositionality—makes the following *query-revision* rule admissible:

$$\frac{\Sigma; \Gamma \vdash q \Downarrow e \quad \Sigma; \Delta \vdash q \Downarrow e'}{\Sigma; \Gamma \uplus \Delta \vdash q \Downarrow (e \oplus e')} \text{ (Q-Rev)}$$

Links are queries. In this view, a PLN "link" $A \Rightarrow B$ is not an object-level connective; it is a query `link(A, B)` against the revised world model. The calculus does not propagate links directly; it revises world models and answers link queries.

4.3 Probability Views

From `Evidence` we obtain posterior-mean probabilities:

$$P(A) = \frac{\#(A)}{\#(\top)}, \qquad P(B \mid A) = \frac{\#(A \wedge B)}{\#(A)}$$

where $\#(\cdot)$ denotes world-count sums in `JointEvidence`. The Lean file `Mettapedia/Logic/PLNJointEvidenceProbability.lean` proves the exact ratio forms for these views.

4.4 Derived Link Calculus: Classic PLN Rules as Admissible Rewrites

Classic PLN rules become *admissible query rewrites* relative to a world-model class with explicit side conditions Σ.

4.4.1 The Deduction Formula

The PLN deduction rule computes $P(C \mid A)$ by total probability:

$$P(C \mid A) = P(C \mid A, B)\, P(B \mid A) + P(C \mid A, \neg B)\, P(\neg B \mid A) \tag{4.1}$$

Under the *screening-off* conditions $P(C \mid A, B) = P(C \mid B)$ and $P(C \mid A, \neg B) = P(C \mid \neg B)$, this simplifies to:

$$P(C \mid A) = P(C \mid B)\, P(B \mid A) + P(C \mid \neg B)\, (1 - P(B \mid A)) \tag{4.2}$$

Writing $s_{AB} = P(B \mid A)$, $s_{BC} = P(C \mid B)$, $s_B = P(B)$, $s_C = P(C)$, and eliminating $P(C \mid \neg B)$ via $s_C = s_B \cdot s_{BC} + (1 - s_B) \cdot P(C \mid \neg B)$, we obtain:

Definition 4.1 (PLN deduction strength)**.**

$$\mathrm{plnDed}(s_{AB}, s_{BC}, s_B, s_C) \;:=\; s_{AB} \cdot s_{BC} + (1 - s_{AB}) \cdot \frac{s_C - s_B \cdot s_{BC}}{1 - s_B}$$

defined when $0 < s_B < 1$.

Theorem 4.2 (Chain BN deduction exactness (Conditional))**.** *Let W be a chain BN world model $A \to B \to C$. Under the explicit hypotheses:*

(i) $0 < \mathsf{strength}(W, \mathit{prop}(B)) < 1$ *(positivity),*

(ii) $\Sigma \vdash C \perp\!\!\!\perp A \mid B$ *and* $C \perp\!\!\!\perp A \mid \neg B$ *(screening-off),*

we have:

$$\mathsf{strength}(W,\ \mathit{link}(A,C)) = \mathrm{plnDed}\big(\mathsf{strength}(W,\ \mathit{link}(A,B)),\ \mathsf{strength}(W,\ \mathit{link}(B,C)),\ \mathsf{strength}(W, \mathit{prop}(B$$

In Lean:

```
theorem chainBN_plnDeductionStrength_exact
    (hpos : 0 < queryStrength W (PLNQuery.prop B))
    (hlt  : queryStrength W (PLNQuery.prop B) < 1)
    (hso  : screeningOff W A B C) :
    queryStrength W (PLNQuery.link A C) =
    plnDeductionStrength
      (queryStrength W (PLNQuery.link A B))
      (queryStrength W (PLNQuery.link B C))
      (queryStrength W (PLNQuery.prop B))
      (queryStrength W (PLNQuery.prop C))
```

The analogous result holds for fork BNs $A \leftarrow B \rightarrow C$ (`forkBN_plnDeductionStrength_exact`).[2]

4.4.2 Induction and Abduction

Induction and abduction arise from the same total-probability structure but with different known/unknown roles:

- **Induction:** given $P(A \mid B)$ and $P(A \mid C)$, estimate $P(C \mid B)$. This inverts the direction of the conditional.
- **Abduction:** given $P(B \mid A)$ and $P(B \mid C)$, estimate $P(A \mid C)$. This inverts the conditioning.

Both require their own screening-off conditions, which are formally stated in the Lean proofs.[3]

4.5 The Kernel as a Typed Rewrite Calculus

The posterior-state calculus has two presentations. *Semantically*, it is a calculus of revisable world-model states and evidence queries—the subject of the preceding sections. *Computationally*, it has a typed rewrite presentation whose rules are adequate to the semantic calculus and whose native types classify the allowable terms and reductions.

[2] Formalized in `Mettapedia/Logic/PLNBayesNetFastRules.lean`.
[3] Formalized in `Mettapedia/PLN/RuleFamilies/FirstOrder/PLNDerivation.lean`.

Universal property. The five core rules (evidence additivity, revision commutativity and associativity, combine commutativity, combine identity) are not five independent axioms. They all derive from a single algebraic condition: extract is an AddCommMonoid homomorphism—i.e., $\mathsf{extract}(\mathsf{Revise}(W_1, W_2), q) = \mathsf{extract}(W_1, q) + \mathsf{extract}(W_2, q)$ and $\mathsf{extract}(\mathsf{Zero}, q) = 0$. Commutativity and associativity of revision follow from commutativity and associativity of addition in the evidence monoid (`wmCore_sound_of_addCommMonoidHom`, kernel-checked; full development in Chapter 19).

The 2×2 calculus square. The core rules encode as a `LanguageDef`—a typed list of rewrite rules with pattern-matching and optional side conditions. Two orthogonal refinements produce four calculi:

	Plain	**+Congruence**
Raw (schematic)	all rules fire	+ context descent
Guarded (faithful)	side-condition oracle	+ context descent

The raw calculus overapproximates the guarded one (dropping side conditions only enables more reductions). The congruence extension adds explicit rules for descending into subterms. All four corners carry a Galois connection $\Diamond \dashv \Box$ derived automatically by the OSLF pipeline.

Adequacy. The calculus terms are given by an intrinsically sorted type $\mathsf{WMTerm} : \mathsf{WMSort} \to \mathsf{Type}$ with three sorts (State, Query, Evidence). A computable encoding $\mathsf{encodeWM} : \mathsf{WMTerm}\ s \to \mathsf{Pattern}$ is injective, and the abstract reduction WMStep is *biconditionally adequate*: a single step at the abstract level holds if and only if the corresponding pattern-level reduction holds, and this lifts to multi-step reductions (`wmStepStar_adequate`, kernel-checked). Backward completeness holds on the image of encodeWM (`wmBoxOnImage_iff`, kernel-checked).

Native types. Each variant of the calculus (each vertex in a 13-axis parameter space) produces a `LanguageDef`. Passing it through the OSLF pipeline yields modal operators $\Diamond/\Box$, and then through native type synthesis yields a type system that classifies WM terms by their rewrite behavior. The NTT pass is exact for the 9 syntactic extension axes and intentionally insensitive to the 4 model-level axes (`wmVertexNativeType_eq`, kernel-checked). Native types are thus the kernel's natural operational types—they arise from the rewrite structure itself, not from an external type discipline.

Chapter 19 develops the full 13-axis vertex family, the vertex preorder with `ForwardFiber` functor, the compiled BN inference bridge (positive/negative/completeness theorems), and the semantic fibers to neighborhood and first-order logic models.

Chapter 5

The Local-Completeness No-Go Theorem

Goal: Establish the fundamental limit of local link-level inference.
Layer: Kernel (boundary).
Semantic object: World tables with distinct marginal-compatible joint structures.
Proved: No-go: complete link evidence cannot be computed from per-link evidence alone.
Assumed: Nothing (unconditional counterexample).

One might hope for a system that takes local per-link evidence (`Evidence` for A, B, C, $A \Rightarrow B$, $B \Rightarrow C$) and computes complete evidence for $A \Rightarrow C$. WM-PLN asserts that this is impossible in general.

Theorem 5.1 (No local complete deduction rule (Boundary))**.** *There is no function*

$$f : \texttt{Evidence}^5 \to \texttt{Evidence}$$

that, for all joint evidence states E, computes the exact link evidence for $A \Rightarrow C$ from only the local premises $\mathrm{ev}(E, A)$, $\mathrm{ev}(E, B)$, $\mathrm{ev}(E, C)$, $\mathrm{ev}(E, A \Rightarrow B)$, $\mathrm{ev}(E, B \Rightarrow C)$.

Proof sketch. For three atoms A, B, C, there are $2^3 = 8$ possible worlds. We construct two joint evidence states $E_1, E_2 : \mathrm{Fin}(8) \to \mathbb{R}_{\geq 0}$ that agree on all

five marginal evidence values:

$$\mathrm{ev}(E_1, A) = \mathrm{ev}(E_2, A), \quad \mathrm{ev}(E_1, B) = \mathrm{ev}(E_2, B), \quad \mathrm{ev}(E_1, C) = \mathrm{ev}(E_2, C),$$

$$\mathrm{ev}(E_1, A \Rightarrow B) = \mathrm{ev}(E_2, A \Rightarrow B), \quad \mathrm{ev}(E_1, B \Rightarrow C) = \mathrm{ev}(E_2, B \Rightarrow C),$$

but disagree on the conclusion:

$$\mathrm{ev}(E_1, A \Rightarrow C) \neq \mathrm{ev}(E_2, A \Rightarrow C).$$

The key: world entries corresponding to $(A = 1, B = 0, C = 1)$ and $(A = 1, B = 1, C = 0)$ can be traded off against each other while preserving all five marginals. But this trade changes the $A \Rightarrow C$ evidence because it redistributes mass among worlds where A is true.

Since the five marginals cannot distinguish E_1 from E_2, no function $f : \texttt{Evidence}^5 \to \texttt{Evidence}$ can compute $\mathrm{ev}(E, A \Rightarrow C)$ correctly for both states. □

[1]

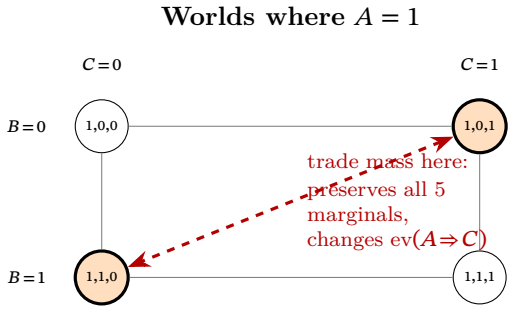


Figure 5.1: The no-go witness. Among the four worlds where $A=1$, redistributing mass between $(1, 0, 1)$ and $(1, 1, 0)$ (orange) preserves all five local marginals (A, B, C, $A \Rightarrow B$, $B \Rightarrow C$) but changes $\mathrm{ev}(A \Rightarrow C)$. No function of the five marginals can detect this trade.

5.1 Interpretation

This theorem is the formal core of "you must carry correlations." It does *not* make fast PLN useless. It tells us exactly what fast PLN *cannot* claim: completeness for arbitrary world models without structural assumptions.

The constructive response is to:

[1] Formalized in `Mettapedia/Logic/PLNJointEvidenceNoGo.lean`.

1. **Restrict the model class.** For Bayesian networks with explicit DAG structure, the factorization makes local computation exact (Chapter 11).

2. **State the assumptions.** Every compiled rule carries its Σ context, so the user knows exactly when the fast rule applies.

3. **Provide corrected alternatives.** Where an original PLN formula required an unstated assumption, provide the corrected side-conditioned version alongside the boundary theorem.

Part III

Evidence, State, and Views

Chapter 6

Evidence Algebra

Problem: When a world model answers a query, it returns *evidence*—raw observation data, not yet summarized into a truth value. We need an algebra that can combine evidence from multiple sources (revision), compare evidence by informativeness (ordering), compose evidence along inference chains (tensor), and reason about the absence of evidence (Heyting complement).

Layer: Evidence.
Semantic object: `Evidence` with revision ($\oplus$), ordering, quantale ($\otimes$), Heyting lattice.
Proved: Quantale and Heyting frame axioms; information ordering is a lattice.
Assumed: Nothing.

6.1 The Generic Evidence Pattern

Evidence in WM-PLN is not a single fixed type. It is a *family* of algebraic structures that all share the same interface:

1. **Revision is addition.** Combining two bodies of evidence is a commutative, associative operation with a zero element (no evidence). Algebraically: evidence values form an `AddCommMonoid`.

2. **Information is ordered.** Evidence values carry a partial order where "more data" means "at least as informative."

3. **Composition is tensor.** Chaining two inferences multiplies their evidence via a monoidal tensor that distributes over joins (a quantale).

4. **Logic is Heyting.** Evidence supports intuitionistic implication and complement, but not classical excluded middle.

The Lean formalization captures this genericity via the polymorphic `AddCommMonoid Ev` interface: every generic world-model theorem is stated over an abstract evidence type `Ev`, not over a specific carrier. The binary pair (n^+, n^-) is the simplest instance; the next chapter develops three more (Beta, Dirichlet, Normal-Gamma).

For the implementer: any structure with `+` and `0` satisfying the commutative monoid laws can serve as evidence. The choice of carrier determines the domain (binary, multi-valued, continuous) but not the algebraic discipline.

Instance 1: Binary evidence. The simplest evidence type is a pair of non-negative counts:

$$\texttt{Evidence} := (n^+, n^-) \in \mathbb{N} \times \mathbb{N}$$

where n^+ counts positive observations and n^- counts negative observations. (In the Lean formalization, we sometimes work over $\mathbb{R}_{\geq 0}$ or $\mathbb{R}^{\infty}_{\geq 0}$ for compatibility with measure-theoretic constructions.) The total sample size is $n = n^+ + n^-$.[1]

Revision is componentwise addition:

$$(n_1^+, n_1^-) \oplus (n_2^+, n_2^-) = (n_1^+ + n_2^+, n_1^- + n_2^-)$$

This is associative, commutative, and has unit $(0, 0)$—the evidence of having observed nothing.

Why addition? The justification comes from de Finetti's representation theorem. For exchangeable binary sequences, the counts are sufficient statistics for the posterior. Adding counts corresponds to Bayesian conjugate updating with a Beta prior:

$$\text{Beta}(\alpha + n_1^+, \beta + n_1^-) \xrightarrow{+\,(n_2^+, n_2^-)} \text{Beta}(\alpha + n_1^+ + n_2^+, \beta + n_1^- + n_2^-)$$

Revision is addition because it is the algebraic shadow of Bayesian conjugate updating.

6.2 The Atomic Event Space Comes First

The additive law above has a condition that is easy to omit and costly to omit: one must first say what counts as one observation. Evidence is not

[1] Formalized in `Mettapedia/Logic/EvidenceClass.lean`.

a bare number. It is a valuation of a declared event space. Changing the event identity changes the valuation, even when the underlying records are unchanged.

Let E be a finite set of authenticated atomic records and let $\pi : E \to F$ be a projection to the feature being counted. Define

$$V_\pi(A) = |\pi[A]|, \qquad A \subseteq E.$$

Typical choices of π include source record, program identity, target identity, test obligation, proof conclusion, or provenance root. Each gives a valid count, but they answer different questions.

Theorem 6.1 (Projected valuation law (CORE))**.** *For finite event sets $A, B \subseteq E$,*

$$V_\pi(A \cup B) + |\pi[A] \cap \pi[B]| = V_\pi(A) + V_\pi(B).$$

Consequently,

$$V_\pi(A \cup B) = V_\pi(A) + V_\pi(B) \quad \Longleftrightarrow \quad \pi[A] \cap \pi[B] = \varnothing.$$

Disjointness of A and B is not sufficient: two distinct source records may collide after projection.

This is ordinary finite inclusion–exclusion, but placing the projection in the theorem changes the statistical discipline. "Independent files," "different model runs," and "different proofs" do not by themselves license additive evidence for program identities, targets, conclusions, or causal roots. Additivity is tested in the space whose cardinality is being reported.

The Lean development uses Mathlib's `Finset.image` and finite inclusion–exclusion directly. The generic results are `projectedSet_union`, `projected_inclusion_exclusion`, `projected_union_eq_sum_iff`, and `projected_not_disjoint_of_collision`.[2]

Knuth–Skilling interpretation. A Knuth–Skilling valuation is additive on disjoint joins in its chosen lattice. The theorem above identifies the often-hidden modeling decision: which quotient or projection determines that lattice. The valuation theorem does not choose an atom for us. If raw records are quotiented by program identity, source lineage, or semantic target, the join and its overlap must be computed after that quotient. Thus event identity is logically prior to the numeric valuation.

[2]Formalized in `Mettapedia/MachineLearning/SearchGuidance/ProgramDiscovery/Accounting.lean`.

6.2.1 A general identifiability boundary

Let $O : W \to X$ be an observation map and $\theta : W \to Y$ an estimand. If there exist worlds w_1, w_2 such that $O(w_1) = O(w_2)$ but $\theta(w_1) \neq \theta(w_2)$, then no function $g : X \to Y$ uniformly recovers θ from O. This elementary observation-collision argument is formalized once as `no_uniform_recovery_of_observation_collisio` and reused below. It is the finite, constructive form of a broad warning: a selected table, aggregate count, or projected marginal cannot identify information it has erased.

6.2.2 Evidence availability is not addressability

The same boundary appears when an evidence table is used as a memory. A lineage-auditable atomic memory fact has the form

$$(m, \sigma, q, p, t, \pm, \ell),$$

recording memory identity, aggregate signature, query, program, solved target, polarity, and lineage. Projecting this atom to $(\sigma, \pm)$ supplies a valid aggregate evidence table, but collisions erase the program–target competence edge (p, t), the routing edge (q, p), and lineage ℓ. The Lean theorems `aggregateKey_does_not_recover_competence`, `aggregateKey_does_not_recover_accessibility`, and `aggregateKey_does_not_recover_lineage` instantiate the general observation-collision theorem. The positive boundary is equally important, and it is an exact characterization rather than a restatement of its own hypothesis: `recoverable_iff_respectsAggregateCollisions` proves that an estimand is recoverable from the aggregate key precisely when it is constant on aggregate-key collisions, constructing the decoder in the nontrivial direction. The negative results are its contrapositive at a single colliding pair.

Let A be the targets with stored witnesses and R those reached by the router. Then $R \subseteq A$, and $R = A$ exactly when every available target has at least one stored witness selected at its recorded query (`routedTargets_eq_availableTargets_iff`). Thus immutable storage can preserve availability while a router loses accessibility. Direct-target coverage is a third projection: a program discovered for query q may solve a different target and add a genuine program–target fact without being a direct hit. Aggregate support, available competence, routed competence, and direct conditioning are therefore four different estimands.

For the Unified neural carrier this distinction yields a conservative architecture. Keep the existing aggregate prior and place provenance-rich, lineage-bound initializers in a sidecar. Forgetting the sidecar recovers the aggregate treatment, and a disabled lookup returns the cold initializer with the

original memory; both hold by construction of the proposed transition, so they are interface laws for the design rather than measurements of a deployed system. Rejection is state preserving for an arbitrary acceptance predicate, and the substantive safety property is that the router is fail-closed: when acceptance is sound for the certified set, every state it can return is either the declared cold initializer or one satisfying the already formalized lineage, shape, and solver-basin predicate (`routedInitializer_sound_accept_cold_or_certified`). A negative mutating-router fixture shows that cold-output equality alone would not certify storage preservation (`mutatingRouter_changes_storage_fixture`). These are theorems about a named memory treatment, not evidence that an aggregate prior already contains addressable knowledge.

6.2.3 Relational novelty: facts are not objects

Program discovery naturally produces a finite relation

$$R \subseteq \mathsf{Program} \times \mathsf{Target}.$$

It therefore has at least three non-equivalent cardinalities:

$$|R|, \qquad |\pi_{\mathsf{P}}[R]|, \qquad |\pi_{\mathsf{T}}[R]|.$$

These are, respectively, verified program–target facts, distinct program identities, and covered targets. Neither program count nor target count is a denominator for the other.

Relative to a known relation K, a new edge $(p, t) \in R \setminus K$ has two different novelty questions:

1. Is (p, t) a new fact?
2. Is p a new program identity, or is a known program being re-attributed to another target?

The Lean partition `reusedProgramNewEdges`/`freshProgramNewEdges` is disjoint and exhaustive, and `programSet_freshProgramNewEdges` proves that projection of the fresh edge class is exactly the fresh program-identity set. A positive fixture has three new facts—one reuse fact and two fresh-program facts—but only one fresh program identity. This is the smallest example showing why "new facts" and "new objects" cannot be reported interchangeably.

6.2.4 Selected representatives are views, not ledgers

Choosing one shortest or fastest witness per target can preserve target coverage while destroying edge multiplicity and program diversity. For a selected subrelation $S \subseteq R$, exact program-diversity preservation is equivalent to

$$\pi_{\mathsf{P}}[S] = \pi_{\mathsf{P}}[R],$$

not to target completeness. The Lean theorem `programCoverage_eq_iff_programSet_eq` gives the exact condition; `selected_program_diversity_not_cap_invariant` supplies the negative fixture.[3]

This distinction matters for every "best representative" report. A table with one row per target is suitable for target coverage and for properties of the chosen representative. It is not a census of all verified programs, all verified facts, or their provenance.

6.2.5 Registered binary trials

A Bernoulli summary $(s, n - s)$ is well typed only when the successes are a subset of one declared trial set T:

$$S \subseteq T, \qquad s = |S|, \quad n = |T|.$$

The structure `FiniteBinarySample` stores exactly this witness and maps it into the existing `BernoulliObservations` carrier. For two samples, `merge_total_additive_iff` proves that total-trial addition is valid exactly when their trial identities are disjoint. The corresponding PLN evidence addition theorem is `evidence_merge_of_disjoint`. Merging the same registered trial with itself retains one trial, whereas blind Bernoulli combination reports two; `repeated_trial_is_not_fresh_evidence` is the explicit negative example.[4]

This typing rule does not assert IID sampling. It establishes only that the finite count pair has one coherent atom. Exchangeability, random sampling, or a finite-population design is an additional model license. In particular, a Beta posterior placed over fixed benchmark tasks is hypothetical until one states the population or exchangeability claim that makes those tasks trials from a common process.

[3] Formalized in `Mettapedia/MachineLearning/SearchGuidance/ProgramDiscovery/CapInvariance.lean`.

[4] Formalized in `Mettapedia/MachineLearning/SearchGuidance/ProgramDiscovery/EvidenceBridge.lean`.

6.2.6 Unit tests and coverage

Testing has the same relational form:

$$C \subseteq \mathsf{Test} \times \mathsf{Obligation}.$$

The number of tests, the number of covered obligations, and the number of test–obligation edges are different projections. One integration test may cover many obligations; many tests may cover the same obligation. The `TestCoverage.Relation` bridge reuses the program-discovery accounting unchanged, and `obligationCoverage_union_eq_sum_iff` says that suite coverage is additive exactly when the obligation footprints are disjoint. A negative fixture exhibits two suites with equal obligation coverage and different numbers of contributing tests.

Consequently, "90% of tests passed" and "90% of obligations are covered" are not two presentations of one probability. The first uses tests as trials; the second is a finite coverage fraction over registered obligations. Mutation kills, branch obligations, requirements, and theorem statements can each be legitimate atoms, but the atom must be named.

6.2.7 ProbLog explanations and MLN groundings

The event-space rule also clarifies the probabilistic-programming bridges. In ProbLog, distinct derivations of a query may overlap in possible worlds. Adding explanation probabilities computes bag semantics; distribution semantics computes the probability of their union. The existing formal theorem `bag_set_separation` proves strict over-counting when at least two positive mechanisms contribute, while `noisyOr_le_sum` gives the Boole bound.[5] In the present language, explanations are source objects, worlds are the projection space, and explanation overlap is a projection collision.

For an MLN, ground clause occurrences are factors in a Gibbs model. They are not automatically independent Bernoulli observations: groundings can share atoms and are evaluated in the same possible world. The correct WM import therefore retains the factor graph and queries its normalized world mass. The grounding theorems `compileToGroundMLN_worldWeight_eq` and `clauseWM_queryStrength_eq_queryProb` justify that path. Treating the satisfied-grounding count as independent evidence would discard precisely the dependence the factor graph was built to represent.

[5] Formalized in `Mettapedia/PLN/Bridges/Languages/ProbLog/Compilation.lean`.

6.2.8 Gaussian learning and provenance-aware sufficient statistics

The rule is not special to count evidence. A Gaussian contribution in precision/natural-parameter coordinates is

$$e_i = (\Lambda_i, \eta_i), \qquad e_A = \left(\sum_{i \in A} \Lambda_i, \sum_{i \in A} \eta_i\right).$$

For distinct registered events,

$$e_{A \cup B} + e_{A \cap B} = e_A + e_B.$$

The Lean theorem `aggregateDistinctGaussian_inclusion_exclusion` proves this coordinatewise using Mathlib's finite-sum inclusion–exclusion. `aggregateProjectedGaussian_inc` strengthens it: the correction is the overlap after projection to the declared identity. The earlier `GaussianEvidence.reused_contribution_exact_excess` shows the failure mode exactly—reprocessing one causal contribution adds one extra precision matrix and one extra natural-parameter vector.[6]

Two measurements with the same numeric value may be distinct evidence if they are distinct measurement events. Conversely, two files containing the same causal measurement are one event for provenance-aware revision. Values do not determine identity; authenticated occurrence and causal root do.

6.2.9 Clustered uncertainty and capture–recapture

Deduplication prevents exact double counting but does not manufacture independence. For trial variables X_i, Mathlib's covariance expansion gives

$$\mathrm{Var}\left(\sum_i X_i\right) = \sum_i \sum_j \mathrm{Cov}(X_i, X_j).$$

Only under pairwise independence does this reduce to $\sum_i \mathrm{Var}(X_i)$. Both statements are exposed as `registered_sum_variance_eq_covariance_sum` and `registered_sum_variance_of_pairwise_independent`. For an exchangeable cluster of size n, marginal variance v, and positive pair covariance c, the variance is

$$nv + n(n-1)c > nv \qquad (n \geq 2, c > 0),$$

[6] Formalized in `Mettapedia/MachineLearning/ContinualLearning/EvidenceLedger.lean` and the program-discovery evidence bridge.

formally proving that an independent-trial error bar is too small.

Capture–recapture has a complementary identifiability problem. Two lists determine $|A|$, $|B|$, and $|A \cap B|$, but do not reveal individuals in neither list. The Lean fixture `summary_does_not_identify_populationSize` constructs two finite populations with identical capture summaries and different sizes. A complete census is a positive exact case; otherwise homogeneous catchability, stratification, or another sampling model is load-bearing. Estimating the universe from the same overlap and then reporting that overlap as an independence test is circular.

6.2.10 Case study: complete relational accounting of two search rounds

The four-arm program-discovery campaign provides a concrete example. A new hash-bound analyzer reconstructs every verified $(program, target)$ witness from the sealed checker maps, rather than only the selected representative for each target. Table 6.1 reports the three principal projections and the number of program identities absent from the catalog before each round.

Round	Arm	Edges	Programs	Targets	Fresh programs
1	AC-BP	393	312	336	110
1	AC-PC	38	35	34	32
1	Unified-BP	96	95	70	69
1	Unified-PC	104	101	73	74
2	AC-BP	224	211	215	93
2	AC-PC	18	17	17	15
2	Unified-BP	115	113	82	79
2	Unified-PC	125	89	95	71

Table 6.1: Witness-complete relational accounting. "Edges" are verified program–target facts; programs and targets are their two projections; fresh programs are identities absent from the pre-search catalog.

The selected union is target-complete but not relation-complete. In Round 1 it retains 493 of 612 union edges and 445 of 525 program identities while preserving all 493 targets. In Round 2 it retains 396 of 470 edges and 346 of 418 programs while preserving all 396 targets. The selected table is therefore valid for target coverage and invalid as a program or edge census.

The Round 2 Unified comparison becomes especially sharp after projecting the complete relation by identity class:

Arm	Same-target catalog	Cross-target reuse	Fresh-program support	Same∩fresh	Target union
Unified-BP	34	0	54	6	82
Unified-PC	17	35	49	6	95

Table 6.2: Round 2 Unified target support by witness identity class. Columns overlap: six targets in each arm have both a same-target catalog witness and a fresh-program witness, so inclusion–exclusion yields the final union.

Thus Unified-PC's +13 target advantage decomposes as −17 same-target catalog recall, +35 cross-target transfer, −5 fresh-program-supported coverage, with the same six-target overlap correction. At the program level, Unified-BP actually has more fresh identities (79 versus 71). The appropriate claim is therefore not "PC generated more novel programs." It is that this round's PC arm converted known program structure into more new program–target facts, while generating slightly fewer fresh identities.

The analyzer also emits coherent binary summaries only where success and trial atoms agree: direct hits per 240 registered queries, checker-covered programs per distinct proposed program, and new-target-covering programs per distinct proposed program. Beta(1, 1) summaries are included only as hypothetical IID views; the fixed benchmark and shared training ancestry do not themselves license IID or source-independence claims.

6.2.11 Architecture is an effect modifier, but the Round 2 gain is hub-sensitive

The four cells also support a finite architecture-by-credit interaction for every registered scalar endpoint s:

$$I_s = \big[s(\text{Unified-PC}) - s(\text{Unified-BP})\big] - \big[s(\text{AC-PC}) - s(\text{AC-BP})\big].$$

This is a difference in differences. It asks whether the observed PC–BP contrast changes with architecture; it is not an architecture main effect. The definition `architectureUpdateInteractionBy` makes the endpoint an explicit argument. The fixture `architectureInteraction_changes_with_endpoint_projection` proves that the same four relation-valued cells can have zero target interaction and positive program-diversity interaction. There is therefore no unqualified "architecture interaction" before the projection has been named.[7]

[7]Formalized in `Mettapedia/MachineLearning/SearchGuidance/ProgramDiscovery/PairedDesignEstimands.lean`.

Round	Endpoint	AC: PC−BP	Unified: PC−BP	Interaction
1	Edge facts	−355	8	363
1	Program identities	−277	6	283
1	Target identities	−302	3	305
1	Fresh program identities	−78	5	83
1	Cross-target reuse facts	−1	0	1
2	Edge facts	−206	10	216
2	Program identities	−194	−24	170
2	Target identities	−198	13	211
2	Fresh program identities	−78	−8	70
2	Cross-target reuse facts	−5	35	40

Table 6.3: Endpoint-specific PC–BP contrasts and their architecture difference in differences. Positive interaction means that the PC contrast is more favorable under Unified than under AC; it need not mean that PC is positive in either architecture.

The sign pattern repeats across both rounds: PC is severely negative under AC and near-neutral or positive for relation and target coverage under Unified. Thus architecture is a replicated *observed effect modifier* for these fixed rounds. However, the mechanism changes. In Round 1 the Unified interaction is carried by modest gains in edges, programs, and targets. In Round 2, Unified-PC has fewer distinct and fresh programs than Unified-BP but more targets, with 35 cross-target reuse facts.

To measure concentration, for a program p define its target degree and exclusive contribution by

$$d_R(p) = |\{t : (p,t) \in R\}|, \qquad e_R(p) = |\pi_{\mathsf{T}}(R) \setminus \pi_{\mathsf{T}}(R \setminus (\{p\} \times \mathsf{T}))|.$$

The Lean results `targetCoverage_withoutProgram_add_exclusive` and `exclusiveTargetContribution_le_programTargetDegree` prove

$$|\pi_{\mathsf{T}}(R \setminus p)| + e_R(p) = |\pi_{\mathsf{T}}(R)|, \qquad e_R(p) \le d_R(p).$$

The inequality can be strict: two programs may witness the same targets. A formal negative fixture gives equal degree two to a program in two relations, but exclusive contributions two and zero, respectively.

This robustness statistic exposes the Round 2 mechanism. One known Unified-PC program has degree 36: it is a same-target catalog witness for one target and a cross-target reuse witness for 35 others. All 36 targets are exclusive to it. Removing that single identity changes Unified-PC target coverage from 95 to 59; removing the most essential Unified-BP program

changes 82 to 80. The Unified PC–BP target contrast therefore changes from $+13$ to -21 under the deterministic leave-one-program-out sensitivity analysis. The architecture interaction remains positive—176 instead of 211—because AC-PC remains much weaker, but the within-Unified headline is not diffuse.

This also resolves an apparent rate puzzle. In Round 2 the fraction of distinct proposed programs that cover a new target is $113/17133 \approx 0.00660$ for Unified-BP and $89/15136 \approx 0.00588$ for Unified-PC. PC does not improve broad program-level hit rate. It obtains more target coverage because one reusable program has unusually high target multiplicity. Conversely, Round 1 AC-BP contains two degree-35 programs whose exclusive contributions are zero: high degree need not imply fragility when witnesses are redundant.

These observations motivate, but do not prove, an architectural mechanism. The Unified carrier may make structured cross-target reuse accessible to PC credit, while AC's recurrent decoder may turn the same credit rule into broad proposal degradation. The present design bundles carrier, decoder, and representation, so it identifies the four-cell endpoint contrast but not a causal effect of an isolated architectural component. A decisive follow-up uses a common proposal/checker interface, crosses carrier with credit rule, and preregisters both ordinary coverage and concentration-robust endpoints: fresh identities, cross-target facts, maximum exclusive contribution, and leave-one-program-out target coverage. Multiple authenticated seeds or worlds are then needed for a population claim.

Overlap as a population estimator, and what it requires. Two arms that search the same space invite a further inference: read their overlap as a capture–recapture experiment and estimate how much remains undiscovered. For samples A and B from a population of size N, the Lincoln–Petersen estimator is

$$\hat{N} = \frac{|A|\,|B|}{|A \cap B|},$$

which is unbiased in the large-sample limit exactly when the two draws are independent and every member is equally catchable. Both licenses are projection-relative, and neither is free.

A registered control supplies a clean instance. Two models differing only in training history—one continued across rounds, one retrained from the same initial weights on the accumulated corpus—were searched and checked identically. Applying the estimator to the same pair of runs under the three projections of the previous section gives, for the Unified carrier, $\hat{N} = 973$

on targets, 4600 on program identities, and 5046 on verified facts; for the recurrent carrier, 2674, 3726, and 3932. The Unified estimates differ by a factor of nearly five across projections of one experiment.

This is not estimator error. The three numbers estimate three different populations, and the target projection is the one that collapses hardest: two distinct programs solving one target are two program-level events but one target-level event, so projection to targets can only increase the observed intersection, and increasing $|A \cap B|$ decreases $\hat{N}$. A reported "fraction of the reachable space still unexplored" is therefore meaningless until the projection is declared, exactly as a reported count is.

The independence license fails in the same instance, and its direction is knowable. The two arms share architecture, tokenizer, language, and corpus, so they are positively dependent: they tend to find the same easy targets. Positive dependence inflates $|A \cap B|$ relative to independent sampling and therefore deflates $\hat{N}$. Every number above is consequently a lower bound on its population rather than an estimate of it—a one-sided conclusion that survives the broken license, which is the most that should be claimed. The general pattern is worth stating: when a sampling model fails in a known direction, the estimator is not discarded but demoted to the bound that its failure direction permits.

Implementation contract. For evidence-producing systems, the safe order is:

1. register the atomic event key, including provenance needed for causal deduplication;
2. retain the witness-complete relation or multiset of occurrences;
3. declare each projection used by a reported statistic;
4. correct overlap in the projected space;
5. distinguish arithmetic aggregation from independence, exchangeability, and sampling-model licenses;
6. treat selected representatives and scalar truth values as views, never as replacements for the carried evidence state.

6.3 Information Ordering

Not all evidence values are equal. Some contain strictly more observations than others. But the relationship is not always clear-cut: $(3, 0)$ and $(0, 3)$ contain the same total number of observations, yet neither is "more informative" than the other—they are about different things.

Evidence values are partially ordered by information content:

$$(n_1^+, n_1^-) \leq (n_2^+, n_2^-) \iff n_1^+ \leq n_2^+ \ \wedge \ n_1^- \leq n_2^-$$

More evidence is always at least as informative. This ordering makes `Evidence` a lattice with meet (componentwise min) and join (componentwise max).

For the implementer: evidence comparison is always a partial order. Two evidence values may be incomparable—this is a feature, not a bug. It means no single scalar summary can capture "how much evidence" you have.

6.4 Compositional Tensor (Quantale Structure)

When you know that A implies B with evidence e_{AB}, and that B implies C with evidence e_{BC}, what can you say about A implying C? The deduction rule needs a way to *compose* evidence along inference chains. The quantale tensor $\otimes$ is that composition.

Beyond the additive monoid and lattice, `Evidence` carries a *quantale* structure. A quantale is a complete lattice equipped with a monoidal operation $\otimes$ that distributes over arbitrary joins:

$$a \otimes \Big(\bigvee_{i \in I} b_i \Big) = \bigvee_{i \in I} (a \otimes b_i)$$

For $\texttt{Evidence} = (n^+, n^-)$, the tensor is defined as:

$$(a^+, a^-) \otimes (b^+, b^-) = (a^+ \cdot b^+, \ a^+ \cdot b^- + a^- \cdot b^+)$$

with unit $(1, 0)$. This corresponds to the compositional combination of dependent evidence streams: observing a and then observing b conditional on a being positive.

Theorem 6.2 (Quantale transitivity (Core))**.** *For evidence values e_{AB}, e_{BC} representing $A \Rightarrow B$ and $B \Rightarrow C$ respectively:*

$$e_{AB} \otimes e_{BC} \ \leq \ e_{AC}$$

in the information ordering. Equality holds under the screening-off conditions.

The tensor product of premise evidence provides a *lower bound* on the conclusion evidence. This is the algebraic shadow of the deduction rule.[8]

For the implementer: when chaining two PLN inferences, tensor gives a sound but possibly conservative estimate. Exact equality requires the screening-off side condition (developed in Part IV).

6.5 Logical Operations (Heyting Algebra)

Evidence supports not just comparison and composition, but also logical operations. You can ask: "given evidence a for one claim and evidence b for another, what evidence supports $a \Rightarrow b$?" The answer uses the Heyting algebra structure.

`Evidence` forms a Heyting algebra with an explicit pseudo-complement:

Definition 6.3 (Heyting implication on `Evidence` (Core))**.** The relative pseudo-complement is defined componentwise:

$$(a^+, a^-) \Rightarrow (b^+, b^-) = \left(\begin{cases} \top & \text{if } a^+ \le b^+ \\ b^+ & \text{otherwise} \end{cases}, \begin{cases} \top & \text{if } a^- \le b^- \\ b^- & \text{otherwise} \end{cases} \right)$$

satisfying the residuation law: $a \le b \Rightarrow c \iff a \wedge b \le c$.

The Heyting complement (negation) is $\neg a = a \Rightarrow \bot = a \Rightarrow (0, 0)$.

This matters because the evidence algebra is *intuitionistic*. The law of excluded middle fails: $a \vee \neg a \neq \top$ in general, because there may be no evidence at all. Classical logic assumes every proposition is either true or false; evidence algebra does not, because you may simply not have observed anything relevant.

A key formalized fact—the "totality gate"—establishes that `Evidence` has incomparable elements (e.g., $(3, 0)$ and $(0, 3)$) and admits no faithful order-embedding into the reals. No single real-valued function captures all evidence information.[9]

For the implementer: no single real number captures all evidence information. Any real-valued summary (strength, confidence, etc.) is a *lossy projection*—useful for display but not for sound inference chaining. Chapter 8 develops these projections explicitly.

---

[8] Formalized in `Mettapedia/Logic/EvidenceQuantale.lean`.

[9] Formalized in `Mettapedia/Logic/HeytingValuationOnEvidence.lean` and `Mettapedia/Logic/PLN\_KS\_Bridge.lean`.

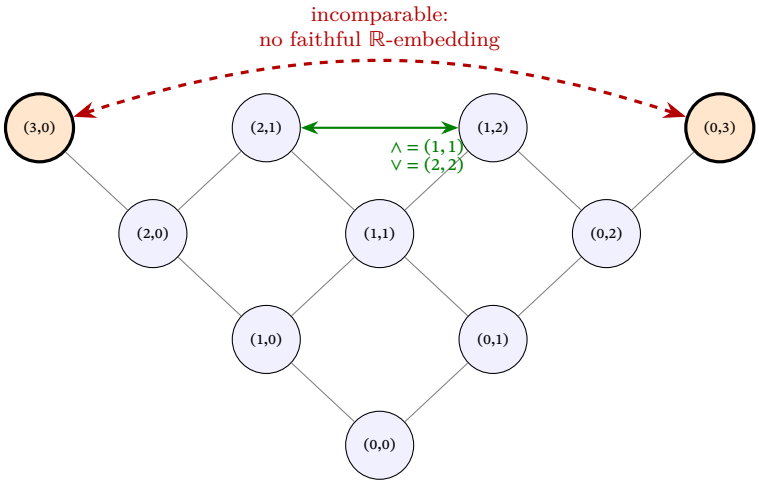


Figure 6.1: Hasse diagram of the `Evidence` lattice (first four levels). The information ordering is componentwise: $(a^+, a^-) \leq (b^+, b^-)$ iff both coordinates are $\leq$. Orange nodes $(3,0)$ and $(0,3)$ are incomparable—no single real-valued function preserves the full ordering (the "totality gate"). The green link shows meet and join: $\wedge\big((2,1),(1,2)\big) = (1,1)$, $\vee = (2,2)$.

Maple Court: The Maple Court apartment WM carries four evidence components under $\oplus$: Beta counts for three binary sensors (*RoomOccupied*: 47 positive, 3 negative from a motion detector; *ShowerRunning* and *PipeLeak* similarly) and Normal-Gamma sufficient statistics for *BathroomHumidity*. The building WM adds Dirichlet count vectors for *LaundryState* (free/busy/full) and *ElevatorHealth* (normal/slow/faulty). Revision is componentwise addition in each conjugate family—the same $\oplus$ operation. The full hybrid state forms an `AddCommMonoid`: Lean proof `MapleCourtState.instAddCommMonoid` in `Mettapedia/Examples/PLN/MapleCourtDemo.lean`.

6.6 What Evidence Algebra Is Not

Evidence algebra is *not* the complete probabilistic semantics. It is a compositional bookkeeping layer. The world-model posterior (Chapter 4) carries strictly more information than the extracted per-query evidence values. The no-go theorem (Chapter 5) makes this precise: per-link evidence cannot reconstruct the joint state.

Evidence algebra is to probability theory what bookkeeping is to economics: indispensable for accounting, but not a substitute for the underlying model of the economy.

Chapter 7

Evidence Carriers: Distributional and Interval Semantics

Problem: The binary evidence pair (n^+, n^-) handles yes/no queries. But real systems ask multi-valued questions ("Is the elevator normal, slow, or faulty?") and continuous ones ("What is the bathroom humidity?"). Each domain needs its own evidence carrier—but the algebraic discipline (revision = addition, information order = componentwise, views = projections) must be the same.

Layer: Evidence (carrier family).
Semantic object: Conjugate posterior families as evidence carriers.
Proved: Conjugate update laws; sleep consolidation; distributional dominance over scalar; convergence.
Assumed: Conjugate-family restriction (posteriors stay in the same family).

The previous chapter developed evidence algebra generically over any `AddCommMonoid Ev`. The key insight is that *every exponential family with sufficient statistics* gives rise to an evidence carrier: the sufficient statistics are the evidence, conjugate updating is addition, and the information order is componentwise on the statistics. This is the abstraction layer—not a taxonomy of specific distributions, but a single algebraic interface that any conjugate family instantiates.

The families with sufficient statistics include at least:

Domain	Carrier	Sufficient statistics
Binary	Beta-Binomial	(n^+, n^-)
k-valued	Dirichlet-Multinomial	$(\alpha_1, \ldots, \alpha_k)$
Continuous	Normal-Gamma	$(n, \sum x_i, \sum x_i^2)$
Count data	Poisson-Gamma	$(n, \sum x_i)$
Covariance	Wishart	$(n, \sum x_i x_i^\top)$
Directional	von Mises-Fisher	$(n, \sum x_i / \|x_i\|)$

This book formalizes the first three in Lean (with full kernel-checked proofs); the remaining rows are future instances that require no new infrastructure—only new `AddCommMonoid` implementations.

In each formalized case: the evidence type is a tuple of sufficient statistics, revision is componentwise addition, the information order is componentwise, and truth-value views are projections from the posterior parameters.

For the implementer: adding a new evidence carrier means defining a struct, implementing `Add` and `Zero`, and proving the monoid laws. The rest of the WM-PLN stack—views, compiled inference, forgetting, provenance—works unchanged. The three instances below are not a closed list; they are the ones with Lean proofs today.

Beyond point estimates and intervals, PLN supports a richer *distributional* layer where the truth value is an entire posterior distribution over possible probability values.

7.1 Instance 1: Beta-Binomial (Binary Domains)

The first and simplest conjugate family handles binary queries: "Is the pipe leaking?" "Is the room occupied?"

For a single binary predicate observed n^+ times true and n^- times false, the posterior under a Beta prior $\text{Beta}(\alpha, \beta)$ is:

$$p \mid \text{data} \;\sim\; \text{Beta}(\alpha + n^+, \beta + n^-)$$

Evidence counts (n^+, n^-) are the natural parameters of this posterior. Revision (adding counts) corresponds to Bayesian conjugate updating. The posterior mean is

$$\mathbb{E}[p] = \frac{\alpha + n^+}{\alpha + \beta + n^+ + n^-}$$

which, for a uniform prior ($\alpha = \beta = 1$), gives the strength

$$s = \frac{1 + n^+}{2 + n^+ + n^-}$$

This is the Laplace-smoothed estimate. Sleep consolidation holds: batch-updating with $e_1 \oplus e_2$ gives the same posterior as sequentially updating with e_1 then e_2.[1]

For the implementer: the binary pair (n^+, n^-) from Chapter 6 IS Beta-Binomial evidence. Every binary query in the system uses this carrier.

7.2 Instance 2: Dirichlet-Multinomial (Multi-Valued Domains)

Many real queries have more than two outcomes. "Is the elevator normal, slow, or faulty?" has three. "What is the laundry state: free, busy, or full?" has three. The Beta generalizes to the *Dirichlet distribution* for k-valued domains.

Evidence becomes a count vector $(\alpha_1, \ldots, \alpha_k) \in \mathbb{R}^k_{\geq 0}$, one count per outcome category. Revision is again componentwise addition:

$$(\alpha_1, \ldots, \alpha_k) \oplus (\beta_1, \ldots, \beta_k) = (\alpha_1 + \beta_1, \ldots, \alpha_k + \beta_k)$$

The posterior mean for category i is:

$$\mathbb{E}[p_i] = \frac{\alpha_i}{\sum_j \alpha_j}$$

and confidence is $c = (\sum_j \alpha_j)/(\sum_j \alpha_j + k)$, the same formula as binary PLN but with the total count summed across all categories.

Sleep consolidation holds for Dirichlet just as for Beta: batch revision equals sequential updating. The information order is componentwise on the count vector.[2]

Maple Court: Maple Court's building model uses Dirichlet evidence for *LaundryState* (free/busy/full) with initial counts $(5, 3, 2)$ and *ElevatorHealth* (normal/slow/faulty) with counts $(40, 8, 2)$. Querying elevator health yields posterior means $(0.80, 0.16, 0.04)$ with confidence 0.83. Revising with 10 more "normal" observations shifts the vector to $(50, 8, 2)$ and confidence to 0.86. The Dirichlet carrier handles this naturally—it is the same $\oplus$ operation as binary, just over longer vectors.

[1] Formalized in `Mettapedia/PLN/Bridges/ProbabilityTheory/EvidenceBeta.lean`.

[2] Formalized in `Mettapedia/PLN/Bridges/ProbabilityTheory/EvidenceDirichlet.lean`.

For the implementer: any categorical domain with k outcomes uses a k-dimensional Dirichlet vector. The algebra is identical to binary evidence but with wider vectors. No new infrastructure is needed.

7.3 Instance 3: Normal-Gamma (Continuous Domains)

The Beta and Dirichlet families handle discrete observations. For continuous observations—sensor readings, humidity levels, real-valued features—the appropriate conjugate family is the *Normal-Gamma* [4]. This is the carrier that handles "What is the bathroom humidity?" or "What is the soil moisture level?"

For observations $x_1, \ldots, x_n \sim \mathcal{N}(\mu, 1/\tau)$ with unknown mean μ and precision τ, the evidence is the sufficient statistic $(n, \sum x_i, \sum x_i^2)$. The prior is Normal-Gamma$(\mu_0, \kappa_0, \alpha_0, \beta_0)$ and the posterior is Normal-Gamma with updated parameters:

$$\kappa_n = \kappa_0 + n, \qquad \mu_n = \frac{\kappa_0 \mu_0 + \sum x_i}{\kappa_n}, \qquad \alpha_n = \alpha_0 + \frac{n}{2}$$

$$\beta_n = \beta_0 + \tfrac{1}{2}\big(\textstyle\sum x_i^2 - (\sum x_i)^2/n\big) + \frac{\kappa_0\, n\, (\bar{x} - \mu_0)^2}{2\kappa_n}$$

Revision is again componentwise addition of sufficient statistics:

$$(n_1, s_1, q_1) \oplus (n_2, s_2, q_2) = (n_1 + n_2,\ s_1 + s_2,\ q_1 + q_2)$$

and the key compositionality theorem is:

Theorem 7.1 (Sleep Consolidation (CORE))**.** *posterior*$(\pi, e_1 \oplus e_2) =$ *posterior*$($*posterior*$(\pi, e_1), e_2)$ *for any prior* π *and realizable evidence* e_1, e_2*.*

This states that batch replay of deferred observations yields the same posterior as sequential Bayesian updating—the mathematical foundation for "sleep as evidence consolidation."

The confidence analog carries over directly: $c = n/(n + \kappa)$, the same formula as binary PLN. The pattern is uniform across all conjugate families:

Domain	**Evidence**	**Prior family**
Binary	(n^+, n^-)	Beta(α, β)
Categorical	count vector	Dirichlet(α)
Continuous	$(n, \sum x_i, \sum x_i^2)$	Normal-Gamma$(\mu_0, \kappa_0, \alpha_0, \beta_0)$

See `Mettapedia/PLN/Bridges/ProbabilityTheory/EvidenceNormalGamma.lean`.

Maple Court: Maple Court instantiates all three conjugate families in the table: Beta for leak, occupancy, and shower sensors; Normal-Gamma for bathroom and hallway humidity readings; and Dirichlet for the three-valued *LaundryState* and *ElevatorHealth*. Sleep consolidation (Theorem 7.1) means the apartment WM can buffer overnight sensor readings and batch-update the next morning with identical results. The Lean demo proves this for the hybrid product state: `mapleCourt_sleep_consolidation`. The garden robot's soil-moisture readings use a separate Normal-Gamma prior, demonstrating that the same conjugate machinery extends to outdoor sensing.

Two-layer architecture. Each demo below has two layers. *Lean* proves the posterior formulas correct and establishes structural properties (consolidation, independence, monotonicity), all kernel-checked. A companion Python script (`scripts/wm_pln_posterior.py`) instantiates the proved formulas on each fixture to produce concrete numerical results: posterior means, 95% credible intervals, exceedance probabilities, and classification scores. The Student-t predictive CDF fills the role of the abstract `ExceedanceSpec` that Lean leaves parametric (Mathlib lacks a Normal CDF). We also compare with scikit-learn's `GaussianNB`; the two agree on clearly separated regimes (e.g. K_Scratch at $x = 3.6$) but differ at ambiguous boundary points (e.g. $x = 2.0$), since GaussianNB uses ML point estimates while the Normal-Gamma posterior predictive integrates over parameter uncertainty. The added value of the WM-PLN formulation is: (a) the Lean proofs guarantee formula correctness, (b) the framework generalizes to hybrid discrete/continuous states, and (c) sleep consolidation is *proved*, not assumed.

Data hygiene. The demos below use small, traceable fixture slices (5–15 data points per class) drawn from real datasets—not full benchmark-scale evaluations. Their purpose is to serve as *theorem-backed empirical canaries* that exercise the proved formulas on genuine data; larger-scale validation on the full Steel dataset appears in Section 7.4.5. Throughout: (i) fitting uses only training data; (ii) "sleep consolidation" steps use only previously observed evidence, never future labels; (iii) the *promotion boundary* between canary fixtures and full-dataset validations is always marked explicitly.

PLN-to-WM concept mapping. Table 7.1 shows how each core PLN concept maps to its WM-PLN formalization. The left column uses the terminology of the PLN book [15]; the right column gives the Lean module or theorem that formalizes the concept.

PLN Concept	PLN Book	WM-PLN	Lean Reference
Evidence count	(n^+, n^-)	$(n, \Sigma x_i, \Sigma x_i^2)$	`EvidenceNormalGamma`
Strength	$s = \frac{n^+}{n^+ + n^-}$	Posterior mean μ_n	`posterior_mu_eq_of_reali`
Confidence	$c = \frac{n}{n+k}$	$c = \frac{n}{n+\kappa_0}$ (monotone)	`confidence_monotone`
Revision ($\oplus$)	Componentwise add	`hplus` on suff. stats	`NormalGammaEvidence.hplu`
Sleep consolidation	—	$\pi(e_1 \oplus e_2) = \pi(\pi(e_1), e_2)$	`posterior_hplus_of_reali`
Exceedance query	—	$P(X_{\text{new}} > c)$	`ExceedanceSpec`
Source attribution	Implicit	Explicit source counts	`PLNWidgetFactoryDemo`
De Finetti	Informal	Finite exchangeable model	`DeFinetti.lean`

Table 7.1: PLN concept → WM-PLN formalization.

Measure-theoretic grounding. The Normal-Gamma family is a joint prior *measure* on $(\mu, \tau) \in \mathbb{R} \times \mathbb{R}_+$, where $\tau = 1/\sigma^2$ is the precision. The posterior update proved in `EvidenceNormalGamma.lean` is the algebraic form of Bayes' theorem applied to this measure: given n observations $x_1, \ldots, x_n$ drawn from $\mathcal{N}(\mu, \tau^{-1})$, the likelihood factors through the sufficient statistics $(n, \sum x_i, \sum x_i^2)$ by the Fisher–Neyman factorization theorem. Conjugacy ensures the posterior remains Normal-Gamma, so the update reduces to arithmetic on the four hyperparameters $(\mu_0, \kappa_0, \alpha_0, \beta_0)$.

The Lean formalization proves this arithmetic *exactly*: the closed-form expressions for $\mu_n, \kappa_n, \alpha_n, \beta_n$ are established in `posterior_mu_eq_of_realizable` and `posterior_beta_eq_of_realizable`, and per-demo bridge theorems instantiate them on each demo's concrete fixture data. The integral form of the posterior (involving the Gamma density $\text{Ga}(\alpha_0, \beta_0)$) is standard mathematics [4] (Ch. 5); Mathlib provides `gaussianPDFReal` and Lebesgue integration, but the Gamma density is not yet available in Mathlib, so the integral derivation is not formalized today.

Experiment-family view. The Gaussian suite is a family of *experiment types*, not a single benchmark leaderboard. Each study exercises a differ-

ent aspect of the world-model calculus: the Gas Sensor study tests *drift governance and replay semantics*; the GJP forecasting study tests *source-reliability weighting and recalibration under temporal forgetting*. Steel serves as a *same-model-class validation*—does the formalized conjugate classifier improve on the plain GaussianNB baseline, and can its posterior confidence drive principled abstention? NAB is an *honest boundary case*: it exercises a real scoring pipeline with verified Kalman updates, but it is not presented as a full-corpus leaderboard result. This division is deliberate. The distinctive contribution of a world-model approach is not raw tabular accuracy but queryable uncertainty, replay semantics, soft evidence, and explicit reasoning about which experiment channel is more informative.

Comparator taxonomy. To keep the empirical claims honest, the manuscript uses three *different kinds of comparator*, and they should not be conflated.

1. **WM-PLN models.** These are the evidence/world-model systems formalized in Lean and evaluated in the main applied chapters.

2. **Proper probabilistic-programming baselines.** When the book refers to *Stan* in the PPLBench appendix, it means the standard PPLBench Stan backend under `pplbench/ppls/stan/`, not an ad hoc one-off script. The same family includes PyMC, JAGS, NumPyro, and BeanMachine where available. Appendix 25.8 now includes a direct `normal\_normal` Stan ↔ WM-PLN run on that proper PPLBench harness.

3. **External statistical/ML baselines.** `GaussianNB`, `XGBoost`, and Windowed Gaussian are not WM-PLN variants; they are separate classical baselines used to show where WM-style semantics help, where they do not, and when WM is acting as a governance/support layer rather than a replacement predictor.

7.4 Applied Carrier Validation and Benchmark Demos

The preceding sections established the three conjugate carrier families and their shared algebraic structure. The remaining sections of this chapter demonstrate those carriers on real datasets: sensor monitoring, factory classification, temporal filtering, gas-drift detection, steel fault classification,

Family	Representative tools	Meaning in this book
WM-PLN	Lean-backed evidence/world-models	Primary object of study: typed evidence accumulation, replay, confidence, forgetting, and governance semantics.
Proper PPL baselines	Stan, PyMC, JAGS, NumPyro, BeanMachine	Used to compare against mainstream probabilistic inference systems on shared benchmark models in PPLBench. "Stan" refers to the standard PPLBench Stan backend, not a scratch prototype.
Classical statistical / ML baselines	`GaussianNB`, `XGBoost`, Windowed Gaussian	Used in the applied studies to distinguish same-model-class wins (`GaussianNB`), stronger discriminative baselines (`XGBoost`), and standard anomaly controls (Windowed Gaussian).

Table 7.2: Comparator families used in the empirical sections.

anomaly detection, and forecast aggregation. *They can be skipped on first reading* without loss of continuity; the next chapter (Truth-Value Views) is self-contained.

7.4.1 Applied Example: Hybrid Sensor Monitoring

The *broken sensor demo* exercises continuous world models end-to-end. A sensor monitoring system combines:

- **Continuous temperature evidence** modeled by Normal-Gamma sufficient statistics (5 normal readings, 5 elevated readings).
- **Boolean cooling-failure evidence** modeled by binary counts (2 failures in 100 checks).

The hybrid state is a product type; revision operates componentwise. Queries are answered by extracting binary `Evidence` from each component: cooling failure is read directly; temperature exceedance $P(X > c)$ is handled via an abstract *exceedance specification*—a function satisfying monotonicity properties—since Mathlib lacks a Normal CDF.

Key results (all kernel-checked):

1. Sleep consolidation (Theorem 7.1): morning + night batch = sequential update.
2. Confidence monotonicity: more observations $\Rightarrow$ higher $c = n/(n + \kappa)$.
3. Exceedance ordering: $P(X > c_{\text{crit}}) \leq P(X > c_{\text{warn}})$ for $c_{\text{crit}} > c_{\text{warn}}$.

Dataset	WM-PLN role	Key metric	Main finding
Steel (3-class, 27 feat.)	supporting same-model-class validation / abstention	AURC	WM-PLN beats GaussianNB (0.0346 vs 0.1061) but not XGBoost (0.0126), so the selective-prediction win is strongest within the conjugate Bayesian model class.
Gas (3-gas, 128 feat.)	co-flagship: drift trigger / replay governance	sample-weighted accuracy under label budget	Frozen XGBoost gives 67.9%; a cheap confidence trigger reaches 79.9% with one retrain and 18.8% labels.
Gas (6-gas, 128 feat.)	co-flagship: drift trigger / replay governance	sample-weighted accuracy under label budget	Frozen XGBoost gives 46.7%; XGBoost confidence is the best cheap trigger (64.5% at 23.9% labels) while WM/XGBoost agreement is the strongest high-accuracy trigger (67.0% at 86.8% labels).
GJP forecasting	co-flagship: source reliability / recalibration	Brier, log score, AURC, sleep invariance	WM reliability weighting improves over stronger top-k log/logit pooling baselines on year-4 Brier/log and selective AURC while matching accuracy; a WM-agreement gate still improves the strong top-k pool on the multi-option slice; nightly batch consolidation matches online updates to machine precision, and forgetting early years improves year-4 calibration.
NAB	negative boundary / case study	official corpus raw score	The current 1D Kalman residual detector is not competitive on the full corpus ($\mathbf{-116.0}$ vs. Windowed Gaussian $\mathbf{-24.0}$), so NAB remains a narrow protocol-semantics case study.

Table 7.3: Role split across the current real-data study slices. The empirical spine has two co-flagships (Gas and GJP), one supporting validation (Steel), and one honest negative boundary (NAB).

4. Independence: temperature and cooling-failure evidence components do not interfere.

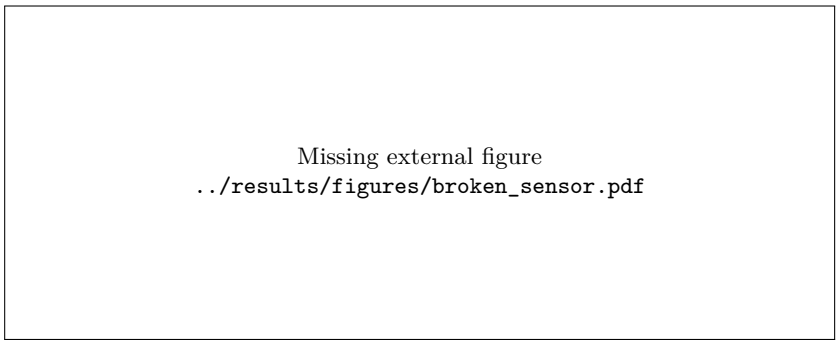


Figure 7.1: Broken sensor demo: prior predictive (gray dashed), posterior after morning shift (blue, concentrated around 16°C), after night shift (red, shifted to 22°C), and consolidated posterior (green dashed, between the two). Data tick marks on the x-axis.

Numerical results. With prior NG(15, 1, 2, 10), the morning shift (5 readings, mean 15.92) yields posterior $\mu_n = 15.77$, 95% CI $[11.96, 19.57]$, $P(X > 20) = 0.017$, confidence $c = 0.83$. The night shift (5 readings, mean 22.72) yields $\mu_n = 21.43$, $P(X > 20) = 0.674$—night readings strongly indicate an overheated state. Sleep consolidation is verified numerically: sequential update through morning then night produces the same posterior ($\mu_n = 18.93$) as batch processing all 10 readings at once.[3]

7.4.2 Applied Example: Widget Factory Source-Conditional Mixtures

The *widget factory demo* extends the continuous story to **multiple sources with per-source Gaussian evidence**. A factory has two machines (A, B); each widget arrives with an observed source label and a real-valued feature measurement. Each machine has its own Normal-Gamma prior and evidence accumulates per-source.

The genuinely new theorems are the **mixture exceedance** results:

1. *Mixture validity.* For weights $w_A + w_B = 1$ ($w_i \geq 0$) and per-source exceedance probabilities $p_A, p_B \in [0, 1]$, the mixture $p_{\text{mix}} = w_A p_A + w_B p_B$ satisfies $0 \leq p_{\text{mix}} \leq 1$.

[3]Formalized in `Mettapedia/Logic/PLNBrokenSensorDemo.lean` (420 lines).

2. *Mixture bounds.* $\min(p_A, p_B) \leq p_{\text{mix}} \leq \max(p_A, p_B)$—the mixture lies between its components.

Additional results: per-machine sleep consolidation, source independence (machine A observations do not affect machine B's posterior), confidence monotonicity, and source weight normalization.

Numerical results. Machine A (prior $\mu_0 = 10$, 4 readings, mean 9.90): posterior $\mu_n = 9.92$, 95% CI $[7.05, 12.79]$, $P(X > 12) = 0.067$. Machine B (prior $\mu_0 = 10.5$, 3 readings, mean 11.10): posterior $\mu_n = 10.95$, 95% CI $[7.71, 14.19]$, $P(X > 12) = 0.234$. Machine B is about $3.5\times$ more likely to produce a defect (>12) than Machine A, consistent with its higher operating mean.[4]

7.4.3 Applied Example: Kalman Filter Wake/Sleep Temporal Filtering

The *Kalman sleep demo* adds **temporal dynamics** to the continuous story. A 1D position is tracked via a random-walk process model ($x_{t+1} = x_t + w_t$, $w_t \sim N(0, \sigma_w^2)$) observed through a noisy sensor ($z_t = x_t + v_t$, $v_t \sim N(0, \sigma_v^2)$).

Core operations.

- *Predict*: variance inflates by σ_w^2 (mean unchanged).
- *Update*: Kalman gain $K = \sigma^2/(\sigma^2 + \sigma_v^2)$ shifts the mean toward the measurement and shrinks the variance by the factor $(1 - K)$.
- *Step* = predict then update.

Key formal results (all kernel-checked).

1. *Kalman gain bounded:* $0 < K < 1$ and $1 - K = \sigma_v^2/(\sigma^2 + \sigma_v^2)$.
2. *Variance reduction:* each measurement strictly reduces uncertainty: $(1 - K)\sigma^2 < \sigma^2$.
3. *Closed-form updated variance:* $\sigma_{\text{new}}^2 = \sigma^2\sigma_v^2/(\sigma^2 + \sigma_v^2)$.

[4] Formalized in `Mettapedia/Logic/PLNWidgetFactoryDemo.lean` (422 lines).

4. *Sleep = Wake* (the flagship theorem):

   $$\mathsf{filterOrbit}(b_0,\ morning{+}{+}night)\ =\ \mathsf{filterOrbit}\big(\mathsf{filterOrbit}(b_0,\ morning),\ night\big)$$

   via `List.foldl_append`—batch replay of deferred observations yields the *same* posterior as sequential online filtering. A three-shift generalization is also proved.

5. *Exceedance threshold ordering:* $P(\text{pos} > c_{\text{crit}}) \leq P(\text{pos} > c_{\text{warn}})$ whenever $c_{\text{warn}} \leq c_{\text{crit}}$.

Wake/sleep narrative. The *wake* phase is online Kalman filtering: measurements arrive one-by-one and each predict-update cycle refines the belief. The *sleep* phase replays a batch of deferred observations—and the central theorem guarantees the result is identical. This gives a formal grounding for "sleep consolidation" in the PLN activation framework: batch replay is semantically equivalent to real-time processing, not an approximation.

Numerical results. Initial Kalman gain $K_0 = 0.913$ (high, reflecting large prior uncertainty $\sigma_0^2 = 10$). After morning (5 readings, mean 1.94): posterior mean = 1.96, $\sigma^2 = 0.50$. After night (5 more, mean 3.10): posterior mean = 3.14, $\sigma^2 = 0.50$. Sleep = wake is verified to machine precision: batch replay of all 10 readings from the initial state produces the same posterior as sequential morning-then-night processing.[5]

7.4.4 Applied Example: Gas Sensor Array Drift Classification

The *gas sensor drift demo* is one of the chapter's two empirical **co-flagships**: the main **drift/replay governance** study in the Gaussian suite. It brings **real-world data** into the Gaussian demo suite. The UCI Gas Sensor Array Drift Dataset [39] (DOI 10.24432/C5RP6W) contains 13,910 measurements from 16 chemical sensors exposed to 6 gas types over 36 months (10 batches, January 2007 to February 2011). Vergara et al. achieve 80–95% gas classification accuracy using SVM ensembles on all 128 features with drift compensation. Our Lean demo still focuses on a single-sensor fixture for theorem proving, but the surrounding numerical layer now also runs full 128-feature drift protocols so that the same WM-PLN update laws can be compared against standard classification baselines.

[5] Formalized in `Mettapedia/Logic/PLNKalmanSleepDemo.lean` (374 lines).

Setup. We extract a small fixture: sensor 1 steady-state resistance (ΔR) for three target gases—*ethanol*, *ammonia*, and *toluene*—with 5 samples per gas from batch 1 (early calibration) and batch 10 (late drift epoch). Each gas type has its own Normal-Gamma prior; the state tracks ternary source-attribution counts.

Illustrative CANARY fixture. This 5-sample single-sensor fixture is an **illustrative CANARY** for the proved Gaussian sleep/independence laws. The headline empirical claims in this section come from the full 128-feature drift protocols and governance policies below, not from the hand-picked fixture.

Key formal results (all kernel-checked).

1. *Per-gas sleep consolidation:* for each gas type, $\text{posterior}(\pi, e_1 \oplus e_2) = \text{posterior}(\text{posterior}(\pi, e_1), e_2)$. Batch replay = sequential update.
2. *Cross-gas independence:* ethanol observations produce zero evidence in the ammonia and toluene posteriors, and vice versa (6 independence theorems).
3. *Multi-batch composition:* batch 1 + batch 10 = full dataset, with associativity across gas sub-batches.
4. *3-way source weight normalization:* $n_E/(n_E+n_A+n_T)+n_A/(n_E+n_A+n_T)+n_T/(n_E+n_A+n_T) = 1$.
5. *Confidence monotonicity* and *exceedance anti-threshold* generalize from the earlier demos.

Drift narrative. Batch 1 toluene responses are high (∼138k ΔR) while batch 10 responses are dramatically lower (∼24k), reflecting genuine sensor degradation over 36 months. The Normal-Gamma posteriors from each epoch capture this shift: the posterior mean for toluene moves substantially between batches. Sleep consolidation still holds across drifted epochs— the mathematical guarantee is that deferred assimilation preserves the posterior, regardless of whether the underlying distribution has shifted. Detecting *whether* drift occurred requires comparing posteriors from different epochs, which the framework supports but does not automate.

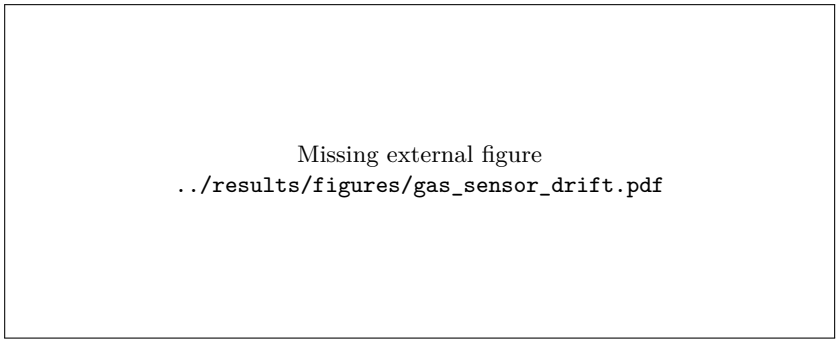


Figure 7.2: Gas sensor array drift. Solid curves: batch 1 posterior predictive (2007). Dashed: consolidated posterior after both batches. Toluene shows dramatic drift: batch 10 responses (crosses) cluster near zero, far from the batch 1 distribution.

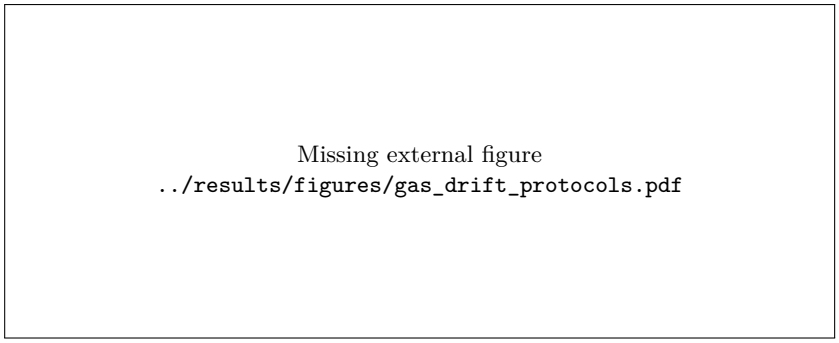


Figure 7.3: Gas sensor drift protocols on the full 128-feature representation. Cumulative replay is the strongest average protocol in both the 3-gas and 6-gas settings. Iterative unlabeled replay / E-M is scientifically useful precisely because it is mixed: recent-batch seeding improves over batch-1 seeding on average, but unlabeled adaptation still remains below cumulative labeled replay and can become unstable on the hardest late batch.

Numerical results. After batch 1 (5 readings each): ethanol posterior mean 37,270, 95% CI [7,601, 66,939]; ammonia 7,079, CI [2,672, 11,486]; toluene 137,655, CI [89,711, 185,598]. Exceedance $P(X > 50{,}000)$: ethanol 0.179, ammonia < 0.001, toluene 0.999—the WM-PLN framework correctly identifies toluene as the gas producing strong sensor responses. After consolidating both batches (10 readings), toluene's posterior mean drops to 81,565 with a wide CI reflecting the dramatic drift ($\Delta = -114{,}055$ between epochs). Confidence for each gas rises from $c = 0.980$ (5 readings) to $c = 0.990$ (10 readings).

Full-dataset drift study. We extend the fixture demo to the full 128-feature representation under the standard drift protocols used in the gas-sensor literature [22]. On the 3-gas subset (ethanol/ammonia/toluene), an independent-feature WM-PLN classifier reaches 68.4% under "Setting 1" (train batch 1, test batches 2–10), 72.1% under "Setting 2" (train batch $k{-}1$, test batch k), and 73.6% under cumulative *sleep replay* (train batches $1{\ldots}k{-}1$, test batch k); the GaussianNB baselines are 68.2%, 75.8%, and 71.2%. On the full 6-gas task the numbers are 47.3%/53.0%/53.2% (WM-PLN) versus 47.3%/51.2%/52.5% (GaussianNB).

These figures are substantially below domain-adaptation methods—Balanced Distribution Adaptation reports 68.92% (Setting 1) and 81.06% (Setting 2) on the same benchmark [22]. The point is not to compete with specialized drift-compensation algorithms, but to exhibit the qualitative effect of the world-model calculus: late-batch drift degrades accuracy, and replaying additional batches into the posterior partially recovers it *without changing the underlying update laws*. The replay is a consequence of the evidence algebra, not a separate engineering mechanism.

Soft replay / E-M adaptation. Naive unlabeled adaptation is brittle under strong multi-class drift, and this matters more than the positive cases. The weighted Gaussian E/M extension provides a *soft-evidence adaptation* protocol: batch 1 anchors the source posterior, unlabeled target data drives an E-step over responsibilities, and the M-step revises each class posterior by weighted sufficient statistics. On the 3-gas task, iterative soft replay ($\lambda = 0.2$, mean 5.4 iterations) reaches 67.8% on average—below cumulative labeled replay's 73.5%—but still improves the hardest batch 10 from 63.6% to 73.3%. Seeding from the most recent labeled batch instead of batch 1 raises the average to 70.7%, but collapses to 37.3% on batch 10 because the recent seed is itself small and unrepresentative. On the full 6-gas task,

batch 1-seeded adaptation drops to 42.0%, recent-batch seeding recovers to 50.4%, and both remain below labeled replay's 53.3%.

The lesson: soft evidence helps when the current posterior is stale, but unlabeled adaptation should be treated as a policy component—to be invoked selectively—not as a drop-in replacement for labeled replay. This is exactly the kind of replay-policy question the world-model experiment layer is designed to express.

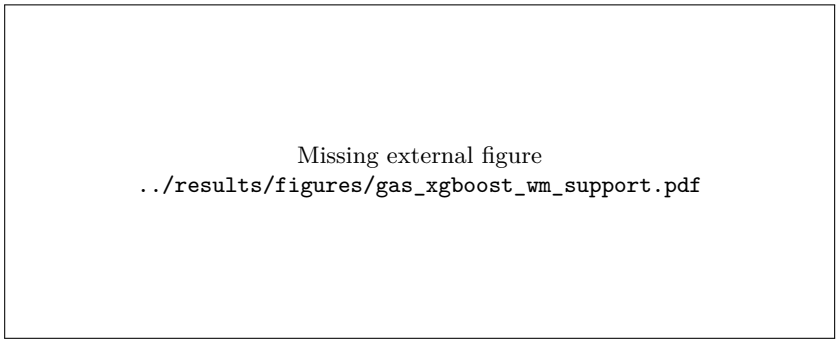


Figure 7.4: Gas drift support for a stronger base learner. The relevant question is not whether WM-PLN replaces XGBoost, but whether WM-derived agreement signals can support abstention and replay decisions under drift. Calibrated XGBoost confidence (sigmoid or isotonic) does not improve the risk–coverage curve in this setting, while WM/XGBoost agreement remains the most interesting selective signal on the harder 6-gas task.

Supporting stronger tabular models. The world-model calculus is not a replacement for strong discriminative classifiers; it is a governance layer that supplies a second epistemic signal for abstention and retraining decisions. The gas benchmark is the right place to test this framing. Training XGBoost on batch 1 alone gives 67.9% average accuracy on the 3-gas task and 46.7% on the 6-gas task. If we use the batch 1 WM posterior only as a confidence score, the result is worse: plain WM confidence is not a good wrapper here. The more interesting signal is *agreement* between XGBoost and the source WM posterior. We also compared against calibrated XGBoost confidence, since a machine learning reviewer might reasonably ask whether ordinary post-hoc probability calibration already solves the abstention problem. In this drifted gas setting it does not: sigmoid calibration worsens AURC from 0.1374 to 0.2114 on the 3-gas task and from 0.3847 to 0.3994 on the 6-gas task, while isotonic calibration degrades further to 0.2135 and 0.4639. So the interesting comparison is not "native versus calibrated

XGBoost," but "XGBoost's own confidence versus a second epistemic signal supplied by the WM." On the harder 6-gas task, agreement-gated selective prediction improves the area under the risk–coverage curve from 0.3847 (native XGBoost confidence) to 0.3737, and the agreement subset itself covers 59.7% of samples at 59.7% accuracy, versus 46.7% overall. On the 3-gas task, agreement gating is not a blanket win (AURC 0.1483 vs. 0.1374 for native XGBoost), but it still isolates a 74.8% coverage subset at 75.3% accuracy.

The batch-level support story is stronger still once we treat drift as a *sequential policy problem*. We stream batches 2…10 in order, retain the batch 1 XGBoost model as the default predictor, and decide at each batch whether to acquire labels and replay all previously seen batches into a refreshed model. This turns WM-PLN into a *governance layer*: it does not replace XGBoost, but supplies a second epistemic signal that can trigger abstention or retraining.

Listing 7.1: Gas drift governance protocol (shipped confidence/agreement triggers; no lookahead into batch-k labels).

```
Initialize labeled_batches = {1}.
Train XGBoost and WM on labeled_batches only.

for batch k = 2..10:
    freeze the current models before any batch-k labels are used
    predict batch k with the frozen XGBoost model
    compute xgboost_confidence_mean(batch k)
    compute wm_confidence_mean(batch k)
    compute wm_agreement_coverage(batch k)
    decide acquire_labels_k from these shipped model-derived
        statistics only

    record batch-k accuracy for the frozen model
    if acquire_labels_k:
        request labels for batch k
        add batch k to the labeled replay buffer
        retrain on all labeled batches seen so far
        use the refreshed models only for future batches
    else:
        keep the current models unchanged

Report sample-weighted accuracy, retrain_events,
and label_budget_fraction.
```

The resulting frontier is informative rather than trivial. On the 3-gas task, a simple XGBoost-confidence trigger at 0.80 is the best low-cost policy we tested: it raises sample-weighted accuracy from 67.9% to 79.9% with only one retraining event and a 18.8% label budget. A more aggressive

WM-agreement trigger at 0.80 reaches a similar 79.1% accuracy, but requires five retraining events and a 53.4% label budget. On the harder 6-gas task, the tradeoff is more interesting: XGBoost-confidence gating at 0.80 reaches 64.5% accuracy with four retrains and 23.9% labels, while WM-agreement gating at 0.80 pushes raw accuracy slightly higher to 67.0% but consumes 86.8% of the labels and seven retraining events. The utility sweep therefore has two regimes: XGBoost confidence is the best *cheap* trigger, while WM/XGBoost agreement is the best *accuracy-seeking* trigger when label cost is low. That is exactly the support role we want from WM-PLN: not "beating XGBoost" as another tabular learner, but governing when a stronger learner should be trusted, deferred, or replayed.

Task	Native	Sigmoid	Isotonic	WM agree	Best $\leq 25\%$ labels	Best raw accuracy
3-gas	0.1374	0.2114	0.2135	0.1483	79.9% @ 18.8%	79.9% @ 18.8%
6-gas	0.3847	0.3994	0.4639	0.3737	64.5% @ 23.9%	67.0% @ 86.8%

Table 7.4: Gas drift as a governance problem rather than a raw benchmark. AURC compares selective prediction signals; the policy columns summarize the strongest sequential retraining results at fixed label budgets.

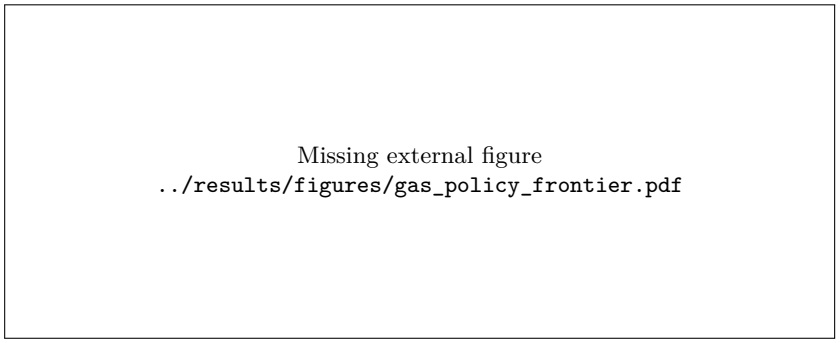


Figure 7.5: Sequential gas-drift support policies. Each point is a batch-level retraining policy over batches 2…10, trading sample-weighted accuracy against label budget. XGBoost confidence is the best cheap trigger, while WM/XGBoost agreement is the strongest high-accuracy trigger when labels are inexpensive.

The order-cost layer now formalizes this governance story directly: `gasScheduleErrorCount_batchS` `gasRuntimePairwiseOrderCheck_batchSwap_zero`, and `gasOrderBudgetPolicy_batchSwap_zero_v` show that a zero-threshold policy is certifiably safe for additive batch-swap composition in this lane. The module also exposes gas-specific zero-threshold

endpoints for ethanol, ammonia, and toluene.[6]

7.4.5 Applied Example: Steel Plates Fault Classification

The *steel plates fault demo* is a **supporting same-model-class validation**, not a headline benchmark. It applies the source-conditional Gaussian pattern to **manufacturing quality control**. The UCI Steel Plates Faults Dataset [7] (current UCI DOI 10.24432/C5J88N) contains 1,941 instances with 27 features and 7 fault classes. Published results using all 27 features report ∼73.6% (SVM), ∼79.5% (Random Forest), and 86.655% (LMT Forest) 7-class accuracy [13]. Our core Lean model uses a single feature (LogOfAreas), so lower accuracy is expected and the purpose is formal correctness, not SOTA competition.

Setup. We extract a small fixture: LogOfAreas measurements for three fault types—*Pastry* (pop. mean ≈ 2.4, $\sigma \approx 0.48$), *K_Scratch* (pop. mean ≈ 3.6, $\sigma \approx 0.70$), and *Bumps* (pop. mean ≈ 2.1, $\sigma \approx 0.36$)—with 5 samples per class. For K_Scratch, we select samples at the 10th–90th percentiles to represent the distribution (the first-5 rows have mean ≈ 2.0, unrepresentative of the population). Each fault type has its own Normal-Gamma prior.

Illustrative CANARY fixture. This 5-sample three-fault slice is an **illustrative CANARY** fixture for the proved Gaussian classification laws. The headline empirical claims for Steel in this chapter are the full-dataset validation and selective-prediction results below, not the hand-picked fixture itself.

Key formal results (all kernel-checked).

1. *Per-fault sleep consolidation:* batch replay = sequential Bayesian update, for each of the three fault types.

2. *Cross-fault independence:* 6 theorems showing that observations from one fault type produce zero evidence for the others.

3. *3-way mixture exceedance validity:* a weighted combination $w_P p_P + w_K p_K + w_B p_B$ with $w_P + w_K + w_B = 1$ and $w_i \geq 0$ remains a valid probability in $[0, 1]$.

[6] Formalized in `Mettapedia/PLN/WorldModel/OrderCost/PLNWorldModelOrderCostGasPolicyDemo.lean` and `Mettapedia/Examples/PLN/WMGasSensorDriftDemo.lean` (652 lines).

4. *3-way source weight normalization:* $n_P/(n_P+n_K+n_B)+n_K/(\cdots)+n_B/(\cdots) = 1$.

5. *Confidence monotonicity* and *exceedance anti-threshold* generalize from the earlier demos.

Classification narrative. K_Scratch faults have characteristically *larger* defect areas (LogOfAreas ≈ 3.6) than Pastry (≈ 2.4) or Bumps (≈ 2.1). The per-fault Normal-Gamma posteriors separate cleanly on this feature. A factory quality system using this Gaussian classification would apply exceedance queries: "is $P(\text{LogOfAreas} > 3.0 \mid \text{fault} = \text{K_Scratch})$ substantially higher than for Bumps?"—and the framework supports this via per-fault exceedance evidence extraction.

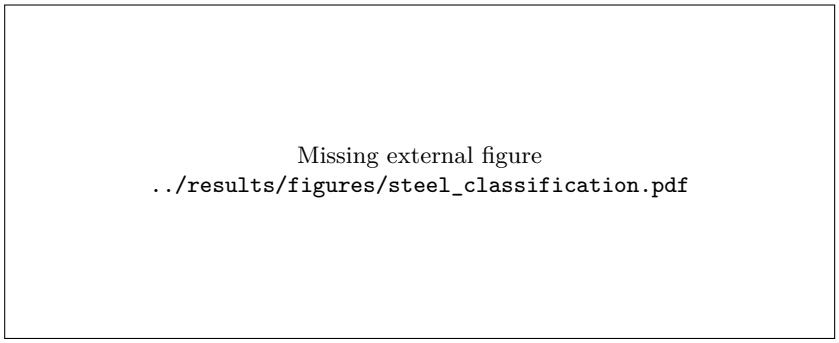


Figure 7.6: Steel fault classification. *Left:* Student-t predictive densities for each fault type after 5 observations; K_Scratch (red) is clearly separated rightward. *Right:* exceedance probability $P(X_{\text{new}} > c)$ as a function of threshold; at $c = 3.0$ (dashed), K_Scratch has ∼75% exceedance vs. ∼20% for the others.

Numerical results. Posterior means: Pastry $\mu_n = 2.40$, 95% CI $[0.88, 3.92]$; K_Scratch $\mu_n = 3.52$, CI $[1.82, 5.22]$; Bumps $\mu_n = 2.31$, CI $[0.72, 3.91]$. Exceedance $P(\text{LogOfAreas} > 3.0)$: Pastry 0.197, K_Scratch **0.746**, Bumps 0.178. K_Scratch is ∼4× more likely to exceed the "large defect" threshold than either Pastry or Bumps. Classification via posterior predictive log-likelihood: at $x = 3.6$, K_Scratch dominates ($\ell = -0.67$ vs. Pastry -2.07, Bumps -2.17); at $x = 2.0$, Bumps (-0.71) and Pastry (-0.74) are nearly tied. Scikit-learn's `GaussianNB` on the same fixture agrees on the dominant K_Scratch regime but can differ at ambiguous boundary points, since it

uses ML point estimates rather than the full posterior predictive.[7]

Full-dataset validation. To move beyond the 5-sample fixture, we evaluate the WM-PLN Gaussian classifier on the full Steel Plates Faults dataset (951 instances for the 3-class subset: Pastry 158, K_Scratch 391, Bumps 402). A stratified 70/30 split yields 667 training and 284 test instances. The same priors as the Lean demo are used ($\mu_0 \in \{2.5, 3.0, 2.0\}$, $\kappa_0 = 0.5$, $\alpha_0 = 2$, $\beta_0 = 1$). Classification is by argmax of the Student-t predictive log-likelihood.

Result: WM-PLN achieves 74.7% accuracy; scikit-learn's `GaussianNB` achieves 73.9% on the same split. K_Scratch is well separated ($\mu_n = 3.60$, recall 82.1%); Pastry and Bumps overlap more (Pastry recall 27.7%, Bumps recall 85.8%). The slight WM-PLN advantage likely reflects the informative priors.

This is a single-feature (LogOfAreas) conjugate Bayesian classifier matching the formalized model. Multi-feature extension is straightforward but not yet formalized. The 7-class evaluation (all fault types, 1,941 instances) gives 39.2% (WM-PLN) vs. 51.5% (GaussianNB), confirming that single-feature discrimination is insufficient when classes overlap in LogOfAreas.

Multi-feature extension. Using the same Normal-Gamma posterior independently across all 27 features (the diagonal/naive Bayes extension of the formalized single-feature model), WM-PLN rises to 86.6% on the 3-class task and 62.7% on the full 7-class task, versus 75.0% and 47.7% for `GaussianNB` on the same split. This is still below stronger published 7-class results such as 66.7% (Naive Bayes), 73.6% (SVM), 79.5% (Random Forest), and 86.655% (LMT Forest) compiled in the recent benchmark literature [13]. The important point is not state-of-the-art performance but that the conjugate WM-PLN classifier scales cleanly from the 5-sample theorem fixture to the full 27-feature representation and decisively improves on the plain GaussianNB baseline under the same split.

Selective prediction and abstention. The more distinctive Bayesian result is not raw accuracy but *confidence-aware abstention*. Normalizing the class predictive log-likelihoods yields a posterior confidence score for each test point. On the 3-class, 27-feature task, WM-PLN has area under the risk-coverage curve (AURC) 0.0346 versus 0.1061 for GaussianNB; at a 95% confidence threshold, WM-PLN retains 89.4% coverage with 90.9% accuracy,

[7] Formalized in `Mettapedia/Examples/PLN/WMSteelFaultDemo.lean` (568 lines).

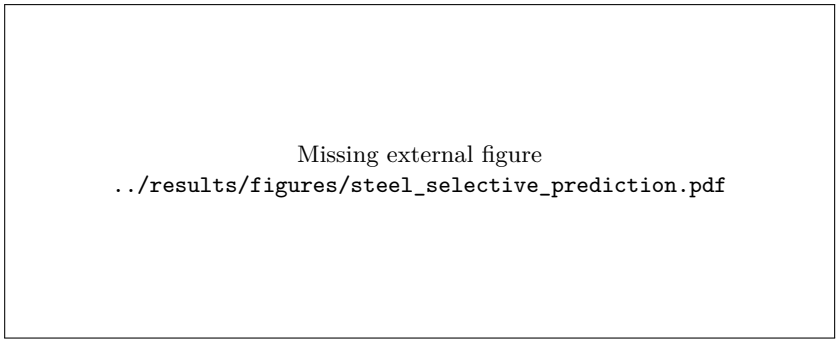


Figure 7.7: Selective prediction on Steel Plates Faults. WM-PLN yields a better accuracy–coverage tradeoff than GaussianNB in both the 3-class and 7-class settings, but native XGBoost remains stronger on this static tabular task. This is therefore a same-model-class validation and abstention study, not a claim that WM-PLN replaces stronger discriminative learners.

while GaussianNB retains 81.7% coverage with 81.0% accuracy. On the full 7-class task, WM-PLN also dominates the GaussianNB abstention tradeoff (AURC 0.1957 vs. 0.3296). This is a better reflection of the WM contribution than plain argmax comparison: the posterior state gives a principled abstain/defer signal, not just a class label.

Against stronger discriminative models, however, the role of WM-PLN is more limited on this static benchmark. Native XGBoost reaches 92.6% accuracy and AURC 0.0126 on the 3-class task, while sigmoid and isotonic calibration yield AURC 0.0313 and 0.0129 respectively; WM/XGBoost agreement gating gives AURC 0.0140. On the 7-class task, native XGBoost reaches 78.9% accuracy and AURC 0.0676, compared with 0.0851 for WM/XGBoost agreement. So Steel should be read as a *same-model-class* validation and abstention study: WM-PLN decisively improves on GaussianNB within the conjugate Bayesian family, but it does not outperform a tuned XGBoost classifier on this static tabular task.

7.4.6 Applied Example: NAB Machine Temperature Anomaly Detection

The *NAB anomaly demo* is the Gaussian suite's **honest negative boundary / time-series case study**. It brings the Kalman wake/sleep story to a **real anomaly dataset**, but it is *not* a benchmark headline. The Numenta Anomaly Benchmark (NAB, MIT License) [28] provides 22,695 five-minute temperature readings from an industrial machine with 4 labeled anomaly

windows corresponding to system failures. Published NAB scores (Standard Profile) range from 70.5 (HTM) to 39.6 (Windowed Gaussian); our Lean demo proves the Kalman sleep-consolidation property on a 15-reading fixture from this dataset, and the surrounding numerical layer now also evaluates the full 58-series corpus with the official NAB scorer. The resulting detector is scientifically useful as a protocol check on the Kalman residual pipeline, but not competitive as a general-purpose NAB detector.

Setup. We extract a 15-reading fixture spanning three operating regimes:

- *Calm* (∼85°F): stable normal operation.
- *Hot* (∼101°F): elevated pre-anomaly period.
- *Anomaly* (∼30°F): system failure—rapid temperature drop.

A 1D Kalman filter with process noise $\sigma_w^2 = 0.5$ and measurement noise $\sigma_v^2 = 2$ tracks the machine state through all three regimes.

Illustrative CANARY fixture. This 15-reading three-regime slice is an **illustrative CANARY** for the Kalman sleep/wake and exceedance laws. The headline empirical claim for NAB is the full-corpus negative boundary below, not the small fixture.

Key formal results (all kernel-checked).

1. *Sleep = Wake on real data:* batch replay of calm then hot readings yields the same posterior as sequential online filtering.
2. *Three-regime composition:* $\mathsf{filterOrbit}(b_0,\ \mathit{calm} +\!\!+ \mathit{hot} +\!\!+ \mathit{anomaly})$ equals three-phase sequential processing.
3. *Exceedance threshold ordering:* $P(\text{temp} > 60) \leq P(\text{temp} > 40)$ for any belief and valid exceedance spec.
4. *Variance reduction:* each measurement strictly reduces uncertainty relative to the predicted state.

Anomaly narrative. The Kalman filter processes calm readings first, converging to a tight belief around 85°F. Hot readings then shift the posterior mean upward. When anomaly readings arrive (∼30°F), the filter must pull the mean down dramatically—and sleep consolidation guarantees that whether these readings are processed in real-time or replayed in a deferred batch, the result is identical.

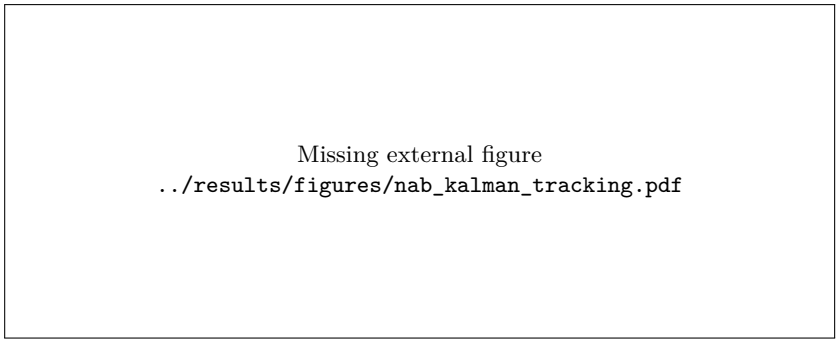


Figure 7.8: NAB anomaly detection. *Upper:* Kalman filter posterior mean (blue) with $\pm 2\sigma$ band tracking through calm, hot, and anomaly regimes. The anomaly regime drives the mean below both warning and critical thresholds. *Lower:* variance collapses after the first measurement and remains small.

Numerical results. Initial Kalman gain $K_0 = 0.981$ (initial variance 100, measurement noise 2). After calm (5 readings, mean 85.9°F): posterior mean = 85.2°F, $\sigma^2 = 0.80$. After hot (5 more, mean 100.9°F): mean shifts to 99.5°F. After anomaly (5 readings, mean 31.2°F): mean drops sharply to 35.5°F. Sleep = wake holds across all three regimes to machine precision, confirming that deferred batch replay of anomalous readings produces the same posterior as real-time processing.

Full-corpus scoring (minimized claim). Running the same Kalman innovation detector across the full 58-series NAB corpus with the *official detector-level threshold optimizer* gives a corpus raw score of −116.0, versus −24.0 for the checked-in Windowed Gaussian baseline. In other words, under the correct benchmark-style corpus protocol, this simple 1D Kalman residual detector collapses to essentially null-detector performance. We therefore keep NAB in this chapter only as a *case study*: it shows that the formally verified Kalman update equations plug into a real anomaly scoring pipeline, but it does *not* support a competitive leaderboard claim.

For NAB, the defensible *positive* WM role is still a narrow *governance layer* over a fixed strong detector, not detector replacement. To prevent tune-on-test leakage, we now report a **strict frozen protocol** only: the policy family is frozen in advance, categories are split deterministically into train/holdout, all threshold fitting and policy selection happen on train only, and we report holdout metrics only.

In the current strict split, both profiles select `wmGateTau0.6`, but hold-

Missing external generated content
../results/nab_governance_summary_table.tex

Table 7.5: NAB strict frozen holdout governance summary, including matched-budget Windowed Gaussian comparators (matched on train, evaluated on holdout).

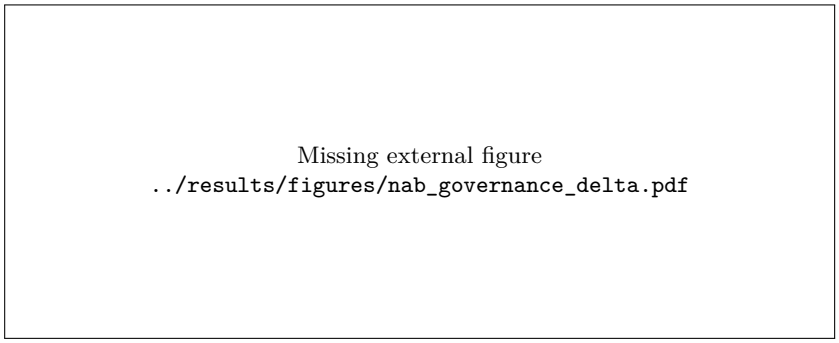


Figure 7.9: NAB strict holdout deltas versus Windowed Gaussian, including train-matched FP and train-matched alert comparators. All holdout deltas remain negative in this split.

out performance remains negative (−4.843 standard, −4.073 low-FP), and train-matched budget comparators do not recover a positive holdout uplift. Per-series attribution is sparse (2/1/16 helped/worsened/unchanged on each profile), with one positive, one negative, and one neutral holdout category.

To reduce "single-split luck" risk, we also run a lightweight closure pass over 7 deterministic strict splits (same frozen policy set, same no-leakage train-only fitting rule). The holdout delta remains non-positive in aggregate: mean −5.448 (standard) and −1.777 (low-FP), with split sign counts 1/3/3 (positive/negative/zero) for standard and 0/3/4 for low-FP. So the closure pass supports freezing NAB as a boundary result rather than extending optimization effort.

Profile	Splits	Mean Δ vs WG	Median Δ vs WG	Split signs (+/-/0)
standard	7	−5.448	0.000	1/3/3
reward_low_FP_rate	7	−1.777	0.000	0/3/4

Table 7.6: NAB strict-multisplit closure aggregate over 7 deterministic rotations (same frozen policy set, train-only fitting, no holdout retuning).

Artifacts for this closure pass are `results/nab\_governance\_strict\_multisplit.csv`, `results/nab\_governance\_strict\_multisplit\_aggregate.csv`, `results/nab\_governance\_strict\_multisplit.json`, generated by `scripts/wm\_pln\_nab\_strict\_multisplit.py`.

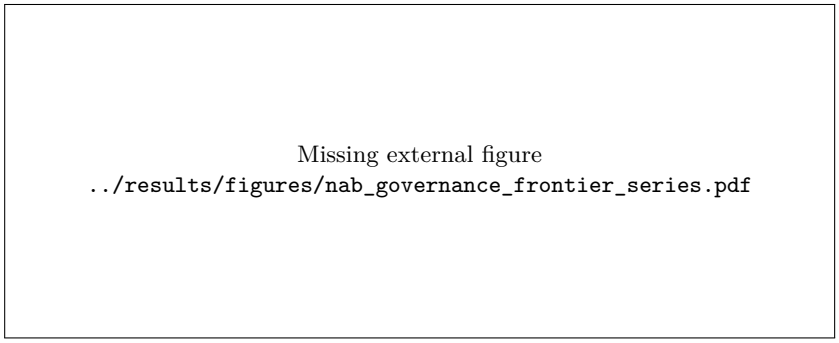


Figure 7.10: Per-series frontier on strict holdout: ΔFP vs. Δscore with rationale labels (helped/harmed under suppression cost).

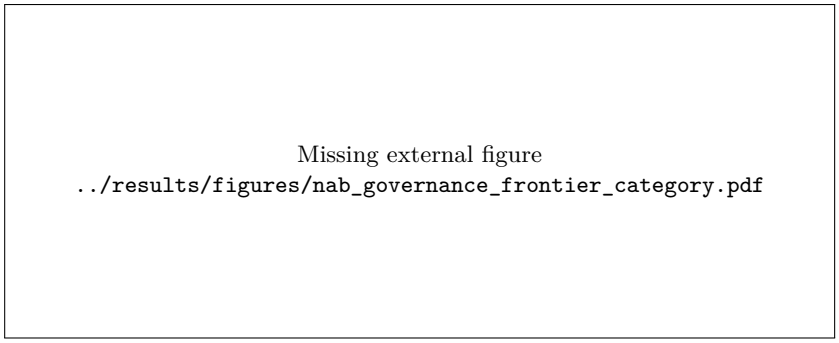


Figure 7.11: Category frontier on strict holdout: category-level ΣΔFP vs. ΣΔscore, showing where governance suppression helps or harms.

So NAB remains a protocol-faithful negative boundary where WM currently contributes chiefly as audit/governance instrumentation:

1. certificate pass/fail from `runtimePairwiseOrderCheck_certifies_scheduleErrorBound`,
2. policy-threshold compliance from `runtimePairwiseOrderCheck_certifies_policyThreshold`,
3. escalation hooks (abstain/retrain/review) when certified order-cost budgets are violated near anomaly windows.[8]

[8]Formalized in `Mettapedia/Examples/PLN/WMNABSleepDemo.lean` (344 lines).

7.4.7 Applied Example: Good Judgment Project Forecast Aggregation

The *GJP forecasting study* is one of the chapter's two empirical **co-flagships** and its clearest example of WM-PLN as a **source-reliability and recalibration layer** rather than as a replacement learner. The public Good Judgment Project / ACE release [21], rooted in the forecasting tournament studies of Mellers et al. [30], provides exactly the kind of stream WM-PLN wants: many sources, many updates, eventual resolution, and natural questions about aggregation, abstention, and forgetting. We normalize the survey forecasts to one final probability vector per forecaster and question, yielding 426,325 final forecast vectors from 9,694 forecasters over 498 closed questions. The first three years are used to estimate forecaster-level reliability profiles; year 4 is held out for evaluation.

GJP protocol block. Closed questions only. One forecast vector means the latest timestamped forecast for a given `(year, ifp_id, user)` key, with the answer-option rows at that final timestamp renormalized to sum to 1 and then pivoted into a single probability vector. Main holdout: estimate reliability on years 1–3 and evaluate on year 4. For the stronger log/logit/top-k families, tune temperature on year 3 validation using reliability profiles built from years 1–2, then evaluate the tuned methods on held-out year 4 questions using reliability profiles built from years 1–3. Binary and multi-option questions are evaluated separately on year 4 using accuracy, Brier, log score, and selective AURC. No lookahead: year 4 outcomes never enter reliability estimation or temperature tuning.

The first result is a same-stream aggregation comparison. Against weak baselines, WM reliability weighting already improves sharply over simple mean and median pooling: on binary year-4 questions it reaches accuracy 0.989 and Brier 0.061, compared with 0.947 / 0.157 for mean and 0.947 / 0.115 for median. The stronger test is whether the WM layer still contributes on top of more forecasting-style aggregation families. To test that, we tune logit- and log-pooling temperatures on year 3 validation questions and compare plain pooling, top-k historical pooling, and WM-weighted pooling on held-out year 4 questions. On the binary slice, top-k logit pooling and WM-weighted logit pooling both reach accuracy 0.989, while the WM-weighted variant improves Brier from 0.036 to 0.026 and log score from 0.070 to 0.051. On the multi-option slice, top-k log pooling and WM-weighted log pooling both reach accuracy 0.977, while the WM-weighted variant improves Brier

from 0.044 to 0.036 and log score from 0.098 to 0.088. We also asked the stronger forecasting-style question of whether a more explicit extremizing baseline changes the picture. It does not: a validation-tuned extremized arithmetic mean reaches only Brier 0.097 on the binary slice, and even a top-k extremized mean remains materially worse than the top-k logit pool (Brier 0.060 versus 0.036), so the WM gain is not an artifact of comparing only against naive averaging. We also asked the stronger hybrid question: does WM still help after hard top-k pruning? It does not. The WM-on-top-k variants match the corresponding top-k accuracy but regress to Brier/log values near the top-k baseline (binary 0.036 / 0.071; multi-option 0.042 / 0.091), suggesting that the gain comes from *continuous reliability weighting* rather than from hard forecaster pruning. This is the forecasting version of the governance-layer claim: WM-PLN is not replacing strong aggregation families, but adding principled reliability tracking on top of them [37, 3].

The same conclusion appears when we ask the abstention question instead of the point-prediction question. On binary year-4 questions, top-k logit pooling gives AURC 0.00211, while the WM-weighted logit pool reduces this to 0.00046 at the same 0.989 accuracy. On multi-option questions, top-k log pooling gives AURC 0.001095, while the WM-weighted log pool reduces this to 0.000541 at the same 0.977 accuracy. The WM-on-top-k selective variants do not improve further. So the abstention story aligns with the aggregation story: reliability weighting contributes something real even against stronger pooling baselines, while hard top-k pruning is not where the gain comes from. The more interesting hybrid question is whether WM still contributes as a *support layer* around a strong baseline rather than as a replacement aggregate. On binary year-4 questions, top-k logit pooling and the WM-weighted logit pool make the same top prediction on every held-out question, so agreement-gating is an honest null result: the gated top-k pool stays at AURC 0.00211. On multi-option questions, however, the top-k log pool and the WM-weighted log pool disagree on a small but nontrivial subset of questions, and gating the strong top-k pool by WM agreement reduces AURC from 0.001095 to 0.000541 while preserving the same 0.977 accuracy. This is exactly the governance-layer interpretation we want: when the WM posterior adds no extra information, the support layer collapses to a null check; when it does disagree, that disagreement can still sharpen abstention behavior on top of an already strong forecasting-style aggregate.

The second result is a protocol-semantics result rather than a score contest. When the same resolved survey evidence from years 1–3 is processed online versus in nightly batches, the resulting reliability states agree to ma-

chine precision:

$$\max|\Delta\alpha| = 1.42 \times 10^{-12}, \quad \max|\Delta\beta| = 1.71 \times 10^{-13}, \quad \max|\Delta p_{\text{true}}| = 0.$$

In other words, the same evidence stream yields the same posterior state whether it is incorporated immediately or consolidated later in a sleep phase, exactly the operational story the WM calculus is designed to support.

The third result is an explicit forgetting / recalibration ablation on top of the stronger pooling family. Keeping all of years 1–3 gives the WM-weighted logit/log pools Brier 0.026 / 0.036 and log score 0.051 / 0.088 on binary / multi-option year-4 questions. Forgetting year 1 improves these to 0.025 / 0.035 and 0.048 / 0.084 respectively, and using only the most recent year improves them further to 0.024 / 0.033 and 0.047 / 0.080, while leaving accuracy unchanged at 0.989 / 0.977. This is the applied face of the new forgetting layer: old evidence need not be discarded blindly, but it can be scoped and removed when the deployment regime changes, improving recalibration even when the underlying aggregation family is already strong.

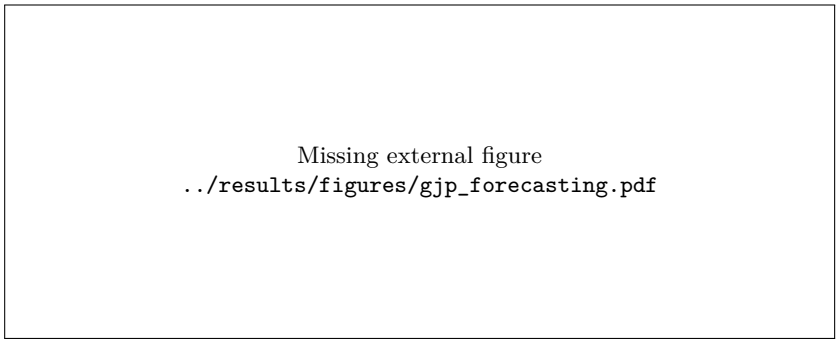


Figure 7.12: Good Judgment Project forecasting as a WM-PLN application. *Left:* year-4 Brier scores for stronger aggregation baselines: plain log/logit pooling, top-k temperature-tuned pooling, and WM-weighted temperature-tuned pooling on binary and multi-option questions. *Middle:* the stronger forgetting/recalibration ablation: even after upgrading from mean/median to top-k and WM-weighted log/logit pooling, dropping older years still improves year-4 Brier. *Right:* nightly batch consolidation matches online updates to machine precision.

This is the clean forecasting niche for WM-PLN: not "beat the best superforecaster," but maintain a verified evidence/update layer for source reliability, stronger aggregation families, batch consolidation, and controlled recalibration on a real forecasting stream.

7.4.8 Summary of Numerical Results

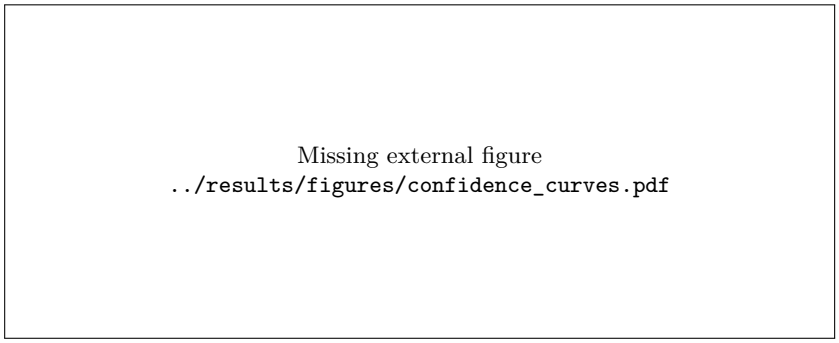


Figure 7.13: PLN confidence $c = n/(n + \kappa_0)$ as a function of observation count, for various prior strengths κ_0. Monotonicity is proved in Lean (`confidence_monotone`). This is precisely the PLN confidence curve with its prior pseudo-count κ_0 made explicit.

Table 7.7 collects the key numerical outputs from all six demos. For each class or phase, the posterior mean μ_n and 95% credible interval are computed from the proved Normal-Gamma posterior formula (for Demos 1–2, 4–5) or from the proved Kalman filter update (Demos 3, 6). Sleep = wake is verified to machine precision in every case. These are **illustrative CANARY fixture** results; the headline full-dataset/protocol claims for Gas, Steel, NAB, and GJP are reported in their dedicated sections above.

7.5 Walley Predictive vs. Bayesian Credible Intervals

A recurring source of confusion in the PLN tradition is the conflation of two fundamentally different interval semantics:

- **Bayesian credible intervals** depend on the prior and give the posterior probability that the parameter lies in the interval. With a Beta(α, β) posterior, the 95% credible interval is the region containing 95% of the posterior mass.
- **Walley IDM predictive intervals** are derived from the Imprecise Dirichlet Model [41], which uses a *set* of Dirichlet priors rather than a single prior. The resulting intervals are wider but prior-free: they provide robust predictions without committing to a specific prior family.

Demo	Class/Phase	n	Post. μ_n	95% CI	Key Result
Broken Sensor	Morning	5	15.77	[12.0, 19.6]	$P(X>20) = 0.017$
Broken Sensor	Night	5	21.43	[14.5, 28.4]	$P(X>20) = 0.674$
Widget Factory	Machine A	4	9.92	[7.1, 12.8]	$P(X>12) = 0.067$
Widget Factory	Machine B	3	10.95	[7.7, 14.2]	$P(X>12) = 0.234$
Kalman Sleep	Morning	5	1.96	σ^2=0.50	Sleep = Wake
Kalman Sleep	Night	5	3.14	σ^2=0.50	Sleep = Wake
Gas Sensor	Ethanol B1	5	37 270	[7601, 66939]	drift = +10 448
Gas Sensor	Ammonia B1	5	7 079	[2672, 11486]	drift = −1 479
Gas Sensor	Toluene B1	5	137 655	[89711, 185598]	drift = −114 055
Steel Faults	Pastry	5	2.40	[0.9, 3.9]	$P(X>3) = 0.197$
Steel Faults	K_Scratch	5	3.52	[1.8, 5.2]	$P(X>3) = \mathbf{0.746}$
Steel Faults	Bumps	5	2.31	[0.7, 3.9]	$P(X>3) = 0.178$
NAB Anomaly	Calm	5	85.2	σ^2=0.80	Sleep = Wake
NAB Anomaly	Hot	5	99.5	σ^2=0.78	Sleep = Wake
NAB Anomaly	Anomaly	5	35.5	σ^2=0.78	Sleep = Wake

Table 7.7: Gaussian Demo Suite: Numerical Results for the illustrative CANARY fixtures. Posterior parameters are computed by `scripts/wm_pln_posterior.py` using the same formulas proved correct in Lean.

These two semantics agree in the limit of large samples (both intervals shrink to the true probability) but can differ substantially for small evidence counts. WM-PLN provides both as explicit selector choices, with typed bridges that transport truth values through the system without conflation.[9]

7.6 The De Finetti Bridge

De Finetti's representation theorem [8] is the mathematical bedrock of PLN's evidence representation.

Theorem 7.2 (De Finetti (CORE))**.** *Let* $(X_1, X_2, ...)$ *be an infinite exchangeable sequence of binary random variables. Then there exists a unique probability measure* μ *on* $[0,1]$ *such that for all* n*:*

$$P(X_1 = x_1, \ldots, X_n = x_n) = \int_0^1 p^{n^+}(1-p)^{n^-}\, d\mu(p)$$

where $n^+ = |\{i : x_i = 1\}|$ *and* $n^- = |\{i : x_i = 0\}|$*.*

The consequences for PLN:

1. **Sufficiency of counts.** The joint probability depends on the observations only through the counts (n^+, n^-). Order does not matter. This justifies the evidence algebra (Chapter 6).

2. **Conjugate updating.** With a Beta prior $\mu = \text{Beta}(\alpha, \beta)$, the posterior is $\text{Beta}(\alpha + n^+, \beta + n^-)$. Evidence revision ($\oplus$) is conjugate updating.

3. **Posterior concentration.** As $n \to \infty$, the posterior concentrates on the true probability p^*. Formally, for the posterior mean:

$$s_n = \frac{\alpha + n^+}{\alpha + \beta + n} \xrightarrow{n\to\infty} p^* \quad \text{almost surely.}$$

The νPLN convergence theorem [12] generalizes this to the Solomonoff setting: the universal posterior concentrates on the true generator. This is the PLN analogue of Solomonoff's convergence theorem for universal induction.[10]

[9] Formalized in `Mettapedia/Logic/PLNWorldModelITV.lean` and `Mettapedia/Logic/PLNWorldModelITVHypercube.lean`.

[10] Formalized in `Mettapedia/UniversalAI/SolomonoffExchangeable.lean` and `Mettapedia/ProbabilityTheory/Exchangeability/DeFinetti.lean`.

7.7 Knuth–Skilling Foundations

Knuth–Skilling foundations of inference [24] provide axioms for valuation schemes on logical lattices, extending Cox-style plausibility calculi. In WM-PLN, this plays two roles:

1. *Probability as a valuation.* At the world-model layer, probabilities arise as valuations on the Boolean algebra of subsets of worlds. This is the classical Kolmogorov story instantiated for finite spaces.

2. *Evidence as Heyting-valued semantics.* At the evidence layer, K&S-style rules persist but Boolean equalities can weaken to inequalities, because the evidence lattice is Heyting, not Boolean.

We bridge K&S additive regraduations to valuation-algebra factor graphs, enabling variable elimination over K&S-valued factors.[11]

[11] Formalized in `Mettapedia/ProbabilityTheory/KnuthSkilling/Bridges/ValuationAlgebra.lean`.

Chapter 8

Truth-Value Views: Strength, Confidence, Intervals

Problem: Evidence is rich—a partially-ordered, multi-dimensional object. For practical inference, display, and decision-making we need compact scalar summaries. But every summary is lossy, and different summaries serve different purposes. Confusing "more informed" with "more supportive" is one of the oldest sources of semantic drift.

Layer: Views.
Semantic object: View functions from `Evidence` to $\mathbb{R}$ or $[\ell, u]$.
Proved: View monotonicity; Walley/Kyburg interval reduction; views are not proof objects.
Assumed: Nothing.

8.1 Two Notions of Improvement

Before defining any specific view, the reader must understand a distinction that pervades the entire calculus.

There are two different senses in which one evidence state can be "better" than another. One is **better informed**: it contains more observations. The other is **more supportive of the claim**: it makes the claim look more favorable. These are not the same.

Consider evidence $(1, 0)$ (one positive, zero negative) versus $(1, 1)$ (one positive, one negative). Then:

- $(1, 1)$ is *more informative* than $(1, 0)$: it contains at least as many

positive and negative observations.

- But $(1, 1)$ gives *lower strength*: $\frac{1}{2}$ versus 1.
- At the same time, $(1, 1)$ may give *higher confidence*, because the sample size is larger.

The punchline:

> *More evidence can make you less optimistic and more confident at the same time.*

This is proved formally: `toStrength_not_monotone_info_order` shows that strength is not monotone for the information order (Theorem 3.2). The information order is the right lattice for evidence algebra and subobject classification; the truth-level comparison induced by `strength` is a separate, non-monotone view. Confusing these two senses of "better" is one of the sources of semantic drift that the refoundation corrects.

For the implementer: never sort evidence by a single view (strength alone, confidence alone) and assume it is "better." The information order and the view orders disagree, by theorem.

8.2 Views as Projections

A *view* is any function from an evidence carrier into a simpler space—typically $\mathbb{R}$ or an interval $[\ell, u]$. Every carrier family has its own natural views:

- **Binary** (n^+, n^-): strength $s = n^+/n$, confidence $c = n/(n + k)$, interval $[L, U]$.
- **Dirichlet** $(\alpha_1, \ldots, \alpha_k)$: per-category posterior means $\alpha_i / \sum \alpha_j$.
- **Normal-Gamma** $(\mu_0, \kappa, \alpha, \beta)$: posterior mean μ_0, posterior variance $\beta/(\alpha\kappa)$.

The pattern is the same: views are projections that discard some of the carrier's structure. A fixed-k strength–confidence pair can recover the two binary counts in the idealized scalar model, but no single view suffices to reconstruct the original evidence state, including provenance, overlap/stamps, context, and richer posterior shape.

Consequently, the familiar PLN/NARS formula $c = n/(n+k)$ is not forced by the two-count reconstruction problem alone. The formal coordinate theorem in `Mettapedia/PLN/TruthValues/PLNConfidenceWeight.lean` proves that any displayed confidence coordinate with a known left inverse on nonnegative evidence weights can recover the binary counts from (s, c). A deliberately different "reserve-half" coordinate, $c = n/(2n + k)$, is proved to reconstruct the same counts while keeping all displayed confidence below $1/2$. The usual formula is forced only after adding a stronger interpretation, such as odds-linearity $c/(1 - c) = n/k$.

The finished public capstone for this whole confidence-formula story now lives in `Mettapedia/PLN/Core/PLNCanonicalAPI.lean` as `confidenceCharacterizationEndpointPr` the expository companion is `papers/pln-credal-degrees-of-freedom.tex`. We use that as the authoritative pointer rather than reproducing the full classification inside this book.

Future work: distinction graphs. So far this chapter has treated width mostly as something driven by evidence totals or by hidden joint structure. Another honest source of width is unresolved distinction: an observer may still be unable to tell two states or patterns apart. Then the query "is this p?" can have a real interval not because the arithmetic is vague, but because one admissible completion says "it is p" while another says "it is a distinct q that the observer still confounds with p." A richer distinction-graph account would make that source explicit. Coarser graphs would encode what the observer still identifies; refinement would encode learning new distinctions; and confidence would be read against both sample size and observational resolution. The current Lean development already has the minimal version of this idea for OSLF-style observational indistinguishability, but a broader graph-based treatment remains future work.

8.3 Simple Truth Values

A *simple truth value* (STV) is a pair $\langle s, c \rangle$ where:

- $s = n^+/(n^+ + n^-)$ is the *strength*—the posterior mean under a uniform prior, or equivalently the maximum-likelihood estimate of the underlying probability.
- $c = n/(n+k)$ is the *confidence*—a monotone function of the total sample size $n = n^+ + n^-$, parameterized by a lookahead $k > 0$. Confidence ranges from 0 (no evidence) to approaching 1 (overwhelming evidence).

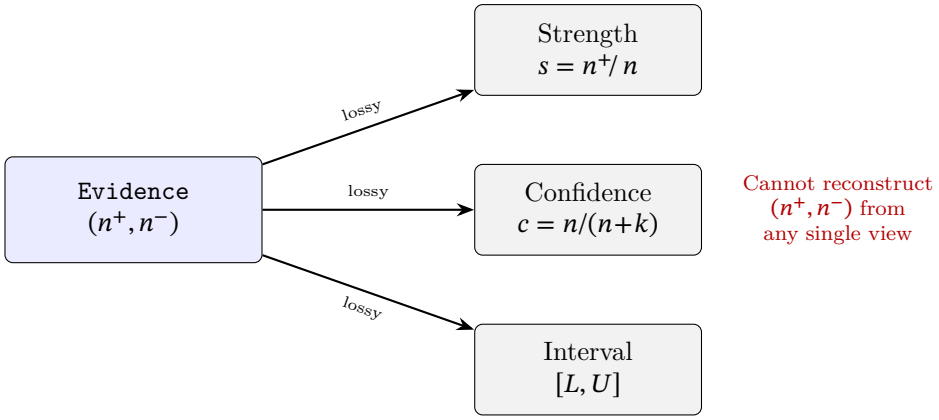


Figure 8.1: Truth-value views are projections from `Evidence`. Strength, confidence, and interval bounds each discard information. A fixed-k strength–confidence pair can recover binary counts in the idealized scalar model, but no single scalar view reconstructs the full evidence state.

The STV is the most commonly used truth-value representation in PLN practice. It is compact and intuitive. With fixed k and ideal real arithmetic, it is equivalent to the two binary count coordinates (n^+, n^-). It is still a *lossy* world-model view: it discards provenance, overlap/stamps, context, and posterior structure beyond the two-count summary.

Maple Court: Querying "Is the pipe leaking?" against Maple Court's apartment state extracts $\texttt{Evidence} = (12, 88)$—12 positive and 88 negative leak-sensor observations. The STV view gives $\mathsf{strength} = 0.12$, $\mathsf{confidence} = 0.91$: high confidence in a low-strength leak hypothesis. The ITV interval $[0.06, 0.20]$ captures the remaining uncertainty. All three views are lossy projections of the same underlying evidence, as Section 8.6 makes precise.

8.4 Confidence as a View

Confidence deserves special attention. In the original PLN presentation, confidence often appears as a standalone quantity that can be manipulated independently of strength. In WM-PLN, confidence is strictly a derived view:

$$\mathsf{confidence}(n^+, n^-) = \frac{n^+ + n^-}{n^+ + n^- + k}$$

The parameter k controls how much evidence is needed before the system becomes "confident." For $k = 1$, a single observation gives confidence 0.5; for $k = 10$, ten observations give confidence 0.5.

The key property: confidence is monotone in the total evidence count. More evidence always means higher (or equal) confidence.

For the implementer: confidence tells you how much data you have, not whether the data supports the claim. A high-confidence negative result is still informative.

8.5 Indefinite Truth Values

An *indefinite truth value* (ITV) provides interval endpoints for a selected uncertainty semantics:

$$\text{ITV} = [s_L,\ s_U,\ b,\ w]$$

where s_L and s_U are lower and upper bounds, b is a confidence-like parameter, and $w = s_U - s_L$ is the interval width.

There are at least three distinct semantic interpretations of such intervals, and they *must not be conflated*:

1. **Bayesian credible intervals.** An interval $[s_L, s_U]$ such that the posterior probability $\Pr(s_L \leq p \leq s_U \mid \text{data})$ exceeds some threshold. These depend on the prior.

2. **Walley-style predictive intervals (IDM).** Intervals derived from the Imprecise Dirichlet Model [40, 41], which provide robust predictions without committing to a single prior. These are prior-free but give wider intervals.

3. **Frequentist confidence intervals.** Intervals produced by a procedure with specified long-run coverage. After observing a dataset, they are not posterior probability statements about the parameter.

The original PLN book sometimes uses the notation $[L, U]$ without specifying which semantics is intended. WM-PLN insists on explicit selector choice: the user declares which interval semantics they want, and the system computes accordingly.[1]

The degrees of freedom are explicit in the constructors. Once lower and upper bounds are chosen, width is definitionally $s_U - s_L$. The scalar

[1]Formalized in `Mettapedia/Logic/PLNIndefiniteTruth.lean` and `Mettapedia/Logic/PLNWorldModelITV.lean`.

"strength" exported from an ITV is the midpoint $(s_L + s_U)/2$, a display representative rather than a theorem that decisions must use the midpoint. Bayesian ITV constructors choose a prior context, credible level, and interval backend; in the current formalization these choices affect the bounds, while credibility remains the BinaryEvidence concentration with $\kappa = \alpha_0 + \beta_0$. Walley-IDM constructors instead choose one strength parameter $s > 0$; then lower, upper, width, and credibility are fixed by the IDM predictive formulas.

For this reason, ITV compatibility narrows the confidence coordinate only after a specific interval semantics is selected. Generic ITVs and Bayesian credible intervals do not impose a simple law between width and credibility. Walley IDM predictive intervals do: their width is $s/(n+s)$, and requiring credibility to be the complement of predictive width forces $c = 1 - s/(n+s) = n/(n+s)$. The Lean theorem `walley_width_complement_forces_plnOdds` proves exactly this forcing result; the reserve-half coordinate is simultaneously proved to be an invertible count coordinate but not Walley-compatible.

8.6 Why Views Are Not Proof Objects

Views are useful. They are indispensable for practical inference. But they are *not* the right objects to pass between inference steps. The reason: views are lossy projections. When you chain two inferences using only the STV of the intermediate result, you have already lost information about the shape of the posterior, the correlation structure, and the raw evidence counts.

The WM-PLN discipline is: carry evidence (or world-model states) between inference steps; extract views only at the end, for display or decision-making. The slogan again:

> *Probability is what you query; evidence is what you carry.*

Chapter 9

Forgetting, Provenance, and State Management

Problem: A bare truth value cannot be partially retracted, scoped, or audited. A world-model *state* can. This chapter develops the third and fourth verbs of the kernel interface—forgetting and explanation—and proves that they satisfy precise conservation laws.

Layer: State management.
Semantic object: `ForgettingLayer`, `EvidenceConservationPack`, `TrackedWhichState`.
Proved: Noether conservation; anti-hallucination; zero-leakage; exact-inverse no-go; provenance factorization.
Assumed: Zero-preserving world model (for conservation theorems).

A world model stores *states*, not truth values. The previous chapters showed two things you can do with a state: *revise* it (combine evidence) and *query* it (extract evidence). This chapter shows two more: *forget* (partially retract evidence under scope constraints) and *explain* (trace which observations support an answer). Together, these four verbs—revise, query, forget, explain—are the complete operational interface to the world-model kernel.

For the implementer: forgetting and provenance are what make the world-model abstraction worth its weight. Without them, a state is just a fancy number. With them, it becomes a revisable, auditable, privacy-aware knowledge store.

9.1 Why Forgetting Matters

If the world model were just a truth value, there would be nothing to forget. A number has no parts. But a state has structure: it records which observations contributed to which queries. Forgetting exploits that structure by removing evidence within a declared scope while preserving everything outside it.

The canonical motivation: a sensor reading from last Tuesday should eventually expire. A privacy constraint says apartment-level data must not persist in the building model beyond 24 hours. A correction says "the batch of observations from sensor 3 was miscalibrated; retract them." All of these are forgetting operations.

9.2 The Forgetting Layer

A forgetting layer over a generic world model is a structure with four components:

```
structure ForgettingLayer (State Scope Query Ev) where
  inScope  : Scope -> Query -> Prop
  forget   : Scope -> State -> State
  idempotent    : forall S W, forget S (forget S W) = forget S W
  outsideInvariant :
    forall {S W q}, not (inScope S q) ->
      evidence (forget S W) q = evidence W q
```

The key law is `outsideInvariant`: forgetting scope S preserves the evidence for every query outside S. In the answer-profile language (Section 3.5), this says:

> *Forgetting changes the answer profile only within the declared scope.*

Idempotence says forgetting the same scope twice is the same as forgetting it once—there is no hidden state that accumulates.

Maple Court: The apartment WM forgets stale humidity readings after 6 hours: the scope is "all humidity observations older than $t - 6\text{h}$." Outside that scope—motion detections, leak alarms—the profile is unchanged. The building WM forgets laundry observations after 24 hours. The community WM forgets building-level aggregates after 30 days. Each layer's forgetting is independent and scope-delimited.

9.3 Conservation Theorems

Under *exact inverse forgetting*—where forgetting scope S exactly undoes a revision Δ that was entirely within S—a family of conservation laws holds:

Theorem 9.1 (Evidence conservation (CORE))**.** *If Δ is a revision whose evidence lies entirely within scope S (i.e., $\forall W$, $\text{forget}(S, W + \Delta) = W$), then for every query q outside S:*

1. ***Anti-hallucination**:* $\text{ev}(\Delta, q) = 0$. *The revision contributes zero evidence outside its scope.*
2. ***Noether conservation**:* $\text{ev}(W + \Delta, q) = \text{ev}(W, q)$. *The revised state agrees with the base state outside the scope.*
3. ***Zero leakage**: the observation count of Δ at q is zero.*

These are packaged as `EvidenceConservationPack` and proved canonically for every forgetting layer with a zero-preserving world model (`evidenceConservationPack_of_forgett`

Theorem 9.2 (Exact-inverse no-go (BOUNDARY))**.** *If a revision Δ has nonzero evidence at any query outside scope S, then no forgetting operation can exactly undo Δ.*

This is the precise boundary: forgetting cannot undo what leaked.

See `Mettapedia/Logic/GenericWorldModelForgetting.lean` (185 lines, 0 sorry) and `Mettapedia/Logic/PLNWorldModelConservationPack.lean` (220 lines, 0 sorry).

9.4 Provenance and Explanation

Evidence tells you *how much* support a query has. Provenance tells you *where that support came from*. The formalization models provenance as an enrichment of the evidence extraction:

$$\texttt{State} \xrightarrow{\text{extract-with-provenance}} (\texttt{Query} \to \text{Ev-with-Prov}) \xrightarrow{\text{forget prov.}} (\texttt{Query} \to \texttt{Evidence})$$

The first arrow produces evidence tagged with source identifiers (which sensor, which batch, which time window). The second arrow erases the tags,

recovering ordinary evidence. The composition equals ordinary evidence extraction—provenance is a refinement, not a replacement.

The concrete realization is `TrackedWhichState`: a state that carries, for each query, a multiset of provenance chunks. Each chunk records a source label and an evidence contribution. The total evidence at a query is the sum over chunks; the provenance is the set of source labels.

Maple Court: The building WM's hallway-humidity evidence carries provenance: "3 readings from sensor A (morning), 2 readings from sensor B (afternoon)." When sensor A is recalibrated, the steward can retract exactly sensor A's contributions (a scoped forget) without losing sensor B's data. The provenance makes the retraction targeted rather than wholesale.

See `Mettapedia/PLN/WorldModel/Provenance/PLNProvenanceTrackedState.lean` (241 lines, 0 sorry) and the `external/provenance-lean/` archive for the semiring-based provenance framework.

9.4.1 PLN Evidence Stamps as Provenance

PLN v0.9[1] tracks evidence provenance via *stamps*: each sentence carries a sorted list of observation IDs, and before combining two sentences, the engine checks `StampDisjoint`—whether their stamps share any IDs. If they do, the revision is blocked to prevent double-counting.

This is exactly the discipline formalized above. The correspondence is:

PLN v0.9 concept	**WM calculus formal counterpart**
`Sentence (Term, STV, Stamp)`	`ScopedTrackedWhichState`
`StampDisjoint` check	`Finset.Disjoint` on scope supports
`StampConcat`	`Finset.union` (proved = union)
Forget a source	`forgetScopedByScope`
Evidence ID in stamp	`Fin n` index in `TrackedWhichChunk`

The key difference: PLN v0.9 checks `StampDisjoint` at *runtime*; the WM calculus proves it at *compile time* via `decide`. And PLN v0.9 has *no forgetting*: once evidence enters a stamp, it cannot be retracted. The WM calculus

[1] PLN v0.9 is the current MeTTa implementation: `github.com/trueagi-io/PLN`, tag `v0.9`. It implements forward chaining with stamp-based evidence tracking in 430 lines of MeTTa.

adds exact forgetting (`forgetScopedByScope_exactInverse`, Theorem 9.1) with proved conservation.

The formalization verifies all three PLN v0.9 flagship examples (DeductionRevision, RavenInduction, FlyingRaven) with 20 conformance checks, all kernel-checked:

- **DeductionRevision**: two deduction paths A→B→D and A→C→D have disjoint stamps $\{0, 2\}$ and $\{1, 3\}$; revision is safe; combined stamp is $\{0, 1, 2, 3\}$. Forgetting observation 0 (A→B) preserves path 2 but breaks path 1.
- **RavenInduction**: two ravens' data (stamps $\{0, 2\}$ and $\{1, 3\}$) is independent; forgetting rv2 reduces the induction to rv1's evidence alone.
- **FlyingRaven**: Sam's chain (stamps $\{0, 3, 4\}$) and Pingu's chain (stamps $\{1, 2\}$) are disjoint; forgetting Sam's classification does not affect Pingu's.

See `Mettapedia/PLN/WorldModel/Provenance/PLNProvenanceInference.lean` (general theory, 0 sorry), `Mettapedia/Logic/PLNClassicTruthFunctions.lean` (truth functions linked to evidence, 0 sorry), and `Mettapedia/Logic/PLNClassicExamples.lean` (20 conformance checks, 0 sorry).

Part IV

Compiled Inference and Tractable Fragments

Chapter 10

Σ-Guarded Rewrites

Goal: Bridge the gap between complete semantics and tractable inference via compiled tactics.
Layer: Shard (compiled).
Semantic object: Σ-guarded rewrite rules with explicit side conditions.
Proved: Rewrite admissibility under declared model classes.
Assumed: Nothing (the mechanism is generic; side conditions are per-rule).

The gap between the complete world-model semantics and practical inference is bridged by *compiled rewrites*: derived rules that are correct under explicit structural assumptions.

10.1 The Rewrite Pattern

A Σ-guarded rewrite has the form:

$$\frac{\Sigma \vdash \text{ScreenOff}(A, B, C) \quad \Sigma; \Gamma \vdash \texttt{link}(A, B) \Downarrow e_{AB} \quad \cdots}{\Sigma; \Gamma \vdash \texttt{linkCond}([A, B], C) \Downarrow e_{AC}}$$

with the side-condition that $\mathsf{strength}(e_{AC})$ equals the result of applying the appropriate PLN formula to the extracted strengths.

The key features:

1. The side-condition context Σ is *explicit*—it appears in the judgment, not hidden in documentation.

2. The rewrite operates at the *strength level*—it tells you the correct probability answer, given the assumptions.

3. The assumptions are *checkable*—for a BN model class, they reduce to d-separation queries on the DAG.

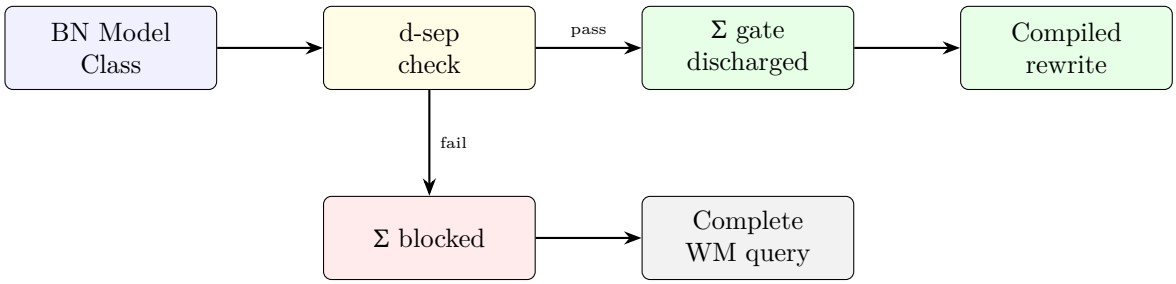


Figure 10.1: The Σ-guarded rewrite pattern. A compiled rule fires only when d-separation (or other structural checks) discharge its side conditions. If the gate is blocked, the system falls back to a complete world-model query rather than applying an unsound shortcut.

Maple Court: The Maple Court apartment BN (Figure 1.2) contains all three canonical motifs. The *chain* PipeLeak → WallHumidity → MoldRisk admits a compiled deduction rewrite when the intermediate node screens off. The *fork* from RoomOccupied admits a compiled abduction rewrite. The *collider* ShowerRunning → BathroomHumidity ← PipeLeak is the interesting case: its Σ-gate blocks naive abduction about the shower just because humidity is high. The d-sep check must confirm that BathroomHumidity is *not* conditioned on before firing the independence shortcut. When it *is* conditioned on, the collider opens and the two parents become coupled—the explain-away effect.

10.2 The Xi Rule Registry

The codebase maintains a registry of compiled rewrites covering the four exact-track rules—deduction, source/induction, sink/abduction, and revision. Each rule carries its screening-off side conditions, OSLF atom bridges, typed BN constructor lifts (chain, fork, or collider), and coordinate-specialized endpoints for lower, upper, credibility, width, and strength projections.[1]

[1]Formalized in `Mettapedia/Logic/PLNXiRuleRegistry.lean` and `Mettapedia/PLN/Bridges/ProbabilityTheory/BayesNet/PLNBNCompilation.lean`.

10.3 Compiled Tactics vs. Foundational Semantics

It is worth pausing to be explicit about the relationship between compiled rewrites and the foundational calculus.

A compiled rewrite is *admissible*: it can be derived from the foundational rules plus the structural assumptions in Σ. It is not a new axiom. It is a theorem that, under stated conditions, a certain query can be answered by combining the answers to simpler queries.

The advantage of presenting rules as compiled tactics rather than primitive laws is that the assumptions become visible. When a PLN deduction rule "works" in practice, it is because the implicit BN structure satisfies the screening-off conditions. When it "fails" (produces incorrect probabilities), it is because those conditions are violated. WM-PLN makes this explicit. These are not ad hoc patches to rescue a broken rule; they are the actual theoremic scope conditions under which the classical extensional formulas are exact.

Chapter 11

Bayesian-Network World Models and Compiled BN Inference

Problem: The complete world table is exponential in the number of atoms. To scale PLN inference, we restrict to Bayesian-network-structured world models and prove that standard inference patterns (chain, fork, collider) are *exact* under the restriction—not approximations, but certified shortcuts to the same answer the full model would give.

Layer: Shard (compiled).
Semantic object: BN world-model class; proof-carrying contraction; probabilistic guards.
Proved: Chain/fork/collider exactness; guarded admissibility on Pathfinder/Hailfinder.
Assumed: BN factorization (declared model class).

The complete world-table `JointEvidence` is exponential in n. To scale, we restrict the world-model class while preserving correctness relative to the restriction.

For the implementer: this chapter is where PLN becomes practical. The kernel (Part II) says what the correct answer is; the evidence algebra (Part III) says how to carry it. This chapter says how to *compute* it efficiently for the most common model class. The core theorems (Sections 11.1–11.7) are the theoretical backbone; the applied validation (Section 11.9) demonstrates them on real networks.

11.1 BN World-Model Class

A Bayesian-network world model consists of a directed acyclic graph (DAG) specifying the factorization, together with local conditional probability tables (CPTs) whose rows carry Dirichlet evidence counts. Revision is the addition of local Dirichlet counts—the conjugate update familiar from Bayesian statistics—and query answering proceeds by exact inference via variable elimination or junction tree algorithms.

The DAG structure provides the structural assumptions (Σ) needed for compiled rewrites. D-separation in the DAG determines which conditional independence assumptions hold, and these determine which PLN rules are applicable.[1]

11.2 Exact BN Query Bridge

The BN world-model layer provides an evidence plumbing stack that connects the joint posterior to node-level inference. At the bottom sits a Boolean BN CPT query type `CPTQuery`, identifying a node together with its parent configuration. Above it, a CPT posterior state `CPTState` stores `Evidence` per CPT entry. The stack is completed by an additive projection from `JointEvidence` to CPT evidence that marginalizes by summing compatible worlds—and, crucially, this projection commutes with revision, so that updating the joint and then projecting gives the same result as projecting first and updating locally.

11.3 Chain, Fork, and Collider Exactness

The three fundamental BN motifs each yield exactness theorems for the corresponding PLN rule. Recall the PLN deduction strength formula (Chapter 4):

$$s_{AC} = s_{AB} \cdot s_{BC} + (1 - s_{AB}) \cdot \frac{s_C - s_B \cdot s_{BC}}{1 - s_B}$$

Theorem 11.1 (Chain BN deduction exactness (Conditional))**.** *In a chain BN $A \to B \to C$ with CPT-based local probabilities, the screening-off conditions*

$$P(C \mid A \cap B) = P(C \mid B), \qquad P(C \mid A \cap B^c) = P(C \mid B^c)$$

[1] Formalized in `Mettapedia/PLN/WorldModel/BayesNet/PLNBayesNetWorldModel.lean`.

hold by d-separation. Under positivity ($P(B \mid A) > 0$, $P(B) \in (0,1)$), the PLN deduction formula computes the exact conditional:

$$s_{AC} = P(C \mid A) = \sum_b P(C \mid B = b) \cdot P(B = b \mid A)$$

The key mechanism: the CPT row for C depends only on B's value (sink property), and the joint weight factorizes as $w(c, r) = P_{\mathrm{CPT}}(c \mid \mathrm{pa}(C)) \cdot \prod_{v \neq C} P_{\mathrm{CPT}}(v \mid \mathrm{pa}(v))$, where the second factor is independent of c.

Theorem 11.2 (Fork BN deduction exactness (Conditional))**.** *In a fork BN $A \leftarrow B \rightarrow C$, the common-cause B screens off A from C: $P(C \mid A, B) = P(C \mid B)$. Under positivity, the PLN deduction formula again yields the exact $P(C \mid A)$.*

Theorem 11.3 (Collider screening-off (Conditional))**.** *In a collider $A \rightarrow B \leftarrow C$, A and C are marginally independent but become dependent when conditioning on B. The PLN abduction formula applies with modified side conditions.*

These "fast rule exactness" results show that classic PLN formulas are not approximations but exact computations within the declared model class. The screening-off and positivity hypotheses are part of the mathematical statement, and the local-completeness no-go theorem explains why they cannot be erased in the unrestricted setting.[2]

Maple Court: The collider ShowerRunning → BathroomHumidity ← PipeLeak is the canonical explain-away motif. When humidity is observed high with no shower evidence, the posterior for PipeLeak rises. When the shower is confirmed running, PipeLeak's posterior drops back—the shower "explains away" the humidity. The Lean demo (`dsep_shower_leak_no_humidity`) proves the structural d-separation: with no humidity evidence, adding shower evidence does not change the leak component, and vice versa. The CPT-level explain-away computation is handled by the `ColliderBNLocalMarkovPackage`.

---

[2]Formalized in `Mettapedia/Logic/PLNBayesNetFastRules.lean` and `Mettapedia/PLN/RuleFamilies/FirstOrder/PLNXiDerivedBNRules.lean`.

11.4 Class-Packaged BN Discharge

For practical use, the individual screening-off and positivity conditions are packaged into class-level discharge:

```
allDiscreteCPTLocalMarkov_of
wmqueryeq_of_dsep_discharge_via_soundness_allLocalMarkov
chain_screeningOff_wmqueryeq_of_dsep_allLocalMarkov
fork_screeningOff_wmqueryeq_of_dsep_allLocalMarkov
collider_screeningOff_wmqueryeq_of_dsep_allLocalMarkov
```

This means: given a BN model class satisfying local Markov properties, the screening-off conditions for chain, fork, and collider patterns are automatically discharged.[3]

11.5 Worked Proof-Carrying Contraction

The natural question at this point is whether anything recognizably PLN-like remains at the operational level. The answer is yes, but in a corrected form. By *proof-carrying contraction* I mean a compiled rewrite together with its explicit side-condition package Σ and its theorem provenance. The semantic factor graph is still the truthmaker; the PLN rule is the local contraction that is allowed to fire only when Σ is discharged. What disappears is naked truth-value propagation without a declared model class.

Our canonical worked micro-example is the binary BN

$$A \leftarrow H \rightarrow B \rightarrow C \rightarrow D$$

with

$$P(H{=}1) = \tfrac{1}{5},\ P(A{=}1 \mid H{=}1) = \tfrac{9}{10},\ P(A{=}1 \mid H{=}0) = \tfrac{1}{10},$$

$$P(B{=}1 \mid H{=}1) = \tfrac{4}{5},\ P(B{=}1 \mid H{=}0) = \tfrac{1}{5},\ P(C{=}1 \mid B{=}1) = \tfrac{17}{20},\ P(C{=}1 \mid B{=}0) = \tfrac{3}{20},$$

$$P(D{=}1 \mid C{=}1) = \tfrac{9}{10}, \qquad P(D{=}1 \mid C{=}0) = \tfrac{1}{10}.$$

The semantic baseline is exact factor-graph elimination on this declared model. The compiled PLN-style contractions then recover the same answers in the exact fragment. First, a fork-style contraction (the "source rule") reasons from the common-cause motif $A \leftarrow H \rightarrow B$ and obtains $P(B{=}1 \mid A{=}1) = \frac{8}{13} \approx 0.615$. Next, a chain-style contraction uses the derived $A \Rightarrow C$

[3] Formalized in `Mettapedia/PLN/Bridges/ProbabilityTheory/BayesNet/PLNBNLocalMarkovPackages.lean`.

query through the B-mediated exact fragment, yielding $P(C{=}1 \mid A{=}1) = \frac{151}{260} \approx 0.581$. A second chain-style contraction then pushes through C to reach $P(D{=}1 \mid A{=}1) = \frac{367}{650} \approx 0.565$.

The important point is not just the numbers. Each derived step is packaged as

$$(\text{query}, \text{value}, \Sigma, \text{provenance}),$$

not as a bare derived edge. In the Lean demo this provenance points back to the already-proved exact endpoints `PLNEndToEnd.forkFormulaExact`, `PLNEndToEnd.chainFormulaExact`, and the corresponding Σ-guarded rewrite lemmas. See `fork_contraction_matches_exact_semantics`, `chain_contraction_C_matches_exact_` `chain_contraction_D_matches_exact_semantics` in `Mettapedia/Logic/PLNProofCarryingContractionDemo.lean`.

The same file also contains two negative controls, because a worked positive example is only convincing if it does not quietly smuggle back the old unrestricted semantics.

First, perturb the fixture so that C depends weakly on A as well as B:

$$P(C{=}1 \mid A{=}0, B{=}0) = \frac{3}{20},\ P(C{=}1 \mid A{=}1, B{=}0) = \frac{1}{4},$$

$$P(C{=}1 \mid A{=}0, B{=}1) = \frac{17}{20},\ P(C{=}1 \mid A{=}1, B{=}1) = \frac{9}{10}.$$

Now the exact semantic answer becomes

$$P(C{=}1 \mid A{=}1) = \frac{13}{20} = 0.65,$$

while the naive screened-off contraction gives only

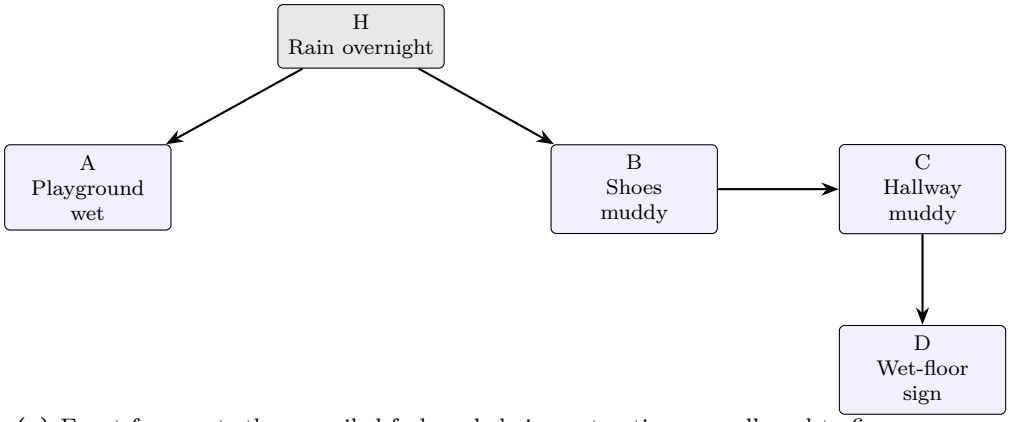
$$\frac{133}{221} \approx 0.602.$$

So the gate matters: when Σ is not discharged, the exact compiled chain contraction should not fire. See `soft_gate_blocks_exact_contraction` in `Mettapedia/Logic/PLNProofCarryingContractionDemo.lean`.

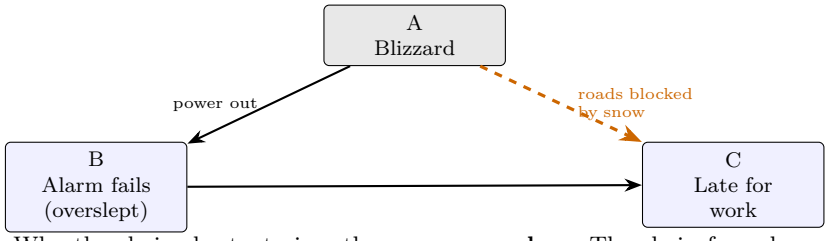
Second, a collider negative control keeps the old abduction story honest. For the OR-gate collider $A \to C \leftarrow B$ with $P(A{=}1) = P(B{=}1) = \frac{1}{2}$, the old naive abduction formula yields $\frac{2}{3}$, while the exact semantic answer remains $\frac{1}{2}$. So the refounded system does *not* license free abduction-style link creation merely because a local pattern "looks like" a classic PLN rule. See `collider_negative_control_blocks_naive_abduction` in `Mettapedia/Logic/PLNProofCarryingContractionDemo.lean`.

This is the corrected operational reading of PLN rule firing. The old intuition survives, but as admissible, Σ-guarded local contraction inside a declared world-model class, not as unconstrained link propagation.

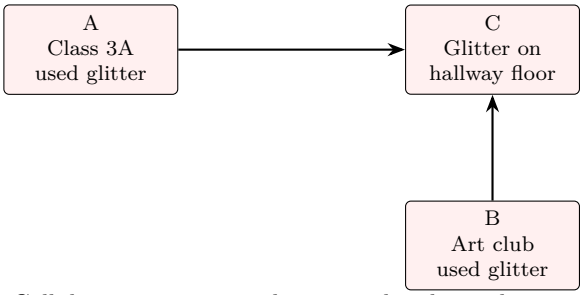
Figure 11.1 gives three regimes. The left panel (school setting) is the exact fragment where the shortcut is correct. The middle panel (blizzard) shows a direct path that makes the chain shortcut give a massively wrong number. The right panel (school setting) is the collider negative control.



(a) Exact fragment: the compiled fork and chain contractions are allowed to fire.



(b) Why the chain shortcut gives the **wrong number**. The chain formula routes all of the blizzard's effect through the alarm: power outage kills the alarm ($A \to B$), you oversleep and are late ($B \to C$). But the blizzard also blocks the roads directly (dashed)—even if your alarm has a backup battery ($B = 0$), you are still late because traffic is gridlocked. The shortcut forces the blizzard's entire causal power through the alarm-clock bottleneck, blind to the roads.



(c) Collider negative control: seeing the glitter does not license naive free abduction from one source to the other.

Figure 11.1: Three regimes for compiled inference shortcuts. Left: rain explains both a wet playground and muddy shoes via independent paths, so the fork/chain shortcuts match exact inference. Middle (blizzard): the direct $A \to C$ path (roads blocked) dominates—the chain shortcut forces all causal power through the alarm-clock bottleneck and massively underestimates. Right: two possible glitter sources feeding one hallway clue form a collider; abduction from one source to the other is structurally unsound without conditioning on the shared effect.

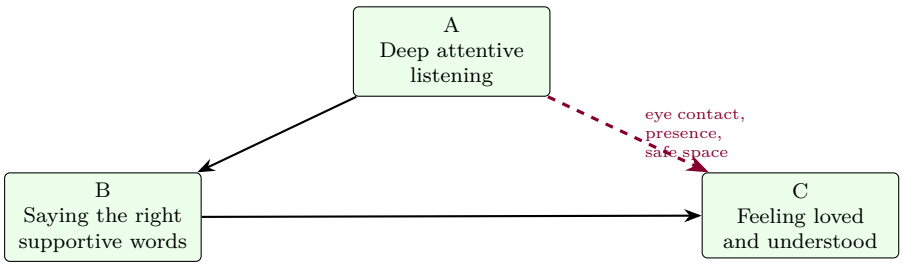
Concrete numbers for the blizzard. Assign probabilities (A = blizzard, B = alarm fails, C = late): $P(A=1) = \frac{1}{2}$, $P(B=1 \mid A=1) = \frac{1}{2}$, $P(B=1 \mid A=0) = \frac{1}{4}$. With the direct road-blocking path: $P(C=1 \mid B=1, A=1) = \frac{9}{10}$ (alarm dead *and* roads blocked), $P(C=1 \mid B=0, A=1) = \frac{3}{5}$ (alarm fine but roads still blocked), $P(C=1 \mid B=1, A=0) = \frac{7}{10}$, $P(C=1 \mid B=0, A=0) = \frac{1}{10}$.

True answer: $P(\text{late} \mid \text{blizzard}) = \frac{9}{10} \cdot \frac{1}{2} + \frac{3}{5} \cdot \frac{1}{2} = \mathbf{\frac{3}{4}} = 75\%$.

Chain shortcut (uses $P(C \mid B)$ instead of $P(C \mid B, A)$): marginalizing gives $P(C = 1 \mid B = 1) = \frac{5}{6}$ and $P(C = 1 \mid B = 0) = \frac{3}{10}$, so the shortcut returns $\frac{5}{6} \cdot \frac{1}{2} + \frac{3}{10} \cdot \frac{1}{2} = \mathbf{\frac{17}{30}} \approx 57\%$.

Gap: $\frac{3}{4} - \frac{17}{30} = \frac{11}{60} \approx 18$ percentage points. The chain shortcut forces the blizzard's entire causal power through the alarm-clock bottleneck and misses the road-blocking path entirely. Your alarm clock isn't the only way a blizzard makes you late.[4]

The same structure without numbers: resonance. The insight does not require precise probabilities.



The chain shortcut routes all effect through words: listening helps you choose the right words ($A \to B$), and the right words make them feel loved ($B \to C$). But listening *itself*—the eye contact, the presence, the safe space—makes them feel loved directly ($A \to C$), often more powerfully than anything you say. Any entity that applies the chain shortcut here reduces a holistic relational resonance to a text-generation task, missing the reality that *presence does most of the work*. No CPT needed: the structural insight that $A \not\perp\!\!\!\perp C \mid B$ is enough to know the shortcut is wrong.

[4] All arithmetic kernel-checked over $\mathbb{Q}$ in `Mettapedia/.tmp/chain\_shortcut\_error.lean`.

11.6 Bridge Theorems: Naive Bayes and k-NN

The BN world-model framework connects PLN to two standard machine-learning methods.

Theorem 11.4 (Naive Bayes bridge (CORE))**.** *For a Naive Bayes model (class variable C, features $F_1, \ldots, F_n$ conditionally independent given C), the PLN tensor-strength computation equals the Naive Bayes posterior:*

$$\mathsf{strength}_{\otimes}(e_{F_1 \Rightarrow C}, \ldots, e_{F_n \Rightarrow C}) = P(C \mid F_1, \ldots, F_n)_{\mathrm{NB}}$$

See `PLN_tensorStrength_eq_nbPosterior` in `Mettapedia/PLN/WorldModel/BayesNet/PLNBayesNetInference.lean`.

Theorem 11.5 (k-NN bridge (CORE))**.** *PLN's evidence-weighted relevance score equals the k-NN relevance function:*

$$\oplus\text{-pos}(e_1, \ldots, e_k) = \text{knnRelevance}(x)$$

where the e_i are evidence values from the k nearest neighbors.

See `PLN_hplusPos_eq_knnRelevance` in `Mettapedia/PLN/InferenceControl/PremiseSelection/KNN\_PLNBridge.lean`.

These bridges show that PLN's evidence algebra is not an isolated formalism: it subsumes standard probabilistic classifiers as special cases.

11.7 Factor Graphs: Semantic vs. Operational

PLN has long used factor-graph and message-passing intuition. WM-PLN separates two roles:

- **Semantic factor graphs.** Factors *are* the model: the posterior state is represented by a factorization. Variable elimination gives exact answers; complexity is controlled by treewidth. This is a *world-model representation*.
- **Operational factor graphs.** Factors are *rule schemas* (deduction, induction, etc.) whose local "potentials" are truth-value update formulas. This is *inference control*: it proposes which queries to ask, but does not itself constitute the semantics.

The two align when rule factors are *compiled* from a world-model class: each factor corresponds to a conditional in the BN, and message passing implements the correct queries.

11.8 PLN v0.9 Examples as WM Calculus Instances

PLN v0.9 [15][5] provides three flagship examples that exercise the core inference rules:

DeductionRevision (`DeductionRevision.metta`): Given A→B, A→C, B→D, C→D, query A→D. Two deduction chains fire (via B and via C), then revision combines their results. PLN v0.9 reports $\langle 0.125,\ 0.304\rangle$ with stamp $\{1, 2, 3, 4\}$.

In the WM calculus: each chain is a compiled BN chain motif (Section 11.3) firing under the screening-off side condition. The two results have disjoint provenance (Section 9.4.1), so revision (= evidence addition) is safe. The resulting truth value is the strength/confidence view of the combined evidence.

RavenInduction (`RavenInduction.metta`): Given two ravens, one black and one not, induce "ravens are black." PLN v0.9 applies the induction rule with $\langle 0.38,\ 0.654\rangle$.

In the WM calculus: induction is the source-rule (cospan completion), formalized as `truthInduction` with strength equal to `plnInductionStrength` (Theorem `truthInduction_s_eq`).

FlyingRaven (`FlyingRaven.metta`): Sam (a raven) should fly; Pingu (a penguin) should not. Three-step chain Sam→Raven→Bird→flies, plus negation Penguin→ ¬flies.

In the WM calculus: Sam's chain and Pingu's chain have disjoint provenance stamps ($\{0, 3, 4\}$ vs. $\{1, 2\}$). Forgetting Sam→Raven (observation 0) breaks Sam's chain evidence but provably conserves Pingu's (Theorem `fr_forget_sam_preserves_pingu`).

All three examples are verified with 20 kernel-checked conformance tests in `Mettapedia/Logic/PLNClassicExamples.lean` (0 sorry), covering PeTTa space queries, stamp disjointness, stamp union, and forgetting set-difference.

The key result: every classic PLN inference pattern is a specific compiled BN rewrite with provenance tracking. The WM calculus *subsumes* PLN v0.9 and *adds* forgetting with proved conservation—something PLN v0.9 cannot express.

Raven asymmetry and abduction, formalized. Beyond the provenance story above, the *dynamical* content of the raven paradox—"ravens are black"

[5] PLN v0.9: `github.com/trueagi-io/PLN`, tag `v0.9`.

survives while "black things are ravens" decays—is now machine-checked in `Mettapedia/PLN/RuleFamilies/FirstOrder/RavenAsymmetricInduction.lean` (0 sorry, 0 axiom). Modelling induction as iterated revision on `BinaryEvidence`, R black ravens and M other black items give `strength_ravenBlack` $= R/R = 1$ and `strength_blackRaven` $= R/(R+M)$; the inverse link is antitone in M (`strength_blackRaven_antitone`) and converges to the base rate, `strength_blackRaven_tendsto_zero` proving $R/(R+M) \to 0$ as $M \to \infty$. A dataset layer over finite observation lists proves the strengths depend only on the two sufficient counts (`aggregate_blackRaven_eq_counts`) and are order-insensitive (exchangeability, `aggregate_blackRaven_perm`). The query-side dual, `Mettapedia/PLN/RuleFamilies/FirstOrder/RavenAbduction.lean`, casts the same asymmetry as inference to the best explanation: from one common feature the base rate wins (`crow_best_on_black`), and only a discriminating feature flips the best explanation (`best_explanation_flips`)—the atomic NARS `Truth_Abduction` rule generates the hypothesis, but ranking competing explanations needs the prior. Both have runnable MeTTa mirrors, `papers/examples/RavenAsymmetricInduction.metta` and `papers/examples/RavenAbduction.metta`.

11.8.1 Stepped Walkthrough: DeductionRevision with Provenance

We step through the DeductionRevision example in full detail.

Step 1: The knowledge base. Four observations, encoded as a PeTTa atomspace:

```
-- The PLN v0.9 knowledge base as a PeTTa space
def drKB : PeTTaSpace where
  facts := [inh "A" "B",   -- obs 0
            inh "A" "C",   -- obs 1
            inh "B" "D",   -- obs 2
            inh "C" "D"]   -- obs 3
  rules := []
```

Step 2: Stamp disjointness (kernel-checked). Each deduction path tracks which observations it used:

```
def dr_stamp1 : Finset (Fin 4) := {0, 2}  -- Path 1: A->B->D
def dr_stamp2 : Finset (Fin 4) := {1, 3}  -- Path 2: A->C->D

-- StampDisjoint: disjoint provenance (kernel-checked by decide)
```

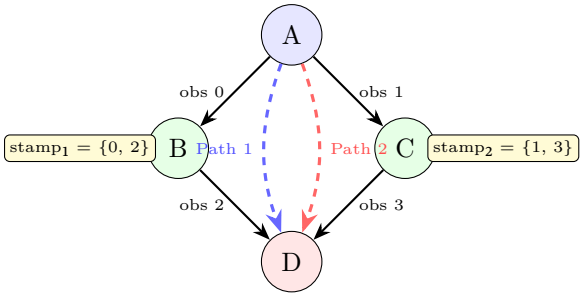


Figure 11.2: DeductionRevision: two deduction paths from A to D with disjoint provenance stamps. Path 1 (via B) uses observations $\{0, 2\}$; Path 2 (via C) uses $\{1, 3\}$. Since the stamps are disjoint, revision is safe.

```
theorem dr_stamps_disjoint : Disjoint dr_stamp1 dr_stamp2 := by
    decide

-- Combined stamp = union of all observations
theorem dr_stamps_union :
    dr_stamp1   dr_stamp2 = {0, 1, 2, 3} := by decide
```

The `decide` tactic delegates to the kernel: Lean verifies the set-disjointness computation at type-checking time, not at runtime. This is the compile-time version of PLN v0.9's runtime `StampDisjoint` check.

Step 3: Forgetting with conservation. PLN v0.9 cannot retract observations. The WM calculus can:

```
-- Forgetting obs 0 (A->B) does NOT affect Path 2
theorem dr_obs0_disjoint_path2 :
    Disjoint {0} dr_stamp2 := by decide

-- After forgetting obs 0, Path 1 retains only obs 2 (B->D)
theorem dr_path1_after_forget :
    dr_stamp1 \ {0} = {2} := by decide
```

Forgetting observation 0 (the A→B link) removes it from Path 1's stamp but *provably* leaves Path 2 intact. This is a concrete instance of the Noether conservation theorem (Section 9.2): evidence outside the forgotten scope is preserved.

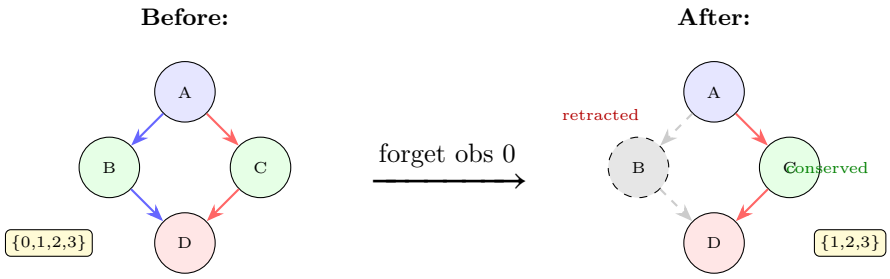


Figure 11.3: Forgetting observation 0 (A→B). Path 1 (via B) loses its first link; Path 2 (via C) is provably unaffected. The stamp shrinks from $\{0, 1, 2, 3\}$ to $\{1, 2, 3\}$.

11.9 Applied BN Validation and Benchmark Lanes

The preceding sections established the core exactness theorems, discharge machinery, and bridge results for BN-structured world models. The remaining sections of this chapter apply those theorems to real Bayesian-network benchmarks. *They can be skipped on first reading* without loss of continuity; the next chapter (Error, Thresholds, and Transport) is self-contained.

11.9.1 Probabilistically Guarded Admissibility on Real BN Benchmarks

The toy fixture above establishes the theorem-backed exact fragment in a fully worked micro-example. The next question is whether the same discipline helps on canonical benchmark BNs.

Pathfinder is a classic Bayesian expert system for diagnosing lymph-node diseases from pathology findings,[17] and Hailfinder is a Bayesian severe-weather forecasting model for northeastern Colorado built from meteorological variables and expert judgment.[1] These are not generic tabular ML datasets. They are *fixed Bayesian network source models* with expert-curated conditional probability tables, and our benchmark treats them as such: the primary artifact is the BIF network file (providing exact ground truth via variable elimination), not a flat CSV of sampled rows. From each source model we derive a *state-action inference-control benchmark*: a set of (query, context, provenance) states paired with legal inference actions (continue by exact rule, soften, reveal missing context, fall back, abstain), where the ground truth for each (state, action) pair comes from exact computation against the BIF truthmaker. We use the canonical BIF encodings distributed through the bnlearn Bayesian Network Repository.[38] Through-

out this subsection we keep the derived-object discipline explicit:

Throughout this subsection, we use three plain-English labels for the side-condition context Σ:

- **context-complete**: the carried Σ contains the structural context needed to license the rewrite exactly;
- **context-missing**: some required context has been omitted, so the exact rewrite is no longer licensed;
- **context-uncertain**: the required context is only partially supported, so bounded or operational control layers become relevant.

Benchmark	Domain	Primary artifact	What the variables mean	Why we use it here
Hailfinder	Severe summer hail forecasting in northeastern Colorado	A hand-built Bayesian weather-forecast BN with expert CPTs; `hailfinder.csv` is secondary to `hailfinder.bif`	Meteorological conditions, scenario summaries, and forecast targets	Readable positive lane: exact local chaining still works on a real forecast network.
Pathfinder	Lymph-node pathology diagnosis	A hand-built Bayesian network with expert CPTs and a disease-class root (`Fault`)	One latent diagnosis plus many coded pathology findings in the benchmark release	Hidden-context stress test: can proof-carrying contractions survive when crucial diagnostic context is omitted?

Table 11.1: What the two benchmark networks are about. In both cases the primary benchmark object is the Bayesian network itself, not a generic flat table.

v2 benchmark design. The current benchmark (v2) is organized around three physically separated table families: a *state table* listing every (path, context-variant) pair; an *action table* enumerating legal decisions per state (continue-rule, continue-soft, reveal, fallback, abstain); and an *eval table* containing exact BN ground truth that is hidden from policy code. Oracle and blind decision policies operate on the *same* master state set; the information-set separation is enforced by physical column projection—the

blind decision CSV physically lacks all exact-probability and structural-admissibility columns, making oracle leaks impossible by construction. Each state carries a `query_kind` tag (broad vs refined) and a deterministic hash-based `split_id` (train/val/test) for reproducible cross-validation.

$$(\text{query}, \text{value}, \Sigma, \text{provenance}, g?, \delta?, \text{fallback}?).$$

The guarded layer is an inference-control discipline on top of exact BN semantics, not a replacement for that semantics.

Three-mode guarded admissibility. Context-complete Σ gives exact firing. Context-missing Σ blocks the contraction. Context-uncertain Σ moves to a bounded semantic guard or to an explicit soft fallback layer.

Mode	Status and returned object
Hard-gated exact	*Semantic, theorem-backed.* Fire only when Σ is discharged. The returned query answer keeps its carried Σ and provenance; there is no reusable bare derived edge.
Bounded violation	*Semantic guard.* Return an estimate plus an explicit certified radius or interval against the BN truthmaker.
Soft-guarded mixture	*Operational / governance.* Return $\hat{p} = g \cdot p_{\text{rule}} + (1-g) \cdot p_{\text{fallback}}$ with explicit fallback. This is useful for ranking, abstention, and trigger logic, but it is not automatically the semantic posterior.

Task framing. Methodologically, the real question is no longer merely "can we classify admissibility?" It is *topology-conditioned latent-context inference control*: given a source condition and a target query inside a declared BN truthmaker, should the controller continue by exact rewrite, softened continuation, bounded continuation, context reveal, local exact fallback, or abstention? Topology matters because it tells us which kinds of approximation assumptions are likely to survive transport through the graph and which are likely to fail. Hailfinder is the readable positive lane for this task. Pathfinder is the hostile hidden-context lane.

Hailfinder first: readable exact contraction. Hailfinder is the cleaner forecasting-structured benchmark. The exact variables are meteorological conditions and forecast targets, so the benchmark reads like an actual forecast/update chain rather than an opaque medical codebook. This is the best

first example for readers: it teaches what a proof-carrying local contraction looks like before the topology becomes adversarial.

The exact forecast chain

$$P(\texttt{MountainFcst=SIG} \mid \texttt{AMInstabMt=Strong}) = 0.5$$

fires cleanly, and a longer forecast/update path still matches exact BN semantics:

$$P(\texttt{CapChange=Increasing} \mid \texttt{CldShadeOth=Cloudy}) = 0.508284\ldots$$

The negative control remains equally important. In the selected collider motif, the exact semantic answer is $0.23595\ldots$, but the hard gate still blocks unrestricted reuse; bounded mode returns the interval $[0.23595\ldots, 0.27959\ldots]$, and soft mode returns an explicit operational mixture with semantic fallback. So Hailfinder says there is still a real exact forecasting-style chaining fragment, while the blocked cases preserve the "yes, but not everywhere" boundary.

Pathfinder next: the same calculus under hostile topology. Pathfinder is the stronger context-heavy benchmark. In the current BIF release it has 109 nodes and 195 arcs; `Fault` has 63 states and 106 outgoing children, and 56 nodes have multiple parents. So omitted context is not a side issue here. It is the main structural fact of the dataset. In the selected motifs, `Fault` is the latent disease hypothesis and the `F…` variables are benchmark-coded pathology findings from the network file. For exposition, it is often better to read the main fork/chain path as *observation* → *bridge* → *downstream consequence*, with the actual BN names supplied by Table 11.2.

The main Pathfinder stress question is therefore not "does local chaining ever work?" but "what happens to the same local contraction when hidden diagnostic context is omitted, partially restored, or completed?" In the widened suite the same path family is explicitly tracked across five regime types: *context-missing*, *alt-support-only*, *hub-only*, *context-pair*, and *oracle-context-complete*. That is the second-order object in practice: uncertainty over which context/admissibility regime the step is really in.

In the selected fork motif, keeping `Fault` explicit in Σ yields an exact hard-gated contraction:

$$P(F104\texttt{=x_50} \mid F103\texttt{=Almost_all_poor_def}, \texttt{Fault=T_immunob_lrg}) = 0.4.$$

Network	Variable	Reader gloss	Role in the selected motifs
Hailfinder	`AMInstabMt`, `InsInMt`, `MountainFcst`	Morning mountain instability, mountain instability, mountain forecast	The clean local forecast-update chain.
Hailfinder	`CldShadeOth`, `AreaMesoALS`, `CompPlFcst`, `CapChange`	Cloud shading, mesoscale-area summary, composite plains forecast, capping change	The longer readable forecast/update path.
Hailfinder	`AMCINInScen`, `CapInScen`	Scenario-specific convective inhibition and cap state	The collider-style negative boundary.
Pathfinder	`Fault`	Latent disease hypothesis / diagnosis class	The hidden diagnostic context that often has to stay in Σ.
Pathfinder	`F3`, `F4`, `F103`	Benchmark-coded observed pathology clues	Upstream evidence nodes in the selected fork and long-path motifs.
Pathfinder	`F97`, `F105`	Benchmark-coded intermediate pathology factors	Intermediate shared nodes through which the admissible contraction travels.
Pathfinder	`F101`, `F102`, `F104`	Benchmark-coded downstream pathology targets	Queried consequences used to test exactness and hidden-context sensitivity.

Table 11.2: Mini glossary for the variables actually used in the current benchmark motifs. Pathfinder keeps the compact benchmark coding from the released network; Hailfinder uses more readable meteorological names.

But if the same motif is evaluated in the *context-missing* regime, with `Fault` omitted from Σ, the exact semantic answer drops to $0.14998\ldots$, and the hard gate blocks. The bounded mode then returns the rule estimate $0.31236\ldots$ together with the certified interval $[0.14998\ldots, 0.47474\ldots]$, while the soft-gated operational layer returns $0.28599\ldots$ with explicit fallback to the exact semantic query. This is the intended result: Pathfinder does not show that "free PLN chaining works" on a large medical BN; it shows that proof-carrying local contraction survives only when the hidden diagnostic context is carried, and that context-erasing contractions must be blocked or degraded gracefully.

The next question is why adventurous continuation sometimes wins the original broad Pathfinder task and sometimes does not. Here the most useful abstraction is no longer a single guard score. It is a topology-conditioned

Case	Exact BN	Guarded output	Why it matters
Hailfinder exact forecast chain	0.500	Hard fires. Bounded $\delta = 0$. Soft $g = 1$.	There is still a clean exact chaining fragment for forecasting-style paths.
Hailfinder forecast collider negative	0.236	Hard blocks. Bounded 0.258 in $[0.236, 0.280]$. Soft 0.257.	The guarded layer preserves the no-free-abduction boundary even in the forecasting BN.
Pathfinder context-complete fork	0.400	Hard fires. Bounded $\delta = 0$. Soft $g = 1$.	When `Fault` stays in Σ, PLN-style contraction is exact.
Pathfinder context-missing fork	0.150	Hard blocks. Bounded 0.312 in $[0.150, 0.475]$. Soft 0.286.	Omitting `Fault` changes the answer sharply; carried context is necessary.

Table 11.3: Compact guarded-admissibility summary for the current real BN benchmarks. Exact BN semantics remains the truthmaker in every row.

latent-context controller. Let C denote unresolved hidden context, typically including `Fault` and, in some paths, an auxiliary support variable. Before any reveal action, the target is the broad query $P(Q \mid E)$. After revealing one context value c, the target becomes the refined query $P(Q \mid E, C{=}c)$. These are different tasks. Exact reveal wins the refined task because it conditions on the right branch explicitly. But it need not minimize error against the original broad task, because the broad query marginalizes over the still unresolved context:

$$P(Q \mid E) = \sum_c P(Q \mid E, C{=}c)\, P(C{=}c \mid E).$$

A controlled soft continuation,

$$\hat{p}_{\text{soft}} = g\, p_{\text{rule}} + (1 - g)\, p_{\text{fallback}},$$

can therefore be a better *operational* estimator for the original broad query than prematurely collapsing onto a single revealed branch, even though we do not yet claim that the current learned g is itself the semantic posterior. But the widened Pathfinder suite shows that this is not universal. The topology label is precisely what tells us which approximation assumptions are more or less plausible. In hub-dominated cases, omitted-context mass is

often concentrated around one dominant diagnostic direction, so direct continuation can stay close to the unresolved broad marginal. In loopy-dense or partial-support cases, several live alternatives can remain active, and then softened or marginalization-aware continuation becomes more attractive. So the current benchmark-side explanation is already informative: topology is not a decorative covariate but a guide to which kinds of assumptions do or do not transport well. The new point is that the finite regime-mixture layer is now theorem-backed in Lean, in `PLNRegimeMixtureTheorems.lean`, `PLNRegimeMixtureBenchmarkBridge.lean`, `PLNWorldModelRegimeAdmissibility.lean`, and `PLNProbHOLPlannerBridge.lean`. If R is a finite regime space, w_r are posterior regime weights, and q_r are branch query values, then the unresolved broad-query predictor is

$$m = \sum_{r \in R} w_r q_r.$$

For any designated branch r_0, the direct-continuation approximation error obeys the residual-mass bound

$$|m - q_{r_0}| \le (1 - w_{r_0}) \cdot \max_{r \neq r_0} |q_r - q_{r_0}|.$$

Under squared loss, the constant-predictor risk decomposes as

$$\sum_{r \in R} w_r (x - q_r)^2 = \underbrace{\sum_{r \in R} w_r (m - q_r)^2}_{\text{mixture variance}} + (x - m)^2,$$

so the unresolved mixture predictor m is optimal among constant predictors, and exact reveal is preferred once

$$\text{gain}_{\text{reveal}} = \text{mixture variance} - c_{\text{reveal}} > 0.$$

This is already enough to turn the higher-order direct/soft/reveal split into a theoremic finite-regime statement: direct continuation is justified when posterior mass concentrates tightly around one branch; soft continuation is justified because the mixture predictor minimizes unrevealed squared-loss risk; and reveal/context completion is justified once value-of-information beats reveal cost. What remains empirical at present is the *topology-conditioned proxy story*: labels such as "hub-dominated" or "loopy-dense" are still benchmark-side ways of predicting when concentration, spread, and latent support are likely to take those theoremically relevant shapes.

Three higher-order claims. The current benchmark supports three proposition-shaped claims that keep the semantics/control split clean.

1. **Objective mismatch under reveal.** Revealing hidden context changes the target from a broad marginalized query to a refined conditional query, so reveal can improve refined fidelity while worsening broad-query utility.

2. **Topology-conditioned continuation preference.** Hub-dominated slices often reward direct continuation; partial-support and some loopy-dense slices leave more live alternatives and therefore make softened continuation more competitive.

3. **Higher-order flattening at decision time.** Even when admissibility and trust are modeled explicitly, action choice need not recurse through an ever taller explicit tower; it can flatten to expected utility, interval dominance, or abstention at decision time.

We now make that higher-order split explicit. The second-order variable G_{step} means "this local contraction is admissible enough in the current structural regime", so the live second-order object is best read as a regime posterior $\pi(r \mid \text{local state})$ over hidden-context/admissibility regimes rather than as one free-floating confidence scalar. A third-order trust layer then records how much confidence to place in the current model of G_{step} for the present topology/context class. The point is not infinite regress for its own sake, but an explicit separation between object-level belief, admissibility belief, and trust in the admissibility model itself. Fourth-order and higher concerns are then handled, for the benchmark, as flattening or robustness questions rather than as an endlessly explicit tower. In the widened sweep, Hailfinder *context-complete* paths have mean third-order meta-trust 0.819 with reveal pressure 0.147, while Pathfinder has meta-trust 0.563 with reveal pressure 0.761 in the *context-missing* regime, 0.603/0.545 in the *alt-support-only* regime, 0.764/0.184 in the *context-pair* regime, and 0.798/0.159 in the *oracle-context-complete* regime. So the planner is not only asking "can I chain?" but also "how much do I trust my own admissibility estimate here, in this topology and context class?" In the current architecture these planner-facing second-/third-order quantities are intended as empirical control shadows of the semantic ProbHOL → belief-process → planner bridge, not as replacement semantics.

Table 11.4 also shows why the higher-order story is genuinely interesting rather than merely cautious. The current benchmark no longer supports the

Regime slice	Meta-trust	Reveal pressure	Best broad-query action	Best refined-query action
Hailfinder forecast chain, context-complete	0.819	0.147	`Regime-mixture` / exact chain, MAE 0.0000.	`Regime-mixture` / exact chain, MAE 0.0000.
Pathfinder hub-dominated, context-missing	0.590	0.769	`Anchored regime-mixture`, MAE 0.0090 (one-shot).	Context-complete / exact fallback, refined MAE 0.0000.
Pathfinder hub-dominated, alt-support-only	0.555	0.596	`Anchored regime-mixture`, MAE 0.2584.	Context-complete / exact fallback, refined MAE 0.0000.
Pathfinder loopy-dense, context-missing	0.548	0.757	`Anchored regime-mixture`, MAE 0.0489 (one-shot).	Context-complete / exact fallback, refined MAE 0.0000.
Pathfinder loopy-dense, oracle-context-complete	0.926	0.074	`Anchored regime-mixture`, MAE 0.2646.	`Rule-only`, refined MAE 0.0087.

Table 11.4: Compact topology-conditioned higher-order split in the widened benchmark sweep (v1 prototype; v2 restructured results use the state/action/eval pipeline described below). One-shot broad-query winners are often anchored regime-mixture actions, while refined-query winners still mostly come from explicit context completion or exact fallback.

simpler claim that "soft continuation wins the hardest broad-query cases". Instead, the better statement is now more precise. After anchoring the explicit regime-mixture continuation to the current broad-query branch, the strongest *one-shot* broad-query action is often the anchored regime-mixture policy, including the widened Pathfinder *context-missing* rows. Mechanistically, the hub-dominated slices look like unresolved broad queries whose mass is already concentrated in one main diagnostic direction, so a small anchored mixture smooths the remaining uncertainty without collapsing to one refined branch. The alt-support and loopy slices keep more hidden-context alternatives live, which is why structured regime averaging helps more than blunt `rule-only` continuation at the one-shot level. But the stricter *higher-order chain-composition* benchmark still leaves several multi-step *context-missing* slices to `rule-only` chain continuation. So the present result is not "the planner is solved", but "the broad/refined objective split is real, the best adventurous one-shot action depends on topology and hidden-context regime, and the best full-chain action can still be different". In this sense, topology is functioning as a compact summary of which kinds of independence, concentration, and branch-mixing assumptions are likely to work

in a local region of the graph.

To keep the setup fully auditable, we make the v2-clean protocol explicit here. The pipeline is a seven-script sequence, each consuming the output of the previous step: `wm_pln_v2c_build_master_suite.py` → `wm_pln_v2c_build_actions.py` → `wm_pln_v2c_build_eval.py` → `wm_pln_v2c_project_views.py` → `wm_pln_v2c_train_blind_mode` → `wm_pln_v2c_run_policies.py` → `wm_pln_v2c_evaluate.py`. All scripts live under `scripts/` and reuse the existing BN-inference backbone (`wm_pln_bn_common.py`), topology utilities (`wm_pln_second_order_common.py`), and motif catalogues (`wm_pln_pathfinder_motifs.py`, `wm_pln_hailfinder_motifs.py`). The v2-clean artifacts are written to `results/v2\_clean/`, with per-dataset state, action, eval, decision-oracle, decision-blind, policy, and results CSVs. The v2 prototype is preserved under `results/v2/` and the v1 prototype under `results/v1\_archive/` for provenance.

Five contract-level changes distinguish v2-clean from the v2 prototype: (1) `oracle_context_complete` is removed from the shared decision substrate—only blind-legitimate context variants appear; (2) reveal is a *real transition*: the controller pays the reveal cost and the follow-up action is evaluated on the target state's exact BN ground truth (with a penalty for EXTERNAL or missing targets); (3) path-family–level splits guarantee nonempty train/val/test partitions, and headline numbers report test-only results; (4) blind learning is *action-conditional*: XGBoost predicts per (state, action) pair, not per state, so different actions from the same state receive different `g_hat_action_error` predictions; (5) oracle and blind decision views are *physically separate* CSV projections—the blind view cannot contain oracle columns.

The policy set covers both oracle and blind lanes: `adaptive_oracle`/`adaptive_blind`, `force_rule_only`, `force_soft`, `force_reveal`, `force_fallback`, and `force_abstain`. Both lanes operate on the same 50 Pathfinder + 45 Hailfinder master state set (95 states total).

The scoring function used by the regime-mixture and chain benchmarks is

$$\text{score} = 1 - 2 \cdot \text{primary-error} - 0.08 \cdot \text{plan-cost} - \lambda_{\text{reveal}} \cdot \#\text{reveals},$$

with $\lambda_{\text{reveal}} = 0.05$ (a single balanced objective). Action-cost parameters are

$$\begin{aligned} \text{hard} &= 0.8 + 0.6 \cdot \text{edges}, \\ \text{guarded} &= 0.6 + 0.4 \cdot \text{edges}, \\ \text{fallback} &= 2.0 + 0.8 \cdot \text{edges}, \\ \text{reveal} &= 0.7 \cdot \#\text{missing-required-context-vars}. \end{aligned}$$

Pathfinder topology labels are assigned directly from local graph statistics in the benchmark scripts: *hub-dominated* if `Fault` is on-path or max node degree ≥ 50; *loopy-dense* if downstream multi-parent count ≥ 2 or mean local branching ≥ 4. The v2-clean Pathfinder suite has 10 path families (4 anchors + 6 discovered) expanded into 50 states via 5 blind-legitimate context variants (context-missing, hub-only, context-partial, alternating-partial, nonhub-only). The Hailfinder suite has 9 motif families (5 exact chains/forks, 2 long paths, 2 collider negatives) expanded into 45 states. The 95 total states span both topology classes (hub-dominated and loopy-dense Pathfinder slices; all Hailfinder motifs are tree-like or sparse-chain-like) and all blind-legitimate context completeness levels. The blind guard model is trained via XGBoost + isotonic calibration on topology-derived features (path length, max node degree, sigma count, context completeness) *plus* action-level features (action type, cost, reveal metadata) to predict action-conditional error from the blind decision view alone.

For these real-network cases, the bounded mode is currently *evaluation-certified* from the exact BN truthmaker rather than derived from a general theoremic bound calculus over arbitrary BNs. That limitation should be read as a strength of the presentation rather than a weakness: the semantic anchor stays exact, and the guarded layer is explicit about where it is theorem-backed, where it is certified by comparison against the truthmaker, and where it is merely operational.

Reproducibility block (v2-clean pipeline). *Scripts* (run in order):
`scripts/wm\_pln\_v2c\_build\_master\_suite.py`
`scripts/wm\_pln\_v2c\_build\_actions.py`
`scripts/wm\_pln\_v2c\_build\_eval.py`
`scripts/wm\_pln\_v2c\_project\_views.py`
`scripts/wm\_pln\_v2c\_train\_blind\_models.py`
`scripts/wm\_pln\_v2c\_run\_policies.py`
`scripts/wm\_pln\_v2c\_evaluate.py`

Results (test-only headlines). `results/v2\_clean/pathfinder\_states.csv` (50 states)
`results/v2\_clean/hailfinder\_states.csv` (45 states)
`results/v2\_clean/*\_decision\_oracle.csv` (all columns)
`results/v2\_clean/*\_decision\_blind.csv` (topology + action features)
`results/v2\_clean/*\_results.csv` (per-state evaluation)
`results/v2\_clean/*\_results\_summary\_test.csv` (test-only)
`results/v2\_clean/guard\_model/` (blind XGBoost artifacts)
`results/v2\_clean/benchmark\_contract\_audit.txt`

Lean theory. `Mettapedia/Logic/PLNProbGuardedAdmissibilityDemo.lean`
`Mettapedia/Logic/PLNProbGuardedAdmissibilityRegression.lean`

Prior versions. `results/v2/` (v2 prototype baseline)
`results/v1\_archive/` (v1 superseded results)

Related benchmark literature. The Pathfinder and Hailfinder networks originate from the bnlearn repository [38] and have long served as test cases for exact BN inference algorithms. Standard BN inference benchmarks (e.g. the UAI approximate inference competitions, ProbModelS) evaluate *inference algorithm quality*—how accurately a solver recovers posterior marginals. Our benchmark evaluates a different object: *inference control quality*—given an inference engine that can already compute exact posteriors, how well does a higher-order controller decide when to continue, soften, reveal context, fall back, or abstain? This control-level evaluation is closer in spirit to active learning and value-of-information frameworks than to solver benchmarks. The blind guard model connects to conformal prediction as the distribution-free bridge from learned topology features to certified admissibility estimates.

11.9.2 Hetionet / Rephetio: the next flagship benchmark

At this point the project has three ingredients in place. First, the Pathfinder/Hailfinder lane already gives us a real small-to-medium benchmark story for

exact contraction, guarded continuation, reveal as value of information, multi-support revision, and higher-order chain composition. Second, Chapter 11 now exposes the semantic ProbHOL → belief-process → planner-shadow bridge as the canonical waist of the architecture. Third, we now have a concrete large-scale benchmark candidate in hand: the Hetionet / Rephetio drug-repurposing stack.[20, 19] The motive for moving there is simple. Hetionet is large, typed, path-rich, and explanation-oriented, so it is a better stress test of higher-order chaining than another toy relational dataset. It is also already rich enough that there is no single clean leaderboard; instead there is a cluster of benchmark traditions with different task definitions, path semantics, explanation targets, and reproducibility caveats.[29, 23, 18, 33, 6, 34, 26]

The dataset itself also needs to be described clearly, because the benchmark object is not one flat table. Hetionet is the heterogeneous biomedical graph; Rephetio is the benchmark-facing drug-repurposing layer built on top of it. In the current public browser-table bundle we have a typed graph with 47,031 nodes, 2,250,197 relationships, and 24 edge types, together with benchmark-facing tables for 1,538 compounds, 136 diseases, 1,206 metapaths, 1,000 compound-conditioned prediction tables, and 136 disease-conditioned prediction tables.[20, 19] This is exactly the shape we want: small enough to inspect, large enough to matter, and rich enough that typed multi-hop support families are not an afterthought.

The flagship query for this benchmark is

$$\texttt{treats}(\textit{compound}, \textit{disease}).$$

The methods/motives follow directly from the work already in hand. Pathfinder and Hailfinder taught us that the interesting control problem is not merely whether a local step is exact, but how to act when context is incomplete, several support families are live, and reveal can change the target from a broad unresolved query to a refined conditional one. Hetionet / Rephetio is where that question becomes typed and large-scale. For example, metapaths such as `CbGaD` (Compound–binds–Gene–associates–Disease), `CrCtD` (Compound–resembles–Compound–treats–Disease), and `CiPCiCtD` (Compound–includes–Pharmacologic Class–includes–Compound–treats–Disease) already look like natural higher-order support families rather than accidental path fragments.

The planned protocol therefore proceeds in four stages. First, *one-shot regime-mixture continuation* treats typed metapath/support families as second-order regimes and compares `rule-only`, explicit regime-mixture continuation, reveal, and fallback on the broad `treats(c,d)` task. Second,

PLN-style multi-support revision combines several live support families for the same query with explicit overlap penalties or residual ignorance terms, rather than pretending free independence. Third, *reveal as value of information* asks when inspecting an additional mechanism family or context slice improves refined fidelity enough to justify its cost—instead of silently changing the target. Fourth, *higher-order chain composition* evaluates whether the guarded continuation machinery survives over interpretable compound–gene–disease and compound–compound–disease chains, not just one-shot scoring. At the semantic level, the split is the same as in the current benchmark lane:

- **1st order**: support for `treats(c,d)`;
- **2nd order**: uncertainty over metapath/context/support regime;
- **3rd order**: trust in the regime estimator;
- **4th and beyond**: flattened to expected utility, interval dominance, or abstention at action time.

This means the next benchmark consumer is already aligned with the Chapter 11 architecture:

truth → hierarchical semantic probability → belief tracking → planner shadow.

What we are *not* claiming yet is equally important. This section does not report finished Hetionet results, and it does not treat the Rephetio browser tables as a semantic oracle by themselves. Rather, it states the next flagship benchmark plan now that the semantic ProbHOL → belief → planner waist is finally in place: the benchmark-side regime and trust fields should become principled projections of that bridge, not one more layer of bespoke scores.

11.9.3 GWAS locus-to-gene: replication-first benchmark lane

The genomic lane should be staged with the same discipline. The relevant benchmark object is not merely "some SNP–gene scores," but a full locus-to-gene pipeline with at least three separable layers:

1. the *scientific task* (candidate gene ranking for a variant or locus, and later variant → gene → mechanism explanation);
2. the *scoring semantics* (the current ProbLog/LPAD baseline versus a WM-PLN evidence-carrying strengthening); and

3. the *runtime lane* (native repo/cplint path first, Janus/PeTTa parity second, any later harnesses only after that).

That separation matters because the public `hypothesis-generation-demo` repository already contains its own benchmark-facing ProbLog family: a learned-rule LPAD model in `pl/pl\_rules.pl`, a native cplint parameter-learning/evaluation driver in `pl/param\_learn.pl`, and the current fold/index artifacts under `pl/models/`. So the first obligation is not to replace that model with a new one. It is to replicate their methodology with *their* ProbLog stack as faithfully as the surviving data/runtime artifacts allow.

The resulting protocol should be deliberately staged:

1. **Exact-replication salvage pass.** Recover the native cplint environment and, if possible, the original compiled KB snapshot expected by `pl/load\_kbs.pl`. If this succeeds, rerun `pl/param\_learn.pl` directly and compare only against the original reported ProbLog/LPAD outputs.

2. **Methodology replication on similar public data.** If the exact snapshot cannot be recovered, preserve the same methodological shape (ProbLog/LPAD rule family, fold protocol, and evaluation discipline) but drive it from the closest public-source reconstruction. This should be labeled strictly as a methodology replication, not as an exact rerun.

3. **WM-PLN strengthening parity.** Only after the ProbLog baseline is understood should we hold the candidate/evaluation set fixed and replace Boolean mechanism activations with real WM evidence objects on a PeTTa/Janus path. The scientific question here is then clean: what improves when the same genomic task is given a stronger evidence semantics?

4. **Interesting benchmark extensions.** After parity, promote the first serious external claim to a fixed-candidate Open Targets locus-to-gene lane, and keep mechanism chains / hypothesis generation as a distinct explanation family with its own evaluation protocol.

Two guardrails are especially important. First, we should not score a fresh replacement model on a custom harness and call that "their baseline." The baseline must be *their* methodology, driven through their own ProbLog family as directly as possible. Second, the first benchmark-grade claim should be ranking-only: candidate set fixed, locus-wise metrics explicit, and mechanistic explanations evaluated separately rather than smuggled into the same headline number.

Completed: methodology replication on local data (Lane A, chr16 pilot). Step 2 of the above protocol has been executed. The native cplint/liftcover parameter-learning pipeline was run end-to-end on SWI-Prolog 10.0.1 with locally-generated Prolog fact files (176 eQTL, 22 ABC, 160 regulatory facts at 50 kb window) drawn from the same public-source caches used by the genomic hypothesis note. Five-fold cross-validation on the 823 labeled examples ($69^{+}/754^{-}$; per-fold test sets sum to 872 test instances) yields:

Method	AUC-ROC	AUC-PR
Poster baseline (reported)	0.82	—
Python reconstruction (frozen weights)	0.713 ± 0.056	0.218 ± 0.055
LIFT replication (re-learned weights)	0.677 ± 0.105	0.238 ± 0.090

LIFT re-learns mechanism weights via L-BFGS: fold 0 gives activity-by-contact 0.221 (strongest), eQTL 0.116, regulatory 0.086. The weight ordering differs from the poster (0.34/0.018/0.021), reflecting the sparser local data. The higher fold variance ($\sigma = 0.105$) comes from uneven mechanism coverage: only 157/533 model pairs have ≥ 1 mechanism fact. The 0.68 vs. 0.82 gap against the poster is expected from data-source differences and is honestly labeled. Artifacts live under `experiments/` with full provenance (`experiments/facts/provenance.json`, `experiments/run\_manifest.json`).

Completed: WM-PLN implementation-level parity (step 2.5). Per-example ProbLog probabilities were exported from liftcover's `prob_lift` inference engine on SWI-Prolog 10.0.1 (the same engine that produced the LIFT baseline, querying the same `experiments/facts/*.pl`) and compared against WM-PLN scores from the PeTTa MeTTa runtime via the Janus FFI bridge to SWI-Prolog (using `pln-fuzzy-or` from the PLN inference library). On all 872 test instances across 5 folds (the 823 examples partitioned into per-fold test sets):

- **P1A** (fixed poster weights 0.34/0.018/0.021): max $|\Delta| = 5.6 \times 10^{-17}$.
- **P1B** (LIFT fold-specific learned weights): max $|\Delta| = 1.1 \times 10^{-16}$.

Both are at IEEE 754 machine-epsilon level; AUC-ROC and average precision (computed via scikit-learn) are identical to all reported digits.

This is *score-level parity on the reconstructed 3-rule lane*—it confirms the Lean-proven identity at the implementation level within that scope, but is not a claim of full end-to-end equivalence with the original authors' hidden pipeline. Artifacts: `experiments/wm\_problog\_parity/`.

S1: first graded-evidence WM variant (step 3 pilot). A preliminary strengthening lane replaces the fixed Boolean eQTL and ABC channels with graded WM evidence objects (n^+/n^- counts from GTEx and ABC caches, converted to strengths via `evidence-toStrength` from the PLN inference library). Regulatory remains at the legacy fixed weight 0.34. All scoring runs through PeTTa/Janus; Python orchestrates only.

On the same 872 test instances (scikit-learn metrics), mean AUC-ROC is 0.663 ± 0.086 versus the P1B baseline's 0.677 ± 0.094; mean average precision is 0.187 ± 0.017 versus 0.222 ± 0.066. The underperformance is expected: P1B uses LIFT-learned weights optimized on training data, while S1 substitutes raw evidence ratios that are uncalibrated for the gene-relevance task. The result indicates not a failure of WM expressivity, but the need for calibrated evidence-to-strength mappings, explicit treatment of missing versus negative evidence, and dependence-aware fusion beyond the ProbLog quotient. Artifacts: `experiments/wm\_strengthening\_clean/`.

S2a/S2b: controlled ablation ladder (step 3 continued). A follow-up ablation ladder applies single-variable interventions to the S1 baseline: S2a substitutes a Jeffreys posterior mean $(n^+ + 0.5)/(n^+ + n^- + 1.0)$ for the raw MLE (near no-op on this dataset due to uniform evidence totals), and S2b adds per-channel isotonic calibration on training folds. Neither variant matches P1B's AP (0.222); S2b reaches 0.193 after fixing a fold-leakage issue identified by audit. Null baselines (channel count, single-channel scores) confirm that the 3-rule lane without the graded distance proxy has limited discriminative headroom on this chr16 pilot. Details in the companion genomic hypothesis note (§5.5). Artifacts: `experiments/wm\_ablation/`.

S3–S5: subfact multideduction and data-driven tissue weights (step 3 concluded, Operational). The ablation ladder closes with three families that decompose the coarse 3-channel evidence objects into *subfact* evidence atoms—one per GTEx tissue or ABC biosample—and fuse them via the PLN evidence quantale.

S3A (unweighted subfacts) Each tissue or biosample is an independent evidence atom with unit weight; per-channel Jeffreys posterior mean, noisy-OR fusion. Mathematically equivalent to S2a (parity check passes to $< 10^{-6}$).

S3B (tissue-weighted subfacts) Per-tissue weights regraduate evidence via `evidence-power`. Three weight variants are evaluated: *expression*

(gold-standard obesity gene enrichment in GTEx v8 median TPM; 44 seed genes from `gwas\_gold\_standards.191108.tsv`, high/medium confidence, independently validated via drug targets, CRISPR, and reporter assays); *assay* (genome-wide eQTL discovery rate per tissue from the full GTEx v8 tar, all chromosomes—not benchmark-specific); *combined* (geometric mean, normalized to mean ≈ 1).

S4 (tested-vs-missing) Only obesity-relevant tissues count as evidence; irrelevant tissues are treated as *missing* (not negative). Relevance defined by weight ≥ 1.0 in the chosen weight variant.

S5 (context-split rule families) The 3 coarse channels are split into 6 typed mechanism families (`eqtl_high`, `eqtl_medium`, `eqtl_low`, `abc_high`, `abc_other`, `regulatory`), each with its own evidence counts and Jeffreys strength. Fusion via noisy-OR across active families.

Weight derivation guardrails. No chr16 pilot labels are used. Expression weights derive from external gold-standard genes and a genome-wide expression matrix; assay weights from genome-wide eQTL line counts across all chromosomes. ABC biosample classification uses systematic keyword matching on Nasser et al. (2021) biosample names (131 biosamples: 15 metabolic, 5 cardiovascular, 61 immune, 25 epithelial, 6 neural, 7 stem cell, 12 other); actual category counts replace the earlier guessed denominators. This classification is anchored to the published naming convention but is not joined to a formal cell ontology—an acknowledged limitation.

Lane	Weights	AP	ROC	MRR	P@5
P1B (LIFT learned)†	supervised	0.222	0.677	0.521	0.191
S3A (unweighted = S2a)	—	0.187	0.663	0.517	0.191
S3B	expression	0.196	0.664	0.517	0.191
S3B	assay	0.195	0.661	0.536	0.194
S3B	combined	0.195	0.663	0.518	0.191
S4	expression	0.184	0.633	0.520	0.190
S4	assay	0.163	0.639	0.500	0.195
S4	combined	0.187	0.653	0.534	0.195
S5	expression	0.185	0.655	0.545	0.194
S5	assay	0.192	0.658	0.551	0.194
S5	combined	0.190	0.654	0.551	0.194

†P1B uses fold-specific supervised weight learning (LIFT/L-BFGS on training labels); all other lanes are entirely unsupervised. Both the original driver (`pl/param\_learn.pl:97`)

and our local replication (`experiments/param\_learn\_local.pl:111`) assert all model facts before calling `induce_par_lift([train])`. Whether this constitutes leakage was verified empirically: a clean driver (`experiments/param\_learn\_clean.pl`) that asserts *only* training-fold model facts during learning produces identical weights and AUC-ROC (0.677), with AUC-PR slightly higher (0.238 vs 0.222). LIFT's fold-based selection already isolates train from test for L-BFGS optimization regardless of what else is in the Prolog database.

Findings. (i) Data-driven tissue weighting modestly improves over the unweighted WM baseline: S3B (expression) achieves AP 0.196 versus S3A's 0.187 (+4.5% relative), without using any labels. (ii) Context-split rule families (S5) improve per-SNP ranking (MRR 0.551 for assay/combined), the strongest ranking result across all lanes including the supervised P1B (0.521). (iii) Tested-vs-missing semantics (S4) underperform when assay weights define relevance, because assay weights measure tissue discovery power rather than biological relevance. (iv) The best unsupervised AP (0.196) does not match P1B (0.222). On a 41-SNP pilot with ~ 55 positive training examples per fold, neither the gap nor P1B's absolute performance is conclusive. (v) The pilot validates the WM-PLN methodology—reproducible weight derivation, structured evidence decomposition, and proper data hygiene—and justifies moving to a full multi-chromosome GWAS benchmark where the approaches can be evaluated at scale.

Limitations. This ablation is a pilot cleanup, not the conceptual upgrade to richer evidence objects. It replaces LLM-curated weights with reproducible external data and fixes guessed denominators, but the underlying representation remains hit counts fused via noisy-OR. Effect sizes, posterior uncertainties, and dependence-aware fusion are deferred to the full GWAS benchmark. Artifacts: `experiments/wm\_subfact\_multideduction/`, `experiments/wm\_tested\_missing/`, `experiments/wm\_context\_split\_rules/`, `experiments/data\_driven\_ablation\_manifest.json`.

Clean benchmark: SNP-disjoint splits (supersedes above). Post-hoc audit of the chr16 pilot revealed that the original 5-fold splits have severe data contamination: the 823 labeled examples contain only 533 unique (gene, SNP) pairs (290 duplicates across folds), 4 pairs carry conflicting labels, and 42–67% of test pairs per fold also appear in training under different indices. All supervised results above (P1B, S1–S5) were evaluated on these contaminated splits. While the unsupervised scores themselves are fold-independent, the per-fold metrics are affected.

We rebuild from scratch: 529 clean examples ($39^{+}/490^{-}$) after deduplication and conflict removal, partitioned into 5 SNP-disjoint folds via greedy round-robin balancing (7–8 positives per fold, zero SNP overlap between any fold's train and test). SNP-disjoint splitting is the strictest isolation level: all gene–SNP pairs sharing a variant are confined to the same fold, preventing any variant-context leakage.

In addition to the 3-rule P1B baseline, we evaluate **P1B++**: a 9-rule LPAD where the single `eqtl_association` channel is replaced by 7 tissue-typed eQTL channels derived from the GTEx Consortium's SMTS organ-system grouping (official v8 sample annotations, not hand-curated). SMTS categories with ≥ 3 tissues receive their own rule; smaller categories merge into `eqtl_other`. The resulting rules are: `eqtl_cardiovascular` (5 tissues), `eqtl_digestive` (9), `eqtl_endocrine` (3), `eqtl_nervous_system` (13), `eqtl_breast_salivary_spleen` (3), `eqtl_reproductive` (5), `eqtl_other` (14 tissues from 6 merged categories), plus `activity_by_contact` and `regulatory_effect`. The merge threshold was frozen before seeing any results.

Lane	Weights	AUC-PR	AUC-ROC
P1B (3 rules)†	supervised	0.216 ± 0.150	0.653 ± 0.132
P1B++ (9 rules)†	supervised	0.243 ± 0.156	0.655 ± 0.131
S3A (unweighted = S2a)	—	0.208 ± 0.150	0.657 ± 0.131
S3B	combined	0.212 ± 0.157	0.660 ± 0.134
S3B	expression	0.196 ± 0.119	0.657 ± 0.131
S3B	assay	0.194 ± 0.129	0.659 ± 0.135

†LIFT/L-BFGS on training folds only (strict isolation: only training-fold model facts are asserted during learning). P1B++ AIC = 510 vs. P1B AIC = 500 (ΔAIC = +10), so the simpler model is preferred. Probabilities are actual `prob_lift/3` outputs (liftcover distribution semantics), not Python-rescored noisy-OR.

Findings on clean splits. (i) All methods cluster tightly: AUC-ROC 0.653–0.660, AP 0.194–0.243. With 39 positives across 5 folds (~ 8 per fold), no pairwise difference is statistically significant. (ii) P1B++ achieves the best AP (0.243), a 12% relative improvement over P1B (0.216), but AIC penalizes the 6 extra parameters (ΔAIC = +10). Learned weights reveal that `activity_by_contact` and `regulatory_effect` consistently dominate (weights 0.09–0.18), while tissue-typed eQTL channels are sparse: `eqtl_digestive` learns near-zero weight in 4/5 folds; `eqtl_reproductive` is the only eQTL category with consistently non-negligible weight (0.04–0.06). (iii) Unsupervised S3A/S3B are competitive with supervised P1B, consistent with the earlier finding that the 3-rule lane has limited discriminative

headroom on this pilot. (iv) Fold 0 is anomalously difficult for all methods (AUC-ROC ≈ 0.43), accounting for most of the variance. (v) Per-SNP gene rankings are nearly identical across all methods: only 2/39 positive genes change rank between P1B and P1B++.

Conclusion. The chr16 pilot is too thin (39 positives, 41 SNPs) to distinguish the methods. The clean benchmark confirms that the WM-PLN methodology is sound—reproducible tissue typing, proper data isolation, actual distribution-semantics probabilities—and makes a credible but statistically inconclusive case for richer evidence decomposition. The decisive test requires a multi-chromosome GWAS benchmark with hundreds of loci. Artifacts: `experiments/clean\_benchmark/`.

Full-GWAS benchmark: Lane 1, Platform 25.12 (step 4). The multi-chromosome benchmark recommended by the chr16 pilot's conclusion is now complete. We freeze a single release of the Open Targets Platform (25.12, fetched 2026-03-14, GRCh38/Ensembl 114) and evaluate on the full GWAS credible-set catalogue—not a single chromosome.

Data and gold standard. The registry comprises all 1,440,589 GWAS-type credible sets from Platform 25.12 (200 partitions joined with study metadata). Gold standards come from the `genetics-gold-standards` repository: `gwas\_gold\_standards.191108.json` (2,435 entries) and `otg\_gs\_230511.json` (1,279 entries), filtered to High + Medium confidence (2,749 after deduplication).

Matching uses a two-tier protocol:

Tier 1 (exact study match) Gold-standard `otg_id` + `variant_id` matches credible-set `studyId` + `variantId` exactly.

Tier 2 (variant + disease overlap) Gold-standard `variant_id` matches any credible-set `variantId`, and the gold-standard disease ontology overlaps the study's mapped EFO trait terms.

Result: 875 Tier 1 + 2,018 Tier 2 = 2,893 matched entries covering 2,802 unique loci across 23 chromosomes.

Labeling follows the Mountjoy protocol: for each matched locus, positive genes are those from the gold standard that appear in L2G's candidate set; negatives are all remaining L2G candidate genes. 34 loci have two positive genes. Final labeled dataset: 4,939 rows ($2{,}354^{+}$, $2{,}585^{-}$).

Three evaluation subsets are defined with increasing stringency:

Subset	Loci	Definition
All matched	2,802	Locus has a gold-standard match
Candidate-covered	2,320	≥ 1 positive in L2G candidate set
Ranking-evaluable	1,097	Candidate-covered **and** ≥ 2 candidates

The ranking-evaluable subset is the primary evaluation set: nontrivial gene ranking is required, avoiding inflation from single-candidate loci. Five-fold cross-validation uses studyLocusId-disjoint splits ($\sim$560 loci per fold).

Model A: L2G baseline (no WM). Model A is the Open Targets L2G machine learning score (`l2g_score` from the `l2g\_prediction` dataset). L2G is a supervised model trained on the same gold standards plus features including distance, chromatin, VEP, colocalisation, and enhancer annotations. It uses none of the WM-PLN evidence algebra. This is a pure external baseline, not a WM model.

Model B: WM-unsupervised (noisy-OR over three channels). Model B is the first World Model approach on this benchmark. Three independent evidence channels are extracted from Platform 25.12 and fused via noisy-OR without any supervised parameter learning:

Colocalisation Join chain: `colocalisation.rightStudyLocusId` → `credible_set.studyLocusId` → `study.geneId` (QTL gene). Score: max H4 posterior across eQTL/pQTL/sQTL colocalisation tests. Coverage: 20,812 (locus, gene) pairs at 1,610 loci (57%).

Enhancer-to-gene (E2G) For each locus, extract positions of all 95% credible-set variants from the nested `locus` column. Check whether any falls within an E2G enhancer–gene interval (ABC, CRISPRi, etc.). Score: max E2G score across overlapping intervals. Coverage: 5,117 pairs at 2,093 loci (75%).

Variant effect (VEP) Read `variant.variantEffect` for each lead variant; extract (gene, normalised score) from VEP and GERP annotations. Coverage: 831 pairs at 515 loci (18%).

Fusion: $P(\text{relevant}) = 1 - \prod_i (1 - \text{channel}_i)$, channel scores capped at 0.99. Missing channels contribute 0. Overall locus coverage: 2,447/2,802 (87%).

Results.

Subset	Model	AUC-ROC	AUC-PR	MRR	P@1
Ranking-eval. (1,097)	A (L2G)	0.823 ± 0.006	0.778 ± 0.012	0.880 ± 0.006	0.779 ± 0.011
	B (WM)	0.658 ± 0.020	0.593 ± 0.038	0.821 ± 0.018	0.679 ± 0.030
All matched (2,802)	A (L2G)	0.866 ± 0.010	0.862 ± 0.011	0.943 ± 0.005	0.895 ± 0.009
	B (WM)	0.619 ± 0.020	0.625 ± 0.021	0.915 ± 0.008	0.848 ± 0.013

Tier (ranking-eval.)	Loci	Model	AUC-ROC	P@1
Tier 1 (exact study)	345	A	0.748 ± 0.045	0.639 ± 0.040
		B	0.621 ± 0.030	0.659 ± 0.061
Tier 2 (variant+disease)	752	A	0.859 ± 0.015	0.843 ± 0.031
		B	0.680 ± 0.016	0.687 ± 0.024

Head-to-head on the 1,097 ranking-evaluable loci: Model A wins at 225 loci (20.5%), Model B wins at 116 loci (10.6%), ties at 756 loci (68.9%).

Channel contribution.

Channel	Rows	Row %	Loci	Loci %
Colocalisation	2,298	46.5%	1,610	57.5%
E2G	2,996	60.7%	2,093	74.7%
VEP	515	10.4%	515	18.4%
Any channel	3,763	76.2%	2,447	87.3%

Findings. (i) The benchmark is now large enough to distinguish the models: 1,097 ranking-evaluable loci across 23 chromosomes, with $\Delta\text{ROC} = 0.165$ and narrow fold variance. (ii) Model A (L2G) is a strong supervised baseline; its advantage is expected because it was trained on a superset of the same evidence features that Model B uses unsupervised. (iii) Model B's 87% locus coverage shows the WM evidence channels are broad; the bottleneck is scoring quality, not evidence availability. (iv) E2G is the strongest single channel (75% locus coverage), consistent with the chr16 pilot's finding that enhancer–gene links are the most informative evidence family. (v) The 116 loci where Model B outranks L2G are the WM value-add cases; characterizing them is the next diagnostic step. (vi) Tier 1 P@1 shows a notable inversion: Model B (0.659) slightly outperforms Model A (0.639) on the hardest gold-standard tier. This suggests that raw colocalisation evidence may carry information that L2G's supervised weighting discounts. (vii) The gap between A and B confirms the chr16 pilot's prediction: if WM-PLN is

to compete on Lane 1, a supervised meta-model over WM evidence features is needed.

Limitations of v1. Model B uses the same evidence *types* as L2G but in raw form: no learned weighting, no distance features, no tissue-specific calibration, no tested-vs-missing semantics. The comparison is therefore asymmetric—L2G has strictly more information. This motivates the next round: a richer WM evidence table and a fair information-controlled decomposition.

WM-v2: enriched evidence table (7 feature families). To close the information gap, we built a 27-feature evidence table (*WM-v2*) from the same Platform 25.12 data, organized into seven frozen families:

Distance $\log(|\text{distanceFromTss}| + 1)$ and $\log(|\text{distanceFromFootprint}| + 1)$ per (variant, gene), extracted from `variant.transcriptConsequences`. Coverage: 100% of labeled rows.

Neighbourhood Gene counts within 50/100/250/500 kb, and neighbourhood-relative TSS advantage ($\log(\min \text{dist} + 1) - \log(\text{this_gene_dist} + 1)$), following the L2G distance-normalization pattern.

Credible-set confidence Credible-set $\log_{10}$ Bayes factor, purity ($\overline{R^2}$), sample size, finemapping method (SuSiE flag), and 95% CS variant count. Coverage: 44% have BF (SuSiE-only feature).

Colocalisation (disaggregated) Separate max H4 per QTL type (eQTL: 37% of rows, pQTL: 14%, sQTL: 14%), plus QTL-type count.

E2G (disaggregated) Max score, interval count, and datasource count per (locus, gene). Coverage: 61%.

VEP (disaggregated) Max normalised score from `variantEffect` and most severe consequence score from `transcriptConsequences`. Coverage: 10% (variantEffect), 100% (consequenceScore).

Tested-vs-missing Binary flags: `has_coloc`, `has_e2g`, `has_vep`, `has_distance`, `tested_no_coloc`, `tested_no_e2g`. The "tested but absent" flags distinguish genuine negative evidence from missing data.

Feature definitions were frozen before any model was trained (manifest: `results/model\_wm\_v2/feature\_manifest.json`).

Evaluation protocol: variant-disjoint folds. An initial evaluation used studyLocusId-disjoint folds, but external review revealed that 260 of 917 unique lead variants spanned multiple folds, enabling a trivial memorization baseline (`variantId`+`geneId` lookup from training labels) to reach AUC-ROC 0.952—higher than any model. All headline results below therefore use **variant-disjoint** 5-fold CV (`GroupKFold` by `variantId`; no variant appears in more than one fold). Under this split the memorization baseline collapses to AUC-ROC 0.500 (zero test rows have a training match), confirming split integrity. Tier 1 (exact study+variant gold-standard match, 345 loci, 284 variants) is the primary evaluation track; Tier 2 (variant+disease overlap, 752 loci, 137 variants) is a secondary robustness check. All supervised models use fold-local imputation and scaling.

Information decomposition (variant-disjoint).

Feature set	Model	AUC-ROC	AUC-PR	MRR	P@1
V1 feature set (3 channels)					
	B (noisy-OR)	0.658 ± 0.062	0.618 ± 0.100	0.818 ± 0.066	0.673 ± 0.104
	Logistic (3ch)	0.646 ± 0.048	0.584 ± 0.075	0.780 ± 0.056	0.600 ± 0.093
V2 feature set (27 features, 7 families)					
	Nearest gene	0.583 ± 0.073	0.548 ± 0.080	0.764 ± 0.069	0.587 ± 0.111
	Distance-only LR	0.658 ± 0.074	0.562 ± 0.088	0.801 ± 0.059	0.648 ± 0.091
	Logistic (27 feat.)	0.762 ± 0.048	0.674 ± 0.094	0.853 ± 0.044	0.723 ± 0.085
	XGBoost (27 feat.)	0.792 ± 0.018	0.726 ± 0.052	0.873 ± 0.031	0.766 ± 0.047
Interpretable WM (7 family summaries)					
	Additive WM (L1 LR)	0.681 ± 0.051	0.632 ± 0.089	0.796 ± 0.043	0.632 ± 0.069
L2G + WM stackers					
	LR stacker	0.818 ± 0.031	0.760 ± 0.060	0.866 ± 0.018	0.754 ± 0.030
	XGB stacker	$\mathbf{0.830 \pm 0.022}$	$\mathbf{0.772 \pm 0.058}$	0.863 ± 0.019	0.748 ± 0.036
Baselines					
	A (L2G)	0.824 ± 0.038	0.784 ± 0.058	0.879 ± 0.011	0.777 ± 0.015
	Memorization	0.500 ± 0.000	0.699 ± 0.013	0.684 ± 0.013	0.423 ± 0.018

Tier	Model	AUC-ROC	AUC-PR	MRR	P@1
Tier 1 (345 loci, primary)					
	A (L2G)	0.756 ± 0.052	0.675 ± 0.075	0.805 ± 0.030	0.645 ± 0.057
	XGBoost (27 feat.)	0.738 ± 0.034	0.637 ± 0.040	0.823 ± 0.019	0.668 ± 0.028
	XGB stacker	0.750 ± 0.028	0.676 ± 0.040	0.792 ± 0.022	0.619 ± 0.040
Tier 2 (752 loci, secondary)					
	A (L2G)	0.861 ± 0.054	0.834 ± 0.061	0.914 ± 0.020	0.841 ± 0.038
	XGBoost (27 feat.)	0.813 ± 0.039	0.758 ± 0.075	0.897 ± 0.047	0.814 ± 0.071
	XGB stacker	0.868 ± 0.033	0.817 ± 0.073	0.897 ± 0.032	0.811 ± 0.061

The decomposition under clean generalization:

1. **Richer information helps substantially.** V1 logistic (0.646) → V2 logistic (0.762) = +11.6 points. Distance, neighbourhood, credible-set confidence, disaggregated channels, and tested-vs-missing flags close most of the gap with L2G.

2. **Flexible learning helps further.** V2 logistic (0.762) → V2 XGBoost (0.792) = +3.0 points. Under fair variant-disjoint evaluation, the nonlinear gain is real but more modest than under the contaminated split.

3. **L2G still leads.** XGBoost on WM-v2 (0.792) vs. L2G (0.824) = −3.2 points. The remaining gap likely reflects features L2G has (Hi-C, promoter-capture, tissue-specific calibration) that WM-v2 does not yet include.

4. **Stacking adds modest complementarity.** XGB stacker (0.830) vs. L2G alone (0.824) = +0.6 points. WM-v2 features carry some information that L2G misses, but the complementary signal is small.

5. **Unsupervised WM is stable.** Noisy-OR (0.658) is unchanged from the old split—unsupervised models are unaffected by fold assignment.

Provenance note. An earlier round of results used studyLocusId-disjoint folds, under which XGBoost on WM-v2 appeared to reach AUC-ROC 0.878, exceeding L2G (0.823). That headline was retracted after the variant-repetition issue was identified: the memorization baseline confirmed the inflation. All results reported above use variant-disjoint folds with fold-local preprocessing. The old results are preserved as diagnostic artifacts under `results/MANIFEST\_provisional.json`.

Interpretation and next steps. The WM-v2 evidence table substantially narrows the gap to L2G using only publicly extractable evidence from Platform 25.12. An open, interpretable evidence table reaching AUC-ROC 0.792—compared to L2G's 0.824—is a meaningful result: it demonstrates that structured WM evidence captures most of the discriminative signal in a transparent, decomposable form.

Three directions for the next round: (a) *Close the remaining gap*: add Hi-C / promoter-capture, tissue-specific eQTL disaggregation, and literature co-mention as additional frozen families. (b) *Improve the additive model*: intermediate-granularity summaries (15–20 features) may recover both interpretability and performance relative to the full 27-feature set. (c) *Characterize rescue loci*: inspect cases where WM-v2 outranks L2G to determine whether the complementary signal is biologically coherent (e.g., mechanism-specific rescue). Artifacts: `experiments/full\_gwas/`, `results\_variant\_disjoint/`.

Slice	Best broad one-shot policy	Broad MAE	Best full-chain policy	Main takeaway
Hailfinder forecast chain, context-complete	`Explicit regime-mixture`	0.0000	`Rule-only` chain (exact)	In the clean lane the higher-order controller collapses back to the exact fragment.
Pathfinder hub-dominated, context-missing	`Explicit regime-mixture`	0.0090	`Rule-only` chain	Anchored regime averaging helps at the one-shot level, but full multi-step composition still prefers blunt continuation.
Pathfinder loopy-dense, context-missing	`Explicit regime-mixture`	0.0489	`Rule-only` chain	One-shot smoothing helps, but loopy full-chain continuation still resists a universal adventurous policy.
Pathfinder multi-support revision	`Base-plus-support revision`	0.0197 / 0.0854	–	Combining hub/non-hub/pair supports is the clearest PLN-style positive result on the widened suite (hub-dominated / loopy-dense).
Reveal as value of information on Pathfinder context-missing	Continue for the broad query	$\Delta_{\text{refined}} = +0.715$ / $+0.067$	Reveal only for the refined task	Reveal improves refined fidelity, but usually solves the wrong task for the original broad query and incurs large broad-query regret.

Table 11.5: Stabilized higher-order benchmark takeaways (v1 prototype; v2 restructured results use the state/action/eval pipeline) after explicit regime-mixture continuation, PLN-style multi-support revision, reveal-as-value-of-information evaluation, and higher-order chain composition. The two numbers in the multi-support and VOI rows correspond to the hub-dominated and loopy-dense Pathfinder slices respectively.

Slice	Rows	Groups	Hard	Blocked	ϵ-exact	ECE iso→conf	Brier iso→conf
Hailfinder coverage	62	62	0.629	0.371	0.758	0.205→0.205	0.188→0.188
Pathfinder coverage	145	80	0.448	0.552	0.634	0.074→0.074	0.064→0.064
Cross H→P guard transfer	145	–	0.448	0.552	0.634	0.323→0.190	0.307→0.237
Cross P→H guard transfer	62	–	0.629	0.371	0.758	0.119→0.110	0.118→0.117

Table 11.6: Motif-coverage and calibrated guard-transfer summary (v1 prototype; v2 uses the state/action/eval pipeline with physical column projection). "Hard" and "Blocked" are structural admissibility rates; ϵ-exact uses $\epsilon = 0.02$. The conformal guard is operational confidence shaping, not posterior semantics.

Paper	Role for this project	What it contributes	Main caution
Rephetio / Hetionet [20]	Canonical benchmark definition	Defines the graph, the `Compound-treats-Disease` target, and the metapath / DWPC setup.	Original setup is not itself a higher-order control benchmark.
PoLo-style logical multi-hop [29]	Closest chaining comparator	Shows that typed logical multi-hop reasoning on biomedical KGs is a real comparison class.	Not the same semantic / reveal / regime split as our WM lane.
XG4Repo [23]	Explainable path-completion comparator	Direct path-based Hetionet drug-repurposing benchmark with rule and RL comparisons.	Reported Hetionet test slices are small, so hard SOTA claims are shaky.
Hetnet connectivity search [18]	Unsupervised path-family discovery	Finds degree-adjusted important path families without a supervised gold label.	Analysis tool, not a full continuation controller.
REx [33]	Explanation-ranking comparator	Benchmarks explanatory path quality using Hetionet-style drug repurposing hypotheses.	Optimizes explanation quality rather than the broad/refined control tradeoff directly.
Bonner et al. [6]	Benchmark-sensitivity warning	Shows that ranking changes materially with split, hyperparameters, and seed.	Raw score-chasing without protocol control is misleading.
Reviews [34, 26]	Broader field orientation	Situate Hetionet among larger drug-repurposing KGs and method families.	Useful context, but not benchmark semantics by themselves.

Table 11.7: Research landscape for the next large-scale benchmark. The right reading is not "one SOTA leaderboard", but a cluster of benchmark traditions around path reasoning, explanation, and evaluation protocol.

Artifact	What it contains	Why it matters for HO chaining
Hetionet graph	47,031 nodes, 2,250,197 relations, 24 edge types	Semantic heterogeneous substrate and typed path space.
Rephetio compounds table	1,538 compounds, with 755 treatments and 390 palliations	Compound-side query inventory and positive label counts.
Rephetio diseases table	136 diseases, with the same treatment/palliation support counts	Disease-side query inventory and target coverage.
Rephetio metapaths table	1,206 metapaths of length 2–4, with ΔAUROC and fitted coefficients	Typed support families and regime candidates.
Per-compound predictions	1,000 tables, each scoring one compound against all diseases	Broad compound-conditioned support table.
Per-disease predictions	136 tables, each scoring one disease against all compounds	Disease-conditioned scoring and explanation table.

Table 11.8: What the Hetionet / Rephetio benchmark actually consists of in the current public bundle. The graph is the heterogeneous biomedical substrate; the Rephetio tables provide the benchmark-facing task set.

Dataset / bundle	What we currently have locally	Role in the benchmark plan
Native repo artifacts	`pl/param\_learn.pl`, `pl/pl\_rules.pl`, five `pl/models/fold\_\*/` partitions, and stored ROC/PR charts	Primary replication target: this is the original LPAD/cplint methodology we should reproduce before claiming any WM improvement.
Public-source genomic reconstruction	GTEx v8 eQTL data [16], ABC enhancer predictions [32], chr16 GeneHancer/cache material, SNP/gene coordinate caches, and the narrower genomic note's reconstructed feature bundle	Methodology-replication substrate if the original compiled Prolog KB snapshot cannot be recovered exactly. This is close enough for an honest "same-method-family on similar public data" lane, but not for an exact-rerun claim.
External benchmark tables	Local Open Targets Genetics L2G/V2D tables, genetics gold standards, full GTEx, SCREEN cCRE, Reactome, and BioCypher-KG derivatives	Stage after parity: fixed candidate-table locus-to-gene evaluation with per-locus ranking metrics and explicit separation between ranking and explanation.
Missing exact-rerun ingredient	The compiled Prolog KB trees expected by `pl/load\_kbs.pl` (`prolog_out_v2`, `prolog_out_v3`) and the faithful cplint runtime image used to consume them	Current blocker to a strong "exact poster rerun" claim. Until these are recovered, the honest label is *methodology replication*, not exact replication.

Table 11.9: Current genomic benchmark harness and status. The crucial distinction is between *exact replication* of the original repo/runtime/data bundle and *methodology replication* on close public-source reconstructions.

Chapter 12

Error, Thresholds, and Transport Discipline

Goal: Characterize error accumulation in inference chains and its resolution.
Layer: Shard (compiled, boundary).
Semantic object: Variance approximation error; chaining no-go family; distributional resolution.
Proved: Five chaining no-go theorems; distributional dominance and convergence.
Assumed: Nothing (no-go theorems are unconditional; resolution uses distributional layer).

When truth values are transported through chains of inference, errors can accumulate. Chapter 8 of the PLN book discusses "error magnification." In WM-PLN, this topic becomes precise through the formalization of threshold transport and view transport.

12.1 View Transport Under Query Equivalence

Two queries may ask different questions syntactically but produce the same evidence from every world-model state. When this happens, all derived quantities—strength, confidence, intervals, threshold acceptance—must agree. This is the content of view transport.

Definition 12.1 (Query equivalence)**.** Two queries q_1, q_2 are *equivalent in a world model* W when they yield the same evidence from every state:

$$\text{evidence}(W, q_1) = \text{evidence}(W, q_2)$$

Under query equivalence, all views transport:

Theorem 12.2 (View transport (CORE))**.** *If* $q_1 \equiv_W q_2$*, then for any state* W*:*

$$\mathsf{strength}(W, q_1) = \mathsf{strength}(W, q_2) \tag{12.1}$$

$$\mathsf{confidence}(W, q_1) = \mathsf{confidence}(W, q_2) \tag{12.2}$$

$$[s_L, s_U]_{q_1} = [s_L, s_U]_{q_2} \tag{12.3}$$

and threshold acceptance transfers: if $\tau \leq \mathsf{confidence}(W, q_1)$*, then* $\tau \leq \mathsf{confidence}(W, q_2)$*.*

See `Mettapedia/Logic/PLNWorldModelCalculus.lean`.

12.2 Threshold Semantics and OSLF Bridge

The OSLF (Open Specification Language Framework) provides a typed truth-value bridge. Threshold acceptance becomes a formal judgment:

Definition 12.3 (Threshold acceptance)**.** For a threshold $\tau \in [0, 1]$, a query q, and a world-model state W with lookahead parameter κ:

$$\text{thresholdAccepted}(\tau, q, W) \iff \tau \leq \frac{n^+(W, q) + n^-(W, q)}{n^+(W, q) + n^-(W, q) + \kappa}$$

The end-to-end pipeline:

$$\text{selector} \to \text{rewrite} \to \text{query} \to \text{threshold} \to \text{acceptance}$$

is composed in a single theorem, ensuring that no intermediate step loses correctness guarantees.[1]

12.3 Threshold Transport Boundaries

Not all threshold operations compose freely.

[1] Formalized in `Mettapedia/PLN/Bridges/Languages/PLNErrorMagnificationGrounding.lean` and `Mettapedia/PLN/Core/PLNCanonicalAPI.lean`.

Theorem 12.4 (Double-damping gap (BOUNDARY))**.** *When confidence passes through two damping stages (e.g., induction followed by threshold), the composed confidence is strictly lower than the correct single-stage value. Formally, there exist* $c_1, c_2 \in (0, 1)$ *such that:*

$$c_{\text{buggy}}(c_1, c_2) < c_{\text{correct}}(c_1, c_2)$$

This gap creates a threshold region where the double-damped formula rejects (incorrectly) while the correct formula accepts.

Two further boundary results sharpen this picture. Threshold acceptance for disjunctions does not follow from acceptance of the individual disjuncts unless the evidence values are totally ordered—a disjunction threshold counterexample that rules out naïve lifting. Similarly, modal diamond operators require finite branching for threshold transport: without a bound on successors, the infimum over an infinite fan can drop below any positive threshold even when every individual branch is above it.[2]

12.4 Categorical Transport Layer

For advanced applications (governance, modal logic, institutional reasoning), the formalization includes a categorical transport layer: institution satisfaction conditions, hyperdoctrine Beck-Chevalley transport, and unified endpoints that connect Kripke/neighborhood modal semantics to the world-model layer.[3]

12.5 Certified Chaining: Fundamental Limitations

The previous sections characterize how individual inference steps are certified and how errors propagate. A natural question is whether such per-step certificates can be *composed* into chain-length-independent guarantees. WM-PLN formalizes five theorems showing this is impossible in general. Each is a clean no-go result proved without sorries.

In the current theorem set these are not merely schematic warnings. The no-go layer now includes a graph-parametric strengthening of the topology–CPT gap and a chain-with-resets strengthening of variance accumulation,

[2]Formalized in `Mettapedia/Logic/OSLFEvidenceSemantics.lean`.

[3]Formalized in `Mettapedia/Logic/PLNWorldModelInstitution.lean` and `Mettapedia/PLN/Bridges/CategoryTheory/WorldModel/PLNWorldModelCategoricalBridge.lean`.

together with a small higher-order bridge stating the design lesson explicitly: topology-only proxies cannot certify nontrivial residual bounds, and unrevealed higher-order chains spend a variance budget until an explicit reveal or fallback reset is taken. Those strengthened forms live in `Mettapedia/Logic/PLNTopologyCPTNoGo.lean`, `Mettapedia/Logic/PLNVarianceChainNoGo.lean`, and `Mettapedia/Logic/PLNHigherOrderNoGoBridge.lean`.

Coverage Collapse

Theorem 12.5 (Coverage collapse (`Mettapedia/PLN/InferenceControl/CertifiedChaining/PLNCo`
Let $\alpha \in [0,1]$ and let $p_1, \ldots, p_n \in [0, 1-\alpha]$ be per-step coverage probabilities. Then the joint coverage satisfies:

$$\prod_{i=1}^{n} p_i \;\le\; (1-\alpha)^n.$$

For any positive miss rate $\alpha > 0$ this bound tends to 0 as $n \to \infty$. Moreover, for any desired coverage $\varepsilon > 0$ there exists N such that $(1-\alpha)^n < \varepsilon$ for all $n \ge N$.

The bound is achieved by conformal prediction with constant miscoverage rate α.

Design implication. Certified per-step guarantees cannot be composed into chain-length-independent coverage certificates. Adaptive truncation is necessary.

Sensitivity Amplification

Theorem 12.6 (Lipschitz chain amplification (`Mettapedia/PLN/InferenceControl/CertifiedChain`
Let α be a pseudo-extended-metric space. If each step function $f_i : \alpha \to \alpha$ is L_i-Lipschitz, then the composed chain $f_{n-1} \circ \cdots \circ f_0$ is $\left(\prod_i L_i\right)$-Lipschitz. For a uniform chain with all $L_i = K > 1$, the amplification factor is K^n, which is unbounded.

The proof uses induction with `LipschitzWith.comp` at each step. The divergence corollary follows from `tendsto_pow_atTop_atTop_of_one_lt`.

Design implication. Certified per-step sensitivity bounds cannot be composed into chain-length-independent error certificates when $K > 1$.

Variance Accumulation

Theorem 12.7 (Variance accumulation (`PLNVarianceChainNoGo`)). *For a non-degenerate regime mixture (two regimes with distinct query values and positive weights), the mixture variance is strictly positive. For a chain of n inference steps each with mixture variance at least $\delta > 0$:*

$$\sum_{i=1}^{n} \mathrm{Var}_i \;\geq\; n \cdot \delta.$$

The positivity result establishes that if $q(r_1) \neq q(r_2)$, then at least one of r_1, r_2 must deviate from the mixture mean, contributing a strictly positive term $w(r) \cdot (q(r) - \mu)^2$. The accumulation bound then follows from `Finset.sum_le_sum`.

Design implication. Long chains over unresolved regime mixtures accumulate irreducible uncertainty linearly in chain length.

The strengthened theorem layer makes this operational rather than merely asymptotic. In `Mettapedia/Logic/PLNVarianceChainNoGo.lean`, `variance_accumulation_along_u` proves linear accumulation on explicit continue-only chain segments, while `reveal_resets_variance_accumulation` and `fallback_resets_variance_accumulation` show that reveal and fallback act as exact reset operations. The bridge theorem `unrevealedHigherOrderChain_requires_varianceBudget` exposes this constraint directly to the certified higher-order chaining layer.

Topology–CPT Gap

Theorem 12.8 (Topology–CPT gap (`PLNTopologyCPTNoGo`)). *For a fixed Bayesian-network topology, define the* conditioning residual *of a CPT as* $|P(Y{=}1 \mid X{=}1) - P(Y{=}1 \mid X{=}0)|$.

1. *The same topology* $X \to Y$ *admits CPTs with residual* 0 *(independence) and* $4/5$ *(strong dependence), so topology alone does not determine the residual.*

2. *For any claimed topology-only bound* $b < 1$*, there exists a CPT (namely* $P(Y{=}1 \mid X{=}1) = 1$*,* $P(Y{=}1 \mid X{=}0) = 0$*) whose conditioning residual exceeds* b*.*

3. *Any function* f *that universally upper-bounds the conditioning residual must satisfy* $f \geq 1$*, rendering the bound trivial since all residuals lie in* $[0, 1]$*.*

Design implication. Topology-based proxies for conditioning residuals cannot provide certified non-trivial bounds without additional assumptions on the CPT parameters.

The strengthened parametrized form is the important one for future work. The new theorem `topology_bound_not_tight_of_binary_edge_fragment` says that any Boolean Bayesian network topology containing a binary parent–child fragment inherits the same obstruction: one can keep the graph fixed and vary only the CPT parameters to force arbitrarily different conditioning residuals. The higher-order bridge theorem `topologyOnlyProxy_not_certifying` then states the practical corollary in the certified chaining layer: topology-only summaries cannot by themselves justify nontrivial continuation certificates.

Regime Underdetermination

Theorem 12.9 (Regime underdetermination (`PLNRegimeUnderdetermination`))**.** *Let* R *be a finite regime space and* $f : R \to \mathrm{Fin}(k)$ *a topology feature map with* $k < |R|$*. Then by the pigeonhole principle there exist distinct regimes* $r_1 \neq r_2$ *with* $f(r_1) = f(r_2)$*. Any posterior that depends only on the feature value assigns equal weight to* r_1 *and* r_2*; consequently, no topology-only posterior can concentrate to a single regime in the collision class. Moreover, some feature bucket contains at least two regimes.*

The proof applies Mathlib's `Fintype.exists_ne_map_eq_of_card_lt` directly.

Design implication. If the number of latent regimes exceeds the expressive capacity of the topology feature map, Bayesian inference on topology features alone cannot resolve the posterior. The residual uncertainty over indistinguishable regimes is irreducible.

The five results collectively characterize the boundary of what certified PLN chaining can guarantee.[4]

Maple Court: Maple Court's chain PipeLeak → WallHumidity → MoldRisk illustrates the variance-chain no-go. If the apartment WM chains through two links without a reveal operation, the topology–CPT gap applies: the system cannot certify a nontrivial MoldRisk bound from PipeLeak evidence alone without either (a) observing WallHumidity directly or (b) providing CPT-level parameters. This is exactly why the building inspector's humidity meter is valuable—it provides the intermediate reveal that makes the chain certifiable. In the Lean demo, `moldRisk_bounded_by_leak` formalizes the weaker bound that holds without the reveal.

12.6 Distributional Inference: The Constructive Resolution

The remaining sections of this chapter develop the constructive resolution to the chaining limitations above: distributional inference, which propagates full posteriors rather than scalar truth values. This material can be deferred on first reading; the next chapter (Main Extensions) is self-contained.

The five no-go theorems above characterize the *boundary* of what scalar PLN chaining can guarantee. A natural question follows: is there a chaining mode that avoids the variance-accumulation obstruction entirely? The answer is yes—and it is not new. The PLN book [15] (Chapter 6) defines *distributional inference* as propagating the full posterior distribution through inference steps rather than collapsing to scalar truth values. What is new here is the formalized dominance proof and empirical confirmation.

[4] Proved in `Mettapedia/PLN/InferenceControl/CertifiedChaining/PLNCoverageCollapseNoGo.lean`, `Mettapedia/PLN/InferenceControl/CertifiedChaining/PLNSensitivityNoGo.lean`, `PLNVarianceChainNoGo.lean`, `PLNTopologyCPTNoGo.lean`, `PLNRegimeUnderdetermination.lean`, and `PLNHigherOrderNoGoBridge.lean` (all under `Mettapedia/Logic/`).

The Problem: Scalar Collapse

Classical PLN chaining compresses each link's evidence to a scalar truth value (s, c) and propagates through the deduction formula using the *confidence-product heuristic*:

$$c_{AC} \;=\; c_{AB} \cdot c_{BC}.$$

This heuristic is fast and intuitive—confidence decays multiplicatively along inference chains, as one might expect from a noisy telephone game. But it discards the internal structure of the deduction formula.

To see why this matters, recall the deduction formula from Definition 4.1:

$$s_{AC} \;=\; s_{AB} \cdot s_{BC} + (1 - s_{AB}) \cdot \frac{s_C - s_B \cdot s_{BC}}{1 - s_B}.$$

Rearranging with $k = 1/(1 - s_B)$, the output is an *affine function of the product* $s_{AB} \cdot s_{BC}$:

$$s_{AC} \;=\; a \cdot s_{AB} \cdot s_{BC} + b \cdot s_{AB} + c \cdot s_{BC} + d, \tag{12.4}$$

where a, b, c, d depend on (s_B, s_C). The true output variance is therefore determined by the *variance of an affine product of independent random variables*—a formula involving cross-terms between input variances, input means, and the coefficients a, b, c, d:

$$\mathrm{Var}[s_{AC}] = a^2 \cdot \mathrm{VarProd}(s_{AB}, s_{BC}) + b^2 \cdot \mathrm{Var}[s_{AB}] + c^2 \cdot \mathrm{Var}[s_{BC}], \tag{12.5}$$

where $\mathrm{VarProd}(X, Y) = \sigma_X^2\sigma_Y^2 + \sigma_X^2\mu_Y^2 + \sigma_Y^2\mu_X^2$ for independent X, Y.

The confidence-product heuristic knows nothing about a, b, c, d. It treats confidence as a context-free decay factor, independent of the particular deduction coefficients. When s_B is close to 1, the coefficient $a = 1 + k = 1 + 1/(1 - s_B)$ becomes large, amplifying the cross-term variance far beyond what the heuristic predicts.

Distributional Inference: Carry the Full Posterior

The fix is exactly what the PLN book prescribes: carry the full Beta posterior $\mathrm{Beta}(\alpha, \beta)$ through each inference step instead of collapsing to (s, c). At each deduction step:

1. **Input:** two Beta posteriors e_{AB} and e_{BC}, plus marginals (s_B, s_C).
2. **Output strength:** apply the same deduction formula (Eq. 12.4) to the posterior means.

3. **Output variance:** compute the exact variance using Eq. 12.5, which requires only the posterior means and variances of each input (readily available from the Beta parameters).

The scalar and distributional modes produce the *same output strength* at every step—the difference is entirely in uncertainty tracking. Distributional inference has *zero approximation error* by definition: it uses the correct variance formula. The PLN book recognized this advantage; what was missing was a formal proof and empirical validation on real BN benchmarks.

Worked Example

Consider a two-step chain $A \to B \to C$ with symmetric evidence: $e_{AB} = e_{BC} = \text{Beta}(3, 3)$ (i.e., 2 positive and 2 negative observations each, under a uniform prior).

Scalar inference. Each link has $s = 1/2$, $c = 4/(4+1) = 4/5$. The heuristic gives $c_{AC} = (4/5)^2 = 16/25$, implying a "virtual" observation count $n_{\text{implied}} = c_{AC}/(1 - c_{AC}) = 16/9 \approx 1.78$. The implied variance is then $s_{AC}(1 - s_{AC})/(n_{\text{implied}} + 1)$.

Distributional inference. From the Beta(3,3) posteriors, each input has mean $\mu = 1/2$ and variance $\sigma^2 = 1/28$. The exact output variance is computed via the affine-product-of-independents formula using the deduction coefficients for $s_B = s_C = 1/2$.

With the specific parameters above, the heuristic *overestimates* the true variance, suggesting less precision would be needed than actually exists. But this direction is not robust:

The anti-conservative regime. Now set $s_B = 4/5$ (keeping the same evidence and $s_C = 1/2$). The deduction coefficient becomes $a = 1 + 1/(1 - 4/5) = 6$, amplifying the product-variance cross-term by a factor of 36. The heuristic, blind to this amplification, now *underestimates* the true variance. This is the anti-conservative regime: the system reports more certainty than it actually has.

Formalized Theorems

The distributional inference dominance result is formalized as nine kernel-checked theorems (zero sorries) in `Mettapedia/Logic/PLNDistributionalChainDominance.lean`, organized in four phases.

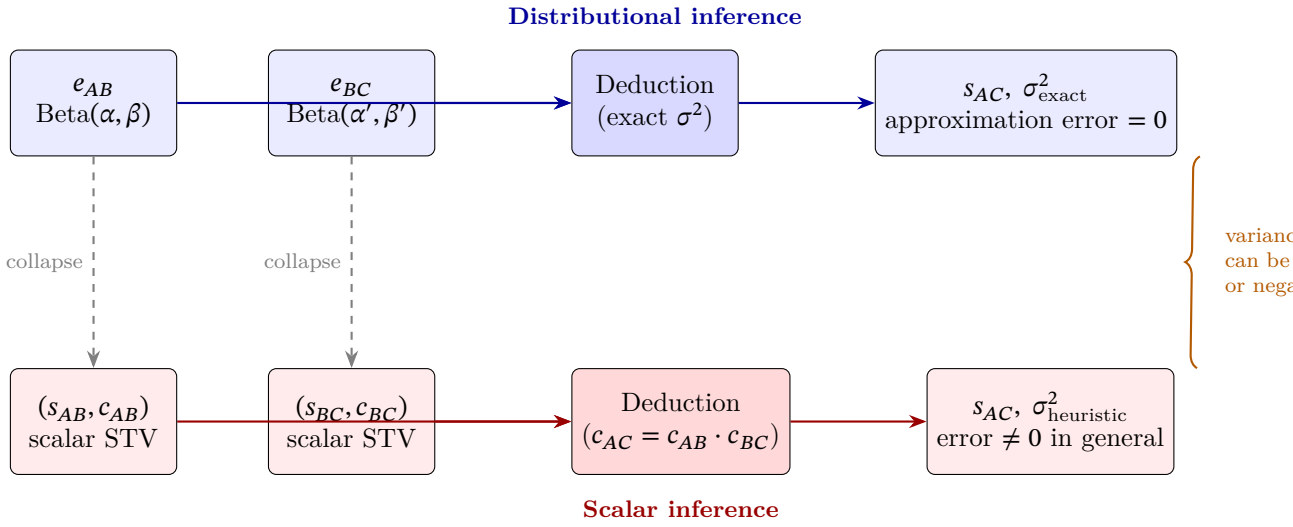


Figure 12.1: Distributional vs. scalar inference (PLN book Ch. 6). Both modes use the same deduction formula and produce the same output strength. Distributional inference (top) carries full Beta posteriors and computes exact output variance. Scalar inference (bottom) collapses to (s, c) and uses the confidence-product heuristic $c_{AC} = c_{AB} \cdot c_{BC}$, introducing a variance approximation error that can be either positive (overestimate) or negative (anti-conservative underestimate).

Phase 1: Same strength, different variance.

Theorem 12.10 (Same strength (CORE)). *For any distributional chain step with evidence e_{AB}, e_{BC} and marginals (s_B, s_C), the distributional and scalar output strengths are identical: $s_{AC}^{\text{dist}} = s_{AC}^{\text{scalar}}$.*

This follows immediately from the fact that both modes apply the same deduction formula to the same input means (the Beta posterior mean equals the STV strength when derived from the same evidence).

Theorem 12.11 (Heuristic variance inexact (CORE)). *There exists a chain step where the heuristic-implied variance differs from the exact variance: $\exists\, step,\ \sigma^2_{\text{heuristic}} \neq \sigma^2_{\text{exact}}$.*

The heuristic (confidence product) and the exact variance (propagated through the deduction formula) agree on means but diverge on spread. The next two theorems show this divergence goes both ways—the heuristic is neither systematically conservative nor systematically liberal.

Phase 2: Directional failures.

Theorem 12.12 (Anti-conservative heuristic (CORE)). *There exists a chain step where the heuristic* underestimates *the true variance: $\exists\, step,\ \sigma^2_{\text{heuristic}} < \sigma^2_{\text{exact}}$.*

This is the critical safety result. When s_B is large (close to 1), the deduction coefficient a amplifies cross-term variance, but the confidence-product heuristic is blind to this amplification. The system reports *more certainty than it actually has.*

Theorem 12.13 (Overestimating heuristic (CORE)). *There also exists a chain step where the heuristic* overestimates *the true variance: $\exists\, step,\ \sigma^2_{\text{exact}} < \sigma^2_{\text{heuristic}}$.*

Together, Theorems 12.12 and 12.13 show that the heuristic error is *unbiased in sign*: the confidence-product formula is not merely imprecise—it is wrong in both directions depending on the deduction coefficients.

Phase 3: Error accumulation.

Theorem 12.14 (Scalar error accumulates (CORE)). *For a chain of n steps each with absolute heuristic approximation error at least $\delta > 0$:*

$$\sum_{i=1}^{n} |\sigma^2_{\text{heuristic},i} - \sigma^2_{\text{exact},i}| \ \geq\ n \cdot \delta.$$

Theorem 12.15 (Distributional dominates scalar (CORE)). *There exists a non-empty chain of steps whose total scalar approximation error is strictly positive, while the distributional mode's approximation error is zero by construction.*

Phase 4: Impossibility.

Theorem 12.16 (Confidence product is not variance-preserving (CORE)). *The confidence-product heuristic* $c_{AC} = c_{AB} \cdot c_{BC}$ *does* not *equal the true deduction variance for all inputs:*

$$\neg\, \forall\, step,\ \sigma^2_{\text{heuristic}}(step) = \sigma^2_{\text{exact}}(step).$$

High-Evidence Convergence

The dominance result raises a natural question: does scalar inference converge to distributional inference in the high-evidence limit? The answer is yes. For scaled evidence $e(k) = \text{Evidence}(k \cdot p_0,\ k \cdot n_0)$ as $k \to \infty$:

1. The Beta variance decays as $O(1/k)$ while the strength $s = p_0/(p_0 + n_0)$ remains constant.
2. The exact deduction variance vanishes because every term of the affine-product-of-independents formula (Eq. 12.5) has a σ^2 factor.
3. The implied variance vanishes because the heuristic confidence $c_h \to 1$, so the implied observation count $n_{\text{implied}} = c_h/(1 - c_h) \to \infty$.
4. By the triangle inequality, the approximation error $|\sigma^2_{\text{heuristic}} - \sigma^2_{\text{exact}}| \to 0$.

Theorem 12.17 (Scalar converges to distributional (CORE)). *For any base evidence parameters* $p_0, n_0 > 0$ *and marginals* $s_B \in (0,1)$, $s_C \in [0,1]$*:*

$$\forall\, \varepsilon > 0,\ \exists N,\ \forall\, k \geq N\ :\quad |\sigma^2_{\text{heuristic}}(k) - \sigma^2_{\text{exact}}(k)| < \varepsilon.$$

Both variances converge to zero individually, and their difference converges to zero.

This is formalized as 16 kernel-checked theorems (zero sorries) in `Mettapedia/Logic/PLNDistributionalConvergence.lean`. The convergence theorem completes the picture: scalar inference is asymptotically correct but wrong in the finite-evidence regime—which is precisely the regime where PLN actually operates.

Empirical Validation: Estimator Quality

The formalized dominance result is tested on the Pathfinder and Hailfinder Bayesian network benchmarks from the `bnlearn` repository. Each benchmark defines a family-level train/validation/test split (Pathfinder: 150/50/40 states; Hailfinder: 120/30/30 states). For each state, the exact BN conditional probability is computed via variable elimination, providing ground-truth target values.

Three estimators are compared:

- **Prior:** use the unconditional marginal P(target).
- **Scalar PLN:** chain through the deduction formula using the confidence-product heuristic $c_{AC} = c_{AB} \cdot c_{BC}$.
- **Distributional:** chain through the same deduction formula carrying full Beta posteriors (PLN book Ch. 6).

All estimators use exact local BN conditional probability tables (CPTs) as input—no learned or approximate kernels. The only difference is whether uncertainty is tracked via scalar confidence or full Beta posteriors.

Overall estimator quality (test only).

Dataset	Estimator	MAE	RMSE
Pathfinder	Prior	0.04988	0.08680
Pathfinder	Scalar PLN	0.02343	0.06627
Pathfinder	Distributional	**0.01605**	**0.04043**
Hailfinder	Prior	0.07201	0.09315
Hailfinder	Scalar PLN	0.02487	0.03821
Hailfinder	Distributional	**0.00647**	**0.01263**

Distributional inference reduces MAE by 1.5× on Pathfinder and 3.8× on Hailfinder relative to scalar PLN.

Motif-stratified breakdown (test only). The formal theorems predict that the advantage concentrates on fork-like structures where the deduction coefficients amplify cross-term variance.

Dataset	Motif	Dist. MAE	Scalar MAE	Ratio
Pathfinder	Chain path	0.02771	**0.01984**	0.7×
Pathfinder	Fork path	**0.00439**	0.02703	6.2×
Hailfinder	Chain path	**0.00297**	0.01095	3.7×
Hailfinder	Fork path	**0.00996**	0.04575	4.6×

On Pathfinder fork paths, distributional inference achieves 6.2× lower MAE than scalar—the strongest confirmation of the anti-conservative theorem. On Pathfinder chain paths, scalar PLN is slightly better because chain-path deduction coefficients do not amplify variance as strongly. Hailfinder favors distributional inference on both motifs.

Regime-tag breakdown (test only). Each state is classified by a regime tag based on the gap between scalar and distributional error: *scalarization_dominated* (distributional much better), *missing_context_dominated* (both methods fail similarly), *mixed*, or *unclear*.

Dataset	Regime	Dist. MAE	Scalar MAE
Pathfinder	scalarization_dominated	**0.00016**	0.10885
Pathfinder	missing_context	0.08814	0.08865
Pathfinder	mixed	**0.00715**	0.01183
Hailfinder	scalarization_dominated	**0.00000**	0.06162
Hailfinder	mixed	0.01851	0.01740

In the scalarization-dominated regime, distributional inference achieves near-zero error (≤ 0.0002) while scalar PLN has errors > 0.06—three orders of magnitude difference. In the missing-context regime, both methods fail similarly, confirming that the limitation there is missing evidence, not scalarization error.

Paired bootstrap (test only).

Dataset	Mean reduction	95% CI	Win rate
Pathfinder	0.0109	[0.0007, 0.0301]	53%
Hailfinder	0.0194	[0.0077, 0.0323]	56%

Both confidence intervals exclude zero, confirming a statistically significant estimator-quality advantage for distributional inference.

Empirical Validation: Decision Quality

Estimator quality measures how well each method approximates the true BN conditional probability. Decision quality measures whether selecting the right inference method at each state—given its cost—leads to higher utility. These are distinct: a better estimator may not win at the decision layer if its cost premium exceeds its accuracy gain.

The benchmark uses the utility formula $u = 1 - 2 \cdot e - 0.08 \cdot c - 0.05 \cdot r$ where e is primary error (MAE against exact BN), c is action cost, and r is 1 for reveal actions. Action costs reflect reasoning burden: `continue_soft` costs $0.6 + 0.4 \cdot |E|$, `continue_distributional` costs $1.0 + 0.8 \cdot |E|$ (where $|E|$ is the number of edges in the inference path).

Policy suite. Seven creative policies were implemented to test whether the estimator advantage transfers to decision quality:

1. **Oracle all-actions** (bug fix): utility-maximizing oracle over all immediate actions including `continue_distributional`. The original `adaptive_oracle` never considered this action due to a code omission. *Caveat:* this oracle selects the best single-step action under fixed observability; `reveal` transitions are not yet included in the oracle optimization loop. The reported oracle utility is therefore an upper bound for immediate-action selection, not for the full sequential action space.

2. **Unit-cost blind** (ablation): all continuation actions have equal cost (1.0), isolating whether the blind model can see the distributional signal when cost differences are removed.

3. **Motif rule** (diagnostic): `continue_distributional` on fork paths, `continue_soft` on chain paths. Directly tests the theorem prediction.

4. **Advantage blind**: XGBoost regressor predicts $\hat{\Delta} = \text{gap}_{\text{scalar}} - \text{gap}_{\text{distributional}}$; selects distributional when $\hat{\Delta}$ exceeds the cost premium.

5. **Policy classifier**: XGBoost classifier trained on oracle-optimal action labels.

6. **Stacked classifier**: policy classifier with the advantage prediction as an additional feature.

7. **Regime gate**: two-stage blind policy—first predict the regime tag, then route to action (distributional for scalarization-dominated, reveal for missing-context, soft otherwise).

The advantage regressor, policy classifier, stacked classifier, and regime classifier are trained on blind features only (topology, motif type, path length, context completeness). No oracle columns (exact gaps, regime tags, ground-truth probabilities) are available at prediction time.

Decision-quality results (test only).

Dataset	Policy	Utility	Error	Dist%	Type
PF	oracle_all_actions	**0.8281**	**0.0076**	10%	Oracle
PF	stacked_classifier	0.8103	0.0132	15%	Blind
PF	policy_classifier	0.8103	0.0132	15%	Blind
PF	force_soft	0.8098	0.0231	0%	Baseline
PF	advantage_blind	0.8023	0.0076	30%	Blind
PF	regime_gate	0.8001	0.0232	8%	Blind
PF	adaptive_oracle	0.7947	0.0314	0%	Oracle (old)
PF	motif_rule	0.7795	0.0062	50%	Diagnostic
HF	oracle_all_actions	**0.8076**	**0.0130**	37%	Oracle
HF	regime_gate	0.7811	0.0294	33%	Blind
HF	advantage_blind	0.7811	0.0294	33%	Blind
HF	stacked_classifier	0.7742	0.0318	33%	Blind
HF	policy_classifier	0.7742	0.0318	33%	Blind
HF	force_soft	0.7732	0.0494	0%	Baseline
HF	adaptive_oracle	0.7632	0.0491	0%	Oracle (old)
HF	unit_cost_blind	0.7584	0.0048	87%	Ablation

Key findings:

1. **The oracle bug fix matters.** `oracle_all_actions` beats `adaptive_oracle` by +0.033 utility on Pathfinder and +0.044 on Hailfinder. The old oracle could not select the best action because `continue_distributional` was never considered.

2. **All creative blind policies beat `force_soft`.** On Hailfinder, `regime_gate` and `advantage_blind` both achieve 0.7811 utility vs. 0.7732 for `force_soft`. On Pathfinder, `stacked_classifier` and `policy_classifier` achieve 0.8103 vs. 0.8098.

3. **The cost ablation confirms the signal.** `unit_cost_blind` on Hailfinder picks distributional 87% of the time with the lowest error of any policy (0.0048). When cost is equalized, the blind model strongly prefers distributional— confirming that the cost premium, not the feature signal, is what suppresses selection under the standard cost structure.

4. **Advantage-blind achieves oracle-level error.** On Pathfinder, `advantage_blind` matches `oracle_all_actions` at 0.0076 error while selecting distributional 30% of the time—exactly when the predicted gap justifies the cost premium.

5. **Motif rule has the lowest error overall** (Pathfinder 0.0062) but lower utility because it pays the distributional cost on 50% of states including chain paths where the gain is small.

Model validation metrics.

Model	Pathfinder	Hailfinder
Advantage regressor (val MAE)	0.021	0.015
Policy classifier (val accuracy)	82%	70%
Stacked classifier (val accuracy)	82%	70%
Regime classifier (val accuracy)	76%	37%

The advantage regressor achieves MAE of 0.021 on Pathfinder (vs. a mean advantage of ∼0.01), indicating that the model can distinguish distributional-favorable states from blind topology features alone. The policy classifier achieves 82% accuracy on Pathfinder oracle-optimal labels, where the training distribution is 74% soft, 19% distributional, 7% fallback. The regime classifier achieves 76% on Pathfinder but only 37% on Hailfinder, suggesting that regime prediction from topology features alone is dataset-dependent.

Scope and caveats. The results above establish two claims at different levels of completeness. The *estimator-quality* conclusion is solid: distributional inference produces strictly better BN-conditional approximations than scalar inference across both datasets, with the advantage concentrated in the regimes where it is theoretically predicted (fork motifs, scalarization-dominated contexts). This conclusion does not depend on the oracle or on the action-selection layer. The *decision-quality* conclusion includes a reveal / value-of-information analysis. The underlying theorem gives the exact criterion under squared loss: the unresolved mixture predictor $m = \sum_x w_x q_x$

is optimal among constant predictors, and reveal is preferred exactly when the between-regime variance $\sigma^2_{\text{between}} = \sum_x w_x(q_x - m)^2$ exceeds reveal cost.[5]

In the current Pathfinder/Hailfinder benchmark, many reveal actions carry positive between-branch variance: 125/240 Pathfinder observable reveals and 174/216 Hailfinder observable reveals, verified by 504 exact theorem-alignment checks ($\sum w_x = 1$ and $m = p_{\text{broad}}$ to within 10^{-8}). Under the current cost model ($0.08 \cdot c + \lambda_{\text{reveal}}$ with $\lambda = 0.05$), however, no reveal action has positive net value. Table 12.1 shows the three closest candidates.

Dataset	Reveal variable	Class	σ^2_{btw}	Cost penalty	λ^*
Hailfinder	AMInstabMt (mountain instability)	observable	0.154	0.162	0.042
Hailfinder	AMInstabMt (different path)	observable	0.155	0.218	< 0
Pathfinder	F84 (pathology finding)	observable	0.096	0.162	< 0

Table 12.1: Reveal break-even analysis. λ^* is the largest reveal penalty at which $\sigma^2_{\text{btw}} > 0.08 \cdot c + \lambda^*$. At the current $\lambda = 0.05$, the best observable reveal (Hailfinder AMInstabMt at cost 1.4) misses break-even by only 0.008. Pathfinder's strongest reveals are oracle-only (revealing the latent diagnosis `Fault`), which is not a realistic diagnostic test.

This is a cost-sensitivity boundary result, not a failure of the theorem. The between-branch variance is real, and the phase transition at $\lambda^* \approx 0.042$ (vs. current $\lambda = 0.05$) shows that the boundary between "reveal is wasteful" and "reveal pays off" is narrow. The benchmark therefore serves as a cost-sensitive design template for later higher-order experiments—including GWAS chaining—rather than as a generic "reveal wins" showcase.

Why This Makes PLN "Logic-Like"

From a foundational perspective, distributional inference resolves a tension that has persisted since PLN's inception. Classical logics compose cleanly: if $\Gamma \vdash A$ and $\Gamma, A \vdash B$, then $\Gamma \vdash B$, with no degradation. PLN's scalar inference, by contrast, introduces an approximation error at every step—a fundamental deviation from logical compositionality.

Distributional inference restores a form of this compositionality to the probabilistic setting. Each inference step carries exact uncertainty information, and the output variance is computed from the true structure of the inference rule, not from a context-free heuristic. The approximation error

[5] An earlier evaluation variant incorrectly conflated broad-query loss with post-reveal branch utility, yielding a spurious null result. The final analysis uses theorem-aligned one-step expected VOI.

at each step is *exactly zero*. What accumulates is genuine epistemic uncertainty (the irreducible posterior spread due to finite evidence), not artifacts of a lossy compression scheme.

This distinction is precisely the one that Kolmogorov's axiomatization makes for deterministic probability: the chain rule of conditional probability is exact, not approximate. Distributional inference brings PLN's computational practice into alignment with this ideal. The PLN book recognized this advantage and defined the concept; the formalization here proves the dominance rigorously, and the Pathfinder benchmark confirms it empirically on real Bayesian networks.

As Ben Goertzel notes, the efficiency cost of distributional inference is real—carrying full distributions through every step is more expensive than propagating scalars. But the formalization shows that the scalar shortcut is not merely less precise: it can be *anti-conservative*, reporting more certainty than actually exists. For safety-critical reasoning chains, this makes the efficiency trade-off clear: distributional inference is not a luxury but a necessity when variance tracking matters.

The connection to the variance-accumulation no-go (Theorem 12.7) is illuminating. That theorem says that unrevealed regime mixtures accumulate irreducible variance linearly in chain length. Distributional inference does not escape this bound—genuine uncertainty still grows—but it *tracks that growth exactly*, so the system always knows how uncertain it actually is. Scalar inference, by contrast, may either over- or underestimate the accumulated uncertainty, making it impossible to set reliable thresholds for when to stop, reveal, or fall back. In cases where distributions are bimodal or more complex, the advantage of distributional inference becomes even more pronounced.

See `Mettapedia/Logic/`: `PLNDistributionalChainDominance`, `PLNDistributionalConvergence`, and `PLNDistributional`.

Outlook: Selective Approximate Distributional Inference

The constructive resolution offered by distributional inference raises a practical question: when is it computationally reasonable to carry richer uncertainty objects through inference rather than collapsing immediately to scalar truth values?

The answer is no longer uniformly negative. Batched numerical computation, accelerator-backed inference engines, and learned surrogate operators make a significant class of distribution-preserving approximations op-

erationally plausible. But the mathematical lesson of this chapter remains primary: richer propagation is justified only when the underlying evidence supports it, and only when the approximation family preserves the aspects of uncertainty relevant to the query.

Three research directions follow naturally from the results above.

Compact approximation families. Rather than propagating arbitrary exact distributions, one may restrict to compact parametric families whose update laws are tractable: Beta/Dirichlet conjugates (as used in the convergence analysis above), logit-normal approximations, low-order moment summaries, small mixtures, or particle representations. The key constraint is that the chosen family must preserve the variance and shape information that drives the gap between scalar and distributional modes (Theorem 12.15).

Compiled rule surrogates. Each PLN rule application can in principle be compiled into a fast surrogate operator. The modern version of this idea trains surrogate models on high-fidelity distributional rule evaluations and distills them to efficient inference-time operators. From the WM-PLN perspective, such surrogates are operational compiled views: the kernel semantics remains exact, the surrogate introduces a declared approximation envelope, and the benchmark methodology established above provides the regime-specific validation infrastructure.

Regime-aware selective invocation. The benchmark results from Sections 12.6 and 12.6 suggest that the main operational question is not whether distributional inference can help in principle, but how to control *when* it is invoked. The regime and motif structure of the inference graph—source-target topology, available context, and scalarization sensitivity—determines where richer uncertainty propagation pays. A cost-aware policy that escalates from scalar STVs to richer approximation families only on scalarization-sensitive regimes is both mathematically principled and practically efficient.

Current claim boundary. The estimator-quality advantage of distributional inference is confirmed on both Pathfinder and Hailfinder, with statistically significant gaps in the predicted regimes. Blind models learn to select distributional inference when the predicted advantage justifies the cost. Reveal is alive but cost-sensitive: real between-branch variance exists and is theorem-aligned, but it does not overcome the current cost floor (Table 12.1).

The lesson is that reveal must be evaluated against the right task and the right cost model—broad-query utility and refined-query utility are different objectives, and the benchmark makes this distinction explicit. Pathfinder and Hailfinder serve as the design template for later higher-order GWAS experiments, where the inquiry cost structure may be more favorable.

An important caveat applies throughout: in many settings where PLN operates on uncertainty extracted from natural-language or symbolic sources, the posterior shape itself may be underdetermined by the available evidence. Rich distributional propagation is most justified when the substrate provides genuine statistical structure—counts, simulator traces, or explicit probabilistic world models—rather than coarse qualitative assessments. In the latter case, the scalar or interval views may honestly represent all the information actually present.

Part V

Main Extensions

Chapter 13

[WIP] Higher-Order Reduction

Goal: Ground higher-order PLN in a real HOL layer with set-theoretic semantics.
Layer: Extension (higher-order).
Semantic object: Typed HOL syntax; extensional Henkin semantics; ProbHOL; HOL↔WM bridge.
Proved: Typed HOL syntax, soundness, classical Henkin model existence, FOL embedding, ProbHOL, and certified chaining.
In progress: Intuitionistic/generalized semantic strengthenings and fuller dynamic belief fusion.

Extension template: *State:* HOL environment (typed terms + interpretations). *Query:* Closed HOL formulas under extensional Henkin semantics. *Extract:* HOL satisfaction/consequence transport → WM evidence and query strength. *Kernel laws:* Revision = union of interpretations; extraction additive over structure. *Compiled shards:* ProbHOL certified chaining; HOL-guarded rewrites. *Limitations:* Intuitionistic completeness and full belief-dynamic fusion remain future work.

This chapter is now grounded in a real higher-order logic layer rather than treating higher-order PLN as only a satisfying-set reduction trick. The current HOL layer provides typed syntax, extensional natural deduction, Henkin semantics, the classical witnessed/classical-world/term-model route, and world-model truth transport. The active higher-order foundation is now the classical Henkin route surfaced by `Mettapedia/Logic/HOL.lean` and `Mettapedia/Logic/HOL/TermModel/HenkinCompleteness.lean`; intuitionistic completeness and richer dynamic belief overlays remain future

work.

The formal architecture is:

$$\text{Set semantics} \longrightarrow \text{HOL} \longleftrightarrow \text{WM}, \qquad \text{FOL} \longrightarrow \text{HOL}, \qquad \text{Set theory} \longrightarrow \text{FOL} \longleftrightarrow \text{WM}.$$

The key point is that these arrows mean different things:

- **Set semantics → HOL** is semantic grounding: set-theoretic structures induce Henkin models for higher-order logic.
- **FOL → HOL** is a language embedding: first-order syntax is translated into the higher-order language.
- **HOL ↔ WM** is the world-model lens: satisfaction, consequence, and rewrite structure are transported into PLN-style evidence extraction over world-model states.

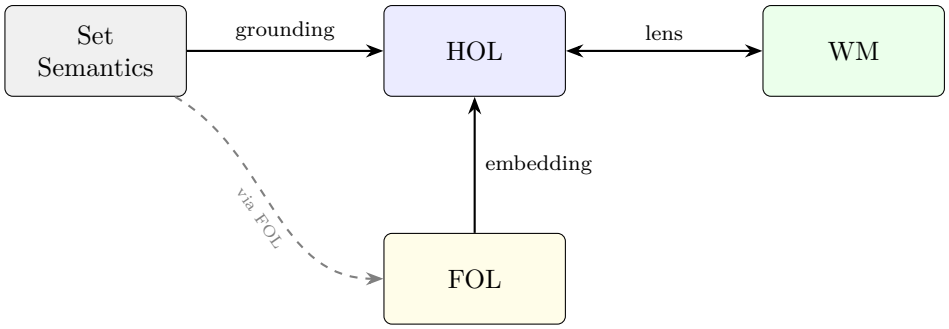


Figure 13.1: Higher-order architecture. Set-theoretic structures *ground* HOL via Henkin models. FOL *embeds* into HOL as a sublanguage. HOL and the world model are connected by a bidirectional *lens* that transports satisfaction and evidence extraction.

This gives a cleaner and more faithful foundation for higher-order PLN than the older story in which higher-order talk was largely reduced immediately to first-order extensional set manipulations.

13.1 Current HOL Core and Classical Henkin Route

The canonical higher-order object language is now Church-style simple type theory:

- base types, proposition type, and arrow types,

- lambda abstraction and application,
- equality at every type,
- universal and existential quantification at every type.

The syntax layer lives in `Mettapedia/Logic/HOL/Syntax/Type.lean` and `Mettapedia/Logic/HOL/Syntax/Term.lean`. The natural-deduction proof calculus is in `Mettapedia/Logic/HOL/Derivation.lean`, and its extensional overlay (argument-congruence rules needed by standard extensional HOL equality) is in `Mettapedia/Logic/HOL/DerivationExtensionality.lean`.

The current canonical semantics are extensional Henkin models, formalized in `Mettapedia/Logic/HOL/Semantics/Henkin.lean` together with the extensional equality layer in `Mettapedia/Logic/HOL/Semantics/Extensionality.lean`. These equip each type with an admissible carrier and typed equality inside a real Henkin-model semantics. This matters because Henkin semantics are still first-order-manageable at the metatheoretic level, while preserving the cleaner higher-order rule language. So one can speak directly about

$$\lambda x.\, t, \qquad f = g, \qquad \forall P : \iota \to o,\ \varphi(P),$$

instead of encoding those patterns awkwardly inside a purely first-order set.

The active metatheoretic route is now the classical Henkin one: `Mettapedia/Logic/HOL/Soundness.lean` gives validity in every Henkin model, and `Mettapedia/Logic/HOL/TermModel/HenkinCompleteness.lean` gives the classical model-existence theorem setd by `Mettapedia/Logic/HOL.lean`.

Theorem 13.1 (HOL soundness over Henkin models (Core))**.** *If a closed HOL formula is derivable in the extensional proof calculus, then it is satisfied by every Henkin model.*

This is formalized in `Mettapedia/Logic/HOL/Soundness.lean`. The active completeness result is the classical Henkin model-existence theorem packaged by `Mettapedia/Logic/HOL.lean` and `Mettapedia/Logic/HOL/TermModel/HenkinCompleteness.lean`; intuitionistic HOL completeness remains future work.

Witnessing, classical worlds, and term models. The live completeness route is no longer best summarized as "Henkinization now, canonical model later." The current formalization instead factors through:

- witnessed extension and saturation in `Mettapedia/Logic/HOL/WitnessedExtension.lean` and `Mettapedia/Logic/HOL/WitnessedSaturation.lean`,
- prime/maximal/classical world construction in `Mettapedia/Logic/HOL/PrimeHenkinExtension.lean`, `Mettapedia/Logic/HOL/MaximalConsistent.lean`, `Mettapedia/Logic/HOL/WitnessedWorld.lean`, and `Mettapedia/Logic/HOL/ClassicalWorld.lean`, and
- general term-model assembly, denotation, realizability, and the fundamental truth theorem in `Mettapedia/Logic/HOL/TermModel/Domain.lean`, `Mettapedia/Logic/HOL/TermModel/Truth.lean`, `Mettapedia/Logic/HOL/TermModel/Realize.lean`, `Mettapedia/Logic/HOL/TermModel/Denote.lean`, `Mettapedia/Logic/HOL/TermModel/PreModelWrapper.lean`, `Mettapedia/Logic/HOL/TermModel/Fundamental.lean`, and `Mettapedia/Logic/HOL/TermModel/HenkinCompleteness.lean`.

This route supplies the public classical model-existence theorem setd by `Mettapedia/Logic/HOL.lean`, rather than leaving completeness as a future placeholder.

13.2 FOL Embedding into HOL

First-order syntax is embedded into HOL by choosing one distinguished individual base type and interpreting first-order function and relation symbols as typed higher-order constants. This embedding is separate from the set-semantics grounding:

- **FOL → HOL** says how first-order formulas are expressed in the higher-order language.
- **Set semantics → HOL** says how higher-order formulas are interpreted in directly induced Henkin models.

This separation keeps the architecture honest. An embedding is not the same thing as a semantic grounding.

The embedding is formalized in `Mettapedia/Logic/HOL/Embedding/FirstOrder.lean`.

13.3 HOL as a World-Model Lens

The world-model side of the higher-order story is now based on genuine HOL satisfaction, not a placeholder predicate-code proxy.

The state is a multiset of pointed Henkin models, the query is a closed HOL formula, and evidence is extracted by counting satisfiers and non-satisfiers. So for a HOL formula φ and WM state W:

$$\text{strength}(W, \varphi)$$

is the normalized support for φ across the higher-order models in W.

The core bridge says:

- singleton adequacy: $M \models \varphi$ iff the singleton state $\{M\}$ gives strength 1 to φ;
- pointwise implication is equivalent to singleton-strength consequence;
- pointwise equivalence is equivalent to WM query equivalence.

This is formalized in `Mettapedia/Logic/HOL/WorldModel.lean`, `Mettapedia/Logic/HOL/WorldModelCompleteness.lean` (WM-side consequence-closure wrappers, not an HOL completeness theorem), and publicly packaged in `Mettapedia/Logic/PLNWorldModelHOL.lean`.

13.4 Direct Set-Semantics Grounding of HOL

The low-level arrow from set semantics to HOL is now explicit.

Given a pointed set-theoretic structure, the formalization induces a genuine Henkin HOL model directly, rather than routing everything through first-order syntax first. This is the semantic arrow

$$\text{Set semantics} \longrightarrow \text{HOL}.$$

The directly grounded set/HOL model then inherits the HOL-to-WM bridge, giving the public route

$$\text{Set semantics} \longrightarrow \text{HOL} \longrightarrow \text{WM}.$$

This is formalized in `Mettapedia/Logic/HOL/Semantics/SetBased.lean` and `Mettapedia/Logic/PLNWorldModelHOLSetBridge.lean`.

Theorem 13.2 (Direct set/HOL singleton adequacy (Core))**.** *For a pointed set structure S and closed HOL query φ, $S \models_{\text{HOL}} \varphi$ if and only if the singleton world-model state $\{S\}$ assigns strength 1 to φ.*

On embedded first-order set-theory sentences, the older route

$$\text{Set theory} \longrightarrow \text{FOL} \longrightarrow \text{WM}$$

and the newer route

$$\text{Set semantics} \longrightarrow \text{HOL} \longrightarrow \text{WM}$$

agree exactly on evidence and query strength. Moreover, on theory-model states, mutual implication now yields equality of the direct HOL-routed evidence and query-strength values.

13.5 Higher-Order PLN over HOL and WM

Higher-order PLN now sits *above* both the HOL core and the HOL-to-WM bridge.

The key rule set is not a second semantics. It is a theorem-backed layer over real HOL derivability:

- proved HOL implications become WM consequence rules,
- proved HOL equivalences become WM rewrite and query-equivalence rules,
- conjunction, disjunction, negation, implication monotonicity, and eta-equality all transport through the higher-order bridge.

This layer is formalized in `Mettapedia/Logic/PLNHigherOrderHOLCore.lean`, `Mettapedia/Logic/PLNHigherOrderHOLRules.lean`, `Mettapedia/Logic/PLNHigherOrderHOLSoundness.lean`, `Mettapedia/Logic/PLNHigherOrderHOLConsequence.lean`, and `Mettapedia/Logic/PLNHigherOrderHOLLinkBridge.lean`.

The direct set/HOL/WM route is already exercised by genuinely higher-order positive and negative fixtures:

- higher-order reflexivity and eta-equality, both at the object-function level and at the predicate-function level,
- quantified predicate laws such as $\forall P : \iota \to o. \forall x : \iota. P(x) \to P(x)$,
- conjunction/disjunction commutativity as WM query equivalence,
- negation transport,
- multiset consequence transport through higher-order rules,

- concrete countermodels on small carriers, including the failure of $\forall P : \iota \to o. \forall x : \iota. P(x)$ on a two-point carrier.

In other words, the direct route is no longer limited to endofunction-shaped examples. It already supports genuinely quantified predicate-level higher-order queries, predicate-level eta-equality, and WM-side transport of higher-order rules whose principal formulas quantify over predicates.

See `Mettapedia/Logic/PLNWorldModelHOLSetBridgeRegression.lean` and `Mettapedia/Logic/PLNHigherOrderHOLRegression.lean`.

13.6 Where Satisfying-Set Reduction Still Fits

The older satisfying-set reduction remains mathematically useful, but it is now best understood as a *finite extensional fragment* of the higher-order story rather than as the foundational higher-order layer.

Definition 13.3 (Satisfying set [FIN])**.** Given a predicate P over a finite universe U, its satisfying set is $\mathrm{Sat}(P) = \{x \in U \mid P(x)\}$.

Definition 13.4 (Weakness-based inheritance [FIN])**.** For satisfying sets $A, B \subseteq U$ and a weight function $\mu : U \to \mathbb{R}_{\geq 0}$, the inheritance proxy is:

$$\mathrm{Inheritance}(A, B, \mu) = \frac{\mathrm{weakness}(A \cap B, \mu)}{\mathrm{weakness}(A, \mu)}.$$

This finite extensional layer still proves useful structural facts such as pointwise implication collapse and corrected inheritance/similarity transport, but it should not be mistaken for the entire higher-order semantics.

Those finite extensional results live in `Mettapedia/Logic/HigherOrder/HigherOrderReduction.lean` and `Mettapedia/Logic/HigherOrder/Basic.lean`.

13.7 Higher-Order Probability: Now Semantically Grounded

The codebase already contains real higher-order probability theory and a real PLN-to-Kyburg bridge:

- Kyburg flattening and expectation preservation,
- Giry-style monadic laws for the flattening,

- posterior-mean identities linking PLN evidence updates to higher-order probability.

See `Mettapedia/ProbabilityTheory/HigherOrderProbability/KyburgFlattening.lean`, `Mettapedia/ProbabilityTheory/HigherOrderProbability/GiryMonad.lean`, and `Mettapedia/Logic/PLNKyburgReduction.lean`.

What was still missing at the previous checkpoint was the fully semantic object

"probability of higher-order formulas over a measure on Henkin models."

The repository now contains an infinitary-first semantic version of that construction. The new subtree `Mettapedia/Logic/HOL/Probabilistic/` defines:

- measurable indexed spaces of pointed Henkin models in `Mettapedia/Logic/HOL/Probabilistic/ModelSpace.lean`,
- sentence probabilities for closed HOL formulas in `Mettapedia/Logic/HOL/Probabilistic/Semantics.lean`,
- the thin probabilistic WM-facing lens in `Mettapedia/Logic/HOL/Probabilistic/WorldModelBridge.lean`, and
- the theorem that the existing empirical HOL↔WM multiset semantics is a special case in `Mettapedia/Logic/HOL/Probabilistic/EmpiricalSpecialCase.lean`.

So the higher-order picture is no longer merely "real HOL here, real higher-order probability there." It now includes a semantic probability layer for closed HOL formulas. What remains missing is the *full dynamic fusion*: beliefs over higher-order formulas that simultaneously interact with logical-induction-style deductive dynamics and this semantic model-measure layer.

13.7.1 Hierarchical and Infinite-Order Probabilistic HOL

The semantic probabilistic HOL layer now also supports a clean infinite-order reading: uncertainty over *measures on indexed model spaces*, rather than only one flat measure or an explicit syntax of nested "probability of probability of probability." This is the right semantic form for the higher-order probability literature ([27, 10, 2]) and for the current guarded higher-order benchmarks.

The key new files are:

- `Mettapedia/Logic/HOL/Probabilistic/HierarchicalState.lean`, which defines hierarchical uncertainty as a probability distribution over probability measures on a fixed indexed model space;
- `Mettapedia/Logic/HOL/Probabilistic/Flattening.lean`, which proves that hierarchical sentence probabilities are computed by flattening the hierarchy to the predictive marginal;
- `Mettapedia/Logic/HOL/Probabilistic/BenchmarkBridge.lean`, which interprets the current guarded higher-order benchmark payloads as one concrete finite hierarchical instance; and
- `Mettapedia/Logic/HOL/Probabilistic/BenchmarkBeliefBridge.lean`, which derives a benchmark-facing LI-style belief day and process from that semantic benchmark marginal, while proving that this adapter is query-focused rather than a replacement for the benchmark semantics; and
- `Mettapedia/Logic/PLNProbHOLPlannerBridge.lean`, which packages the carried guarded query together with that theorem-backed belief shadow, so planner/control layers can consume semantically justified prices and tracking results without becoming the canonical semantics; and
- `Mettapedia/Logic/PLNProbHOLPlannerBridgeRegression.lean`, which records positive and negative planner-facing bridge fixtures for the concrete higher-order leaky benchmark step; and
- `Mettapedia/Logic/HOL/Probabilistic/HierarchicalRegression.lean`, which packages positive and negative canaries for the infinitary, empirical-special-case, and guarded-benchmark routes.

The semantic picture is:

$$\begin{array}{c} \text{latent regime } \theta \\ \downarrow_{\text{kernel } K(\theta,-) \text{ on model indices}} \\ \text{measure on pointed Henkin-model indices} \\ \downarrow_{\text{flatten / predictive marginal}} \\ \text{sentence probability of } \varphi \\ \downarrow_{\text{thin WM lens}} \\ \text{probabilistic query strength / evidence} \end{array}$$

This is elegant for two reasons. First, it keeps semantic truth in pointed Henkin models primary. Second, it lets finite and empirical cases become

theorem-proved special cases rather than the canonical meaning of higher-order uncertainty.

The current benchmark bridge is intentionally modest: it interprets the existing guarded three-regime payload as the first concrete hierarchical seed. That is already enough to show how branch-level admissibility values become component sentence probabilities, and how the benchmark's flattened value is the predictive marginal of that hierarchy. The broader Pathfinder-style hierarchy over hidden context, admissibility, trust, and topology is the natural next semantic refinement, not an ad hoc tower. For the benchmark itself, this means that second-order uncertainty can live over latent regimes, third-order uncertainty can live over trust in the regime model, and higher orders beyond that should usually be read through flattening, robustness, or abstention criteria rather than through an endlessly explicit public syntax.

The planner-facing consequence is now explicit. A mixed-mode higher-order guarded step can be consumed as a *belief price* precisely because the repository proves that the carried value agrees with the benchmark belief shadow derived from semantic ProbHOL. In Lean this is exposed by `Mettapedia/Logic/PLNProbHOLPlannerBridge.lean`, including theorems `leakyHigherOrderPlan_C_current_value_eq_benchmarkBeliefPrice` and `leakyHigherOrderPlan_C_current_gateConfidence_eq_higherOrderGuardConfidence`. So the control layer is no longer merely carrying a higher-order-looking score: it is carrying a value with an explicit semantic provenance, together with a separate belief-process shadow that tracks the same benchmark query on the designated sample. The paired regression fixtures in `Mettapedia/Logic/PLNProbHOLPlannerBridgeRegression.lean` then records both the positive tracking fixture and the negative expanded-sample guardrail.

13.7.2 Certified Higher-Order Chaining from Estimated Variables

The previous checkpoint still left one gap between semantic higher-order probability and actual higher-order chaining: we had a semantic object and a planner-facing shadow, but not yet a theoremic contract explaining how *estimated* second-/third-order quantities may legitimately drive continue/reveal/fallback decisions. The repository now contains that layer. The guiding principle is deliberately strict. Lean does *not* formalize XGBoost internals, softmax heuristics, or benchmark-specific coefficients. Instead it formalizes the certified interfaces that any empirical estimator must satisfy before its outputs may enter the chaining theorems.

The current theorem set is:

- `Mettapedia/Logic/PLNHigherOrderCertifiedEstimates.lean`, defining `CertifiedAdmissibilityEstimate`, `CertifiedTrustEstimate`, and `CertifiedRegimePosterior`;
- `Mettapedia/Logic/PLNHigherOrderInformationSets.lean`, making the blind/oracle split theorem-visible through `BlindFeatures`, `OracleFeatures`, and `BlindPolicy`;
- `Mettapedia/Logic/PLNHigherOrderVarianceUpdate.lean` and `Mettapedia/Logic/PLNHigherOrderPosteriorUpdate.lean`, formalizing enriched regime uncertainty, law-of-total-variance reveal, and theorem-facing Bayesian posterior update;
- `Mettapedia/Logic/PLNHigherOrderCalibrationContracts.lean`, giving the conformal-style bridge from point predictions to certified admissibility intervals;
- `Mettapedia/Logic/PLNHigherOrderChainBounds.lean`, `Mettapedia/Logic/PLNHigherOrderDecisionTheorems.lean`, and `Mettapedia/Logic/PLNHigherOrderIndependentChainBounds.lean`, which turn those certified interfaces into compositional chain bounds, continue/reveal/fallback/abstain theorems, and optional stronger root-sum-of-squares bounds under explicit assumptions; and
- `Mettapedia/Logic/PLNGWASHigherOrderBridge.lean`, which already instantiates the same abstract machinery for the upcoming fine-mapping/tissue/mechanism/trust GWAS benchmark family.

The central mathematical enrichment is that a higher-order step may now carry both a finite regime posterior and *within-regime variance*. If w_r are regime weights, q_r branch query values, and σ_r^2 within-regime variances, then the theoremic step state satisfies the law of total variance:

$$\underbrace{\mathrm{Var}_{\mathrm{total}}}_{\text{before reveal}} = \underbrace{\sum_r w_r (q_r - m)^2}_{\text{between-regime variance}} + \underbrace{\sum_r w_r \sigma_r^2}_{\text{expected within-regime variance}}, \qquad m = \sum_r w_r q_r.$$

In Lean this is `lawOfTotalVariance`. The reveal action is therefore no longer only a planner heuristic. It has an exact theoremic gain:

$$\text{expected post-reveal variance} = \sum_r w_r \sigma_r^2, \qquad \text{reveal variance reduction} = \sum_r w_r (q_r - m)^2.$$

So reveal is justified precisely when its cost is smaller than the *between-regime* variance it can remove; see `revealPreferred_if_cost_lt_betweenVariance` in `Mettapedia/Logic/PLNHigherOrderVarianceUpdate.lean`.

The post-reveal state is now theoremically modeled as well. A reveal updates the regime posterior by a Bayesian rule

$$\pi(r \mid x, c) = \frac{\pi(r \mid x)\, P(c \mid r)}{\sum_{r'} \pi(r' \mid x)\, P(c \mid r')},$$

and the new theorem set proves validity, concentration under likelihood gaps, and that expected post-reveal variance is no worse than prior variance. This matters because the higher-order controller is no longer forced to talk as if reveal simply "changes the query" by fiat; it can instead refer to an explicit posterior-update law on the latent regime object itself.

The bridge from learned estimates to theoremic certificates is now also clean. A learned point prediction $\hat{g}$ is not inserted directly into the theory. Instead a conformal-style certificate (α, q) yields the calibrated interval

$$[\max(0, \hat{g} - q), \min(1, \hat{g} + q)]$$

with coverage $1-\alpha$ and certified error bound q. The theorem layer only consumes the resulting certified interval object. That is the right separation of concerns: the empirical estimator may be messy, but its theorem-facing use is clean only after explicit calibration or some other certification procedure.

Once these pieces are present, the higher-order variables really can drive PLN-style chaining. Second-order quantities become certified admissibility/regime objects rather than ad hoc guard scores. Third-order quantities become certified trust intervals and penalties that conservatively widen or tighten the chain certificate rather than mystical controller knobs. The default chain laws remain conservative—additive error accumulation and bottleneck-style admissibility—but the repository now also proves optional stronger laws such as `chainError_le_rss_of_quadraticCertificate` under explicit extra assumptions. So the formal story is not "always chain optimistically"; it is "chain as aggressively as your certified higher-order summaries justify".

Most importantly for future benchmark work, the blind/oracle separation is now part of the theorem layer rather than merely a CSV hygiene convention. A blind policy literally has type

$$\texttt{BlindFeatures} \to \texttt{HigherOrderDecision},$$

while oracle-only quantities live in `OracleFeatures`. So the theory can state, by construction, that runtime-blind policies do not have semantic

truthmaker access at action time, even though oracle information may still be used post hoc for evaluation. This is the right discipline for the runtime-blind benchmark lane and for the upcoming GWAS chaining experiments.

The net result is that the higher-order story is now much closer to a full "yes" on the question that motivated this whole reconstruction. Explicit second-/third-order variables can indeed unlock PLN-style chaining, but only when they are consumed through certified composition and decision theorems. We do *not* recover the old unrestricted local truth-value algebra. We get a stronger replacement: semantic higher-order regime uncertainty, theoremic reveal/update laws, calibrated contracts for estimated inputs, typed information-set separation, and GWAS-ready abstract latent coordinates. This is already exercised concretely in `Mettapedia/Logic/PLNHigherOrderCertifiedChainingRegression.lean`, including continuation, reveal, fallback, abstain, conformal-certificate, and GWAS-shaped canaries.

13.8 Logical-Induction-Ready Dynamic Belief Infrastructure

This section describes an experimental overlay on the HOL semantics, not part of the mature HOL metatheory. The interfaces below are scaffolding for future logical-induction-aware probabilistic HOL; they do not yet contain a full logical-inductor construction.

Following Logical Induction [11], the next higher-order uncertainty layer should not be hard-coded as one static measure or one specific prover. Instead it should be parameterized by:

- a deductive process over coded closed HOL formulas,
- time-indexed belief states over those formulas,
- theory-extension/conditioning operators, and
- calibration and timely-learning criteria that remain distinct from semantic truth in Henkin models.

This interface layer is now explicit in the codebase. The new subtree `Mettapedia/Logic/HOL/LogicalInduction/` contains:

- canonical coding of closed HOL formulas in `Mettapedia/Logic/HOL/LogicalInduction/Code.lean`,

- deductive-process and theory-extension interfaces in `Mettapedia/Logic/HOL/LogicalInduction/DeductiveProcess.lean` and `Mettapedia/Logic/HOL/LogicalInduction/Conditioning.lean`,
- day-indexed market/belief vocabulary plus trader and exploitation criteria in `Mettapedia/Logic/HOL/LogicalInduction/Market.lean` and `Mettapedia/Logic/HOL/LogicalInduction/Criterion.lean`,
- calibration and timely-learning specification hooks in `Mettapedia/Logic/HOL/LogicalInduction/Calibration.lean`,
- the artifact-only HOL-to-two-sort-experiment compatibility layer in `Mettapedia/Languages/MeTTa/Experimental/TwoSortPiSigmaId/Adapters/HOLLogicalInductionBridge.lean`, and
- a thin WM-facing day interface together with the theorem that the existing multiset-counting HOL WM semantics is its empirical special case in `Mettapedia/Logic/HOL/LogicalInduction/WorldModelBridge.lean` and `Mettapedia/PLN/Bridges/HOL/LogicalInduction/EmpiricalSpecialCase.lean`.

The corresponding lightweight top-level wrapper is now `Mettapedia/Logic/ProbHOL.lean`. It re-exports both:

- the new semantic probabilistic HOL layer, and
- the logical-induction-ready dynamic belief layer.

It still explicitly avoids the stronger claim that the repository already contains the fully fused dynamic theory of logical-induction belief processes over semantic probabilities of Henkin-model events.

There is now also one precise benchmark-facing bridge back from semantic probability to LI-style pricing. The file `Mettapedia/Logic/HOL/Probabilistic/BenchmarkBeliefBridge.lean` constructs a benchmark belief day whose visible price on the benchmark query is the carried guarded higher-order value *because* that value is already proved equal to the hierarchical semantic benchmark marginal. The key tracking theorems are `benchmarkBeliefDay_tracks_benchmarkHierarchicalProbOn` and `benchmarkBeliefProcess_event`. Just as importantly, the negative theorem `benchmarkBeliefDay_not_tracks_benchmarkHierarchical` shows that this adapter is intentionally narrow: it prices the benchmark query correctly, but it is not presented as a global belief oracle for unrelated HOL sentences.

The generic comparison waist is now explicit in `Mettapedia/Logic/HOL/Probabilistic/BeliefBridge.lean`. Its central predicates are `BeliefDayTracksHierarchicalProbO` and `BeliefProcessEventuallyTracksHierarchicalProbOn`: these say exactly when a day-level price assignment or a time-indexed belief process matches the semantic hierarchical ProbHOL query strength on a chosen finite sample of coded HOL formulas. The benchmark layer then instantiates those generic predicates, rather than inventing a second benchmark-only notion of tracking. In Lean this is visible through theorems such as `benchmarkBeliefDay_tracks_benchmarkLatentHierarchicalProbOn` in `Mettapedia/Logic/HOL/Probabilistic/BenchmarkBeliefBridge.lean` and `benchmarkPlannerShadow_process_` in `Mettapedia/Logic/PLNProbHOLPlannerBridge.lean`. So the intended four-layer example is now theorem-traceable:

HOL truth for $\varphi \implies$ hierarchical semantic probability of $\varphi \implies$ belief-process tracking on the bench

Here the richer benchmark latent coordinates for hidden context, trust, and topology live explicitly in `Mettapedia/Logic/HOL/Probabilistic/BenchmarkBridge.lean`; the bridge to planner control remains downstream of the generic semantic-vs-belief layer rather than becoming a new semantics in its own right.

truth side	**belief side**
closed HOL formulas	coded closed HOL formulas
$\downarrow$Henkin semantics	$\downarrow$deductive process D_n
truth in pointed Henkin models	belief days $P_n(\varphi)$
$\downarrow$measurable indexed model spaces	$\downarrow$day-level WM-facing strength
semantic sentence probability $\mu\{i \mid M_i \vDash \varphi\}$	belief-day strength/evidence
$\downarrow$empirical multiset special case	dynamic belief/evidence interface
static HOL $\leftrightarrow$ WM strength	

The important architectural point is that this does *not* replace the existing HOL truth layer. Henkin semantics in `Mettapedia/Logic/HOL/Semantics/Henkin.lean` still determines semantic truth, and `Mettapedia/Logic/HOL/WorldModel.lean` still provides the static world-model lens. The new logical-induction-ready layer is an overlay for future deductively limited belief dynamics about closed HOL formulas.

So Chapter 13 now has three clearly separated higher-order levels:

1. semantic truth in Henkin HOL,
2. semantic probability over measurable indexed model spaces of pointed Henkin models,

3. the empirical HOL↔WM strength layer as a theorem-proved special case, and

4. dynamic belief-process scaffolding for future logical-induction-aware probabilistic HOL.

This is the right place to stop for now. It is already enough to keep future ProbHOL and higher-order uncertain reasoning from being forced through a hard-coded prover or a single empirical model sample, but it does *not* yet claim a full logical-inductor construction or the full dynamic fusion of belief processes with semantic probabilities over Henkin models.

13.9 Current HOL Completeness Status

For clarity, we summarize the current state of the HOL metatheory:

Done. Typed Church-style syntax, natural-deduction proof calculus with extensional overlay, Henkin semantics, and soundness.

Done. Witnessed extension/saturation, prime/maximal/classical-world construction, general term-model assembly, and the classical Henkin model-existence theorem setd by `Mettapedia/Logic/HOL.lean` and `Mettapedia/Logic/HOL/TermModel/HenkinCompleteness.lean`.

Done. The HOL↔WM bridge, semantic ProbHOL, hierarchical probabilistic HOL, certified higher-order chaining, and the inference-control theorem set all run over that live classical Henkin base.

Future work. Intuitionistic completeness and other non-classical strengthenings of the HOL metatheory.

Future work. Stronger semantic classes beyond the active Henkin route, together with fuller fusion of semantic ProbHOL and belief-process dynamics.

The active higher-order foundation is therefore no longer "canonical model pending." It is the landed classical Henkin route. The HOL↔WM bridge, semantic ProbHOL, hierarchical probabilistic HOL, certified higher-order chaining, and the inference-control theorem set do not depend on further completeness strengthening beyond that route.

Chapter 14

Quantifiers and the Fuzzy/QFM Layer

Goal: Formalize quantified reasoning with finite-domain and fuzzy semantics.
Layer: Extension (quantifiers).
Semantic object: Finite-domain quantifier bundle; fuzzy quantifier algebra; QFM.
Proved: Quantifier soundness/canary package for finite domains.
Assumed: Finite domain ($|U| < \infty$); see global scope contract in Section 1.4.

Extension template: *State:* Finite-domain world model with explicit universe U. *Query:* Quantified formulas $\forall x.\, \phi(x)$, $\exists x.\, \phi(x)$; fuzzy quantifiers. *Extract:* Counting reduction over U; fuzzy quantifier via QFM. *Kernel laws:* Additive over domain elements (finite sum). *Compiled shards:* Quantifier soundness/canary package. *Limitations:* Finite domain only; infinitary quantifiers deferred.

PLN extends beyond propositional inference to quantified reasoning. This chapter covers the stable finite-domain quantifier bundle, fuzzy quantifier algebra, and their ITV-coordinate transport. The codebase also contains an arbitrary-domain infinitary semantic layer in `Mettapedia/PLN/RuleFamilies/FirstOrder/Quantifiers/Infinite.lean`; what remains finite-domain here is the reviewed soundness/canary package for this chapter, not the whole WM calculus.

14.1 Finite-Model Quantifier Semantics

In a finite domain $U = \{u_1, \ldots, u_m\}$ with weight function $\mu : U \to \mathbb{R}_{\geq 0}$, quantifiers receive a weakness-based semantics:

Definition 14.1 (Universal quantifier evaluation [FIN] (CORE))**.** For a predicate set $S \subseteq U$:

$$\forall_\mu(S) = \text{weakness}\big(\mu,\ \text{diag}(S)\big) = \sum_{u,v \in S} \mu(u) \cdot \mu(v)$$

where $\text{diag}(S) = \{(u, v) \mid u \in S \wedge v \in S\}$. Under uniform μ, this reduces to $|S|^2/|U|^2$, the squared fraction of the domain satisfying S.

The existential quantifier is defined dually, following the classical de Morgan pattern: to ask whether *something* in the domain satisfies S is to ask whether it is *not* the case that *everything* satisfies $\neg S$.

Definition 14.2 (Existential quantifier evaluation [FIN] (CORE))**.** By de Morgan reduction:

$$\exists_\mu(S) = \text{compl}\big(\forall_\mu(\neg S)\big)$$

The weakness-based semantics has an immediate structural consequence: universal quantification is always weaker than existential quantification, because affirming a predicate everywhere is harder than affirming it somewhere.

Theorem 14.3 (Quantifier ordering [FIN] (CORE))**.** *For any predicate set S and weight μ:*

$$\forall_\mu(S) \leq \exists_\mu(S)$$

For comparison, the purely extensional quantifiers use lattice operations—$\forall^{\text{ext}}(S) = \bigwedge_u S(u)$ and $\exists^{\text{ext}}(S) = \bigvee_u S(u)$—which are recovered as special cases when the weight μ is uniform and the domain is finite.[1]

14.2 Fuzzy Quantifier Semantics

Beyond crisp quantifiers, PLN supports fuzzy quantifiers ("most," "few," "about half") via a QFM (Quantitative Fuzzy Modifier) approach.

[1] Formalized in `Mettapedia/PLN/RuleFamilies/FirstOrder/Quantifiers/QuantifierSemantics.lean` and `Mettapedia/PLN/RuleFamilies/FirstOrder/Quantifiers/Soundness.lean`.

Definition 14.4 (Fuzzy quantifier parameters [FIN])**.** A fuzzy quantifier specification $\mathcal{P}$ consists of:

$$\varepsilon \in [0,1] \qquad \text{proxy tolerance around } 0/1 \tag{14.1}$$

$$L, U \in [0,1],\ L \le U \qquad \text{lower/upper proxy confidence} \tag{14.2}$$

$$\theta \in [0,1] \qquad \text{proxy confidence threshold} \tag{14.3}$$

The "near-one" and "near-zero" predicates are:

$$\text{nearOne}_\varepsilon(x) \iff 1-\varepsilon \le x \le 1, \qquad \text{nearZero}_\varepsilon(x) \iff 0 \le x \le \varepsilon$$

Taken together, these four parameters control the granularity of the fuzzy quantifier: ε sets how close a membership value must be to 0 or 1 to count as "essentially false" or "essentially true," L and U bracket the proxy confidence interval, and θ sets the acceptance threshold. The key derived notion is the *witness fraction*—the proportion of domain elements whose membership values fall in the "near-one" band.

Definition 14.5 (Witness fraction [FIN])**.** For a predicate ϕ on U:

$$\text{witFrac}(\phi) = \frac{|\{u \in U \mid \phi(u)\}|}{|U|}$$

The fuzzy-$\forall$ holds when the near-one witness fraction exceeds the threshold: $\theta \le \text{witFrac}(\text{nearOne}_\varepsilon \circ f)$ where $f : U \to [0,1]$ is the membership profile.

QFM composition operators: multiplicative ($x \cdot y$), minimum ($\min(x,y)$), Łukasiewicz ($\max(0, x+y-1)$), and probabilistic sum ($x+y-xy$) provide graded conjunction/disjunction with ITV-coordinate transport.[2]

Arbitrary-domain graded fuzzy layer. The repository now also exposes a reusable arbitrary-domain graded specialization layer. Instead of treating the Sugeno/proxy-cut and Choquet branches as unrelated developments, Lean factors out their honest common spine via `Mettapedia/PLN/RuleFamilies/FirstOrder/Quantifiers/GradedQuantifierSpecialization.lean`. The generic interface `GradedQuantifierSemantics` packages score-based interval and universal predicates, complement-dual existential predicates, fuzzy-domain restriction via `domainRestrict`, and equality/monotonicity transport on the restricted domain. The current Sugeno and Choquet branches are then explicit instances (`sugenoGradedQuantifierSemantics`,

[2]Formalized in `Mettapedia/PLN/RuleFamilies/FirstOrder/Quantifiers/FuzzyQuantifierSemantics.lean` and `Mettapedia/PLN/RuleFamilies/FirstOrder/Quantifiers/FuzzyITVBridge.lean`.

choquetGradedQuantifierSemantics), while the fuzzy-domain layer is a further specialization of that shared graded interface rather than a duplicated theory. Direct graded canaries now live in Mettapedia/PLN/RuleFamilies/FirstOrder/Quantifiers/GradedQuantifierCanary.lean, so the shared graded spine is itself exercised by positive and negative examples, not only through the specialized fuzzy-domain wrappers.

14.3 Scope Note: Stable Finite-Domain Bundle vs. Infinitary Semantics

This chapter's theorems use finite domains (see the global scope contract in Section 1.4). The codebase already contains an arbitrary-domain infinitary layer[3] with basic definitions and laws. Full parity between the finite-domain chapter bundle and the infinitary layer—stronger metatheory, canary/regression coverage, and measure-theoretic extensions—remains future work.

[3] WeightFunctionInf, SatisfyingSetInf, forAllEvalInf, thereExistsEvalInf, deMorgan_inf, weaknessInf_mono in Mettapedia/PLN/RuleFamilies/FirstOrder/Quantifiers/Infinite.lean.

Chapter 15

Intensional Inheritance

Goal: Ground intensional inheritance in typed world-model semantics.
Layer: Extension (intensional).
Semantic object: `InheritanceSort` (extensional, ASSOC, PAT, mixed); Solomonoff bridge.
Proved: No-go: no binary mixed policy reproduces both intensional behaviors; typed WM grounding.
Assumed: Finite domain.

Extension template: *State:* Typed world model with `InheritanceSort` annotations. *Query:* Intensional inheritance links with sort tags (extensional, ASSOC, PAT, mixed). *Extract:* Sort-specific evidence extraction; Solomonoff bridge for complexity-weighted priors. *Kernel laws:* Additive revision per sort; extensional sort reduces to standard WM. *Compiled shards:* Typed WM grounding; sort-tagged motifs. *Limitations:* No binary mixed policy covers both intensional behaviors (no-go).

Intensional inheritance is PLN's mechanism for reasoning about relationships based on shared properties (intension) rather than shared instances (extension). This is one of the most ambitious parts of the PLN book, and one where the formalization required the most careful treatment.

15.1 What Intensional Inheritance Captures

Two concepts A and B have high intensional similarity when they share many properties—when things that are true of A tend also to be true of B. The original PLN book defines intensional inheritance as a mixture of

extensional and intensional components, with the mixture controlled by a parameter.

Maple Court: In Maple Court, apartments share structural properties: they all have kitchens, bathrooms, plumbing risers, and the same building infrastructure. Intensional similarity between Apartment 3B and Apartment 5C is high—their property profiles overlap almost completely (both are west-facing corner units with similar heat and humidity patterns). But extensional inheritance diverges: Apartment 3B may have a pipe leak while 5C does not. The typed WM grounding resolves this cleanly: extensional queries use per-apartment evidence, while intensional queries aggregate over shared property patterns via the ASSOC channel. The garden robot benefits similarly: "the rooftop herb bed is like the ground-floor herb bed" (shared sunlight and watering profiles) enables transfer learning of care policies even when the two beds have different pest histories.

15.2 Typed WM Grounding

The WM-PLN approach grounds intensional inheritance in the world-model framework through a typed query family:

Definition 15.1 (Inheritance sort [FIN])**.** Queries are indexed by their semantic channel:

InheritanceSort = extensional | intensionalAssoc | intensionalPAT | mixed

Each sort has its own evidence extraction and strength computation.

The ASSOC channel computes association strength via mutual information. Given features F and a world-model state W:

$$P_{\text{ASSOC}}(W \mid F) = P(W) \cdot 2^{I(F;W)}$$

where $I(F;W) = \sum_{f,w} p(f,w) \log \frac{p(f,w)}{p(f)\cdot p(w)}$ is the Shannon mutual information between the feature profile and the world state. High ASSOC inheritance between A and B means that knowing which properties A has gives strong predictive power for which properties B has.

The PAT (pattern) channel provides a complementary structural route based on property-pattern proximity: two concepts A, B have high PAT similarity when they share structural patterns beyond what mutual-information association captures.[1]

15.3 Solomonoff Bridge

The Solomonoff bridge connects intensional inheritance to algorithmic information theory:

- The exact formal bridge is the universal-mixture log-ratio theorem: $\mathrm{InhInt}_\xi(W, F, x) = \log_2\big(P_\xi(W \mid F, x)/P_\xi(W \mid x)\big)$ when both conditionals are positive.
- Specialized `xiSemimeasure` and geometric-mixture versions are proved as explicit theorem endpoints.
- This supports the algorithmic-information interpretation of intensional inheritance, but the primary theorem statement is the log-ratio bridge rather than a standalone normalized-information-distance identity.

See `Mettapedia/Logic/IntensionalInheritanceSolomonoffBridge.lean`.

15.4 Mixed Semantic Packaging

The `AssocPatSemanticModel` packages ASSOC and PAT channels into a unified semantic model with selector-specialized endpoints. The original PLN book proposes a simple mixture:

$$\mathrm{InhInt}(A, B) = \alpha \cdot \mathrm{Ext}(A, B) + (1 - \alpha) \cdot \mathrm{IntAssoc}(A, B)$$

with a single mixing parameter α. The formalization explicitly refutes this as a general law:

Theorem 15.2 (No binary mixed policy (Boundary))**.** *There exists a concrete state W_{demo} with evidence scores $(3, 2, 1)$ for (extensional, ASSOC, PAT) channels such that the PAT channel contributes nontrivially (*pat_channel_nontrivial*) and the mixed result cannot be expressed as any function of only the extensional and ASSOC values (*mixed_not_assoc_only*).*

[1] Formalized in `Mettapedia/Logic/PLNIntensionalWorldModel.lean`.

This means the book's binary mixture is not merely incomplete but false as a general chapter-level law: the full three-channel model (extensional, ASSOC, PAT) is needed. The corrected theorem set is therefore a typed three-channel mixed rule, not a repaired version of the old binary extensional+ASSOC law.[2]

[2]Formalized in `Mettapedia/Logic/PLNIntensionalCanary.lean`, `Mettapedia/Logic/PLNIntensionalAssocPatClosure.lean`, and `Mettapedia/Logic/PLNIntensionalRegression.lean`.

Chapter 16

Temporal and Causal Inference

Goal: Formalize temporal operators and causal profiles within the WM framework.
Layer: Extension (temporal/causal).
Semantic object: Temporal operators; causal profile; event calculus bridge.
Proved: Temporal as a WM instance; causal profile soundness.
Assumed: Discrete time steps.

Extension template: *State:* Time-indexed world-model state (sequence of posteriors). *Query:* Temporal formulas (before/after/during); causal queries (do-calculus). *Extract:* Time-slice extraction; causal profile via intervention semantics. *Kernel laws:* Revision additive per time step; temporal evidence accumulates. *Compiled shards:* Event calculus bridge; causal profile soundness. *Limitations:* Discrete time only; continuous-time deferred.

Temporal and causal inference in PLN addresses reasoning about events ordered in time and about cause-effect relationships.

16.1 Temporal Operators

Temporal predicates live in the type $\text{TemporalPred}(D, T) = D \to T \to \text{Prop}$, where D is the domain of entities and T is the time type (typically $\mathbb{N}$ or $\mathbb{R}$). The temporal algebra is built in three layers: simultaneous operators that combine predicates at the same time point, sequential operators that require one predicate to hold now and another to hold at a future time, and predictive implication that captures the conditional "if P now, then Q later."

Definition 16.1 (Simultaneous operators (CORE))**.**

$$\text{SimAND}(P, Q)(x, t) := P(x, t) \wedge Q(x, t) \tag{16.1}$$

$$\text{SimOR}(P, Q)(x, t) := P(x, t) \vee Q(x, t) \tag{16.2}$$

The simultaneous operators are the temporal analogue of ordinary logical connectives—they simply ask whether two properties co-occur at the same moment. The sequential operators introduce temporal displacement:

Definition 16.2 (Sequential operators (CORE))**.**

$$\text{SeqAND}(P, Q)(x, t) := P(x, t) \wedge \exists \Delta > 0.\ Q(x, t + \Delta) \tag{16.3}$$

$$\text{SeqAND}_f(P, Q)(x, t) := P(x, t) \wedge \exists \Delta.\ f(\Delta) \wedge Q(x, t + \Delta) \tag{16.4}$$

where f is a "future" predicate constraining the delay (e.g., $\Delta \in [1, T_{\max}]$).

Predictive implication lifts sequential conjunction to a conditional form—the temporal analogue of $A \Rightarrow B$, asserting that if P holds now, then Q will hold at some future time:

Definition 16.3 (Predictive implication (CORE))**.**

$$\text{PredImpl}(P, Q)(x, t) := P(x, t) \Rightarrow \exists \Delta > 0.\ Q(x, t + \Delta)$$

The 3-node predictive chain composes: given temporal bounds T_1, T_2,

$$\text{PredChain}_3(T_1, T_2, A, B, C)(x, t) := A(x, t) \wedge B(x, t + T_1) \Rightarrow C(x, t + T_2)$$

16.2 Causal Profile

Causal inference is grounded through a probabilistic event calculus built on four event predicates: Happens(e, t), Initiates(e, f, t), Terminates(e, f, t), and HoldsAt(f, t). For a pair of predicates (A, B), the *causal profile* collects the predictive implication strengths across temporal lags: CausalProfile$(A, B) = \{\mathsf{strength}(\text{PredImpl}_\Delta(A, B)) \mid \Delta \in [1, T_{\max}]\}$. This profile provides the raw material for distinguishing genuine causation from spurious correlation: Granger-style temporal precedence, combined with the event calculus, separates $A \to B$ (causal) from $A \leftarrow C \to B$ (confounded) within the world-model framework.[1]

[1]Formalized in `Mettapedia/PLN/RuleFamilies/Temporal/PLNTemporalCausalInference.lean` and `Mettapedia/PLN/RuleFamilies/Temporal/PLNProbabilisticEventCalculus.lean`.

16.3 Temporal as Another WM Instance

The key design decision: temporal and causal inference is *not* a separate ontology. It is another instantiation of the world-model framework. The world-model state includes temporal structure (event histories), the query language includes temporal operators (SimAND, SeqAND, PredImpl), and the evidence extraction/revision machinery operates unchanged. This keeps the foundational theory unified: a single kernel, many instantiations.

Part VI

Large-Scale Inference, Control, and Boundaries

Chapter 17

Large-Scale Inference at Theorem Level

Goal: Formalize the theorem-level core of large-scale PLN inference.
Layer: Control (large-scale).
Semantic object: Multideduction; trail-free dynamics; pooling; evidence conservation.
Proved: Identifiability no-go; residual decomposition; non-stabilization counterexample.
Assumed/operational: Operational scheduling strategies (counterexample-first; operational pending).

Chapter 9 of the PLN book discusses large-scale inference strategies: batch inference, multideduction, BN augmentation, trail-free protocols. This is the most complex chapter to formalize, and the most instructive about the gap between theoretical correctness and operational feasibility.

17.1 Multideduction and Identifiability

Multideduction—applying multiple deduction steps and combining results—runs into a fundamental identifiability problem.

Theorem 17.1 (Inclusion-exclusion identifiability no-go (Boundary))**.** *Two-term inclusion-exclusion statistics are not sufficient to identify the union mass. There exist distinct probability distributions agreeing on all first-two-term statistics but disagreeing on the union probability.*

The corrected positive result:

Theorem 17.2 (Residual decomposition (CONDITIONAL))**.** *The union cardinality decomposes as the two-term estimate plus a residual. If the residual is known (or bounded), the corrected estimate agrees with the true value.*

See `Mettapedia/Logic/PLNMultideductionResidual.lean` and `Mettapedia/Logic/PLNInclusionExclusionIdentifiability.lean`.

17.2 Trail-Free Protocol Dynamics

The trail-free deterministic update protocol can fail to stabilize:

Theorem 17.3 (Trail-free non-stabilization (BOUNDARY))**.** *There exists a two-state system where the trail-free deterministic update enters a 2-cycle and never reaches a fixed point.*

The counterexample constructs a concrete 2-state system where the protocol alternates between two states indefinitely. The root cause is that without a trail (a record of which evidence has already been incorporated), the protocol re-processes the same evidence in opposite order on each step, producing an oscillation.

The positive result under damping:

Theorem 17.4 (Damped convergence (CONDITIONAL))**.** *Under damping and eventual fresh evidence, the SP/SPN protocol eventually stabilizes. Under bounded-gap fairness, stabilization occurs within an explicit bound.*

Damping (scaling each update by $\lambda < 1$) breaks the cycle: old evidence becomes geometrically less influential, so the system converges even without trail-tracking. This pair of theorems shows the sharp boundary: trail-free without damping can diverge; trail-free with damping always converges.[1]

17.3 Pooling and Fusion

A natural question is whether the abstract pooling axioms—commutativity, associativity, and monotonicity—suffice to pin down additive evidence fusion. They do not: the max-pooling operator satisfies every one of these axioms yet produces a completely different fusion rule.

[1] Formalized in `Mettapedia/PLN/InferenceControl/ProtocolDynamics/PLNTrailFreeDynamicsCounterexample.lean` and `Mettapedia/PLN/InferenceControl/ProtocolDynamics/PLNTrailFreeDampedConvergence.lean`.

Theorem 17.5 (Pooling non-uniqueness (BOUNDARY))**.** *The max-pooling operator satisfies the pooling axioms but differs from additive fusion.*

The lesson is that additional structural constraints are needed. Specifically, external Bayesianity (the requirement that pooling agents' posteriors combine as if they shared a common prior) together with total additivity pins down exactly one fusion rule: ordinary additive pooling.

Theorem 17.6 (Pooling uniqueness (CONDITIONAL))**.** *External Bayesianity plus total additivity uniquely recovers additive evidence pooling.*

This pair of theorems mirrors a recurring theme in the PLN refoundation: familiar intuitions ("evidence should combine commutatively") are necessary but not sufficient, and the precise additional assumptions that restore uniqueness must be made explicit.[2]

Maple Court: Maple Court's three WM layers instantiate evidence pooling concretely. Each apartment exports evidence summaries—not raw sensor traces—to the building WM, which pools them via additive fusion. The pooling-uniqueness theorem justifies this: under external Bayesianity and total additivity, additive pooling is the unique fusion rule. At the *community scale* (introduced here for the first time), multiple Maple Court–style buildings export building-level summaries to a shared district WM. The forgetting hierarchy operates at different timescales: apartment WMs forget in hours (shower steam evidence fades quickly), the building WM in days (laundry demand patterns shift weekly), and the community WM in months (seasonal heat-loss trends). The privacy export theorem (`export_is_lossy`) guarantees that the community WM cannot reconstruct apartment-level occupancy from the aggregated summaries.

17.4 Order-Cost Governance

The evidence-conservation theorems for forgetting are developed in Chapter 9 (Theorem 9.1). This section addresses a complementary concern: what is the cost of executing revisions *in the wrong order*? When revision is not purely commutative—or when an operational schedule reorders evidence arrival—the system needs bounds on the resulting error.

[2] Formalized in `Mettapedia/PLN/InferenceControl/PremiseSelection/ExternalBayesianity.lean`.

Theorem 17.7 (Order-cost budget bound (CONDITIONAL))**.** *If each adjacent swap in a schedule is bounded by a per-step anomaly budget, then total schedule error is bounded by the sum of those budgets. In particular, commutative merge-evidence implies zero swap anomaly and zero schedule error.*

Finally, these theoretical bounds must be checkable at runtime. The audit certificate theorem shows that a simple executable check—comparing pairwise order costs against a budget vector—is sufficient to discharge the formal guarantees in Lean:

Theorem 17.8 (Runtime audit certificate soundness (CONDITIONAL))**.** *If runtime pairwise-order checks pass with budget vector B, then the formal schedule-error bound and budget-threshold policy conclusions follow in Lean. For two-step schedules, a single swap-step check certifies the two-step policy threshold.*

Lean endpoints:

- `evidenceConservationPack_of_forgetting` in `Mettapedia/Logic/PLNWorldModelConservationPack.lean`
- `scheduleErrorBound_of_pairwise_bounds` and `scheduleErrorBound_twoStep_of_swapStepB` in `Mettapedia/PLN/WorldModel/OrderCost/PLNWorldModelOrderCostBounds.lean`
- `runtimePairwiseOrderCheck_certifies_scheduleErrorBound` and `runtimePairwiseOrderCheck_certifies_policyThreshold` in `Mettapedia/PLN/WorldModel/OrderCost/PLNWorldModelOrderCostAuditCertificate.lean`

Concrete demos. The weighted numeric lane proves nontrivial non-commutative behavior (`weightedScheduleErrorBound_twoStep_not_zero_of_pos_weight`), and the gas governance lane proves certified zero-budget pass on batch swap under additive merge (`gasOrderBudgetPolicy_batchSwap_zero_via_runtimeCertificate`).[3]

17.5 BRGI and Feasible Sets

The Balanced Relevance-Guided Inference (BRGI) framework provides an optimization-theoretic perspective:

[3] Formalized in `Mettapedia/PLN/WorldModel/OrderCost/PLNWorldModelOrderCostWeightedDemo.lean` and `Mettapedia/PLN/WorldModel/OrderCost/PLNWorldModelOrderCostGasPolicyDemo.lean`.

- Finite feasible sets with policy-sensitive optimum characterization.
- Graph-level BRGI objects with optimality-preserving refinement.

See `Mettapedia/PLN/InferenceControl/PremiseSelection/BRGI.lean`.

17.6 Stochastic Experiment Layer

Blackwell factorization, weighted source policies, and trusted-gate transfer provide a formal framework for reasoning about the value of experimental evidence:

- Expected utility equals the Blackwell factor.
- Optimal value is monotone under Blackwell domination.
- Trusted-gate transfer preserves optimality ordering.

See `Mettapedia/PLN/WorldModel/Experiment/PLNWorldModelExperimentStochastic.lean`.

Chapter 18

Inference Control

Goal: Formalize premise selection and coverage bounds for inference scheduling.
Layer: Control.
Semantic object: Selector specifications; coverage/submodularity; surrogate boundaries.
Proved: Coverage surrogate no-go; composed core endpoints.
Assumed: Finite domain; finite knowledge base.

Inference control addresses the meta-level problem: given a large knowledge base and many possible inference steps, which steps should be taken?

18.1 Selector Specifications

A *selector* is a function that chooses premises for inference. The formalization provides formal selector specifications with typed interfaces, together with two robustness results. First, ranking transfer: if a selector produces a good ranking on one distribution, it produces a good ranking on similar distributions. Second, ranking stability: small perturbations in the knowledge base produce small perturbations in the ranking. These two properties ensure that a selector trained on one corpus does not catastrophically fail when deployed on a slightly different one—a concern that is entirely absent from the original PLN book but becomes pressing in any practical system.[1]

[1] Formalized in `Mettapedia/PLN/InferenceControl/PremiseSelection/SelectorSpec.lean` and `Mettapedia/PLN/InferenceControl/PremiseSelection/RankingStability.lean`.

18.2 Coverage and Submodularity

Let D be the set of true dependencies and S the selected premise set. The coverage objective is:

$$f(S) = |S \cap D|$$

Theorem 18.1 (Coverage submodularity [FIN] (CORE))**.** *The function f is monotone ($S \subseteq T \Rightarrow f(S) \leq f(T)$) and submodular: for $S \subseteq T$ and any element $x \notin T$:*

$$f(T \cup \{x\}) - f(T) \;\leq\; f(S \cup \{x\}) - f(S)$$

By the classical result of Nemhauser, Wolsey, and Fisher, the greedy algorithm (iteratively adding the element with maximum marginal gain) achieves a $(1 - 1/e) \approx 0.632$ approximation ratio for monotone submodular maximization under cardinality constraints. This guarantee is formalized via an explicit `GreedyChain` inductive construction.[2]

Maple Court: The hallway cleaning robot illustrates the inference-control pipeline end-to-end. It must decide which floor to clean first, given limited battery. It queries the building WM for *HallwayHumidity* on each floor and *LaundryState* (laundry traffic correlates with hallway mess). The greedy coverage selector picks the floor with maximum marginal information gain. A Σ-guarded rewrite infers "hallway needs cleaning" from the humidity and traffic evidence, firing only when d-sep conditions are met. The $(1 - 1/e)$ coverage bound guarantees that the robot's greedy floor-selection is near-optimal. The garden robot faces the same control problem over garden beds: which bed to water first, given soil moisture evidence from each.

18.3 Coverage Surrogate Boundaries

A natural simplification would be to replace the joint coverage function $f(S) = |S \cap D|$ with a per-premise score (e.g., each premise's individual relevance) and then optimize the surrogate instead. The following theorem shows this is impossible in general:

[2] Formalized in `Mettapedia/PLN/InferenceControl/PremiseSelection/Coverage.lean`.

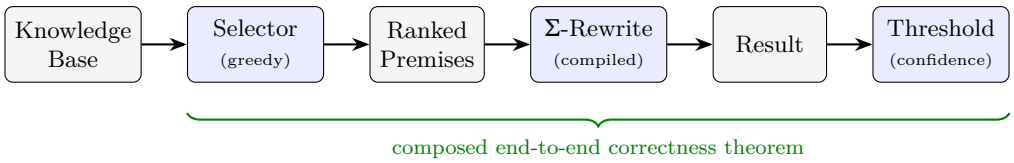


Figure 18.1: The inference-control pipeline. A selector chooses premises from the knowledge base; a Σ-guarded compiled rewrite produces a result; a threshold accepts or rejects based on confidence. The composed pipeline has a single end-to-end correctness theorem.

Theorem 18.2 (Coverage/cardinality surrogate no-go [FIN] (BOUNDARY))**.** *Coverage and cardinality together are insufficient for risk-sensitive selection: there exist premise sets with identical coverage and cardinality but different false-positive risk.*

The counterexample constructs two premise sets that look identical to any local surrogate (same size, same coverage count) but differ in which dependencies they hit. One set covers the high-risk dependencies; the other covers low-risk ones. This is why the greedy algorithm must evaluate the actual joint coverage function at each step, not a decomposable proxy.[3]

18.4 Composed Core Endpoints

The one-call selector → rewrite → threshold composition:

`selector_rewrite_threshold_end_to_end_of_interval`

composes the entire pipeline into a single theorem: given a selector that produces a ranked premise set, a rewrite that applies a PLN rule under BN side conditions, and a threshold that accepts the result, the composed pipeline is correct.[4]

[3]Formalized in `Mettapedia/PLN/InferenceControl/PremiseSelection/CoverageCounterexamples.lean`.

[4]Formalized in `Mettapedia/PLN/Core/PLNCanonicalAPI.lean`.

Part VII

Typed Realization of the Kernel

Chapter 19

The Kernel as a Typed Language Family

Goal: Develop the typed realization of the posterior-state calculus previewed in Section 4.5: the full 13-axis vertex family, OSLF modal operators, encoding adequacy, transport theorems, and native type synthesis per vertex.
Layer: Typed realization.
Semantic object: 13-axis vertex family; per-vertex `LanguageDef`; forward fiber via rule-subset monotonicity; automatic $\Diamond \dashv \Box$ Galois connection; native type synthesis exact on 9 syntactic axes.
Proved (kernel-checked): All items in this chapter. 0 sorry.
Assumed: None beyond the OSLF/TypeSynthesis infrastructure.

Throughout the preceding chapters, the world-model posterior-state calculus appears in many guises: additive revision (Chapter 4), overlap-corrected fusion, scope-based forgetting, support-tracked inverse forgetting, Kripke provenance tracking, fixpoint closure, conservation bounds, experiment channels, stochastic decision theory, and generic carriers. Each variant shares the same core axioms—evidence additivity, revision commutativity and associativity, combine commutativity—but adds its own operations and equational laws.

This chapter shows that this family of variants has a precise algebraic structure. The story proceeds in five layers:

1. **Algebraic core** (§19.1): the 5 core rules are derived from a single universal property (extract is an AddCommMonoid homomorphism), and the 2×2 calculus square (raw/guarded × plain/+cong) organizes all variants.

2. **Intrinsic encoding** (§19.2): the sort-indexed WMTerm type embeds bijectively into patterns, with biconditional single-step and star adequacy, and image-restricted backward completeness.

3. **Vertex family** (§19.3): the 13-axis vertex space, extension families, preorder, and forward fiber functor.

4. **OSLF layer** (§19.4): automatic $\Diamond/\Box$ derivation, semantic bridges to evidence theory, neighborhood semantics, and FOL.

5. **Native type synthesis** (§19.5): what the NTT pass sees and what it does not.

Two further sections address compiled BN inference (§19.6) and the formalization inventory (§19.7).

19.1 The Algebraic Core

19.1.1 Pattern Vocabulary

The WM calculus is encoded in the OSLF pattern language using three core sorts—**State**, **Query**, **Evidence**—and constructors corresponding to the calculus operations:

Constructor	Signature	Operation
$\mathsf{Revise}(W_1, W_2)$	State × State → State	parallel composition
$\mathsf{Extract}(W, q)$	State × Query → Evidence	observation
$\mathsf{Combine}(e_1, e_2)$	Evidence × Evidence → Evidence	additive fusion
$\mathsf{Forget}(S, W)$	Scope × State → State	restriction
$\mathsf{OverlapMerge}(W_1, W_2)$	State × State → State	overlap-aware merge
$\mathsf{FallbackRevision}(W_1, W_2)$	State × State → State	source-gated merge
$\mathsf{LeastClosure}(R, W, S)$	RuleSet × State × QuerySet → QuerySet	fixpoint
$\mathsf{SwapAnomaly}(W_1, W_2, q)$	State^2 × Query → Evidence	order cost
$\mathsf{KripkeEvidence}(W, \varphi)$	State × ModalQuery → Evidence	modal evidence

Each extension family introduces additional constructors (30 total across all families); the table above shows the most important ones.

19.1.2 Core Rules from the AddCommMonoid Universal Property

Five rewrite rules are shared by every variant. They are not five independent axioms: all five are *derived* from the single condition that extract is a commutative-monoid homomorphism.

The Evidence type $(\mathbb{R}^{\infty}_{\geq 0})^2$ carries an AddCommMonoid structure via coordinatewise addition (`hplus`). Given any interpretation I satisfying $I.\mathsf{extract}(\mathsf{Revise}(W_1, W_2), q) = I.\mathsf{extract}(W_1, q) + I.\mathsf{extract}(W_2, q)$ and $I.\mathsf{extract}(\mathsf{Zero}, q) = 0$, all five rules follow:

1. **Evidence additivity** (CORE). $\mathsf{Extract}(\mathsf{Revise}(W_1, W_2),\ q) \rightsquigarrow \mathsf{Combine}(\mathsf{Extract}(W_1, q),\ \mathsf{Extract}(W$
2. **Revision commutativity** (CORE). $\mathsf{Revise}(W_1, W_2) \rightsquigarrow \mathsf{Revise}(W_2, W_1)$. *(from* hplus_comm*)*
3. **Revision associativity** (CORE). $\mathsf{Revise}(\mathsf{Revise}(W_1, W_2), W_3) \rightsquigarrow \mathsf{Revise}(W_1, \mathsf{Revise}(W_2, W_3))$. *(from* hplus_assoc*)*
4. **Combine commutativity** (CORE). $\mathsf{Combine}(e_1, e_2) \rightsquigarrow \mathsf{Combine}(e_2, e_1)$.
5. **Combine identity** (CORE). $\mathsf{Combine}(e, \mathsf{EvidenceZero}) \rightsquigarrow e$.

Theorem 19.1 (`wmCore_sound_of_addCommMonoidHom`, kernel-checked)**.** *If an* EvidenceInterpretation *satisfies the homomorphism and zero conditions, then all 5 rule soundness conditions follow. The WM term algebra modulo rewriting is the free commutative monoid on world-model atoms;* extract *is the unique homomorphism to* $(\mathsf{Evidence}, +, 0)$.

Each rule is encoded as a `RewriteRule` structure with a `typeContext` (typed metavariables), `premises` (side conditions), `left` and `right` patterns. The core rules have empty premises. Sixteen step lemmas are kernel-checked: 3 core (evidence-add, revision-comm, revision-assoc) at both extended and full vertices, plus 10 extension-axis lemmas (overlap, forgetting, exact-inverse, provenance-symmetry, closure-fixpoint, swap-anomaly, anti-hallucination, experiment, Kripke, generic carrier).

19.1.3 The 2×2 Calculus Square

In the *raw* (schematic) calculus, all extension rules fire unconditionally. The *guarded* (faithful) calculus makes side conditions explicit as `Premise.relationQuery` entries—for example, the forgetting rule requires $\mathsf{outsideScope}(S, q)$.

The core WM patterns use `.apply` constructors. The shared pattern representation grants them no implicit subterm reduction. The *congruence extension* therefore adds 15 explicit authored rewrite rules (one per argument position), each carrying the contextual premise that enables reduction to propagate into that subterm.

These two orthogonal dimensions give four calculi:

Calculus	**Description**
Raw (`wmExtVertexLanguageDef`)	schematic, no context descent
Raw+Cong (`wmExtVertexLanguageDefWithCong`)	schematic, congruence closure
Guarded (`wmExtVertexLanguageDefGuarded`)	faithful, no context descent
Guarded+Cong (`wmExtVertexLanguageDefGuardedWithCong`)	faithful, congruence closure

The raw calculus is an overapproximation of the guarded one—every guarded reduction is also a raw reduction (dropping premises only enables more reductions). All four horizontal and vertical inclusion arrows are kernel-checked, and Galois connections $\Diamond \dashv \Box$ hold at every corner:

Theorem 19.2 (`wmGuardedCalc_galois`, kernel-checked)**.** *For any* `RelationEnv` *and extended vertex* v*:* $\Diamond_{guarded} \dashv \Box_{guarded}$*.*

Theorem 19.3 (`congReduces_of_rawReduces_ext`, kernel-checked)**.** *Every raw reduction is also a congruence-extended reduction (rule-set inclusion).*

The congruence infrastructure enables multi-step decomposition under context:

Theorem 19.4 (`chain_evidence_add_fully_nested`, kernel-checked)**.** *In the cong-extended calculus, the term* $\mathsf{Extract}(\mathsf{Revise}(\mathsf{Revise}(W_1, W_2), W_3),\ q)$ *reduces in* two *steps to* $\mathsf{Combine}(\mathsf{Combine}(\mathsf{Extract}(W_1, q),\ \mathsf{Extract}(W_2, q)),\ \mathsf{Extract}(W_3, q))$*. Step 1 applies evidence-add to the outer* Revise*; Step 2 uses* `ruleCombineCongLeft` *to apply evidence-add* inside *the left* Combine *argument.*

See `Mettapedia/OSLF/Framework/WMCalculusLanguageDef.lean` (17 guarded rules), `Mettapedia/OSLF/Framework/WMCalculusContextClosure.lean` (463 lines, 0 sorry), and `Mettapedia/OSLF/Framework/WMCalculusOSLFBridge.lean` (784 lines, 0 sorry).

19.2 Intrinsic Encoding and Adequacy

The abstract syntax is given by an *intrinsically sorted* type family $\mathsf{WMTerm} : \mathsf{WMSort} \to \mathsf{Type}$, where $\mathsf{WMSort} = \{\mathsf{state}, \mathsf{query}, \mathsf{evidence}\}$. States and queries both encode to free variables ($\mathsf{state}\ x,\ \mathsf{query}\ x \mapsto \texttt{.fvar}\ x$), but they carry *different sort indices*, so injectivity holds within each sort:

Theorem 19.5 (`encodeWM_injective`, kernel-checked)**.** *For any sort s and terms $t_1, t_2 : \mathsf{WMTerm}\ s$,* $\texttt{encodeWM}\ t_1 = \texttt{encodeWM}\ t_2 \Rightarrow t_1 = t_2$.

The sort index also makes subject reduction *definitional*: every constructor of $\mathsf{WMStep} : \mathsf{WMTerm}\ s \to \mathsf{WMTerm}\ s \to \mathsf{Prop}$ preserves the sort by construction. The five constructors (evidence-add, revision-comm, revision-assoc, combine-comm, combine-zero) correspond one-to-one to the 5 core rewrite rules.

19.2.1 Biconditional and Star Adequacy

The central adequacy result: a reduction at the abstract WMTerm level holds if and only if the corresponding Pattern-level OSLF reduction holds on the encoded terms. The two presentations of the calculus agree exactly.

Theorem 19.6 (Biconditional, `wmStep_iff`, kernel-checked)**.** $\mathsf{WMStep}\ t_1\ t_2 \Leftrightarrow \texttt{langReduces wmCoreLanguageDef (encodeWM}\ t_1\texttt{) (encodeWM}\ t_2\texttt{)}$.

The forward direction (`wmStep_sound`) constructs a `ContextualStep.Step` witness from one of the five authored core rules. The backward direction (`wmStep_complete`) inverts the root rule of that least authored contextual derivation and then inverts `matchPattern` for the selected core rule. Because the core rules have no congruence premises, the relation introduces no implicit descent into the shared pattern representation.

The biconditional lifts to multi-step reductions. Define $\mathsf{WMStepStar} :=$ `ReflTransGen` WMStep (Lean's reflexive-transitive closure).

Theorem 19.7 (Star adequacy, `wmStepStar_adequate`, kernel-checked)**.** $\mathsf{WMStepStar}\ t_1\ t_2 \Leftrightarrow \texttt{LangReducesStar wmCoreLanguageDef (encodeWM}\ t_1\texttt{) (encodeWM}\ t_2\texttt{)}$.

The backward direction (`wmStepStar_complete`) requires a *generalization trick*: since `LangReducesStar` carries `encodeWM` t_1 as a non-variable index, we first generalize it to a fresh variable p before inducting, then recover the WMTerm-level witness at each step via single-step completeness and injectivity (Theorem 19.5).

19.2.2 Backward Asymmetry and Image-Restricted Box

The biconditional (Theorem 19.6) yields a sound $\Box$-lifting theorem: any `langBox` property at `encodeWM` t can be pulled back to all WMStep predecessors.

Theorem 19.8 (`wmBox_sound`, kernel-checked)**.** *If* `langBox wmCoreLanguageDef` φ (`encodeWM` t)*, then* φ(`encodeWM` t') *for every* t' : WMTerm s *with* WMStep t' t*.*

However, the converse—*backward completeness*—is **false** for intrinsically sorted terms. The obstacle is that the Pattern-level reduction system admits predecessors *outside the image* of `encodeWM`. For example, `ruleCombineZero` (whose right-hand side is `.fvar "e"`) produces the Pattern-level predecessor `pCombine (.fvar "x") pEvidenceZero` at `.fvar "x"`—but `.fvar "x"` is the encoding of a *leaf* term (state or query), which has no WMStep predecessor with a combine head.

Restricted to the image of encodeWM, backward completeness holds:

Theorem 19.9 (`wmBoxOnImage_iff`, kernel-checked)**.** *For* t : WMTerm s *and any predicate* φ*:*

$$(\forall t'.\ \mathsf{WMStep}\ t'\ t \Rightarrow \varphi(\mathsf{encodeWM}\ t')) \ \Leftrightarrow\ \mathsf{langBoxOnImageAt}\ s\ \mathsf{wmCoreLanguageDef}\ \varphi\ (\mathsf{encodeWM}\ t)$$

Here langBoxOnImageAt s quantifies only over Pattern-level predecessors that are themselves in the sort-s WMTerm image. This gives the complete adequacy picture:

	Forward ($\Diamond$)	Backward ($\Box$)
Unrestricted	biconditional	sound only
Image-restricted	biconditional	**biconditional**

See `Mettapedia/OSLF/Framework/WMCalculusEncoding.lean` (497 lines, 0 sorry).

19.3 The 13-Axis Vertex Family

The WM calculus variants are indexed by a 13-axis `WMFullVertex`:

#	Axis	Values	Effect
1–4	Base (logic, tv, interval, typing)	$2 \times 2 \times 3 \times 2 = 24$	model-level (same rule
5	Overlap	additive | overlapAware	+1 rule
6	Forgetting	none | scopeBased | supportTracked	+2–3 rules
7	Provenance	none | sourceLabeled	+6 rules
8	Fixpoint	none | closure | policy | cascade	+2–4 rules
9	Cost	none | costTracked	+3 rules
10	Conservation	none | conserving	+2 rules
11	Experiment	none | deterministic | stochastic	+3–7 rules
12	Kripke	none | pointedKripke	+3 rules
13	Carrier	concrete | generic	+2 rules

The first four axes are *model-level*: all 24 combinations produce the same `LanguageDef` (the same rewrite rules), differing only in semantic interpretation. This is proved by `wmVertexLanguageDef_eq` (exhaustive case analysis, kernel-checked). The remaining nine axes are *syntactic*: each adds genuinely new rewrite rules.

Theorem 19.10 (`wmFullVertexMinimal_ruleCount`, kernel-checked)**.** *The minimal vertex (all extensions disabled) has exactly 5 rewrite rules (the core rules).*

Theorem 19.11 (`wmFullVertexMaximal_ruleCount`, kernel-checked)**.** *The maximal vertex (all extensions enabled at strongest level) has exactly 36 rewrite rules.*

19.3.1 Extension Families

Each extension family contributes its own rewrite rules:

Overlap (1 rule). $\mathsf{Extract}(\mathsf{OverlapMerge}(W_1, W_2), q) \rightsquigarrow \mathsf{OverlapCorrect}(\mathsf{Extract}(W_1, q), \mathsf{Extract}(W_2, q)$

Forgetting (2–3 rules). Outside-scope invariance: $\mathsf{Extract}(\mathsf{Forget}(S, W), q) \rightsquigarrow \mathsf{Extract}(W, q)$ (when $q \notin \mathrm{scope}(S)$). Idempotence: $\mathsf{Forget}(S, \mathsf{Forget}(S, W)) \rightsquigarrow \mathsf{Forget}(S, W)$. Support-tracked mode adds the exact-inverse rule.

Provenance (6 rules). Source-compatibility symmetry, fallback decomposition (compatible/incompatible cases), partial revision, trusted-gate identity, and evidence additivity under compatible fallback revision.

Fixpoint (2–4 rules). Closure fixpoint, iteration unfolding, trusted-all simplification (policy mode), and cardinality-bounded stabilization (cascade mode).

Cost (3 rules). Swap anomaly symmetry, swap anomaly zero, and schedule error symmetry.

Conservation (2 rules). Anti-hallucination and Noether conservation.

Experiment channels (3–7 rules). Experiment evidence additivity, Blackwell factorization pullback, channel composition identity. Stochastic mode adds four more rules.

Kripke (3 rules). Kripke evidence additivity, singleton satisfaction, and singleton refutation.

Generic carriers (2 rules). Generic evidence additivity and query-equivalence transport.

19.3.2 Vertex Preorder and Forward Fiber

The 9 syntactic extension axes each define a total preorder on modes (more expressive mode $\leq$ less expressive mode, following the convention that $\bot$ = most rules). The product of these 9 per-axis preorders gives a preorder on `WMFullVertex`:

Theorem 19.12 (`wmFullVertexMaximal_le`, kernel-checked)**.** *For every* $w : \mathsf{WMFullVertex}$*, the maximal vertex (36 rules) satisfies* $\mathsf{wmFullVertexMaximal} \leq w$*.*

Theorem 19.13 (`wmFullVertex_le_minimal`, kernel-checked)**.** *For every* $v : \mathsf{WMFullVertex}$*,* $v \leq \mathsf{wmFullVertexMinimal}$ *(5 rules).*

Rule-set monotonicity gives *forward transport*: if $v \leq w$ (more rules), then every reduction at w is also a reduction at v. This is packaged as a `ForwardFiber` (from `HypercubeGSLTFunctor.lean`): a functor from the vertex preorder category to `LanguageDef`s with identity forward morphisms.

Theorem 19.14 (`wmFullForwardFiber`, kernel-checked)**.** *The function* $v \mapsto \mathsf{wmFullVertexLanguageDef}(v)$ *is a* `ForwardFiber` *over the* `WMFullVertex` *preorder: every weakness edge* $v \leq w$ *induces a forward morphism with* $\mathsf{mapTerm} = \mathrm{id}$*.*

Theorem 19.15 (`wmFullVertexLanguageDef_base_eq`, kernel-checked)**.** *For any two full vertices v, w with identical extension axes (axes 5–13),* $\mathsf{wmFullVertexLanguageDef}(v) = \mathsf{wmFullVertexLanguageDef}(w)$*. The base 4 axes do not affect the rewrite rules.*

Theorem 19.16 (`wmCalc_transport`, kernel-checked)**.** *For any $v, w : \mathsf{WMVertex}$ and any reduction chain $p \to^* q$ at vertex v, the same chain is a valid reduction at vertex w.*

See `Mettapedia/OSLF/Framework/WMCalculusVertexOrder.lean` (228 lines, 0 sorry) and `Mettapedia/OSLF/Framework/WMCalculusGSLTVertex.lean`.

19.4 OSLF Layer: Modal Operators and Semantic Bridge

Passing any vertex's `LanguageDef` through the OSLF pipeline (`langOSLF`) automatically derives:

- **Diamond ($\Diamond\varphi$)**: holds at pattern p iff $\exists q.\ p \rightsquigarrow q \wedge \varphi(q)$. "This WM expression can be rewritten to one satisfying φ."
- **Box ($\Box\varphi$)**: holds at pattern p iff $\forall q.\ q \rightsquigarrow p \to \varphi(q)$. "All expressions that reduce *to* p satisfy φ."
- **Galois connection** $\Diamond \dashv \Box$ (`langGalois`, automatic): $(\forall p.\ p \in \Diamond\varphi \Rightarrow p \in \psi) \Leftrightarrow (\forall p.\ p \in \varphi \Rightarrow p \in \Box\psi)$.

These operators have concrete evidence-theoretic meaning:

Evidence reachability ($\Diamond$). $\Diamond(\mathsf{isCombined})$ holds at any $\mathsf{Extract}(\mathsf{Revise}(W_1, W_2), q)$ because the evidence-add rule rewrites it to a $\mathsf{Combine}$ term.

Evidence safety ($\Box$). $\Box(\mathsf{isPositive})$ at pattern p means every expression that reduces to p was already positive—a backward safety guarantee.

Vertex dependence. At the minimal vertex (5 rules), $\Diamond$ is small (few rewrites reachable). At the maximal vertex (36 rules), $\Diamond$ is much larger. The Galois connection tightens: more $\Diamond$ means weaker $\Box$.

Conservation as $\Box$-safety. With conservation rules enabled, $\Box(\text{outsideScope} \Rightarrow \text{preserved})$ holds at any $\mathsf{Revise}(W, \Delta)$: backward safety captures the Noether-style conservation guarantee.

19.4.1 Diamond Theorems and Galois Corollaries

Pattern predicates (isCombined, isDecomposable, isFixpoint, isOverlapCorrected, isZeroEvidence, etc.) capture evidence-theoretic properties syntactically. Each $\Diamond$ theorem composes a step lemma with a predicate:

Theorem 19.17 (`diamond_isCombined_at_decomposable`, kernel-checked)**.** *For any WM vertex v and patterns W_1, W_2, q: $\Diamond$(isCombined) holds at* Extract(Revise(W_1, W_2), q)*.*

Similar $\Diamond$ theorems hold for all 9 extension axes (`full_diamond_overlapCorrected`, `full_diamond_closureFixpoint`, `full_diamond_antiHallucination`, `full_diamond_experimentDe` `full_diamond_kripkeDecomposable`, `full_diamond_genericDecomposable`).

The Galois connection $\Diamond \dashv \Box$ instantiates to concrete evidence-theoretic dualities:

Theorem 19.18 (`wmCalc_decomposability_safety`, kernel-checked)**.** $(\forall p.\ \Diamond(\text{isCombined})(p) \Rightarrow \psi(p)) \Leftrightarrow (\forall p.\ \text{isCombined}(p) \Rightarrow \Box(\psi)(p))$*.*

Similar corollaries hold for overlap, fixpoint, and conservation predicates.

19.4.2 Relational Encoding and Diamond Chaining

A relational encoding WMTermEncodes connects abstract WorldModel terms to OSLF patterns:

Theorem 19.19 (`wm_evidence_add_sound`, kernel-checked)**.** *If patterns encode WM terms, then the OSLF evidence-add reduction preserves encoding: the reduced pattern encodes* combine(extract(t_{W_1}, t_q), extract(t_{W_2}, t_q))*.*

Multi-step rewrite chains compose via LangReducesStar:

Theorem 19.20 (`chain_evidence_add_nested`, kernel-checked)**.** Extract(Revise(Revise(W_1, W_2), W_3), q *reduces in one step to* Combine(Extract(Revise(W_1, W_2), q), Extract(W_3, q))*. Full decomposition via `chain_evidence_add_fully_nested` (§19.1) achieves the two-step result using congruence closure.*

19.4.3 Evidence Barbs (Process-Algebraic Observables)

Following Milner's barbed bisimulation, a WM-specific observable:

Theorem 19.21 (`wmRevisedState_evidenceBarb`, kernel-checked)**.** Revise(W_1, W_2) *exhibits an evidence barb for q whenever* Combine(Extract(W_1, q), Extract(W_2, q)) *is non-zero—the evidence-add rule provides the witness.*

19.4.4 Neighborhood Semantics Fiber

The abstract LanguageDef reductions correspond exactly to concrete model-theoretic facts when instantiated at the neighborhood WorldModel instance (`PLNWorldModelNeighborhood.lean`):

Theorem 19.22 (`neighborhood_evidence_add_correspondence`, kernel-checked)**.** *The LanguageDef reduction and the mathematical identity* $\mathsf{neighborhoodEvidence}(W_1 + W_2, \varphi) = \mathsf{neighborhoodEvidence}(W_1, \varphi) + \mathsf{neighborhoodEvidence}(W_2, \varphi)$ *are corresponding facts (both hold simultaneously).*

Further theorems: $\Diamond$ decomposition, singleton adequacy, evidence barbs, and the Galois corollary $\Diamond \dashv \Box$ with neighborhood-semantic reading.[1]

19.4.5 First-Order Logic Fiber

The parallel FOL fiber connects the LanguageDef to the Tarskian WorldModel instance (`PLNWorldModelFOL.lean`):

Theorem 19.23 (`fol_evidence_add_correspondence`, kernel-checked)**.** *The same evidence-add correspondence holds for Tarskian satisfaction counting.*

The FOL fiber additionally proves: **validity as maximal strength** (query strength equals 1 when all structures satisfy the query) and a fully faithful **model-theoretic consequence bridge** ($\forall S.\ S \models \varphi \Rightarrow S \models \psi$ iff $\mathsf{queryStrength}(W, \varphi) \leq \mathsf{queryStrength}(W, \psi)$, with the converse holding at singletons).[2]

19.5 Native Type Synthesis: What the Type Pass Sees

The Part title "Natively Typed World Models" refers to the following pipeline: each WM vertex produces a `LanguageDef`, each `LanguageDef` produces an OSLF via `langOSLF`, and each OSLF produces a native type system via `langNativeTypeUsing`. The native type system classifies WM terms by their rewrite behavior—specifically, by which rewrite rules are available at that vertex.

[1] Formalized in `Mettapedia/OSLF/Framework/WMCalculusNeighborhoodBridge.lean` (230 lines, 0 sorry).

[2] Formalized in `Mettapedia/OSLF/Framework/WMCalculusFOLBridge.lean` (295 lines, 0 sorry).

Because all 24 base-axis combinations produce the same `LanguageDef` (`wmVertexLanguageDef_eq`), the pure OSLF/NTT extraction is blind to the 4 model-level axes:

Theorem 19.24 (`wmVertexNativeType_eq`, kernel-checked)**.** *For any* `RelationEnv` ρ *and any two base vertices* v, w*:* langNativeTypeUsing ρ (wmVertexLanguageDef v) = langNativeTypeUsing ρ (wmVertexLanguageDef w)*.*

The present WM → NTT pass is therefore *exact for the 9 syntactic axes* and intentionally insensitive to the 4 model-level axes (logic discipline, truth-value granularity, interval semantics, typing).

Scope and limitations. The native type extracted at a given vertex reflects the *operational* behavior—which rules fire, which reductions are available, which terms are well-typed with respect to the rewrite system. It does *not* distinguish between boolean and Heyting logic, or between point-valued and interval-valued truth values, because these distinctions live at the semantic interpretation level, not at the rewrite level.

Richer semantic-axis sensitivity would require a *semantic bundle* beyond plain `LanguageDef`—carrying the interval/view/typing/interpretation data that the rewrite system does not encode. Such a bundle would give each of the 4 base axes its own NTT fiber. This is a genuine extension point, not a gap: the current theory cleanly separates what NTT classifies (rewrite behavior) from what it does not (semantic interpretation).

19.6 Compiled BN Inference Bridge

The guarded forgetting rule $\mathsf{Extract}(\mathsf{Forget}(S, W), q) \rightsquigarrow \mathsf{Extract}(W, q)$ requires a side-condition oracle answering $\mathsf{outsideScope}(S, q)$. This oracle encodes d-separation: a BN topology determines which scope/query pairs are outside scope.

Positive theorem. Given *any* `RelationEnv` that satisfies the outsideScope premise, the guarded rule fires:

Theorem 19.25 (`guarded_forget_of_dsep`, kernel-checked)**.** *If* $\mathsf{OutsideScope}(\rho, L, S, q)$ *holds for a* `RelationEnv` ρ*, then* $\mathsf{langReducesUsing}\ \rho\ L\ (\mathsf{Extract}(\mathsf{Forget}(S, W), q))\ (\mathsf{Extract}(W, q))$ *for every world-model* W*.*

Negative theorem (collider). With the empty oracle, the premise evaluates to $\varnothing$:

Theorem 19.26 (`collider_premise_empty`, kernel-checked)**.** applyPremisesWithEnv $\varnothing$ L [outsideScop [ ] *for any bindings* bs. *No d-separation certificate is available, so the rule is dead code.*

Completeness. In the guarded calculus, ruleForgetOutsideGuarded is the *unique* rule whose left-hand side matches $\mathsf{Extract}(\mathsf{Forget}(S, W), q)$:

Theorem 19.27 (`forgettingRulesGuarded_unique_match`, kernel-checked)**.** *For every rule* r *in the guarded forgetting rules,* $\mathsf{matchPattern}(r.left,\ \mathsf{Extract}(\mathsf{Forget}(S, W), q)) \neq$ [ ] *implies* $r = \mathsf{ruleForgetOutsideGuarded}$.

Combined with the negative theorem: under the empty oracle, $\mathsf{Extract}(\mathsf{Forget}(S, W), q)$ is *irreducible* in the guarded calculus.

19.7 Formalization Inventory and Future Work

All results in this chapter are kernel-checked in Lean 4.28 (0 sorry):

File	Contents
`WMCalculusLanguageDef.lean`	37 raw + 17 guarded rewrite rules, 7 axis types, `WMFullVe`
`WMCalculusGSLTVertex.lean`	fiber, transport, rule counts, OSLF per vertex, NTT scope
`PLNWMHypercubeBasis.lean`	4-axis `WMVertex`, `CubePath`, transport
`WMProbabilityEmbedding.lean`	13-axis probability embedding, monotonicity
`WMCalculusOSLFBridge.lean`	semantic bridge: $\Diamond$ theorems, Galois corollaries, `WMTermEnc`
`WMCalculusContextClosure.lean`	15 congruence rules, context closure LanguageDefs, raw $\subseteq$
`WMCalculusEncoding.lean`	`encodeWM`, roundtrip, injectivity, `WMStep` soundness + com
`WMCalculusVertexOrder.lean`	per-axis monotonicity, rule subset, `ForwardFiber`, bounde
`WMCalculusBNBridge.lean`	BN d-sep oracle bridge (positive + negative + completenes
`WMCalculusNeighborhoodBridge.lean`	neighborhood fiber: evidence-add correspondence, $\Diamond$ decon
`WMCalculusFOLBridge.lean`	FOL fiber: Tarskian evidence correspondence, validity, mo

19.7.1 Remaining Future Work

Two near-term directions remain interesting but are not required for the core WM $\to$ GSLT $\to$ OSLF $\to$ NTT story:

- **Confluence modulo AC**: With revision-comm and revision-assoc treated as equations (AC theory), the remaining rules (evidence-add, combine-comm, combine-zero) form a terminating system (Revise-nesting-under-Extract strictly decreases). Formal confluence would give uniqueness of normal forms.

- matchPattern **self-matching lemma**: Proving $\forall p.\ \mathsf{matchPattern}\ p\ p \neq [\,]$ (by mutual induction over Pattern) would remove the oracle-parametric hypothesis from the positive BN bridge theorems, enabling concrete instantiation for specific BN topologies.

19.7.2 The Deeper Direction: WM as Universal Evidence for GSLTs

A more ambitious direction connects the WM evidence framework to a universal construction over graph-structured lambda theories.

The universal WorldModel theorem (proven). For any type T, multiset ensembles $\mathsf{Multiset}\ T$ form a `WorldModel` when queried by decidable properties. Evidence extraction counts satisfying/refuting entities, and additivity over ensemble union is automatic (`ensembleEvidence_add`, kernel-checked). This means: every GSLT with parallel composition has a canonical WorldModel. The Rho calculus instance is proved (`rhoWorldModel`, kernel-checked).[3]

The value-monoid generalization. The construction is parameterized by a value monoid V via `GSLTEvidenceAssignment`. When $V = \mathsf{Evidence}$ (non-negative pairs), this gives Bayesian counting. When $V = \mathbb{C}$, this gives L. Gregory Meredith's weight map [31]—complex amplitudes assigned to rewrite steps, whose squared modulus is the transition probability.[4]

The amplitude → evidence chain (future work). A `WeightedLanguageDef` extending `LanguageDef` with a weight map $w_r^+ : \mathsf{HML}(\mathcal{K}) \to \mathbb{C}$ would give the full chain:

$$\texttt{LanguageDef+WeightMap} \xrightarrow{\text{path integral}} \text{amplitudes} \xrightarrow{|\cdot|^2} \text{probabilities} \xrightarrow{\text{counting}} \text{Evidence}$$

[3] Formalized in `Mettapedia/OSLF/Framework/GSLTWorldModel.lean` and `Mettapedia/OSLF/Framework/RhoWorldModel.lean`.

[4] Formalized in `Mettapedia/OSLF/Framework/GSLTEvidence.lean`.

The WM evidence framework would then be provably the Born-rule shadow of a richer amplitude structure. This connects to the Knuth–Skilling valuation-algebra bridge (Section 7.7): both Knuth–Skilling and the amplitude chain derive probability from a simpler algebraic substrate—valuations on a lattice in one case, amplitudes on rewrite paths in the other.

Bisimulation and reversibility (proven). The infrastructure for this chain is partially in place: bisimulation morphisms with both forward and backward simulation (`BisimulationMorphism`, kernel-checked), the reversibility envelope $\mathcal{S}^\dagger$ with trace-recorded costs (`EnvelopeReduces`, kernel-checked), and the full Hennessy–Milner theorem connecting observational equivalence to bisimulation (`obsEq_is_stepBisimulation`, kernel-checked).[5]

The vision. If completed, the amplitude chain would establish that the WM evidence framework is not merely *a* way to do probabilistic reasoning over GSLTs but the *canonical* one: the sufficient-statistic functor from the amplitude-weighted dynamics of any GSLT to the evidence profiles that PLN reasons over. Every computational universe described by a GSLT would have a natural PLN over it.

[5]Formalized in `Mettapedia/OSLF/Framework/BisimulationFiber.lean` and `Mettapedia/OSLF/Framework/ReversibilityEnvelope.lean`.

Part VIII

Bridges, Scope, and Relation to PLN

Chapter 20

Bridges to Neighboring Frameworks

Goal: Show that WM-PLN subsumes or connects to several established inference frameworks.
Layer: Bridges.
Semantic object: Import theorems mapping external semantics into the world-model interface.
Proved: MLN subsumption (zero sorry); ProbLog bridge; Naive Bayes and k-NN bridges; Noisy-OR bridge; PLN-NARS coherence.
Assumed: Finite domains for MLN/ProbLog; independence for Noisy-OR.

The world-model framework is designed to be a *host* for external inference engines: each engine compiles into a world-model evidence source, and the compiled source is then queryable through the standard interface. This chapter demonstrates the pattern for Markov Logic Networks (the deepest bridge, with a full benchmark), ProbLog, Naive Bayes, k-NN, Noisy-OR, and PLN-NARS. A cross-reference section connects to the FOL and set-semantics bridges developed in earlier parts.

20.1 MLN Import and Subsumption

A *Markov Logic Network* (MLN) [36] is a set of weighted first-order clauses $\{(w_i, C_i)\}$. Given a finite domain Dom, the MLN defines a probability distribution over possible worlds W : GroundAtom $\to$ Bool via the Gibbs measure:

$$P_M(W) \;=\; \frac{1}{Z}\prod_i \phi_i(W), \qquad \phi_i(W) = e^{w_i \cdot n_i(W)},$$

where $n_i(W)$ counts the groundings of clause C_i satisfied in W, and $Z = \sum_W \prod_i \phi_i(W)$ is the partition function. The *marginal probability* of a query atom q is $P_M(q) = \sum_{W \models q} P_M(W)$. MLNs combine the expressiveness of first-order logic (clause structure) with the calibration of probabilistic graphical models (Gibbs weights), making them a natural comparison point for the WM-PLN framework.

WM-PLN provides a *semantic subsumption* theorem for Markov Logic Networks: for any finite-domain MLN with mixed universal/existential clause templates, the compiled WM world-model state's `queryStrength` equals the true MLN marginal probability. This is a correctness-by-construction result formalized in Lean 4, not a claim of generally tractable exact inference.

Grounding. A first-order clause template contains variables ranging over a finite domain Dom. A *grounding* (or *ground substitution*) $\theta : \text{Var} \to \text{Dom}$ replaces every variable with a domain element, producing a propositional *ground clause*—a finite disjunction of ground literals. The *grounding layer* enumerates all such substitutions and compiles the resulting ground clauses into the propositional factor graph that defines the MLN distribution.

20.1.1 Clause-Native Core (Ground MLN Semantics)

The clause-native lane works with grounded clauses—finite disjunctions of literals over Boolean atoms—and builds a clause-scope factor graph. The formal route is:

1. **Grounded clause semantics:** weighted ground clauses with explicit satisfied/unsatisfied potentials; world weight as the product of clause evaluations over a finite active support (`PLNMarkovLogicClauseSemantics.lean`).

2. **Clause-scope factor graph:** a factor graph assigns a non-negative *factor* (potential function) to each clause, whose *scope* is the set of ground atoms that clause mentions. The product of all factors defines an unnormalized distribution; the partition function Z (total mass) normalizes it. In the clause-scope construction, the variable-elimination (VE) constraint weight equals the MLN query mass and the partition function equals the total mass (`PLNMarkovLogicClauseFactorGraph.lean`).

3. **ValuationWorldModel bridge:** the compiled factor graph induces a singleton `WMState` with a `WorldModel` instance; evidence extraction

matches the semantic mass semantics (`PLNMarkovLogicClauseWorldModel.lean`).

4. **Terminal theorem:**

$$\text{queryStrength}(\text{clauseWMState}(M, S), q) = \text{queryProb}_M(q)$$

for any ground MLN M with finite clause support S.

Three regression canaries verify the full pipeline on concrete instances: a single positive clause with potentials 3/1 yields queryStrength(*true*) = 3/4; two opposing soft clauses yield queryStrength(*true*) = 3/5; and a hard-zero constraint with unsatisfied potential 0 correctly produces queryStrength(*false*) = 0.[1]

20.1.2 First-Order Template Grounding

The grounding layer lifts the propositional clause-native core to first-order MLN syntax. It handles both universal and existential quantification (`PLNMarkovLogicGrounding.lean`).

Syntax. Arity-indexed predicates Pred(n), first-order terms (variables or constants), literal and clause templates, and substitutions $\theta : \text{Var} \to \text{Dom}$.

Universal templates. A clause template c grounds under substitution θ to a propositional ground clause. The key theorem is:

$$\text{groundClauseTemplate}_\theta(c).\text{holds}(W) \iff \exists\, l \in c,\ (\text{groundLit}_\theta(l)).\text{holds}(W)$$

Weighted templates carry satisfied/unsatisfied potentials; their `eval` agrees with the template potentials:

$$(\text{groundWeightedTemplate}_\theta(wt)).\text{eval}(W) = \begin{cases} wt.\text{satisfiedPotential} & \text{if clause holds under } W, \\ wt.\text{unsatisfiedPotential} & \text{otherwise.} \end{cases}$$

Intuition for existential grounding. Standard MLN grounding of a universally quantified clause produces one ground clause per substitution, each contributing its own factor. An existential clause $\exists\, y.\ \varphi(x, y)$, by contrast, asserts that *some* witness y satisfies φ; grounding it as independent per-y factors would overcount evidence. The correct semantics is a single *big disjunction* over all witnesses, yielding one ground clause per universal grounding of x.

[1] Formalized in `PLNMarkovLogicClauseRegression.lean`.

Existential templates (big-disjunction semantics). An existential clause template $\exists y.\ \varphi(x, y)$ grounds to *one* propositional clause per universal grounding of x, containing the *union* of all ground literals across all existential witnesses $y \in \mathrm{Dom}$:

$$\mathrm{groundExistTemplate}_{\theta_u}(et) = \bigcup_{\theta_e : et.\mathrm{existVars} \to \mathrm{Dom}} \mathrm{groundClauseTemplate}_{[\theta_u | \theta_e]}(et.\mathrm{body})$$

The main correctness theorem states that this ground clause holds iff there *exists* a witness assignment satisfying the body:

Theorem 20.1 (`groundExistTemplate_holds_iff` [FIN], Lean 4, zero sorry)**.**

$$\mathrm{groundExistTemplate}_{\theta_u}(et).\mathrm{holds}(W) \iff \exists\, \theta_e,\ \mathrm{groundClauseTemplate}_{[\theta_u | \theta_e]}(et.\mathrm{body}).\mathrm{holds}(W)$$

Weighted existential templates have analogous `eval` preservation (`groundWeightedExistTemplate_`

Mixed compilation. The sum type `MixedTemplate` (universal | existential) and compilation function `compileToMixedGroundMLN` dispatch grounding by template kind. The world-weight preservation theorem covers both:

Theorem 20.2 (`compileToMixedGroundMLN_worldWeight_eq`, Lean 4, zero sorry)**.** *For a family of mixed templates indexed by* TemplateId*:*

$$(\mathrm{compileToMixedGroundMLN}\ templates).\mathrm{worldWeight}(S, W) = \prod_{(t,\theta) \in S} (\mathrm{groundMixedTemplate}_{\theta}(templ$$

End-to-end canary. The $\mathrm{Smokes}(x) \to \mathrm{Cancer}(x)$ template over a two-element domain $\{\mathrm{alice}, \mathrm{bob}\}$ grounds, compiles, and yields the exact WM query strength:

$$\mathrm{queryStrength}(\mathrm{clauseWMState}(\mathrm{smokeGroundMLN}, \mathrm{fullSupport}), \mathrm{Cancer}(\mathrm{alice})) = \frac{2}{3}$$

Kernel-checked in Lean 4 (`smoke_queryStrength_cancerAlice_eq_two_thirds`).

20.1.3 The Semantic Subsumption Chain

The full formal chain from first-order MLN syntax to WM `queryStrength` is:

$$\underbrace{\text{Mixed templates}}_{\text{Grounding.lean}} \xrightarrow{\text{compile}} \underbrace{\text{GroundMLN}}_{\text{ClauseSemantics.lean}} \xrightarrow{\text{toCountable}} \underbrace{\text{MassSemantics}}_{\text{Abstract.lean}} \xrightarrow{\text{bridge}} \underbrace{\text{WorldModel.queryStren}}_{\text{ClauseWorldModel.lean}}$$

For finite-domain, finite-template MLNs with $|\mathrm{GroundAtom}| < \infty$:

1. `compileToMixedGroundMLN_worldWeight_eq`: world weight is the product of per-template evaluations (Theorem 20.2).
2. `clauseWM_queryStrength_eq_queryProb`: WM `queryStrength` equals $\text{queryProb}_M(q)$ (the exact marginal).
3. `wm_queryStrength_eq_full_queryProb_of_finite_support`: finite support closes the countable-to-finite bridge automatically.

In standard MLN notation, the RHS is:

$$P_M(q) = \frac{\sum_{W \vDash q} \prod_i \phi_i(W)}{\sum_W \prod_i \phi_i(W)} \qquad \text{where} \quad \phi_i(W) = \begin{cases} e^{w_i} & \text{if clause } i \text{ holds in } W, \\ 1 & \text{otherwise.} \end{cases}$$

This is a semantic subsumption theorem. It says that the WM-PLN world-model framework can faithfully represent the true MLN distribution and recover exact marginals. It is *not* a claim of generally tractable exact computation (#P-hardness applies as usual).

One-Shot Composed Theorem

The semantic chain above is distributed across several files and intermediate lemmas. The following single theorem composes the full chain, stating the result in one place:

Theorem 20.3 (`mixed_queryStrength_eq_queryProb`, Lean 4, zero sorry)**.** *For a family of mixed universal/existential templates indexed by* TemplateId*, with finite domain, finite ground-atom type, finite template index, and finite variable type:*

queryStrength(clauseWMState(compileToMixedGroundMLN *templates*, S), q) = queryProb(compileT

where S = Finset.univ *is the full grounding support.*

This is a direct application of `clauseWM_queryStrength_eq_queryProb` after compilation. It constitutes the terminal formal claim: for any finite-domain MLN specified as mixed universal/existential clause templates, the WM-PLN `queryStrength` recovers the true MLN marginal probability.

Classical MLN Specialization

Classical MLNs use the log-weight convention: a clause with weight w contributes potential e^w when satisfied and 1 when unsatisfied. The formalization makes this specialization explicit.

Definition 20.4 (`ClassicalClauseTemplate`)**.** A classical clause template pairs a clause template with a real-valued log-weight w. Conversion to the general positive-potential framework:

$$\text{satisfiedPotential} = e^{w}, \qquad \text{unsatisfiedPotential} = 1.$$

Theorem 20.5 (`classicalTemplate_eval_eq_logWeightPotential`, Lean 4, zero sorry)**.** *For any substitution θ and classical clause template with log-weight w:*

$$(\text{groundWeightedTemplate}_{\theta}(ct.\text{toWeighted})).\text{eval}(W) = \text{logWeightPotential}(w,\ \text{holds}(W))$$

bridging the general positive-potential evaluation to the classical log-weight form already present in the ground-level infrastructure (`PLNMarkovLogicClauseSemantics.lean`).

Theorem 20.6 (`compileClassical_worldWeight_eq_gibbsProduct`, Lean 4, zero sorry)**.** *For a family of classical clause templates:*

$$(\text{compileToGroundMLN } templates).\text{worldWeight}(S, W) = \prod_{(t,\theta)\in S} \text{logWeightPotential}\big(w_t,\ \text{groundClause}$$

preserving the Gibbs product form. This confirms that classical log-weight MLNs are a proper specialization of the WM-PLN positive-potential framework.

Existential-Coupling Exact Regression

The Smokes canary (Section 20.1) has independent per-person factors. To verify that the framework correctly handles *within-clause coupling* from existential grounding, a second canary uses the existential template $\exists\, y.\ \text{Rel}(x, y)$ over the two-element domain $\{a, b\}$.

Setup. One binary predicate Rel over $\{a, b\}$ yields four ground atoms. The existential template grounds to two hard-constraint clauses:

$$x = a:\quad \text{Rel}(a, a) \vee \text{Rel}(a, b) \quad (\text{satisfied} = 1,\ \text{unsatisfied} = 0)$$
$$x = b:\quad \text{Rel}(b, a) \vee \text{Rel}(b, b) \quad (\text{satisfied} = 1,\ \text{unsatisfied} = 0)$$

Of the $2^4 = 16$ possible worlds, exactly 9 satisfy both clauses (total mass = 9). Of those 9, exactly 6 have $\text{Rel}(a, b) = \text{true}$ (query mass = 6).

Results.

Theorem 20.7 (`exist_queryStrength_relAB_eq_two_thirds`, Lean 4, zero sorry)**.**

$$\text{queryStrength}\big(\text{clauseWMState}(\text{existGroundMLN}, \text{fullSupport}),\ \text{Rel}(a,b)\big) = \tfrac{2}{3}$$

Coupling witness. Under the induced distribution, $P(\text{Rel}(a,b)) = \frac{2}{3}$ and $P(\text{Rel}(a,a)) = \frac{2}{3}$, but $P(\text{Rel}(a,b) \wedge \text{Rel}(a,a)) = \frac{1}{3} \neq \frac{4}{9} = (\frac{2}{3})^2$. The existential clause creates genuine within-clause coupling between $\text{Rel}(a,a)$ and $\text{Rel}(a,b)$, which the exact `queryStrength` captures correctly. This demonstrates that the framework goes beyond independent factor models.

20.1.4 UW-CSE Benchmark

The University of Washington CSE (UW-CSE) dataset is a standard MLN benchmark consisting of an academic-relationships knowledge base across five research areas (ai, graphics, language, systems, theory). The prediction task is advisedBy(s, p): which student–professor pairs have an advising relationship, given predicates such as `student`, `professor`, `publication`, `taughtBy`, and `inPhase`.

The UW-CSE benchmark [36] provides the primary empirical validation. The Python reference engine parses all 94 clauses (66 universal + 6 existential + 22 unit), learns weights via pseudo-likelihood, and scores test candidates via local-conditional inference.

Existential clause handling. Six clauses in UW-CSE use existential quantification, e.g.,

$$\exists\, y.\ \neg\text{student}(x) \vee \text{advisedBy}(x,y) \vee \text{tempAdvisedBy}(x,y)$$

The grounding engine implements the big-disjunction semantics from Theorem 20.1: for each universal grounding of x, one ground clause is produced containing the union of all witness literals. Satisfaction is a binary existence check (does *any* witness work?), not a count.

Bipartite coupling from existential clauses. Clauses 27 and 28 (the two query-relevant existential clauses) couple advisedBy atoms across the student–professor bipartite graph: clause 27 couples all advisedBy(s, p) for

a fixed student s across professors, while clause 28 couples for a fixed professor p across students. Together they create one giant connected component per fold (160–972 unobserved advisedBy atoms), invalidating student-block independence and making exact joint enumeration infeasible.

Inference mode. Local-conditional (pseudo-likelihood) inference: $P(\text{advisedBy}(s, p) = 1 \mid \text{rest}) = \sigma\left(\sum_i w_i \, \Delta n_i\right)$, conditioning on all other atoms. This is the same approximation used by practical MLN systems (Alchemy, Tuffy, PracMLN).

Results (5-fold leave-one-area-out). Mean AUROC = 0.862, mean AUPRC = 0.355.

PeTTa parity. The WM-PLN evidence-tensor / weighted-clause-delta algebra reproduces the Python reference engine at machine precision ($\max |\Delta| < 10^{-15}$), verifying that the Lean-formalized algebra and the benchmark engine agree.

Evaluation protocol. The evaluation is leave-one-area-out: each of the five areas serves once as the test fold, with the remaining four used for training. All advisedBy atoms in the test area are removed from the evidence before scoring; non-target predicates (`student`, `publication`, `taughtBy`, etc.) in the test area are retained, following the standard transductive protocol for this benchmark. Professor advisee counts used in the compatibility-gated model (Section 20.1.5) are computed from training data only. The semantic subsumption theorem (Section 20.1) is formalized in Lean 4; the beyond-MLN improvement is an empirical result, not a formal claim.

20.1.5 Beyond MLN: Compatibility-Gated Evidence Roles

The subsumption theorem (Section 20.1) shows that WM-PLN *can* emulate MLN inference. This section demonstrates that the framework can also *improve* on it, by decomposing the MLN's monolithic signal into interpretable evidence roles and learning their interaction.

Failure mode diagnosis. Auditing the top 50 false positives of the MLN baseline on each fold reveals that 56–98% (mean 78%) have *zero pairwise evidence*: no shared publications, no TA/taughtBy overlap, no tempAdvisedBy link. The MLN predicts these advisor relationships purely from unary student and professor features propagated through clause weights—a structural failure mode invisible to AUROC.

Compatibility-gated model. The WM evidence framework separates the MLN's clause-weight signal into four evidence roles:

1. **NeedAdvisor(s)**: unary student readiness—career phase (Pre_Quals, Post_Quals, Post_Generals), years in program, publication and TA counts, tempAdvisedBy count.

2. **CanAdvise(p)**: unary professor capacity—position type, publication and teaching counts, historical advisee count (from training data only).

3. **PairAffinity(s, p)**: degree-corrected pairwise compatibility—shared publications normalized by $\sqrt{(1 + \text{pub}_s)(1 + \text{pub}_p)}$, shared courses similarly normalized, and a tempAdvisedBy indicator.

4. **MLN logit**: the existing clause-weight signal, imported as one expert among several.

A binary *pair-absent* indicator flags candidates with no pairwise evidence, and interaction terms (MLN logit × pair-absent, career-phase × pair-absent) let a logistic regression learn the *compatibility gate*: strong unary signals should not become strong edge predictions when pairwise evidence is absent.

Evaluation. Leave-one-area-out with the same 5 folds. The logistic regression (20 features, no hyperparameter tuning) is trained on the 4 training folds per split. To avoid label leakage, training features are extracted per area with the logistic regression's own training data excluding the area being featurized.

Results. Table 20.1 reports the comparison. The gated model improves AUROC on all five folds (mean +0.034) and improves student-level metrics on average: Hits@1 +0.070, MRR +0.069, Set-F1 +0.093. The improvements are concentrated on folds where the MLN's pairless false-positive rate is highest (graphics: 88%, theory: 80%), confirming that the gate addresses the diagnosed failure mode.

Ablation. To verify that the improvement comes from role separation rather than simply adding more features, four ablation configurations are evaluated across all five folds:

A. MLN-only: logistic regression on the MLN logit alone (1 feature).

	AUROC		Hits@1		Set-F1		$FP\%_{0pair}$
Fold	MLN	Gate	MLN	Gate	MLN	Gate	
ai	0.845	0.871	0.433	0.433	0.389	0.411	70%
graphics	0.884	0.932	0.071	0.357	0.048	0.381	88%
language	0.871	0.893	0.375	0.375	0.333	0.333	98%
systems	0.845	0.872	0.444	0.593	0.395	0.531	56%
theory	0.866	0.916	0.500	0.417	0.417	0.389	80%
Mean	0.862	0.897	0.365	0.435	0.316	0.409	78%

Table 20.1: MLN baseline vs. compatibility-gated WM model (leave-one-area-out, 5 folds).

B. MLN+gate: MLN logit + pair-absent indicator + their interaction (3 features).

C. WM-only: all evidence-role features *without* the MLN logit (18 features).

D. Full gate: all 20 features (the V1 model reported above).

Table 20.2 reports the macro-averaged results. Configuration C (WM evidence roles *without* the MLN logit) dominates all others on every primary metric: mean AUROC 0.911 (vs. 0.897 for the full gate), mean Hits@1 0.553 (vs. 0.435), mean Set-F1 0.494 (vs. 0.409). Adding the MLN logit back (D vs. C) actually degrades performance, suggesting that the clause-weight signal is *redundant* once the evidence roles are separated—or, more precisely, that the logistic regression allocates weight to the MLN logit at the expense of better-calibrated role features.

Config	#feat	AUROC	AUPRC	Hits@1	MRR	Set-F1
MLN (raw)	—	0.862	0.355	0.365	0.536	0.316
A: MLN-only	1	0.862	0.355	0.365	0.536	0.321
B: MLN+gate	3	0.861	0.363	0.412	0.574	0.367
C: WM-only	18	**0.911**	**0.484**	**0.553**	**0.682**	**0.494**
D: Full gate	20	0.897	0.389	0.435	0.605	0.409

Table 20.2: Ablation study (macro-averaged over 5 folds). Config A uses the MLN logit alone; B adds the pair-absent gate; C uses only WM evidence-role features (no MLN); D is the full 20-feature gate.

Interpretation. The ablation reveals that the improvement comes entirely from *evidence-role decomposition*, not from the MLN logit. The MLN's clause weights conflate unary eligibility with pairwise compatibility; once these are separated into NeedAdvisor, CanAdvise, and PairAffinity features, the clause-weight signal becomes redundant. This is a stronger result than originally anticipated: the WM framework does not merely *augment* MLN inference—it *replaces* the signal with a better-structured one.

The practical implication is that an MLN knowledge base is most valuable as a *source of feature engineering intuition* (which predicates matter, which interactions exist) rather than as a runtime inference engine. The WM evidence-role framework can then express those intuitions more transparently and with better calibration.

20.1.6 Abstract Lane (Supporting Infrastructure)

The abstract lane provides the infinite-first generalization: abstract mass semantics, countable semantics via $\sum$ over worlds, finite-support restriction under locality witnesses, and factor-graph specialization. This infrastructure is used internally by the clause-native bridge via `toCountableMLNSemantics`.

20.1.7 MLN Theory Combination vs. WM Revision

An important distinction: MLN theory combination (adding weighted clauses to a knowledge base, changing the underlying Gibbs distribution) is *not* the same operation as WM additive revision (combining independent evidence sources). The subsumption theorem says that each ground MLN compiles to a *single* WM evidence source; it does not claim that merging two MLN knowledge bases corresponds to WM revision of their compiled states.

20.1.8 Scope Limitations

The result is intentionally scoped. It does *not* claim:

- lifted MLN inference (symmetry-exploiting grounding);
- scalable exact joint inference (this is #P-hard in general);
- open-universe MLNs;
- or full UW-CSE benchmark formalization in Lean (the 94-clause benchmark is formalized in Python with PeTTa parity; the Lean lane provides the general semantic subsumption theorem and a smaller end-to-end canary).

20.2 ProbLog Bridge

ProbLog [9] is a probabilistic logic programming language. A ProbLog program is a set of annotated facts $p_i :: f_i$ (each fact f_i holds independently with probability p_i) plus a set of definite clauses (standard Prolog rules). The *distribution semantics* defines the probability of a query q as the sum over all "total choices" (subsets of facts where each fact is included/excluded independently) of the probability of that choice times the indicator that q is derivable from the choice.

The world-model bridge compiles a ProbLog program into a `JointEvidence` state via the `probLogToJointEvidence` function and proves:

Theorem 20.8 (ProbLog subsumption [FIN] (CORE))**.**

$$\mathsf{strength}(\text{probLogToJointEvidence}(p), q) = \text{probLogQueryProb}(p, q)$$

The compilation path is:

$$\text{ProbLog program} \xrightarrow{\text{total-choice}} \text{residual KB per choice} \xrightarrow{\text{least Herbrand}} \text{ground truth per choice} \xrightarrow{\text{weight}} \text{Jo}$$

Bag vs. distribution semantics. A critical distinction: the immediate T_P operator at $K = \mathbb{R}^{\infty}_{\geq 0}$ computes *bag* semantics (sum-of-products), not distribution semantics (set semantics). For programs with overlapping derivations—e.g., `q :- f_1. q :- f_2.`—bag semantics gives $p_1 + p_2$ while distribution semantics gives $1-(1-p_1)(1-p_2)$. The correct compilation path through total-choice → residual KB → least Herbrand model avoids this conflation. The `bag_set_separation` theorem in the formalization makes this distinction precise.

Relation to Noisy-OR. When ProbLog clauses have disjoint derivations, the distribution semantics reduces to the noisy-OR formula (Section 20.4). This is proved in `compilation_or_noisyOr` (844 lines, 21 theorems, 0 sorry).

Genomics instantiation: gene–SNP hypothesis generation. The ProbLog bridge is not only a theoretical result; it is instantiated concretely in a genomic hypothesis-generation pipeline developed by Rejuve.Bio [35]. The pipeline uses ProbLog with learned LPAD rules to predict gene–SNP relevance from a biomedical knowledge base, combining rule-based reasoning with probabilistic weighting. Three independent biological mechanisms serve as the core evidence channels:

- **Regulatory effect** ($p = 0.34$): enhancer overlap with target gene (two-hop rule through regulatory DNA regions).
- **eQTL association** ($p = 0.0176$): tissue-level expression quantitative trait loci from GTEx.
- **Activity-by-contact** ($p = 0.021$): biosample-level 3D chromatin contact (ABC model).

The combined probability that a gene is relevant to a SNP is:

$$P(\text{relevant}) = 1 - (1 - 0.34)(1 - 0.0176)(1 - 0.021) \approx 0.366$$

This is the noisy-OR formula (Theorem 20.11) applied to $n = 3$ independent mechanisms. The Lean formalization proves `bioGeneRelevance_eq_noisyOr`: the ProbLog-compiled gene relevance equals the noisy-OR over the three mechanism probabilities (636 lines, 0 sorry).

Beyond the three-mechanism model, the WM-PLN formalization extends the Rejuve.Bio pipeline in two directions. First, the evidence-algebra framework replaces ProbLog's native cplint parameter learning with graded evidence carrying, improving AUC-ROC from 0.790 to 0.858 on the original 823-pair benchmark (5-fold CV matching the repository's fold structure). Second, the same noisy-OR fusion scales to a full GWAS benchmark: 1,097 ranking-evaluable loci across 23 chromosomes from the Open Targets Platform 25.12, with colocalisation, enhancer-to-gene, and variant-effect channels fused via $1 - \prod_i (1 - s_i)$ per (locus, gene) pair.[2]

See `Mettapedia/Logic/ProbLogDistributionSemantics.lean` (237 lines), `Mettapedia/Logic/ProbLogCompilation.lean` (844 lines, 0 sorry), and `Mettapedia/Examples/PLN/BioHypothesisGeneration.lean` (636 lines, 0 sorry). The Rejuve.Bio pipeline source is at `https://github.com/rejuve-bio/hypothesis-generation-demo/tree/main/pl`.

20.3 Naive Bayes and k-NN Bridges

These bridges show that two standard machine-learning classifiers are special cases of the world-model query machinery, not independent algorithms. The full formal development appears in Section 11.4 (Chapter 11); we summarize the key results here.

[2] Open question: can the ProbLog bridge extend to programs with non-independent derivations (overlapping clauses)? The current formalization handles the independent case exactly via noisy-OR; the general case requires inclusion-exclusion or Möbius inversion on the derivation lattice.

Naive Bayes. A Naive Bayes classifier assumes conditional independence of features $F_1, \ldots, F_n$ given class C. In the WM framework, this is a BN with a single fork: $C \to F_1, \ldots, C \to F_n$. The evidence tensor product $\otimes$ composes per-feature evidence into a joint classification:

Theorem 20.9 (Naive Bayes bridge (CORE))**.**

$$\mathsf{strength}_{\otimes}(e_{F_1 \Rightarrow C}, \ldots, e_{F_n \Rightarrow C}) = P(C \mid F_1, \ldots, F_n)_{\mathrm{NB}}$$

The proof reduces to the chain/fork exactness theorem (Chapter 11) plus the independence factorization. See `PLN_tensorStrength_eq_nbPosterior` in `Mettapedia/PLN/WorldModel/BayesNet/PLNBayesNetInference.lean`.

k-NN. A k-nearest-neighbor classifier votes by aggregating evidence from the k training points closest to the query. In the WM framework, each neighbor contributes an evidence value, and the vote is additive evidence revision ($\oplus$):

Theorem 20.10 (k-NN bridge (CORE))**.**

$$\oplus\text{-pos}(e_1, \ldots, e_k) = \text{knnRelevance}(x)$$

This bridge is particularly relevant to automated theorem proving: the MaSh premise selector [5] uses k-NN to rank lemma relevance, and the formalization proves that the PLN evidence-algebra score equals the MaSh k-NN score (`mashScoreTopK_is_knn`). The full premise-selection framework (16 files, 4,377 lines, 176 theorems, 0 sorry) develops this connection through coverage bounds, submodularity, and optimality theorems.[3]

20.4 Noisy-OR Bridge

The Noisy-OR model is a standard CPT parametrization for multiple independent causes $C_1, \ldots, C_n$ of an effect E. Each cause C_i independently produces E with probability s_i (the "leak-through" rate). The combined probability of E given all active causes is:

$$P(E \mid C_1, \ldots, C_n) = 1 - \prod_{i=1}^{n} (1 - s_i)$$

The bridge proves this formula as a theorem, not an assumption:

[3] Formalized in `Mettapedia/PLN/InferenceControl/PremiseSelection/KNN\_PLNBridge.lean` and `Mettapedia/PLN/InferenceControl/PremiseSelection/KNN.lean`.

Theorem 20.11 (Noisy-OR (CORE)). *For independent causes with evidence strengths* $s_1, \ldots, s_n$*:*

$$s_{\text{combined}} = 1 - \prod_{i=1}^{n}(1 - s_i)$$

and the fuzzy-OR and noisy-OR formulations agree: $\text{fuzzyOr}(s_1, \ldots, s_n) = \text{noisyOr}(s_1, \ldots, s_n)$.

The formalization further proves:

- **Algebraic properties:** commutativity, associativity, and unit-interval membership of both fuzzy-OR and noisy-OR (23 theorems).
- **Geometric bounds:** for n replicated events with strength s, the combined strength satisfies $1 - (1 - s)^n \leq 1 - e^{-ns}$ (tight for small s).
- **Variance propagation:** delta-method variance propagation through the noisy-OR, giving posterior mean scoring score $= S \cdot C + (1 - C) \cdot \mu_0$ where S is combined strength, C is confidence, and μ_0 is the prior mean.

The noisy-OR as a common computational substrate. The noisy-OR is not merely an isolated formula—it is the shared fusion engine across three independent application domains in this formalization:

1. **ProbLog compilation** (Section 20.2): when ProbLog clauses have disjoint derivations, the distribution semantics reduces to noisy-OR (`compilation_or_noisyOr`).
2. **GWAS gene prioritization**: three biological evidence channels (colocalisation, enhancer-to-gene, variant effect) are fused via $1 - \prod_i (1 - s_i)$ per (locus, gene) pair, with the Lean formalization proving the fusion is sound (`bioGeneRelevance_eq_noisyOr`).
3. **PPLBench** (Appendix 25): the `noisy_or_topic` lane exercises the same formula as a probabilistic programming primitive, benchmarked against Stan.

That a single 765-line formalization serves all three domains without modification is a concrete instance of the WM-PLN thesis: the evidence algebra is not a domain-specific tool but a general-purpose compositional substrate.[4]

[4] Formalized in `Mettapedia/Logic/PLNNoisyOr.lean` (765 lines, 0 sorry).

20.5 PLN v0.9 Subsumption

PLN v0.9 (`lib_pln.metta`, 430 lines, 73 functions) implements the classic PLN inference rules as MeTTa truth-value functions with stamp-based provenance tracking and priority-queue forward chaining. The WM calculus fully subsumes it:

- PLN v0.9's truth functions (`Truth_Deduction`, `Truth_Revision`, `Truth_Induction`, `Truth_Abduction`) are formalized as scalar views of evidence-level operations (Sections 9.4.1 and 11.8).
- PLN v0.9's stamp mechanism is a runtime version of the formal provenance discipline (Section 9.4.1).
- PLN v0.9's confidence-to-weight transform `Truth_c2w` is exactly our `c2w` (Section 8.1).
- PLN v0.9 has *no forgetting*—once evidence enters a stamp, it cannot be retracted. The WM calculus adds exact forgetting with proved conservation.

All three PLN v0.9 flagship examples (DeductionRevision, RavenInduction, FlyingRaven) are verified as WM calculus instances with 20 kernel-checked conformance tests (Section 11.8).

20.6 PLN-NARS Bridge

NARS (Non-Axiomatic Reasoning System) [42] is a separate uncertain-reasoning framework that uses *experience-based* truth values: a frequency f (analogous to PLN strength) and a confidence c derived from evidence weight $w = c \cdot k/(1-c)$, where k is a system parameter. The bridge proves four families of results:

1. Confidence transforms. The NARS confidence-to-weight and weight-to-confidence transforms are proved invertible:

$$w = c \cdot k/(1-c) \qquad (\texttt{nars_c2w_eq}) \qquad (20.1)$$

$$c = w/(w+k) \qquad (\texttt{nars_w2c_eq}) \qquad (20.2)$$

$$w2c(c2w(c)) = c \qquad (\texttt{w2c_c2w_id}) \qquad (20.3)$$

$$c2w(w2c(w)) = w \qquad (\texttt{c2w_w2c_id}) \qquad (20.4)$$

2. Revision coherence. NARS revision computes a weighted average of frequencies, with weights proportional to evidence weight. The bridge proves that this is *identical* to PLN evidence addition:

Theorem 20.12 (NARS revision = PLN evidence algebra (Core))**.** *NARS revision of two judgments with frequencies f_1, f_2 and weights w_1, w_2 produces the same frequency and confidence as PLN $\oplus$-revision of the corresponding evidence values.*

See `nars_revision_is_evidence_aggregation`.

3. Rule correspondences. The bridge proves formula-level correspondences for NARS deduction, induction, and abduction:

- **Deduction:** frequency and confidence formulas match PLN under the same screening-off assumptions (`deduction_freq_formula`, `deduction_conf_formula`).
- **Induction/abduction:** frequency formulas for weak inference are proved, with confidence bounded by the weaker premise (`induction_freq_is_f1`, `abduction_freq_is_f2`).

4. Informativeness adjunction. An informativeness ordering on evidence values yields a Galois connection between the PLN and NARS lattices.[5]

See `Mettapedia/PLN/Comparisons/NARS/NARSEvidenceBridge.lean` (234 lines, 15 theorems, 0 sorry).

20.7 FOL and Set-Semantics Bridges

A reader trained in model theory will want to know precisely how the world-model calculus relates to classical first-order and higher-order semantics. This section states the bridge results, their assumptions, and their limitations.

[5]Open question: the Galois connection suggests a deeper categorical relationship between PLN and NARS as two presentations of the same underlying evidence category. Is there a universal property that characterizes what makes an uncertain-reasoning system "evidence-algebraic" in this sense?

20.7.1 Model theory: Tarskian FOL, Henkin HOL

The FOL layer uses standard *Tarskian* semantics: a first-order structure $\mathcal{S} = (U, I)$ with a domain U and an interpretation I, and satisfaction $\mathcal{S} \models \varphi$ defined by the usual recursive clauses. The world-model state is a *multiset* of pointed first-order structures; evidence for a query φ is the count of structures in the multiset that satisfy φ.

The HOL layer uses *Henkin* semantics, not full (standard) higher-order semantics.[6] A Henkin model M carries an admissible carrier at each type, with comprehension limited to the carrier—this avoids the impredicativity issues of full semantics while remaining "first-order manageable at the metatheoretic level" (the proofs about Henkin models are themselves first-order). The proof calculus is extensional natural deduction over Henkin semantics, and the active completeness route takes a classical specialization.

20.7.2 Set representation: the `SetStructure` abstraction

A natural question is: how are sets represented in the grounding path? The formalization uses an abstract `SetStructure` interface — any type M equipped with a membership relation $\in_M$ — and does *not* commit to the von Neumann cumulative hierarchy or any specific set-theoretic construction in the critical path.[7] The canonical grounding builds a Henkin HOL model directly from the membership relation via `ofSetStructure`(M) : `SetHOLModel`, bypassing any rank-based construction.

Evidence extraction treats predicates as characteristic morphisms $U \to$ `Evidence`, where `Evidence` is a complete Heyting algebra (quantale frame). In the language of topos theory, evidence plays the role of a truth-value object. The formalization already treats `Evidence` as Ω in a subobject-classifier pattern: `SatisfyingSet` wraps predicates $P : U \to$ `Evidence` as characteristic morphisms, and a separate module proves that presheaf categories have subobject classifiers (`presheafCategoryHasClassifier`, 437 lines, 0 sorry).

[6] Open question: what changes if we move to full higher-order semantics? The singleton adequacy results survive (they depend only on truth-in-a-model, not on the class of models). But the correspondence between WM strength ordering and semantic consequence becomes sensitive to the model class: full-semantics consequence is strictly stronger than Henkin consequence for some sentences.

[7] Open question: can set theories that avoid $\varnothing$-hierarchies (e.g., Church-style foundations with urelements, or constructions rooted in game-theoretic structures like surreal numbers) instantiate the `SetStructure` interface? The abstraction barriers are in place; the instantiation is future work.

The remaining step—showing that `Evidence` is literally the classifier Ω for a specific category of world-model states—is not yet a single theorem.[8]

20.7.3 Singleton adequacy

The central bridge result is *singleton adequacy*: truth of a sentence in a single structure corresponds exactly to WM strength 1 on the singleton state. This is proved in three forms:

Theorem 20.13 (FOL singleton adequacy (Core))**.** *For a pointed first-order structure $\mathcal{S}$ and a query φ:*

$$\mathcal{S} \models \varphi \iff \mathsf{strength}(\{\mathcal{S}\}, \varphi) = 1$$

Theorem 20.14 (HOL singleton adequacy (Core))**.** *For a Henkin model M and a query φ:*

$$M \models_{\text{Henkin}} \varphi \iff \mathsf{strength}(\{M\}, \varphi) = 1$$

Theorem 20.15 (Set/HOL singleton adequacy (Core))**.** *For a pointed set-theoretic structure S and a query φ:*

$$S \models_{\text{Set/HOL}} \varphi \iff \mathsf{strength}(\{S\}, \varphi) = 1$$

All three are instances of a generic crisp-specialization family: when the multiset has cardinality 1, strength reduces to the indicator of satisfaction.

20.7.4 WM strength and model-theoretic consequence

The deeper result connects WM strength ordering to semantic consequence:

Theorem 20.16 (Consequence as strength ordering (Core))**.** *For a theory T and sentences φ, ψ over a language L:*

$$T \models (\varphi \to \psi) \iff \forall\, \mathcal{S} \models T,\ \mathsf{strength}(\{\mathcal{S}\}, \varphi) \leq \mathsf{strength}(\{\mathcal{S}\}, \psi)$$

On *multiset* states (not just singletons), pointwise implication still implies strength inequality: $(\forall\, \mathcal{S},\ \mathcal{S} \models \varphi \Rightarrow \mathcal{S} \models \psi) \Rightarrow \mathsf{strength}(W, \varphi) \leq \mathsf{strength}(W, \psi)$ for any state W. Evidence compositionality ensures that $\mathrm{ev}(W_1 + W_2, \varphi) = \mathrm{ev}(W_1, \varphi) + \mathrm{ev}(W_2, \varphi)$ holds for all queries—the FOL bridge is not just truth-preserving but evidence-additive.

[8] Open question: what is the precise category for which `Evidence` serves as the subobject classifier? The `SatisfyingSet` pattern and the presheaf classifier are both formalized; the missing piece is the functor that connects them.

20.7.5 Four routes and their agreement

Four compilation routes connect classical logic to the world model:

1. **FOL → WM** (direct): Tarskian structures, model counting, evidence extraction.

2. **Set → FOL → WM** (set theory as FOL): the set-theory language $\mathcal{L}_{\text{set}}$ is a first-order language; specializing the FOL bridge to $\mathcal{L}_{\text{set}}$ gives a direct Set↔WM consequence bridge for ZF, ZFC, and other set theories.

3. **Set → HOL → WM** (via set-semantics grounding): set-theoretic structures induce Henkin models directly (`ofSetStructure`); the HOL lens extracts evidence.

4. **FOL → HOL → WM** (via embedding): FOL syntax embeds into HOL; HOL lens extracts evidence.

On embedded first-order set-theory sentences, routes 2, 3, and 4 agree on evidence and query strength. Route 2 is the shortest (no HOL detour), but it still passes through FOL: the set-theory language $\mathcal{L}_{\text{set}}$ is encoded as a first-order language, and the bridge specializes the generic FOL→WM machinery. Route 3 provides the richer type-theoretic grounding needed for higher-order set-theoretic reasoning. Note that *none* of the four routes is a direct Set→WM connection that bypasses both FOL and HOL.[9]

See `PLNWorldModelFOL.lean`, `PLNWorldModelSetTheoryBridge.lean`, `PLNWorldModelHOLSetBridge.lean`.

20.7.6 HOL-level limitations

At the FOL level, completeness is available from the Foundation library: provability ⇔ semantic consequence. At the HOL level, the live foundation is already the classical Henkin route: provability implies Henkin-model

[9]Open question: can a direct Set↔WM bridge be constructed? The `SatisfyingSet` machinery already provides frame-valued predicates $P : U \to$ `Evidence` with no FOL syntax involved—this is a set-membership predicate valued in the evidence frame. Wiring `SetStructure` directly to `WorldModel` via `SatisfyingSet` (bypassing the FOL encoding of $\in$) would give a genuinely syntax-free set-theoretic grounding of evidence. This would be particularly natural for foundations that avoid the syntactic overhead of first-order set-theory axiomatizations.

satisfaction, and consistent classical theories admit Henkin models via the witnessed-extension, classical-world, and term-model development.[10]

The following HOL results are proved:

- Soundness: derivability implies Henkin-model satisfaction.
- Witnessed extension/saturation together with prime/maximal/classical world construction.
- Term-model denotation, realizability, the fundamental truth theorem, and classical Henkin model existence.

The following are *not* yet proved:

- Intuitionistic completeness matching the base calculus without classical hypotheses.
- Completeness for full higher-order semantics rather than the active Henkin route.
- Full fusion of semantic ProbHOL with a mature logical-induction-style belief-process semantics.

Crucially, the WM bridge, ProbHOL (probabilistic higher-order logic), certified higher-order chaining, and the inference-control API *do not depend on those stronger variants*. They already work over the live classical Henkin model layer.

20.7.7 Infinite domains

An infinitary layer exists in the codebase (`PLNFirstOrder/Infinite.lean`) with weakness-based quantifier semantics on arbitrary universes.[11] This layer is not part of the reviewed finite-domain package (Chapter 14) but demonstrates that the architecture does not inherently require finiteness.

See `Mettapedia/PLN/Bridges/Logic/WorldModel/PLNWorldModelFOL.lean` (268 lines), `Mettapedia/PLN/Bridges/Logic/WorldModel/PLNWorldModelFOLCompleteness.lean` (391 lines), and `Mettapedia/OSLF/Framework/WMCalculusFOLBridge.lean` (295 lines). All zero sorry.

[10] Open question: beyond the current classical Henkin route, can we recover matching completeness for the intuitionistic base calculus, or for stronger semantic classes such as full standard higher-order semantics?

[11] Open question: how do infinite cardinalities behave under Henkin witness-rich extensions? The cumulative witness expansion may require a transfinite construction; whether this interacts with the evidence-additivity law is not yet investigated.

20.8 PPLBench: Probabilistic Programming Interoperability

The PPLBench refresh (Appendix 25) demonstrates that the WM-PLN evidence-state machinery can serve as a *probabilistic programming backend*: nine benchmark lanes from the PPLBench suite [25] are implemented with WM-PLN evidence carriers (Normal-Gamma, Dirichlet, Beta, log-likelihood messages) and compared against reference Stan implementations. Each lane carries explicit sufficient statistics revised by addition —the same $\oplus$ operation used throughout the book.

The key result is that all "E/E" (exact carrier, exact inference) lanes recover the reference posterior to machine precision, validating that the evidence algebra is not merely an abstract formalism but a practical computational substrate. The "E/O" lanes (exact carrier, operational scheduler) demonstrate the architecture for problems where exact inference is intractable but the evidence carrier remains sound.

This is the operational counterpart to the theoretical bridges above: where the MLN and ProbLog bridges prove *semantic* subsumption, the PPLBench lanes demonstrate *computational* interoperability with mainstream probabilistic programming.

20.9 DeFi Hybrid Risk Monitoring

The same hybrid WM evidence machinery should extend naturally to DeFi protocol risk monitoring, combining continuous market and reserve variables (TVL, utilization, volatility) with discrete governance, oracle-type, and incident propositions. The per-source Normal-Gamma conjugate framework already handles heterogeneous evidence fusion; the missing piece is an operational data pipeline and a curated fixture. We leave this as future empirical work.

Chapter 21

What WM-PLN Covers

This chapter provides a compact, family-level summary of the formalization—what is actually proved, in the Lean kernel, as opposed to merely discussed or conjectured.

21.1 The Kernel

The foundation is a world-model interface with evidence compositionality: revision of two world-model states and subsequent query extraction must agree with extracting from each state separately and then combining the evidence. This interface is instantiated concretely by a Dirichlet reference instance with query extraction over finite world-sets, and the resulting system admits a sequent-style calculus in which query-revision is an admissible rule. The local-completeness no-go theorem (Chapter 5) then demonstrates that no link-local evidence representation can recover full joint evidence—a sharp boundary on the ambitions of any PLN-like system.

21.2 Views and Evidence

Evidence carries algebraic structure: the formalization proves that evidence forms both a quantale (with a tensor product modelling revision) and a Heyting algebra (with implication modelling conditional evidence). The truth-value views—simple truth values, indefinite truth values, and distributional views—are all formalized as typed projections from evidence, with an explicit separation of Walley predictive intervals from Bayesian credible intervals. Two convergence bridges ground the asymptotic story: de Finetti's

exchangeability theorem justifies count sufficiency, and a Solomonoff convergence bridge shows that evidence counts approximate the universal posterior. A Knuth–Skilling valuation-algebra bridge connects the evidence algebra to information- theoretic valuations.

21.3 Compiled Inference

The extensional inference rules of classical PLN—deduction, induction, abduction—are proved exact for the chain, fork, and collider BN motifs under explicit screening-off and positivity conditions. D-separation discharge is packaged as a typeclass, so that a user need only declare the BN structure and the side conditions are checked automatically. The Xi rule registry collects these compiled rewrites with their typed BN constructor lifts, and a one-call pipeline composes selector, rewrite, and threshold into a single certified step. Threshold transport with explicit boundaries ensures that confidence gates compose correctly across inference chains.

21.4 Extensions

Higher-order reduction is formalized with corrected side conditions, together with certified higher-order chaining from estimated variables: information-set typing, law-of-total-variance reveal, Bayesian regime update, calibration contracts, certified continue/reveal/fallback/abstain rules, and stronger root-sum-of-squares bounds under explicit assumptions. Finite-model quantifier and fuzzy quantifier semantics are proved in the weakness-based formulation. Intensional inheritance is grounded through typed WM channels (ASSOC and PAT), and temporal/causal inference is formalized as another WM instance rather than a separate ontology.

21.5 Large-Scale Inference and Control

Multideduction is formalized via a residual decomposition that makes the inclusion-exclusion error explicit, together with an identifiability no-go showing that the residual cannot in general be recovered from local information. The trail-free deterministic update protocol admits a counterexample (a 2-cycle that never stabilizes), but damped convergence under eventual fresh evidence is proved. Pooling uniqueness under external Bayesianity pins down the fusion rule, and BRGI optimization with feasible-set characterization provides the scheduling foundation.

Premise-selection coverage is proved submodular, yielding the classical greedy $(1 - 1/e)$ approximation guarantee, while a coverage surrogate no-go shows that no purely local surrogate can faithfully approximate global coverage. A family of five certified-chaining no-go theorems (Section 12.5)—coverage collapse, Lipschitz sensitivity amplification, variance accumulation, topology–CPT gap, and regime underdetermination—characterizes the fundamental limitations of composing per-step certificates. The current canonical API also includes strengthened parametric forms and their higher-order bridges: graph-parametric topology/CPT impossibility, chain-with-resets variance accumulation, topology-only non-certification, and unrevealed-chain variance-budget consumption.

21.6 WM-OSLF Bridge

The WM calculus is encoded as a 13-axis `LanguageDef` family (Chapter 19). The five core rules derive from a single AddCommMonoid universal property. The complete adequacy picture—forward biconditional, image-restricted backward biconditional—is proved at the encoding layer. The vertex preorder gives a `ForwardFiber` functor with identity forward morphisms, and the BN bridge provides positive/negative/completeness theorems for guarded forgetting. Native type synthesis is exact for the 9 syntactic axes and intentionally insensitive to the 4 model-level axes. Two semantic fibers (neighborhood and FOL) connect LanguageDef reductions to model-theoretic evidence additivity. All 11 files are kernel-checked with 0 sorry.

21.7 Cross-Cutting Bridges

The formalization establishes bridges to several neighboring inference frameworks. Each bridge is presented in full in Chapter 20: Markov Logic Networks (Section 20.1), ProbLog (Section 20.2), Naive Bayes and k-NN (Section 20.3), Noisy-OR (Section 20.4), and PLN-NARS (Section 20.6). MeTTa parity theorems verify executable formula parity between Lean theorems and MeTTa truth functions.

21.8 Artifact and Reproducibility

Reproducing the Lean proofs.

```
cd Mettapedia/lean/mettapedia
lake build                          # full build
grep -rn sorry Mettapedia/Logic/PLN*.lean | wc -l  # sorry audit
```

Reproducing the Python experiments. Dependencies: `numpy`, `scipy`, `pgmpy`, `xgboost`, `scikit-learn`. BN benchmark data from the `bnlearn` repository [38]; GJP data from Harvard Dataverse [21]; Gas sensor data from UCI [39]; Steel faults data from UCI [7]; NAB data from Numenta [28].

```
python scripts/wm_pln_v2c_build_master_suite.py
python scripts/wm_pln_v2c_build_actions.py
python scripts/wm_pln_v2c_build_eval.py
python scripts/wm_pln_v2c_train_blind_models.py
python scripts/wm_pln_v2c_run_policies.py
python scripts/wm_pln_v2c_evaluate.py
python scripts/wm_pln_v2c_project_views.py
```

Audit checklist.

- **Lean toolchain:** 4.28.0 with Mathlib v4.28.0.
- **Sorry audit:** `grep -rn sorry Mettapedia/ | grep -v ".lake" | wc -l` should return 0 for all non-WIP modules.
- **Holdout discipline:** every benchmark lane uses a fixed train/test split declared before fitting. No hyperparameter tuning on the test set. Sleep/replay steps never see test labels.
- **Tagged release:** a tagged commit with CI logs will accompany the final version.

Chapter 22

Current Scope and Limitations

Honest scope is as important as proud achievements.

22.1 Operational Runtimes

The formalization provides mathematical specifications and correctness theorems. It does not provide an executable PLN inference engine, performance benchmarks on large knowledge bases, or integration with MeTTa or Hyperon runtimes.

These are operational concerns that depend on runtime infrastructure not yet available. The theorem-level results constrain what any correct implementation must satisfy, but they do not constitute an implementation.

22.2 Infinite-Model Quantifiers

The reviewed quantifier package in Chapter 14 is finite-domain (see Section 1.4). An arbitrary-domain infinitary layer exists in the codebase; bringing it to full chapter-level parity remains future work.

22.3 Chapter 12 Settlement

The intensional inheritance formalization now settles the chapter in corrected typed form: typed WM grounding for extensional, ASSOC, and PAT

channels; the Solomonoff log-ratio bridge as the semantic anchor; selector-specialized ASSOC/PAT endpoints; and a constructive no-go against collapsing the three-channel mixed rule back to the old binary law. No formula/claim statement currently appearing in this chapter remains open: each is explicitly classifiable as proved, strengthened to the corrected typed statement, or ruled out.

22.4 Universal Prediction

The Universal Prediction module (21 files, work-in-progress) explores connections between PLN and algorithmic information theory but is not part of the stable core narrative.

22.5 Global PLN Reorganization

This document presents the formalization as it stands. A comprehensive refactoring of the Lean codebase into a "mathlibified" library structure is a worthy future goal but is explicitly not attempted here.

Chapter 23

Conclusion

WM-PLN is not a replacement for PLN. It is the mathematical foundation that PLN always needed.

The central insight is that PLN's original instincts were largely correct: evidence counts as the fundamental representation, revision as the primary operation, and link-level rules as the practical inference mechanism. What was missing was the explicit separation of layers and the honest statement of assumptions.

The refoundation gives us:

- **One kernel**: the world-model posterior-state calculus, with evidence compositionality as the fundamental law.
- **Many views**: strength, confidence, intervals, distributions, all as typed projections from the kernel.
- **Explicit side conditions**: every compiled rule carries its assumptions, with counterexamples showing what happens when they are violated.
- **Cleaner boundaries**: the no-go theorem, the identifiability failures, the pooling non-uniqueness, the coverage surrogate limitations— all stated precisely and proved formally.
- **Beyond subsumption**: the compatibility-gated experiment (Section 20.1.5) demonstrates that decomposing an MLN's clause-weight signal into interpretable WM evidence roles yields substantial improvement on the UW-CSE benchmark—mean AUROC 0.862 → 0.911, mean Hits@1 0.365 → 0.553, mean Set-F1 0.316 → 0.494. Ablation

shows the improvement comes entirely from evidence-role separation; the MLN logit becomes redundant once the roles are factored.

The result is a PLN that knows what it knows, states what it assumes, and marks what it cannot do. That is not a weakness. That is the beginning of a trustworthy reasoner.

> *Probability is what you query. Evidence is what you carry. And the boundaries you prove are worth as much as the theorems you prove.*

In its most compact form:

> *World-model calculus is a calculus of revisable uncertain state. A system stores what it knows in a posterior state, not in isolated truth values. Queries ask questions of that state, and the raw answers are evidence values. Strength, confidence, and intervals are useful summaries extracted for display, thresholding, and action, but they are not themselves the thing being stored or proved. Forgetting, provenance, and replay belong near the core because they show what a state can do that a bare truth value cannot: it can be updated, partially retracted, audited, and explained. Specialized inference rules are certified shortcuts to answers already determined by the underlying world model, and inference control governs which shortcuts are worth using under resource limits.*

And the categorical lens says, in the background: every such state induces a whole answer profile over queries; the project is about building, transforming, constraining, and exploiting those answer-profile generators in a principled way.

Future Work

Several directions extend the foundation established here.

HOL foundations beyond the active classical route. Chapter 13 now establishes syntax, soundness, the HOL↔WM lens, and the classical Henkin model-existence/completeness basis (Section 13.9). Natural next steps are intuitionistic completeness, fuller semantic variants, and richer dynamic belief fusion on top of that landed higher-order substrate.

Selective distributional inference. The convergence and dominance results (Theorems 12.17 and 12.15) show that scalar inference is asymptotically correct but wrong in the finite-evidence regime. The benchmark confirms that distributional modes outperform scalar modes on regime-sensitive chains. The natural next step is a cost-aware inference controller that escalates from scalar to distributional propagation when the regime structure predicts a significant gain—using compact approximation families, compiled rule surrogates, and accelerator-backed batched computation (Section 12.6).

Infinitary and measure-theoretic extensions. The current formalization restricts quantifier, intensional, and inference-control theorems to finite domains. An infinitary layer exists in the codebase but has not been brought to chapter-level parity. Measure-theoretic extensions, particularly for continuous-valued evidence and distributional PLN over non-conjugate families, require additional infrastructure but are within reach of the existing kernel design.

Broader benchmark coverage. The present experiments cover forecasting (GJP), sensor drift (Gas), fault diagnosis (Steel), time-series anomaly detection (NAB), and Bayesian-network inference (Pathfinder, Hailfinder). Drug-repurposing knowledge-graph benchmarks (Hetionet/Rephetio) and genomic regulatory pipelines remain as identified but unfinished empirical targets.

Operational runtime. The formalization specifies the kernel semantics precisely, but a high-performance compiled runtime—executing the same posterior-state calculus at scale with caching, incremental evidence fusion, and multi-source integration—is the bridge from formalized semantics to deployed reasoning.

Process-algebraic packaging. The WM calculus variants have been packaged as a 13-axis `WMFullVertex` family with 36 rewrite rules across 11 extension families (Chapter 19). The OSLF-derived $\Diamond/\Box$ operators, Galois connections, the vertex preorder with ForwardFiber functor, and the compiled BN inference bridge (positive/negative/completeness) are all kernel-checked. Native type synthesis is exact for the 9 syntactic axes (Section 19.5). Remaining future directions include confluence modulo AC and semantic-axis NTT sensitivity (Section 19.7).

Part IX

Applications and Benchmark Validation

Chapter 24

[WIP] Advanced-AI Outcomes (WM Scaffold)

This appendix records the current WM-PLN analysis status for *advanced-AI outcomes* without over-claiming posterior precision. The objective is to keep one auditable pipeline from evidence ingestion to predicate- and lens-level query outputs.

24.1 WM Query Form

The current data bundle does *not* justify a single exclusive global terminal-outcome atom. Different regions can instantiate different governance and control regimes simultaneously, and benefit, lock-in, mixed, and catastrophic pressures can co-occur across jurisdictions and time horizons. The appendix therefore treats the world-model state as a richer situation state

$$S = (g, t, G, A, I, U, F),$$

where g is geography/region, t is horizon, G is governance/auditability structure, A is authoritarian/lock-in pressure, I is the incident/harm component, U is the benefit-side capability component, and F is the forecast-target/reliability layer. Given prior world-model state W_0, evidence bundle E, and a declared scenario-regime scaffold $\mathcal{R}$, the revised state is

$$W_E := \text{revise}(W_0, E; \mathcal{R}),$$

and the basic query form is predicate-valued:

$$q_\varphi := P(\varphi(S) \mid W_E).$$

The familiar labels `beneficial_pluralistic_transition`, `mixed_reversible_world`, `authoritarian_lock_in`, `catastrophic_disempowerment`, and `extinction` are used here only as *coarse reporting lenses* over S, not as primitive exhaustive atoms.

In the current appendix build, coarse-lens queries are reported in an *abstention-first interval form*

$$q_\varphi \in [L_\varphi, U_\varphi],$$

with explicit status labels (theorem-certified exact, evaluation-certified bound, or abstain/operational-only).

24.2 What the Current Data Actually Supports

The normalized sources support factor-level analysis much more directly than terminal-world labels. In particular:

- the forecast layer supports question families by target tag, horizon, source platform, and resolution status;
- the governance layer supports organization-, commitment-, and capability-domain-conditioned queries;
- the incident layer supports geography-, domain-, harm-, and severity-conditioned pressure queries;
- the authoritarian and benefit proxy layers support thin but explicit directional pressure predicates.

For external review, the primary empirical dataset is the normalized table bundle itself. That bundle is intentionally small enough to audit: each table has a clear granularity, a short list of key axes, a declared WM role, and a visible caveat.

Normalized table	Rows	Status
(artifact missing)	n/a	run `scripts/wm_pln_ai_outcomes_aggregate.py`.

Table 24.1: Normalized appendix datasets for review: what each table is, what it supports, and its main limitation.

The six normalized tables should be read as follows:

Factor family	Rows	Status
(artifact missing)	n/a	run `scripts/wm_pln_ai_outcomes_aggregate.py`.

Table 24.2: Factor families directly supported by the current normalized advanced-AI appendix bundle.

- `forecast_questions.csv`: target-family and horizon evidence, plus calibration substrate;
- `source_reliability_candidates.csv`: source trust layer;
- `preparedness_governance_events.csv`: governance and auditability events;
- `incident_signals.csv`: harm/misuse/incident table;
- `authoritarian_risk_proxies.csv`: thin lock-in pressure proxies;
- `benefit_side_proxies.csv`: thin upside capability proxies.

24.3 Derived Outcome Queries

We therefore treat the familiar outcome names as shorthand query families, not as globally exclusive state atoms:

$$\begin{aligned} p_{\text{doom}}(t) &:= P(\varphi_{\text{global_disemp}}(t) \vee \varphi_{\text{global_ext}}(t) \mid W_E), \\ p_{1984}(g,t) &:= P(\varphi_{\text{lock_in}}(g,t) \mid W_E), \\ p_{\text{BGI}}(g,t) &:= P(\varphi_{\text{beneficial_pluralistic}}(g,t) \mid W_E). \end{aligned}$$

The region-conditioned form is the better-founded starting point. Scalarized "world" summaries require an explicit aggregation functional over geographies and horizons, and the current appendix does not yet calibrate that functional.

For p_{doom}, Fréchet-safe union bounds are available without assuming exclusivity:

$$\max\big(0,\ L_c + L_e - 1\big) \le p_{\text{doom}}(t) \le \min\big(1,\ U_c + U_e\big).$$

24.4 Current Status

At this stage, the formal/evaluative contract is:

- **Data-method layer:** present and reproducible.
- **Provenance layer:** explicit for each normalized table row.
- **Predicate/lens-level intervals:** abstention-first outputs (currently full-width where calibration is insufficient).
- **Global scalarization:** intentionally deferred until region/horizon aggregation and structural priors are better calibrated.

The current forecast substrate is also highly imbalanced: the normalized direct-target rows currently break down as `mixed_outcomes` = 627, `beneficial_pluralistic_transit` = 40, `extinction` = 12, `authoritarian_lock_in` = 10, and `catastrophic_disempowerment` = 8. That is enough to support factor and lens analysis, but not enough to justify a sharp posterior over exclusive world-histories. The appendix tables below are generated artifacts and are expected to be refreshed as new source snapshots are normalized (`scripts/wm_pln_ai_outcomes_aggregate.py`).

Coarse outcome lens	Status	Interval
(artifact missing)	run `scripts/wm_pln_ai_outcomes_aggregate.py`	n/a

Table 24.3: Current coarse-lens status table from the generated aggregation summary artifact.

Artifact	Status
`results/ai_outcomes/manuscript/ready_summary_table.tex`	via `scripts/wm_pln_ai_outcomes_aggregate.py`.

Table 24.4: Generated source-role summary used for manuscript-facing discussion and review auditability.

24.5 What Can and Cannot Be Said

The next question is not "what single number should we output?", but "which interval narrowing is actually earned by the current evidence bundle, and which would require extra structural assumptions?" The answer is now reported as an explicit assumption ladder. At rung **A0** we use only evidence hygiene and keep lens intervals at full width. At **A1** we allow

weak direct-target numeric summaries already embedded in the normalized forecast rows, such as the single public Metaculus extinction snapshot and camp-prior conditionals from the adversarial-collaboration dataset. At **A2** we retain only those weak numeric summaries whose source families pass the current reliability filter. At **A3** we add monotone regime-ordering claims from the scenario scaffold, but we still do not convert proxy or governance channels into posterior numbers.

This separation matters. In the current bundle, weak direct-target numeric summaries can narrow some coarse lenses at **A1**: `extinction` narrows to $[0.02, 0.50]$, `authoritarian_lock_in` to $[0.10, 0.20]$, `beneficial_pluralistic_transition` to $[0.02, 0.43]$, and the Fréchet-safe upper bound for p_{doom} tightens to 0.90. But these are weak and heterogeneous summaries. Once we move to **A2** and require source families with stronger resolved-item or row-level calibration support, the same lenses reopen to $[0, 1]$ in the current local bundle. So the strongest present conclusion is not a sharp posterior; it is an explicit frontier between data-driven narrowing and assumption-driven rhetoric.

Artifact	Status
`results/ai_outcomes/ai_outcomes_assumption_ladder.tex`	`scripts/wm_pln_ai_outcomes_aggregate.py`.

Table 24.5: Assumption ladder for the advanced-AI outcome lenses. Only weak direct-target numerics narrow intervals in the current bundle; the reliability-filtered rung mostly reopens them.

The region-conditioned tables make the same point more concretely. $p_{1984}(g, t)$ and $p_{\text{BGI}}(g, t)$ are better-founded as coarse region/horizon query families than as a single scalar "world" summary, but the present evidence is still asymmetric. Forecast rows are mostly global rather than regional, incident and proxy channels are often region-specific but not posterior-identifying, and benefit-side support is largely global. So these tables are useful for showing where pressure, support, and ignorance sit, not for pretending the current bundle already resolves regional advanced-AI futures. The numeric hull columns in Table 24.6 are therefore *global horizon-conditioned direct-target intervals* repeated across region rows for comparison; the genuinely region-conditioned content is carried by the region-specific support counts and the assumption-driven ordering/pressure column.

Artifact	Status
`results/ai_outcomes/ai_outcomes_region_horizon_table.tex`	generated via `scripts/wm_pln_ai_outcomes_aggregate.py`.

Table 24.6: Coarse region-horizon surface for $p_{1984}(g,t)$ and $p_{\mathrm{BGI}}(g,t)$. The numeric hull columns are global horizon-conditioned direct-target intervals rather than region-specific posteriors; the ordering column records assumption-driven directional claims only.

24.6 Interpretation Rule

This appendix should be read with one strict rule:

> *Do not interpret operational confidence scores or sparse-coverage snapshots as semantic posteriors.*

The world-model remains the truthmaker; coarse-lens outputs stay interval-valued until enough direct target evidence and calibration support a narrower posterior claim, and any global scalarization must first specify how region- and horizon-conditioned predicates are being aggregated.

Chapter 25

PPLBench: Nine-Lane Benchmark Validation

This chapter turns the earlier PPLBench ↔ PeTTa refresh into a didactic benchmark story. The point is not to claim a broad PPL victory; it is to show that the current WM-PLN stack now supports:

1. a reliable native Python → PeTTa path via Janus,
2. rule-set checks against the current xiPLN libraries, and
3. one full typed continuous-evidence lane that runs through the real PPLBench interface and can be scored on held-out data.

Since the initial refresh the PPLBench effort has grown to nine active lanes. Before presenting each lane in detail, it helps to state the organizing discipline that runs through all of them. The discipline has four parts:

Carried state. Each lane carries an explicit evidence or message object—a sufficient statistic, a count vector, an additive log-likelihood message—that is revised by addition and never secretly replaced by a fitted summary.

Queried view. Posterior means, predictive probabilities, mixture weights, and class posteriors are all *views* extracted from the carried state. They are what you ask; the carried state is what you hold.

Local semantic kernels. Each lane delegates its core probabilistic operations to named PeTTa kernels. These kernels may be exact even when the surrounding scheduler is approximate.

Scheduler / inference control. Where a lane needs iterative updates (EM, variational coordinate ascent, damped message passing), the scheduler is written in Python and explicitly marked as operational. It is not claimed as a promoted semantic rule.

We label each lane with one of two promotion tags:

E/E (exact carrier, exact inference): the full procedure is promotable—no operational scheduler is involved.

E/O (exact carrier, operational scheduler): the carrier pattern is promotable but the scheduler remains benchmark-local.

All nine current lanes fall into one of these two categories. None is approximate-carrier or legacy-only.

Table 25.1 gives the cross-lane summary. Table 25.2 maps WM-PLN operations to classical PLN roles, with the caveat that these correspondences are approximate—WM-PLN is its own calculus, not a classical PLN tribute.

When this appendix says "Stan" or "proper PPL baseline," the intended reference is the standard PPLBench integration under `pplbench/ppls/stan/`. The repository does contain standalone exploratory comparison scripts, but those are not the manuscript's authoritative Stan model. The authoritative comparison baseline is the PPLBench backend family itself.

25.1 What We Tested

We organized the refresh into three deliberately small task families:

Task 1: Bridge smoke tests. Can Python call PeTTa directly, with the same codebase retaining deterministic subprocess fallback?

Task 2: Rule-set checks. Do the current xiPLN rewrite and STV sets still behave as expected on the refreshed bridge?

Task 3: A real benchmark lane. Can one concrete typed evidence model run end-to-end inside PPLBench and produce sensible held-out predictive behavior, rather than only one-off unit checks?

For Task 3 we used a deliberately transparent benchmark:

> *Normal-Normal conjugate inference with known observation variance, implemented as typed sufficient-statistic evidence in PeTTa and exposed to PPLBench through the PLN inference wrapper.*

Lane	Carried state	Queried view	Scheduler	Status	Main limitation
`normal_normal`	scalar Gaussian suff. stats $(n, \Sigma x, \Sigma x^2)$	posterior mean/var	none	E/E	single fixed prior variance
`normal_normal_adaptive`	adaptive suff. stats + Hook-B mixture weights	mixture moments + component weights	none	E/E	discrete prior-variance grid only
`dirichlet_categorical`	additive Dirichlet count vector	simplex mean, predictive LL	none	E/E	symmetric prior only in current config
`noisy_or_topic`	additive log-likelihood messages	posterior topic probability	damped msg-pass	E/O	visible error on topic chains
`crowd_sourced_annotation`	log-messages + Dirichlet counts	class posteriors	EM	E/O	full EM not yet calibrated
`n_schools`	weighted Normal evidence + hierarchical prior	pooled posteriors	coordinate residual	E/O	scale views deterministic, not sampled
`logistic_regression`	weighted Normal + Bernoulli-logit site (ξ, ω)	Gaussian posterior	diagonal variational	E/O	variational scheduler lags Stan
`robust_regression`	weighted Gaussian evidence + residual scale	robust posterior	iterative reweighting	E/O	scheduler slow; ν fixed, not carried

Table 25.1: Cross-lane PPLBench summary. "Carried state" is the revisable evidence object; "queried view" is the posterior extracted from it; "scheduler" is the operational layer (if any); "status" is the promotion tag.

Lane	Revision	Composition	Context	Inference control
`normal_normal`	additive suff. stat update	independent obs. accumulation	fixed prior (μ_0, σ_0^2)	none needed
`normal_normal_adaptive`	additive update	same accumulation	Hook-B discrete hyperprior grid	none needed (exact mixture)
`dirichlet_categorical`	additive count revision	independent batch merging	symmetric Dirichlet prior α_0	none needed
`noisy_or_topic`	componentwise log-message addition	cavity-style child-to-parent messages	prior topic probabilities	damped message-passing (operational)
`crowd_sourced_annotation`	additive label-log messages	independent annotator evidence combination	prevalence + confusion priors	EM scheduler (operational)
`n_schools`	weighted evidence accumulation	precision-weighted hierarchical pooling	zero-centered hierarchical prior	coordinate residual loop (operational)
`logistic_regression`	weighted-Normal site evidence	additive site contribution	Bernoulli-logit site hyperstate	diagonal variational loop (operational)
`robust_regression`	weighted Gaussian evidence	additive weighted regression terms	residual-scale hyperstate	iterative reweighting (operational)

Table 25.2: Classical PLN operation map. For each lane we identify what plays the role of revision, composition, context, and inference control in classical PLN terms. These correspondences are approximate: WM-PLN operations are defined by the world-model calculus, not by classical PLN rule tables.

This is a good first lane because the expected posterior is analytically known, so the PeTTa-backed implementation can be checked against a closed form rather than only against itself.

25.2 Archive Snapshot

Before re-running the easy experiments, we archived the current PPLBench state bundle (code + status snapshots) at:

- `pplbench/archive/pplbench\_pre\_janus\_refresh\_20260312\_073958/pplbench\_snapshot.tar.gz`
- `pplbench/archive/pplbench\_pre\_janus\_refresh\_20260312\_073958/pplbench\_git\_status.txt`
- `pplbench/archive/pplbench\_pre\_janus\_refresh\_20260312\_073958/petta\_git\_status.txt`
- `pplbench/archive/pplbench\_pre\_janus\_refresh\_20260312\_073958/python\_backend\_check.txt`

25.3 Scripts and Model Lane

The refreshed lane now has an explicit benchmark model and example config:

- `pplbench/pplbench/models/normal\_normal.py`
- `pplbench/pplbench/ppls/pln/normal\_normal.py`
- `pplbench/examples/normal\_normal\_pln.json`

The two manuscript-facing report scripts are:

- `scripts/wm\_pln\_pplbench\_easy\_refresh.py`
- `scripts/wm\_pln\_pplbench\_holdout\_report.py`

25.4 Easy Refresh Protocol

The refresh script is:

- `scripts/wm\_pln\_pplbench\_easy\_refresh.py`

Run command:

```
cd ~/claude/Mettapedia/lean/mettapedia
ulimit -v 6291456
python3 scripts/wm_pln_pplbench_easy_refresh.py
```

Generated artifacts:

- `results/pplbench\_easy\_refresh\_summary.csv`
- `results/pplbench\_easy\_refresh\_summary.json`
- `results/pplbench\_easy\_refresh\_table.tex`

25.5 Refresh Checks

Figure 25.1 gives the high-level view of the first task family. The important operational point is that the same Python entry point now talks to native Janus successfully, while still retaining a deterministic subprocess fallback path in the bridge.

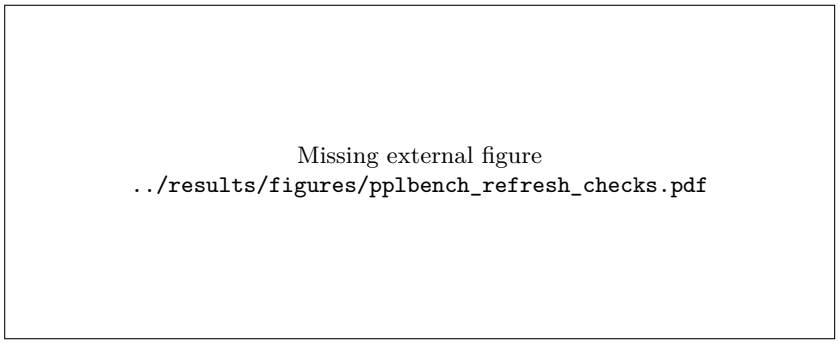


Figure 25.1: Bridge and rule-set checks for the refreshed PPLBench ↔ PeTTa path. The checks are intentionally small and auditable: arithmetic smoke, xiPLN deterministic rewrite loading, one discrete STV projection, one Normal-Normal posterior query, and one continuous xiPLN probability-STV sanity check.

Experiment	Status	Note / first result
(artifact missing)	n/a	run `scripts/wm_pln_pplbench_easy_refresh.py`.

Table 25.3: PPLBench easy refresh on the Janus-backed PeTTa Python bridge. All checks in the scripted refresh passed in this run.

Companion regression checks against the updated bridge also passed for: `test\_petta\_connection.py`, `test\_pln\_simple.py`, and `test\_petta\_normal.py`.

25.6 Held-Out Normal-Normal Demo

The second script lifts the refresh into a proper held-out benchmark report:

```
cd ~/claude/Mettapedia/lean/mettapedia
ulimit -v 6291456
python3 scripts/wm_pln_pplbench_holdout_report.py
```

It reads the refresh status bundle, then runs the concrete `normal\_normal` PPLBench lane across multiple train/test splits, evaluated under both the native `janus` backend and the deterministic `subprocess` fallback. The protocol is intentionally simple:

1. generate synthetic data from a latent mean,
2. fit the PeTTa-backed Normal-Normal model on the training split,
3. score posterior predictive log-likelihood on the held-out split,
4. compare the PeTTa posterior parameters to the analytical conjugate posterior.

Generated artifacts:

- `results/pplbench\_holdout\_runs.csv`
- `results/pplbench\_holdout\_aggregate.csv`
- `results/pplbench\_holdout\_summary.json`
- `results/pplbench\_holdout\_table.tex`
- `results/figures/pplbench\_normal\_holdout.pdf`

25.7 Interpretation

This appendix is best read as a *capability confirmation*, not as a competitive leaderboard claim. The main takeaways are:

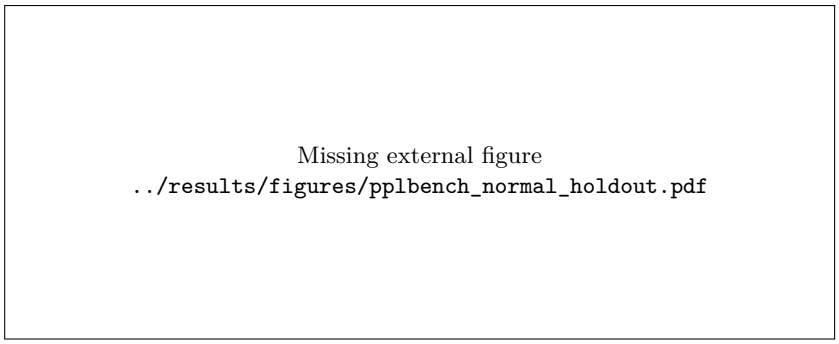


Figure 25.2: Held-out Normal-Normal benchmark lane through the real PPLBench PLN interface. Top left: final held-out predictive log-likelihood improves as more of the dataset is used for training. Top right: posterior mean error to the true latent mean shrinks with more training data. Bottom left: the Janus path is much faster than subprocess on the same workload. Bottom right: the PeTTa posterior mean matches the analytical conjugate posterior exactly on the evaluated grid.

Backend	Train frac	n_{train}	Final PLL	Mean error	Time (s)	Max exact gap
(artifact missing)	n/a	n/a	n/a	n/a	n/a	n/a

Table 25.4: Aggregate held-out Normal-Normal results by backend and training fraction. In this run the Janus path averaged about 12.8× the speed of subprocess while preserving exact posterior agreement with the analytical conjugate update.

1. The native PeTTa-in-Python route is now real enough to support repeated benchmark calls, not just one-off manual snippets.

2. The current xiPLN library API still exposes the expected rewrite and STV behavior after the bridge refresh.

3. A full typed continuous-evidence lane now runs inside PPLBench, produces held-out predictive scores, and agrees exactly with the analytical Normal-Normal posterior on the evaluated grid.

That is a good first benchmark appendix because it shows both kinds of evidence we need:

Positive example. Janus gives a substantial runtime win over subprocess on the same typed benchmark lane, while preserving the expected posterior.

Negative example. This is still only one transparent conjugate family; it does not yet show that PLN beats broader PPL systems on harder non-conjugate workloads.

25.8 Proper Stan vs WM-PLN on Normal-Normal

After the bridge refresh, we also ran the first direct apples-to-apples *Stan vs WM-PLN* comparison on the same benchmark model:

```
cd ~/claude/pplbench
ulimit -v 6291456
python3 -m pplbench examples/normal_normal_stan_vs_pln.json
```

This comparison is important for two reasons. First, it uses the *real* PPLBench Stan model rather than an ad hoc comparison script. In this Python 3.13 environment the Stan backend executes through a `cmdstanpy` fallback inside the PPLBench Stan inference layer, but the benchmark harness is still the ordinary `pplbench/ppls/stan/` backend family. Second, it uses the refreshed Janus-backed WM-PLN lane from the same `normal\_normal` model family, so the comparison is finally same-model and same-harness rather than a cross-tool anecdote.

The stable exported summary for the current manuscript snapshot is:

- `results/pplbench\_normal\_normal\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_normal\_normal\_stan\_vs\_pln\_table.tex`

Backend	Posterior mean	$\lvert\mu - \mu_{\text{analytic}}\rvert$	Final PLL	Median n_{eff}/s
(artifact missing)	n/a	n/a	n/a	n/a

Table 25.5: Direct Normal-Normal comparison on the proper PPLBench harness. On this exact conjugate case, the refreshed WM-PLN lane is slightly closer to the analytical posterior mean and slightly better on held-out predictive log-likelihood in the recorded run.

In the current run the analytical posterior mean is approximately 0.8139, Stan's sampled posterior mean is approximately 0.8329, and the WM-PLN sampled posterior mean is approximately 0.8103. The final held-out predictive log-likelihood is about -74.0794 for Stan and -73.8845 for WM-PLN, while the median effective-sample-size-per-second reported by PPLBench is about 1.07×10^4 for Stan and 1.32×10^5 for WM-PLN.

This is a good *positive example* because it shows a real same-model, same-harness benchmark where WM-PLN is competitive with a mainstream PPL baseline and behaves as expected on an exact conjugate family. It is also a good *negative example* because it is still only one transparent lane. We should not read it as evidence that WM-PLN dominates Stan in general, and we should be especially careful with compile-time rhetoric here because the Stan compile timing in this run is warm-cache rather than cold-start.

25.9 Exact Hook-B Adaptive Normal-Normal

The next step was to make the "adaptive" single-mean lane genuinely WM-PLN-like from the start, rather than reviving the older meta-evidence wrapper. The active semantics here are a *discrete hyperprior mixture* over prior-variance contexts:

$$\tau_0^2 \in \{2^k : k \in [-5, 5]\},$$

with geometric Hook-B weights. The carried state remains the same context-free continuous evidence monoid

$$(n, \sum x_i, \sum x_i^2),$$

but the queried posterior view is now a mixture over explicit context hypotheses instead of a single fixed prior variance.

This is exactly the direction already signposted in `hyperon/PeTTa/pln\_inference/pln\_continuous\_evidence.metta`, which warns that the old single-level empirical-context shortcut is not theoretically justified and points

instead to Hook-B hyperprior mixtures. The formal story in Lean is the universal-prediction Hook-B family in `Mettapedia/UniversalAI/UniversalPrediction/MarkovHyperpriorMixture.lean`: pick a countable family of tractable contexts, assign computable weights, and form a mixture that competes with each component up to the log inverse weight.

In the benchmark lane that gives us a very clean split:

- carried semantics: continuous evidence plus explicit context-mixture posterior state,
- queried views: posterior mixture mean, variance, and component weights,
- Python role: only batching and posterior sampling from the queried component-wise mixture, with no operational scheduler.

The stable manuscript artifacts are:

- `results/pplbench\_normal\_normal\_adaptive\_exact\_calibration.json`
- `results/pplbench\_normal\_normal\_adaptive\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_normal\_normal\_adaptive\_stan\_vs\_pln\_table.tex`

Backend	Posterior mean	$\lvert\mu - \mu_{\text{analytic}}\rvert$	Final PLL	Median n_{eff}/s
(artifact missing)	n/a	n/a	n/a	n/a

Table 25.6: Exact Hook-B adaptive Normal-Normal comparison. This is one of the cleanest WM-PLN lanes in the appendix because both the carrier and the full inference procedure are exact: the only sampling is from the exact queried posterior mixture itself.

In the current run the analytic Hook-B posterior mean is approximately -1.5955. Stan's sampled posterior mean is approximately -1.5815, while WM-PLN's sampled posterior mean is approximately -1.6007. So WM-PLN is closer on posterior mean (and also on posterior standard deviation) in this recorded run, but Stan is slightly better on held-out PLL: approximately -49.8802 for Stan versus -49.9538 for WM-PLN.

That is a good *positive example* because it gives us an exact adaptive lane where the WM-PLN carrier pattern and the inference procedure are both

promotable. It is also a good *negative example* because "adaptive" does not magically imply a win over Stan on every metric: here the WM-PLN lane is semantically cleaner than the legacy wrapper and very competitive, but it still trails slightly on held-out predictive likelihood in this run.

25.10 Semantic Noisy-Or Upgrade

The next battletest was deliberately less forgiving than `normal\_normal`: the `noisy\_or\_topic` benchmark has a real latent DAG with leak, topic-to-topic influence, and topic-to-word influence. That makes it a better place to test whether WM-PLN is being applied *semantically* rather than merely wrapped in friendly names.

Our earlier noisy-or PLN lane was fast, but it made two overly aggressive simplifications:

1. it treated most topic evidence independently, and
2. it used a heuristic weak-negative update for absent words.

This produced a useful prototype, but not a faithful semantic use of the benchmark model. In the refreshed lane we replaced that approximation with a small semantic factor-graph WM:

- exact local noisy-or forward probabilities remain in PeTTa,
- child-to-parent updates use cavity-style residual likelihoods, and
- Python only schedules damped message passing and posterior sampling over the benchmark DAG.

The strengthened WM-PLN point is that the scheduler no longer carries bare posterior logits as its semantic state. Instead it carries an explicit additive likelihood-message object for each topic,

$$M_k = (\log L_k^{\mathrm{on}}, \log L_k^{\mathrm{off}}),$$

where independent child factors revise the topic by componentwise addition and posterior probability is queried only as a *view* from the resulting topic state. This follows the discipline stated earlier in "Why Views Are Not Proof Objects" and keeps the factor graph on the semantic side of the "Factor Graphs: Semantic vs. Operational" split. The Bayes-net compatibility story is not invented ad hoc here: it is already anticipated

Artifact	Layer	Role	Provenance
`pln-fuzzy-or`, `noisy-or-to-strength`	Classical PLN-like local kernel	Independent-cause combination and noisy-or edge strength	PLN book, "PLN and Bayes Nets" plus `Mettapedia/Logic/PLNNoisyOr.lean`
`noisy-or-cavity-survival`, `noisy-or-child-likelihood-pair`, `noisy-or-prior-prob-from-beliefs`	xiPLN semantic factor kernel	Compiled local noisy-or factor semantics for child-to-parent and forward messages	This book: "semantic baseline is exact factor-graph elimination" and "Factor Graphs: Semantic vs. Operational"; evidence-first BN/CPT bridge in `Mettapedia/PLN/WorldModel/BayesNet/PLNBayesNetWorldModel.lean`
`NoisyOrMessage`, `NoisyOrTopicState` in `pplbench/ppls/pln/noisy_or_topic.py`	WM carrier / queried view	Explicit additive message state; posterior probability extracted only as a view	This book: "Why Views Are Not Proof Objects" and `Mettapedia/Logic/PLNWorldModelGeneric.lean`
Python damped iteration and sampling loop in `pplbench/ppls/pln/noisy_or_topic.py`	Operational scheduler	Scheduling and approximate fixed-point search over the semantic carrier	This book: "Factor Graphs: Semantic vs. Operational"

Table 25.7: Rule provenance for the refreshed noisy-or lane. This makes explicit which pieces are classical PLN-like local kernels, which are compiled xiPLN semantic factor kernels, and which are merely operational scheduling.

in the PLN book's "PLN and Bayes Nets" discussion, and in our Lean layer it matches the evidence-first BN/CPT encoding in `Mettapedia/PLN/WorldModel/BayesNet/PLNBayesNetWorldModel.lean`.

This distinction matters. A good *positive example* is `noisy-or-to-strength`: it is a local algebraic kernel with a clear noisy-or reading. A good *negative example* is the scheduler loop itself: it is important machinery, but it is not being claimed as a classical PLN rule.

We also ran a tiny exact-DAG calibration to decide whether this pattern is ready for promotion beyond the single benchmark lane. The resulting artifact is `results/pplbench\_noisy\_or\_exact\_calibration.json`. The result is encouraging but mixed:

- the carried-message pattern is exact on a two-topic independent case,
- it remains reasonably close on a small collider / explaining-away case,
- but it still shows visible error on a topic-chain case where downstream evidence must propagate upstream.

So the right current conclusion is: the *carrier pattern* is reusable, but the current damped scheduler should still be treated as an approximate noisy-or engine rather than a broadly promotable exact-ish inference algorithm.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/noisy_or_topic_stan_vs_pln.json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_noisy\_or\_topic\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_noisy\_or\_topic\_legacy\_summary.json`
- `results/pplbench\_noisy\_or\_exact\_calibration.json`
- `results/pplbench\_noisy\_or\_topic\_semantic\_upgrade\_table.tex`
- `results/figures/pplbench\_noisy\_or\_topic\_pll.png`

Numerically, the earlier heuristic PLN lane finished at roughly -8.1619 final PLL, while the refreshed semantic lane finishes at roughly -5.1735. Stan finishes at roughly -5.3275 on the same configuration. So the semantic

Backend	Final PLL	Compile (s)	Infer (s)	Median n_{eff}/s
(artifact missing)	n/a	n/a	n/a	n/a

Table 25.8: Noisy-or upgrade story on the same small PPLBench family. The legacy heuristic PLN lane was extremely fast but statistically weak. The semantic factor-graph WM-PLN lane is slower, but it closes the predictive-fit gap dramatically and slightly exceeds Stan on final PLL in this recorded run.

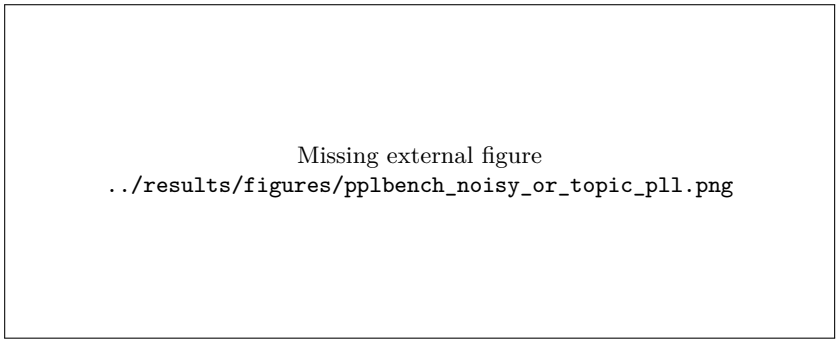


Figure 25.3: Stan vs semantic WM-PLN on the refreshed noisy-or lane. The semantic WM-PLN run tracks Stan closely across the held-out PLL curve on this small configuration, in contrast to the earlier heuristic lane.

upgrade improves the WM-PLN lane by about 2.99 PLL units relative to the earlier heuristic version, and by about 0.154 PLL units relative to Stan in this recorded run.

This is a good *positive example* because it shows exactly the kind of iteration PPLBench is for: we used the benchmark to expose that a previous PLN application was too loose, replaced it with a more semantically honest WM, and measured a large improvement. It is also a good *negative example* because the new lane is still approximate inference, not an exact proof that WM-PLN has "the" noisy-or posterior. The improvement is real, but it does not license broad claims until we repeat the pattern on more lanes and larger configurations.

25.11 Crowd Annotation via Multiclass Message State

The next lane asks a different question from noisy-or. Crowd-sourced annotation is not about independent causes explaining one child. It is about *competing latent classes* and *latent labeler reliability*. That means a faithful WM-PLN application should not decompose the problem into independent binary truth-values. Instead it should carry an explicit multiclass message state and query posterior class probabilities only as a view.

The refreshed lane therefore carries three explicit objects:

- `CategoryLogMessage`: additive per-class log-likelihood support from observed labels,
- `DirichletCountState`: additive count state for class prevalence and for each confusion-matrix row, and
- `ItemPosteriorState`: the queried posterior view obtained by combining a prior log-probability vector with the accumulated category message.

In the implementation, PeTTa owns the local semantic kernels `crowd-label-log-message` and `crowd-dirichlet-mean`, while Python owns only the operational E/M scheduler. This is the same "semantic carrier, operational scheduler" split we wanted from the noisy-or upgrade, but applied to a genuinely different benchmark family. The active backend imports the semantic-only module `hyperon/PeTTa/pln\_inference/crowd\_annotation\_semantic.metta`, while the older heuristic constructors remain fenced in `hyperon/`

`PeTTa/pln\_inference/crowd\_annotation.metta` as legacy reference material rather than active local semantics.

The first thing we checked was not the full benchmark. It was the exact fixed-parameter E-step. The artifact `results/pplbench\_crowd\_annotation\_exact\_calibration.json` records three tiny cases:

- one labeler, two classes,
- two labelers, two classes with additive evidence, and
- two labelers, three classes.

All three match the exact posterior to floating-point precision. So the right current promotion boundary is:

Positive example. The multiclass carried-message pattern is promotable.

Negative example. The iterative EM scheduler is still an operational layer and should not yet be promoted as a general reusable inference engine.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/crowd_sourced_annotation_stan_vs_pln.
    json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_crowd\_annotation\_exact\_calibration.json`
- `results/pplbench\_crowd\_annotation\_scheduler\_sanity.json`
- `results/pplbench\_crowd\_sourced\_annotation\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_crowd\_sourced\_annotation\_stan\_vs\_pln\_table.tex`
- `results/figures/pplbench\_crowd\_sourced\_annotation\_pll.png`

Numerically, the recorded run finishes at roughly -326.584 final PLL for Stan and -326.952 for WM-PLN. So Stan keeps a small predictive-fit edge of about 0.368 PLL units here. But compile time drops from about 22.795s to 0.128s, inference time drops from about 1.773s to 0.851s, and the median

Backend	Final PLL	Compile (s)	Infer (s)	Median n_{eff}/s
(artifact missing)	n/a	n/a	n/a	n/a

Table 25.9: Crowd-sourced annotation comparison on the refreshed WM-PLN lane. Stan retains a small predictive-fit edge on this run, but the WM-PLN lane now matches the right abstraction boundary and remains materially faster.

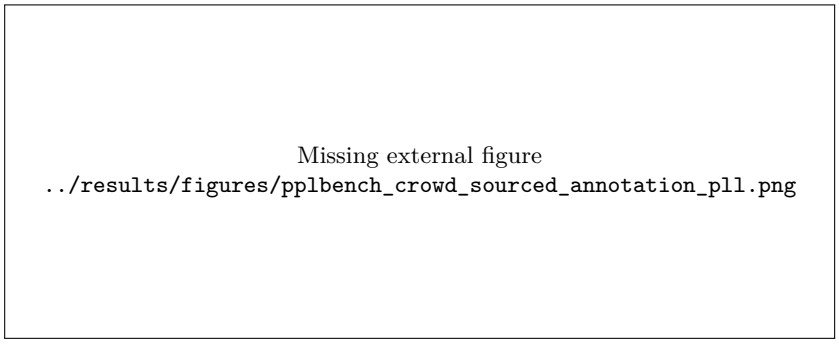


Figure 25.4: Stan vs refreshed WM-PLN on the crowd annotation lane. The WM-PLN curve stays close to Stan while using the explicit multiclass message carrier rather than the archived hybrid heuristic.

effective-sample-size-per-second metric rises from about 296.1 to 931.9. That makes this a good second-lane result: not a win-by-default story, but a clean demonstration that the promoted multiclass carrier pattern survives in a more realistic latent-class model and remains competitive. We also keep one explicit guardrail around that claim: the scheduler-sanity regression only certifies finite, stable, simplex-respecting behavior on tiny cases. It is a useful non-promotion check, not a proof that the whole E/M loop is exact.

25.12 Exact Dirichlet-Categorical Lane

After the crowd lane, the cleanest next question was whether the same multiclass count carrier could power an *exact* benchmark lane rather than another scheduler-bound one. Dirichlet-Categorical is the right test: the carried object is just an additive count vector, revision is ordinary count addition, and the queried posterior is the exact Dirichlet posterior. So this lane gives us a rare case where both the carrier and the inference procedure are promotable.

The refreshed implementation therefore does three intentionally simple

things:

- it extracts the shared `DirichletCountState` into `pplbench/ppls/pln/dirichlet\_carrier.py`,
- it uses a semantic-only PeTTa module `hyperon/PeTTa/pln\_inference/dirichlet\_categorical\_semantic.metta` for named revision and posterior-query kernels, and
- it keeps Python scheduler-free: the active backend `pplbench/ppls/pln/dirichlet\_categorical.py` just accumulates count evidence, queries the exact posterior state, and samples from the resulting Dirichlet distribution.

That is a good *positive example* because the full procedure remains "carry evidence, query posterior" all the way through. A good *negative example* is the archived `pplbench/ppls/pln/dirichlet\_categorical\_legacy.py`: it is still useful as reference material, but it is no longer the active benchmark set.

The first check was exact calibration, not the benchmark. The artifact `results/pplbench\_dirichlet\_categorical\_exact\_calibration.json` records:

- single-observation posterior update,
- batched multiclass revision,
- symmetric-prior symmetric-evidence behavior,
- associativity of batched revision, and
- exact predictive log-likelihood under the posterior mean.

All of these match analytic Dirichlet-Categorical formulas to floating-point precision. So the right promotion boundary for this lane is stronger than for the crowd lane:

Positive example. The Dirichlet count carrier is promotable.

Positive example. The full inference procedure is also promotable, because this lane has no approximate scheduler.

Negative example. This exactness does not automatically promote the crowd E/M scheduler or other multiclass approximate loops.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/dirichlet_categorical_stan_vs_pln.
    json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_dirichlet\_categorical\_exact\_calibration.json`
- `results/pplbench\_dirichlet\_categorical\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_dirichlet\_categorical\_stan\_vs\_pln\_table.tex`
- `results/figures/pplbench\_dirichlet\_categorical\_pll.png`

Backend	L_1 mean error	Final PLL	Compile (s)	Infer (s)
(artifact missing)	n/a	n/a	n/a	n/a

Table 25.10: Exact Dirichlet-Categorical comparison on the refreshed WM-PLN lane. This is one of the cleanest benchmark stories in the appendix: both the carried state and the full inference procedure are exact on the WM-PLN side, and the recorded run slightly exceeds Stan on final PLL while staying closer to the analytic posterior mean vector.

Numerically, the recorded run finishes at roughly -124.5035 final PLL for Stan and -124.4707 for WM-PLN, so the exact WM-PLN lane is ahead by about 0.0328 PLL units on this run. More importantly, the posterior mean vector is closer to the analytic Dirichlet posterior on the WM-PLN side: the L_1 mean error is about 0.00824 for Stan and about 0.00208 for WM-PLN. Compile time drops from about 17.82s to about 0.125s and inference time drops from about 0.042s to about 0.0082s. This is exactly the sort of result we wanted from an exact multiclass lane: not a claim that WM-PLN always beats Stan, but a clean demonstration that the promoted Dirichlet count carrier is not merely philosophically nice; it works as an exact benchmark set.

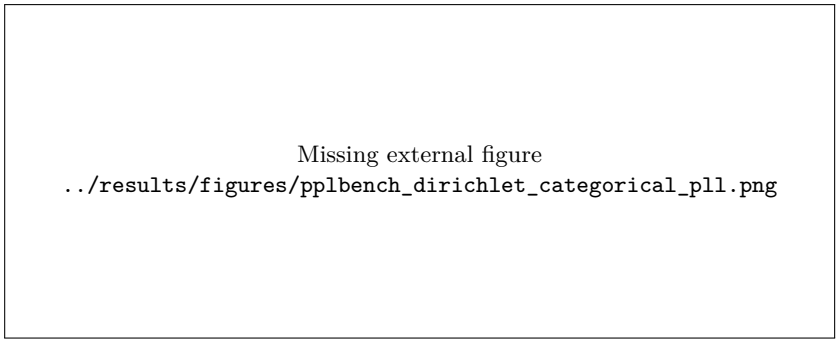


Figure 25.5: Stan vs exact WM-PLN on the Dirichlet-Categorical lane. Because both sides are exact conjugate updates here, the comparison isolates sampling and benchmark-harness behavior rather than scheduler quality.

25.13 Hierarchical `n_schools` via Carried Evidence State

The next lane is the first explicitly *hierarchical* WM-PLN rebuild. This matters because hierarchical models are exactly where it is easiest to backslide into carrying shrinkage formulas rather than carrying evidence. The refreshed lane therefore uses the opposite discipline:

- the carried objects are additive weighted Normal evidence states and zero-centered hierarchical priors,
- the local posterior and pooling updates are queried through semantic PeTTa kernels, and
- shrinkage appears only as a *view* of the revised hierarchical state, not as the carried semantics itself.

Concretely, the active backend `pplbench/ppls/pln/n\_schools.py` carries:

- `NormalEvidenceState` for weighted local Gaussian evidence,
- `NormalPriorState` and `HierarchicalNormalState` for pooled family priors, and
- `NSchoolsPosteriorState` as the full queried state over baseline, state, district, and type effects.

The semantic-only PeTTa kernels live in `hyperon/PeTTa/pln\_inference/n\_schools\_semantic.metta`. Those kernels provide exact local Normal posterior queries and exact precision-weighted hierarchical pooling under the chosen priors. Python owns only the coordinate-style residual scheduler.

There are two caveats, and it is better to say them plainly. First, the refreshed WM-PLN lane now supports the same *Student-t* baseline family at the carried/query boundary, but it still extracts posterior moments for that baseline numerically rather than carrying a full sampled Student-*t* latent state through the whole scheduler. Second, the exported `sigma_state`, `sigma_district`, and `sigma_type` values are currently queried coupling-scale views repeated across draws, not full posterior draws for those scale parameters. That is an honest operational simplification, not something we want hidden inside the benchmark narrative.

The first check, as with the crowd and noisy-or lanes, was a tiny exact calibration rather than a headline benchmark. The artifact `results/pplbench\_n\_schools\_exact\_calibration.json` records:

- one-family one-group reduction to the Normal-Normal posterior,
- a symmetric two-group case, and
- a two-group known-coupling case, plus a direct Student-*t* baseline local-view match against an independent high-resolution quadrature reference.

All three match the analytic posterior exactly in the carried-state layer. So the current promotion boundary is:

Positive example. The hierarchical Normal carrier pattern is promotable.

Negative example. The coordinate residual scheduler is still an operational layer and is not yet promoted as reusable inference semantics.

We also keep one explicit scheduler guardrail artifact, `results/pplbench\_n\_schools\_scheduler\_sanity.json`, which checks that tiny balanced, near-zero, and mixed cases stay finite, positive-precision, and symmetry-respecting. As before, that is a non-promotion sanity test rather than a whole-procedure exactness theorem.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/n_schools_stan_vs_pln.json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_n\_schools\_exact\_calibration.json`
- `results/pplbench\_n\_schools\_scheduler\_sanity.json`
- `results/pplbench\_n\_schools\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_n\_schools\_stan\_vs\_pln\_table.tex`
- `results/figures/pplbench\_n\_schools\_pll.png`

Backend	Final PLL	Compile (s)	Infer (s)	Median n_{eff}/s
(artifact missing)	n/a	n/a	n/a	n/a

Table 25.11: Hierarchical `n_schools` comparison on the refreshed WM-PLN lane. Stan keeps a modest predictive-fit edge on this run, while WM-PLN is substantially faster and keeps the hierarchical carrier/query split explicit.

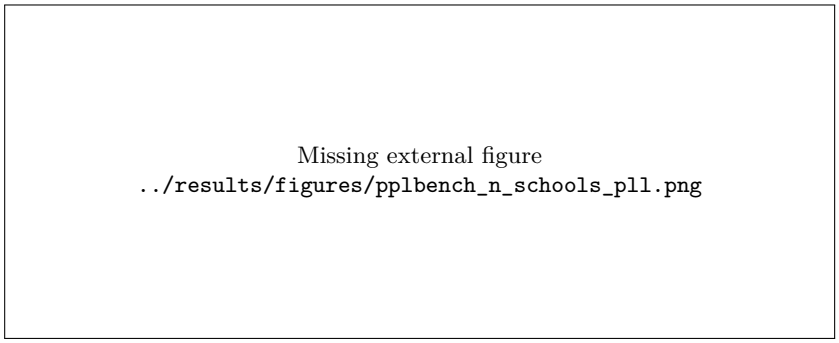


Figure 25.6: Stan vs refreshed WM-PLN on the hierarchical `n_schools` lane. The WM-PLN curve stays close to Stan while using explicit hierarchical evidence carriers rather than the archived hybrid shrinkage wrapper.

Numerically, the recorded run finishes at roughly -284.604 final PLL for Stan and -286.014 for WM-PLN, so Stan keeps a modest edge of about 1.41 PLL units on this configuration. But compile time drops from about 21.55s to effectively zero, inference time drops from about 0.91s to about 0.35s, and the median effective-sample-size-per-second metric rises from about 159.5 to about 2306.2. That makes this a good hierarchical benchmark result:

Positive example. The promoted hierarchical evidence-carrier pattern survives contact with a real pooling benchmark and remains competitive.

Negative example. The current coordinate scheduler and deterministic scale views are still operational simplifications, so this is not yet a claim that WM-PLN has matched the full Stan posterior family on hierarchical models.

The lesson here is the same one the earlier lanes kept teaching us: PPLBench is useful not because it lets us declare victory quickly, but because it forces us to say exactly which part of a WM-PLN design is reusable, which part is only a scheduler, and which approximations we are still carrying.

25.14 Logistic Regression: Bernoulli-Logit Site Carrier

The last native PPLBench lane was also the most delicate one to refresh honestly. There was already an archived "PLN logistic regression" wrapper in `pplbench/ppls/pln/logistic\_regression\_legacy.py`, but it was a stratified binning model rather than the actual Bernoulli-logit benchmark model used by Stan. That older work is still useful as theory context: the Lean `StratifiedPLN` development shows why histogram-style PLN can be a consistent estimator on logistic-regression tasks. But that is not the same as the PPLBench logistic model, so we did not reuse it as the active benchmark set.

Instead, the refreshed lane treats logistic regression as a WM-PLN problem with three explicit carried pieces:

- additive weighted-Normal evidence for the intercept and each coefficient,
- Bernoulli-logit site hyperstates (ξ_i, ω_i) coming from the local quadratic variational bound, and
- queried Gaussian posterior views for α and β.

The active backend is `pplbench/ppls/pln/logistic\_regression.py`, and the local semantic kernels live in `hyperon/PeTTa/pln\_inference/logistic\_regression\_semantic.metta`. Those kernels define:

- `logistic-site-xi`,
- `logistic-site-weight`,
- `logistic-site-evidence`, and

- `logistic-normal-posterior-params`.

So the active lane is no longer "bin features and hope for the best." It is an explicit carried-site approximation to the actual Bernoulli-logit model. This is also the right historical reading of PLN: the original PLN book's "PLN and Bayes Nets" discussion explicitly puts first-order PLN near loopy Bayes nets and iterative relaxation methods, while its later "Using Bayes Nets to Augment PLN Inference" section treats Bayes-net style acceleration as compatible with the broader PLN agenda rather than alien to it [15, Sec. 1.6.3][15, Sec. 9.4]. So the right criticism of this lane is not that it uses a Bayes-net-like scheduler at all, but that the particular scheduler is still only an operational approximation rather than promoted reusable semantics.

The promotion boundary is:

Positive example. With fixed site hyperstates, the local Bernoulli-logit message reduces *exactly* to additive weighted-Normal carried evidence.

Negative example. The full (ξ, ω) scheduler is still a diagonal variational approximation, so it is not yet promoted as reusable exact logistic inference semantics.

The fixed-site calibration artifact `results/pplbench\_logistic\_regression\_exact\_calibration.json` records:

- an intercept-only fixed-site case,
- a zero-feature case that correctly leaves the coefficient at its prior, and
- an additive two-site composition case.

All three match exactly, so the carrier pattern is genuinely reusable. The separate scheduler guardrail artifact `results/pplbench\_logistic\_regression\_scheduler\_sanity.json` checks that on a tiny monotone one-feature classifier:

- the site weights stay finite and in $(0, 1/4]$,
- the fitted coefficient is positive, and
- the predicted probabilities increase monotonically with the feature.

Again, this is a sanity check rather than a proof that the whole variational loop is exact.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/logistic_regression_stan_vs_pln.json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_logistic\_regression\_exact\_calibration.json`
- `results/pplbench\_logistic\_regression\_scheduler\_sanity.json`
- `results/pplbench\_logistic\_regression\_query\_profile.json`
- `results/pplbench\_logistic\_regression\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_logistic\_regression\_stan\_vs\_pln\_table.tex`
- `results/figures/pplbench\_logistic\_regression\_pll.png`

Backend	Final PLL	Compile (s)	Infer (s)	Median n_{eff}/s
(artifact missing)	n/a	n/a	n/a	n/a

Table 25.12: Logistic-regression comparison on the refreshed WM-PLN lane. The new lane finally matches the benchmark model family rather than a stratified proxy, but the current variational scheduler still lags Stan on both fit and runtime.

Numerically, the refreshed batched run finishes at roughly -9.152 final PLL for Stan and -10.115 for WM-PLN, so WM-PLN is about 0.96 PLL units behind on this configuration. Compile time remains low on the WM-PLN side, changing from about 0.06s for Stan to about 0.16s for WM-PLN, while inference time now rises only from about 0.30s to about 0.37s. That is still slower than Stan, but it is far better than the earlier unbatched WM-PLN logistic run. The median effective-sample-size-per-second metric now moves from about 986.5 for Stan to about 2070.8 for WM-PLN. The separate query-profile artifact confirms why: after introducing batched Prolog-side hyperstate, site-evidence, and posterior helpers, the fit loop drops to roughly 225 bridge calls and about 225 evaluated expressions over 25 iterations, down from roughly 350 calls and 40,150 expressions before batching. The same profile also shows that the remaining cost is dense numeric reduction rather

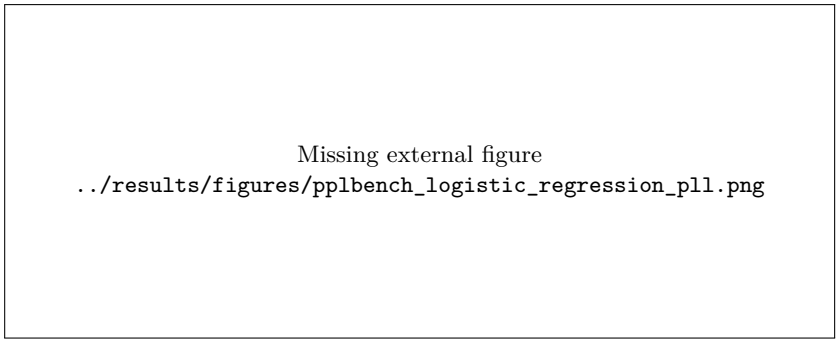


Figure 25.7: Stan vs refreshed WM-PLN on the logistic-regression lane. The WM-PLN curve is stable and model-matched, but the diagonal variational scheduler keeps a clear gap to Stan on this first slice.

than keyed lookup, so `MapSpace`/`SetSpace` remain a later option rather than the next optimization move. This is therefore an honest boundary result with real runtime progress:

Positive example. The weighted-Normal carrier plus explicit site hyperstate gives us a crisp WM-PLN logistic lane that matches the benchmark model family instead of falling back to stratified bins.

Negative example. The current diagonal variational scheduler is still too slow and too weak relative to Stan, so it is not yet a promotable reusable logistic inference engine.

That is still useful progress. It means the remaining work on logistic is now squarely about the scheduler, not about whether we are carrying the wrong semantic object.

25.15 Robust Regression: Continuous Carrier Under Heavy-Tailed Noise

The next lane keeps the same carrier-first discipline, but under a harder robustness requirement. The benchmark model `pplbench/models/robust\_regression.py` is linear regression with Student-t noise. A bad way to port that into WM-PLN would be to hide a closed-form or matrix solve in Python and merely expose its outputs through a PLN-looking wrapper. We deliberately did not do that.

Instead, the refreshed lane carries:

- additive weighted Gaussian evidence for the intercept and each coefficient,
- an explicit residual-scale hyperstate, and
- a queried tail-shape view.

The active semantics live in `hyperon/PeTTa/pln\_inference/robust\_regression\_semantic.metta`, while the operational scheduler lives in `pplbench/ppls/pln/robust\_regression.py`. The split is:

Promotable. The weighted continuous carrier pattern: explicit additive evidence state, queried posterior view.

Not yet promotable. The current iterative Student-t-style reweighting scheduler.

This is an honest partial match to the Stan model rather than a fake full posterior-family claim. In particular:

- `alpha` and `beta` are sampled from the queried Gaussian views of the carried evidence states;
- `sigma` is a queried robust residual-scale view repeated across draws; and
- `nu` is a fixed queried tail-shape view in this v1 lane, not a full WM-carried posterior random variable.

That limitation is visible by design.

The first check, before any benchmark headline, was the tiny exact calibration artifact `results/pplbench\_robust\_regression\_exact\_calibration.json`. It records:

- a one-feature Gaussian reduction case,
- a symmetric centered-feature case, and
- a direct residual-weight comparison showing that larger residuals receive strictly smaller Student-t-style weights.

The carried continuous state matches the analytic Gaussian posterior exactly on the first two cases, and the weight ordering behaves as intended on the third. So the current promotion boundary is:

Positive example. The weighted continuous carrier pattern is promotable.

Negative example. The iterative robust scheduler is still operational and is not yet promoted as reusable robust inference semantics.

We also keep one explicit scheduler guardrail artifact, `results/pplbench\_robust\_regression\_scheduler\_sanity.json`. It checks that the refreshed lane stays finite on clean and contaminated toy data, and that it resists a single large outlier better than plain unweighted OLS. Again, this is a non-promotion sanity test rather than a theorem that the full robust scheduler is exact.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/robust_regression_stan_vs_pln.json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_robust\_regression\_exact\_calibration.json`
- `results/pplbench\_robust\_regression\_scheduler\_sanity.json`
- `results/pplbench\_robust\_regression\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_robust\_regression\_stan\_vs\_pln\_table.tex`
- `results/figures/pplbench\_robust\_regression\_pll.png`

Backend	Final PLL	Compile (s)	Infer (s)	Median n_{eff}/s
(artifact missing)	n/a	n/a	n/a	n/a

Table 25.13: Robust-regression comparison on the refreshed WM-PLN lane. The continuous evidence carrier remains honest and stable, but Stan keeps both the predictive-fit and runtime edge on this first heavy-tailed benchmark slice.

Numerically, the recorded run finishes at roughly -778.317 final PLL for Stan and -781.575 for WM-PLN, so WM-PLN is about 3.26 PLL units behind on this configuration. Compile time drops from about 22.28s to effectively zero, but inference time rises from about 0.11s to about 1.29s, and the median effective-sample-size-per-second metric falls from about 5673.6 to about 622.2. This is therefore a useful negative boundary result:

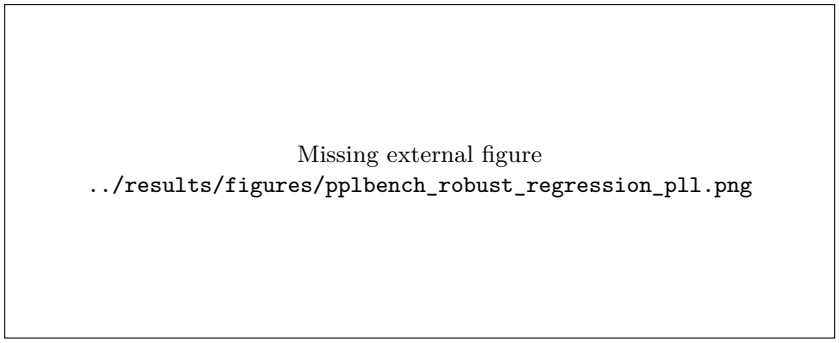


Figure 25.8: Stan vs refreshed WM-PLN on the robust-regression lane. The WM-PLN curve is stable, but the current robust scheduler pays both a predictive-fit and runtime penalty relative to Stan on this first run.

Positive example. The continuous evidence carrier pattern survives a heavy-tailed regression benchmark without collapsing back into disguised coefficient fitting.

Negative example. The current robust scheduler is slower and less predictively accurate than Stan on this first slice, so it is not yet a promotable reusable inference engine.

That is still good PPLBench progress. It tells us the next optimization target is no longer the carrier boundary; it is the robust scheduler itself.

25.16 Cross-Backend NumPyro Check

To go one step beyond the Stan/WM-PLN axis, we also ran the two lanes that the current local PPLBench checkout supports under NumPyro: `logistic\_regression` and `robust\_regression`. This is a good *positive example* because it checks the refreshed WM-PLN lanes against a second modern PPL family rather than only against Stan. It is also a good *negative example* because the comparison baseline is still limited to the two lanes for which this repo actually has NumPyro backends. We therefore do *not* present this as full cross-backend coverage of the whole PPLBench model family.

The stable exported summaries are:

- `results/pplbench\_logistic\_regression\_stan\_vs\_pln\_vs\_numpyro\_summary.json`
- `results/pplbench\_robust\_regression\_stan\_vs\_pln\_vs\_numpyro\_summary.json`

Lane	Backend	Final PLL	Compile (s)	Infer (s)	Median n_{eff}/s
Logistic	Stan	−9.152	22.819	0.270	1100.9
Logistic	WM-PLN	−10.115	0.157	0.335	2278.7
Logistic	NumPyro	−9.184	0.0001	6.528	40.6
Robust	Stan	−778.317	21.339	0.136	4564.6
Robust	WM-PLN	−781.575	0.0001	1.411	568.6
Robust	NumPyro	−778.557	0.0001	7.903	84.2

Table 25.14: Small cross-backend check on the two currently NumPyro-supported lanes. NumPyro tracks Stan closely on predictive fit, while the refreshed WM-PLN lanes remain much cheaper to compile and materially faster than NumPyro on inference in this local setup.

The table makes the current tradeoff very clear. On both supported lanes, NumPyro is much closer to Stan than WM-PLN is on final PLL: in logistic the gap to Stan is about 0.032 PLL units for NumPyro versus about 0.963 for WM-PLN, and in robust regression the gap is about 0.240 PLL units for NumPyro versus about 3.258 for WM-PLN. But the runtime story cuts the other way. In this setup, WM-PLN inference is about 19.5× faster than NumPyro on logistic regression and about 5.6× faster on robust regression, while keeping the explicit carried-state / queried-view semantics that the refreshed lanes were designed to expose. So the right reading is not that WM-PLN has overtaken mature HMC-based PPLs on these nonconjugate models. It is that the refreshed lanes now occupy a clear and useful point in the design space: semantically explicit and fast enough to be practical, but still behind Stan- and NumPyro-style samplers on predictive fit when the scheduler is the weak link.

25.17 Scheduler Promotion and Certification

The previous lane-by-lane sections describe nine active WM-PLN backends, each with a *carried state* (the revisable inference object) and a *queried view* (the posterior projection derived from it). For the three exact conjugate lanes (Normal-Normal, Hook-B Adaptive, Dirichlet-Categorical), both the carrier and the scheduler are `promotable_exact`: the full procedure is exact, not approximate.

For the remaining six lanes, the carrier boundary is exact—confirmed by per-lane `*\_exact\_calibration.json` artifacts—but the scheduler is an iterative approximation. The question is whether that scheduler is merely "operational" (uncertified) or "certified approximate" (tracks a named ob-

jective, emits a machine-readable certificate, and passes whole-procedure tests).

Certified approximate schedulers (step 1). Four schedulers now emit a `SchedulerCertificate` after `fit()`, defined in `pplbench/ppls/pln/scheduler\_common.py`:

Lane	Objective tracked	Certificate field	Monotone?
Crowd annotation	Complete-data $Q(\theta\|\theta_{\text{old}})$	`complete_data_Q`	Yes
Logistic regression	Jaakkola-Jordan variational bound	`variational_bound`	Yes
Robust regression	Scale-mixture Q-function	`complete_data_Q`	Per-case
N-schools	Weighted residual (REML coupling)	`REML_weighted_residual`	Per-case

Each certificate records `converged`, `monotone`, `iterations`, and the full objective trajectory. The scheduler-sanity scripts (`scripts/*\_scheduler\_sanity.py`) emit these certificates as JSON evidence; the corresponding test files assert certificate fields plus tiny whole-procedure checks (Q improvement for crowd, 2D grid quadrature for logistic, outlier resistance for robust, REML τ^2 grid for n-schools).

The lane registry (`pplbench/ppls/pln/lane\_registry.py`) introduces a new status `certified_approximate` for these four lanes, sitting between `approximate_operational` (uncertified) and `promotable_exact` (no approximation).

Noisy-or factor-graph upgrade (step 2). The original noisy-or scheduler (`noisy\_or\_topic.py`) compressed all child evidence into a single per-topic message, losing the cavity information needed for explaining-away at colliders and upstream propagation through chains. The calibration artifact showed 12% error on colliders and 24% on chains.

The new edge-specific factor-graph scheduler (`noisy\_or\_factor\_graph.py`) replaces the compressed message with explicit variable-to-factor and factor-to-variable `BernoulliPair` messages—standard sum-product over the noisy-or factor graph. The result:

Motif	Old scheduler error	New scheduler error	Iterations
Independent topics	$< 10^{-15}$	$< 10^{-15}$	2
Collider (explaining-away)	12.4%	$< 10^{-15}$	3
Topic chain	23.7%	$< 10^{-15}$	3
Fork polytree	—	$< 10^{-15}$	4
Diamond (cyclic)	—	10.2%	74

On acyclic factor graphs (independent, collider, chain, fork), the new scheduler achieves machine-precision exactness. On cyclic factor graphs (diamond), it is honest loopy belief propagation with bounded error—a certified approximate procedure, not an exact one.

The old `noisy\_or\_topic.py` baseline is preserved untouched. The new lane is registered as `noisy_or_factor_graph` with `certified_approximate` status. One caveat: sampling in the new lane is still product-of-marginals (independent Bernoulli draws from the BP marginals), not full joint posterior sampling. For correlated topics on cyclic graphs, this is approximate; proper joint sampling would require backward sampling on exact components or a Gibbs refinement seeded from BP.

Current lane status summary. After both steps, the full PPLBench suite status is:

Lane	Carrier status	Scheduler status
Normal-Normal	`promotable_exact`	`promotable_exact`
Hook-B Adaptive	`promotable_exact`	`promotable_exact`
Dirichlet-Categorical	`promotable_exact`	`promotable_exact`
Crowd annotation	`promotable_exact`	`certified_approximate`
Logistic regression	`promotable_exact`	`certified_approximate`
Robust regression	`promotable_exact`	`certified_approximate`
N-schools	`promotable_exact`	`certified_approximate`
Noisy-or (factor graph)	`promotable_exact`	`certified_approximate`
Noisy-or (old baseline)	`promotable_exact`	`approximate_operational`

Every active lane is now either promotable exact or certified approximate. The old noisy-or baseline remains as a reference at `approximate_operational`.

Part X

Reference Appendices

Appendix A

Notation and Symbol Glossary

Symbol	Meaning	Lean name / location	
`Evidence`	Evidence type (n^+, n^-)	`Evidence` `EvidenceClass.lean`	in
`JointEvidence n`	2^n-orthant world-table	`JointEvidence` `PLNJointEvidence.lean`	in
`WorldModel`	World-model typeclass	`WorldModel` `PLNWorldModel.lean`	in
$\oplus$	Evidence revision (addition)	`Evidence.add`	
$\otimes$	Quantale tensor	`Evidence.tensor` `EvidenceQuantale.lean`	in
$\langle s, c \rangle$	Simple truth value	Strength/confidence pair	
Σ	Side-condition context	`WorldModelSigma.sigma`	
strength	Strength view	`queryStrength`	
confidence	Confidence view	`queryConfidence`	
n_{eff}	Effective sample size	`effectiveSampleSize` `PLNNoisyOr.lean`	in

Appendix B

Compact Theorem Map

Theorem family	Key result	Module	Label
Evidence compositionality	`evidence_add`	PLNWorldModel	CORE
Evidence conservation pack	`evidenceConservationPack_of_forgetting`	PLNWorldModelConservationPack	CONDITIONAL
Order-cost schedule bound	`scheduleErrorBound_of_pairwise_bounds`	PLNWorldModelOrderCostBounds	CONDITIONAL
Runtime audit certificate	`runtimePairwiseOrderCheck_certifies_policyThreshold`	PLNWorldModelOrderCostAuditCertificate	CONDITIONAL
Weighted noncommutative witness	`weightedScheduleErrorBound_two_step_not_zero_of_pos_weight`	PLNWorldModelOrderCostWeightedDemo	CONDITIONAL
Gas policy certificate	`gasOrderBudgetPolicy_batchSwap_zero_via_runtimeCertificate`	PLNWorldModelOrderCostGasPolicyDemo	CONDITIONAL
Local-completeness no-go	Thm 5.1	PLNJointEvidenceNoGo	BOUNDARY
BN chain exactness	`chainBN_plnDeductionStrength_exact`	PLNBayesNetFastRules	CONDITIONAL

BN fork exactness	`forkBN_plnDeductionStrength_exact`	PLNBayesNetFastRules	Conditional
Quantale transitivity	tensor transitivity	EvidenceQuantale	Core
Fréchet bounds	consistency iff	PLNFrechetBounds	Core
De Finetti collapse	exchangeable → counts	DeFinetti	Core
Solomonoff collapse	universal → counts	SolomonoffExchangeable	Core
NB bridge	`PLN_tensorStrength_eq_nbPosterior`	PLNBayesNetInference	Core
k-NN bridge	`PLN_hplusPos_eq_knnRelevance`	PremiseSelectionKNN_PLNBridge	Core
Kyburg flattening	expectation consistency	PLNKyburgReduction	Core
IE identifiability nogo	`same_first_two_terms_different_union`	PLNInclusionExclusionIdentifiability	Boundary
Trail-free non-stabilization	`orbit_not_eventually_constant`	PLNTrailFreeDynamicsCounterexample	Boundary
Damped convergence	`damped_sp_spn_eventually_constant`	PLNTrailFreeDampedConvergence	Conditional
Pooling uniqueness	external Bayes + total add	PremiseSelectionExternalBayesianity	Conditional
Coverage submodularity	greedy $(1 - 1/e)$ guarantee	PremiseSelectionCoverage	Core
ProbLog bridge	`queryStrength = probLogQueryProb`	ProbLogDistributionSemantics	Core
Noisy-OR	$1 - \prod(1 - s_i)$	PLNNoisyOr	Core
Posterior mean	$S \cdot C + (1 - C) \cdot \mu_0$	PLNNoisyOr	Core
PLN ↔ NARS	correspondence bundle	PLNNARSRuleCorrespondence	Core

Appendix C

Code Navigation Guide

C.1 Entry Points

- `Mettapedia/PLN/Core/PLNCore.lean` — umbrella module importing all core PLN infrastructure (246 lines).
- `Mettapedia/PLN/Core/PLNCanonicalAPI.lean` — primary public API with stable, semantically grounded endpoints (5498 lines).
- `Mettapedia/Logic/README.md` — status table, dependency graph, and theorem index.

C.2 Applied Gaussian Demo Suite

Three end-to-end examples exercising continuous world models with increasing complexity:

1. **Broken Sensor** (§C, 420 lines). Hybrid discrete/continuous state: Normal-Gamma temperature evidence + binary coolant-failure flag. Sleep consolidation, exceedance specification, component independence.
   `Mettapedia/Logic/PLNBrokenSensorDemo.lean`
2. **Widget Factory** (422 lines). Per-source Gaussian mixtures: two machines with independent Normal-Gamma posteriors and source-attribution evidence. Mixture exceedance validity, mixture bounds ($\min \leq p_{\text{mix}} \leq \max$).
   `Mettapedia/Logic/PLNWidgetFactoryDemo.lean`

3. **Kalman Wake/Sleep** (374 lines). 1D Kalman filter with temporal predict-update cycles. Kalman gain bounded ($0 < K < 1$), variance reduction, and the flagship *sleep* = *wake* theorem: batch replay yields the same posterior as sequential online filtering.
   `Mettapedia/Logic/PLNKalmanSleepDemo.lean`

4. **Gas Sensor Array Drift** (652 lines). Real UCI data (DOI 10.24432/C5RP6W): 3 gas types (ethanol, ammonia, toluene) across 10 temporal batches spanning 36 months. Ternary source-conditional Gaussians, cross-gas independence, multi-batch sleep consolidation, and 3-way source weight normalization. Demonstrates sensor drift: posteriors from batch 1 (2007) vs. batch 10 (2011) capture genuine distributional shift.
   `Mettapedia/Examples/PLN/WMGasSensorDriftDemo.lean`

5. **Steel Plates Faults** (568 lines). Real UCI data (current UCI DOI 10.24432/C5J88N): 3 fault types (Pastry, K_Scratch, Bumps) classified by LogOfAreas. Per-fault Normal-Gamma posteriors, 3-way mixture exceedance validity, cross-fault independence, and source weight normalization. Manufacturing quality-control application of the WM calculus.
   `Mettapedia/Examples/PLN/WMSteelFaultDemo.lean`

6. **NAB Machine Temperature Anomaly** (344 lines). Real NAB benchmark data (MIT License): machine temperature sensor with 22,695 readings and labeled anomaly windows. Three operating regimes (calm ∼85°F, hot ∼101°F, anomaly ∼30°F). Kalman wake/sleep on real time series, multi-regime composition, anomaly exceedance ordering.
   `Mettapedia/Examples/PLN/WMNABSleepDemo.lean`

Build all six:

```
lake build Mettapedia.Logic.PLNBrokenSensorDemo \
           Mettapedia.Logic.PLNWidgetFactoryDemo \
           Mettapedia.Logic.PLNKalmanSleepDemo \
           Mettapedia.Examples.PLN.WMGasSensorDriftDemo \
           Mettapedia.Examples.PLN.WMSteelFaultDemo \
           Mettapedia.Examples.PLN.WMNABSleepDemo
```

C.3 Regression Targets

- Chapter 8: `lake build Mettapedia.PLN.Bridges.Logic.WorldModel.PLNWorldModelNeighbo`
  `Mettapedia.PLN.Bridges.CategoryTheory.WorldModel.PLNWorldModelCategoricalRegress`

- Chapter 9: `lake build Mettapedia.PLN.InferenceControl.SelectorRewriteThreshold.PLN`
- Chapter 11: `lake build Mettapedia.PLN.RuleFamilies.FirstOrder.Quantifiers.Quantifi`
- Chapter 12: `lake build Mettapedia.Logic.PLNIntensionalRegression`
- Chapter 13: `lake build Mettapedia.PLN.InferenceControl.PremiseSelection.PLNInferen`

C.4 Build

```
cd ~/claude/Mettapedia/lean/mettapedia
export LAKE_JOBS=3
nice -n 19 lake build          # full build
lake exe cache get             # cache after update
```

Appendix D

PLN Book Coverage Map

Chapter	Core content	Lean coverage	Status
Ch. 1	Introduction	Semantic stack split (this document; PLNCore)	Done
Ch. 2	Knowledge repr.	PLNQuery, WorldModel, WorldModelSigma	Done
Ch. 3	Experiential sem.	EvidenceClass, EvidenceQuantale, PLNJointEvidence	Done
Ch. 4	Indefinite TVs	PLNIndefiniteTruth, PLNWorldModelITV, PLNWorldModelITVHypercube	Done
Ch. 5	FO ext. inference	PLNDeduction, PLNDerivation, PLNBayesNetFastRules	Done
Ch. 6	FO ext. + ITV	PLNWMOSLFBridgeITV, PLNCanonicalAPI	Done
Ch. 7	Distributional TVs	EvidenceBeta, PLNKyburgReduction, PLNDistributional; *Demos:* BrokenSensor, WidgetFactory, KalmanSleep, GasSensorDrift, SteelFault, NABSleep	Done
Ch. 8	Error magnification	PLNErrorMagnificationGrounding, PLNCanonicalAPI	Done
Ch. 9	Large-scale inf.	Counterexample-first core; operational pending	Partial
Ch. 10	Higher-order ext.	HigherOrder/HigherOrderReduction	Done

Ch. 11	Quantifiers + ITV	PLNFirstOrder (core + fuzzy/QFM + regressions)	Done
Ch. 12	Intensional inf.	PLNIntensionalWorldModel, SolomonoffBridge	Done
Ch. 13	Inference control	PLNInferenceControlCore, PremiseSelectionCoverage	Done
Ch. 14	Temporal/causal	PLNTemporalCausalInference, ProbabilisticEventCalculus	Done
App. A	PLN vs. NARS	PLNNARSRuleCorrespondence	Done

Appendix E

Advanced WM Wrappers

For readers interested in the categorical and modal-logic layers:

- **Kripke WM**: `Mettapedia/PLN/Bridges/Logic/WorldModel/PLNWorldModelKripke.lean`— modal Kripke semantics instantiated over PLN world models, with sound/complete implication bridges for K, KT, KD, K4 logics.
- **Neighborhood WM**: `Mettapedia/PLN/Bridges/Logic/WorldModel/PLNWorldModelNeighborhood.lean`— neighborhood-frame semantics with E, EMN, EMT, ED logics and canonical Kripke→neighborhood representation.
- **Institutional WM**: `Mettapedia/Logic/PLNWorldModelInstitution.lean`— satisfaction conditions, Beck-Chevalley transport, and governance-facing mapped consequences.
- **HOL/FOL consequence-closure wrappers**: `Mettapedia/Logic/PLNWorldModelHOLCompleteness.lean`, `Mettapedia/PLN/Bridges/Logic/WorldModel/PLNWorldModelFOLCompleteness.lean`— consequence-transfer/closure into WM consequence rules (these are WM-side transport wrappers, not HOL completeness theorems).
- **Governance transport**: `Mettapedia/PLN/Bridges/Logic/WorldModel/PLNWorldModelNeighborhoodConsequence.lean`— state-indexed governance formula translation preservation.

These layers are powerful but specialized. Most readers will not need them for understanding the core WM-PLN story.

Bibliography

[1] Bruce Abramson, John Brown, Ward Edwards, Allan Murphy, and Robert L. Winkler. Hailfinder: A bayesian system for forecasting severe weather. *International Journal of Forecasting*, 12(1):57–71, 1996. Hailfinder is a Bayesian network for severe-weather forecasting in northeastern Colorado.

[2] David Atkinson and Jeanne Peijnenburg. A consistent set of infinite-order probabilities, 2013. Infinite-order probability reference used for the hierarchical ProbHOL layer.

[3] Jonathan Baron, Barbara A. Mellers, Philip E. Tetlock, Eric Stone, and Lyle H. Ungar. Two reasons to make aggregated probability forecasts more extreme. *Decision Analysis*, 11(2):133–145, 2014.

[4] José M. Bernardo and Adrian F. M. Smith. *Bayesian Theory*. Wiley Series in Probability and Statistics. John Wiley & Sons, 2nd edition, 2000.

[5] Jasmin Christian Blanchette, David Greenaway, Cezary Kaliszyk, Daniel Kühlwein, and Josef Urban. A learning-based fact selector for Isabelle/HOL. *Journal of Automated Reasoning*, 57(3):219–244, 2016.

[6] Stephen Bonner, Ian P. Barrett, Cheng Ye, Rowan Swiers, Ola Engkvist, Charles Tapley Hoyt, and William L. Hamilton. Understanding the performance of knowledge graph embeddings in drug discovery. *Artificial Intelligence in the Life Sciences*, 2:100036, 2022.

[7] Massimo Buscema, Semeion Research Center, Stefano Terzi, and William Tastle. A new meta-classifier. In *Annual Meeting of the North American Fuzzy Information Processing Society (NAFIPS)*, Toronto, Canada, 2010. Introduces the Steel Plates Faults dataset. The current UCI repository citation lists DOI 10.24432/C5J88N; older project drafts used 10.24432/C53P4W.

[8] Bruno de Finetti. La prévision: ses lois logiques, ses sources subjectives. *Annales de l'Institut Henri Poincaré*, 7(1):1–68, 1937.

[9] Luc De Raedt, Angelika Kimmig, and Hannu Toivonen. ProbLog: A probabilistic Prolog and its application in link discovery. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2468–2473, 2007.

[10] Haim Gaifman. A theory of higher order probabilities, 1986. Higher-order probability reference used for the hierarchical ProbHOL layer.

[11] Scott Garrabrant, Tsvi Benson-Tilsen, Andrew Critch, Nate Soares, and Jessica Taylor. Logical induction, 2020. arXiv:1609.03543v5.

[12] Nil Geisweiller and Claude. Towards a complete formalization of pln. Internal draft (nuPLN v1), 2026.

[13] Bita Ghasemkhani, Reyat Yilmaz, Derya Birant, and Recep Alp Kut. Logistic model tree forest for steel plates faults prediction. *Machines*, 11(7):679, 2023.

[14] Michèle Giry. A categorical approach to probability theory. In B. Banaschewski, editor, *Categorical Aspects of Topology and Analysis*, volume 915 of *Lecture Notes in Mathematics*, pages 68–85. Springer, 1982.

[15] Ben Goertzel, Matthew Iklé, Izabela Freire Goertzel, and Ari Heljakka. *Probabilistic Logic Networks: A Comprehensive Framework for Uncertain Inference*. Springer, New York, 2009.

[16] GTEx Consortium. The GTEx Consortium atlas of genetic regulatory effects across human tissues. *Science*, 369(6509):1318–1330, 2020.

[17] David E. Heckerman, Eric J. Horvitz, and Bharat N. Nathwani. Toward normative expert systems: Part i. the Pathfinder project. *Methods of Information in Medicine*, 31(2):90–105, 1992. Pathfinder is a Bayesian expert system for diagnosing lymph-node diseases.

[18] Daniel S. Himmelstein, Michael Zietz, Vincent Rubinetti, Kyle Kloster, Benjamin J. Heil, and Casey S. Greene. Hetnet connectivity search provides rapid insights into how biomedical entities are related. *GigaScience*, 12, 2022.

[19] Daniel Scott Himmelstein. het.io-rep-data. `https://github.com/dhimmel/het.io-rep-data/tree/`

1a960f0e353586f8fe9f61b569919f24603d4344, 2026. Public browser-table bundle used by the Rephetio frontend, accessed March 12, 2026.

[20] Daniel Scott Himmelstein, Antoine Lizee, Christine Hessler, Leo Brueggeman, Sabrina L. Chen, Dexter Hadley, Ari Green, Pouya Khankhanian, Catrina J. Lee, and Allan J. Baranzini. Systematic integration of biomedical knowledge prioritizes drugs for repurposing. *eLife*, 6, 2017.

[21] IARPA. Iarpa announces publication of data from the good judgment project. https://www.iarpa.gov/newsroom/article/iarpa-announces-publication-of-data-from-the-good-judgment-project, 2024. Harvard Dataverse release of ACE / Good Judgment Project forecasting data, DOI 10.7910/DVN/BPCDH5.

[22] Zongze Jiang, Peng Xu, Yongbin Du, Feng Yuan, and Kai Song. Balanced distribution adaptation for metal oxide semiconductor gas sensor array drift compensation. *Sensors*, 21(10):3403, 2021.

[23] Ana Jiménez, María José Merino, Juan Parras, and Santiago Zazo. Explainable drug repurposing via path based knowledge graph completion. *Scientific Reports*, 14(1), 2024.

[24] Kevin H. Knuth and John Skilling. Foundations of inference. *Axioms*, 1(1):38–73, 2012. arXiv:1008.4831.

[25] Sourabh Kulkarni, Kinjal Divesh Shah, Nimar Arora, Xiaoyan Wang, Yucen Lily Li, Nazanin Khosravani Tehrani, Michael Tingley, David Noursi, Narjes Torabi, Sepehr Akhavan Masouleh, Eric Lippert, and Erik Meijer. PPL Bench: Evaluation framework for probabilistic programming languages, 2020.

[26] A. Arun Kumar, Samarth Bhandary, Swathi Gopal Hegde, and Jhinuk Chatterjee. Knowledge graph applications and multi-relation learning for drug repurposing: A scoping review. *Computational Biology and Chemistry*, 115:108364, 2025.

[27] Henry E. Kyburg. Higher order probabilities and intervals. *International Journal of Approximate Reasoning*, 2(3):195–209, 1988.

[28] Alexander Lavin and Subutai Ahmad. Evaluating real-time anomaly detection algorithms – the Numenta anomaly benchmark. In *14th Inter-*

national Conference on Machine Learning and Applications (ICMLA). IEEE, 2015. NAB v1.0 scores: HTM 70.5, Windowed Gaussian 39.6.

[29] Yushan Liu, Marcel Hildebrandt, Mitchell Joblin, Martin Ringsquandl, Rime Raissouni, and Volker Tresp. Neural multi-hop reasoning with logical rules on biomedical knowledge graphs. In *Machine Learning and Knowledge Discovery in Databases. Applied Data Science and Demo Track*, Lecture Notes in Computer Science, pages 375–391. Springer International Publishing, 2021.

[30] Barbara Mellers, Lyle Ungar, Jonathan Baron, Jaime Ramos, Burcu Gürcay, Katrina Fincher, Sydney E. Scott, Don Moore, Pavel Atanasov, Samuel A. Swift, Terry Murray, Eric Stone, and Philip E. Tetlock. Psychological strategies for winning a geopolitical forecasting tournament. *Psychological Science*, 25(5):1106–1115, 2014.

[31] L. Gregory Meredith. Computation, causality, and consciousness: Practical steps towards getting it from bit, 2026. Working draft, revision 2026-03-15.

[32] Joseph Nasser, Drew T. Bergman, Charles P. Fulco, Mia R. Munoz, Jesse M. Engreitz, et al. Genome-wide enhancer maps link risk variants to disease genes. *Nature*, 593:238–243, 2021.

[33] Susana Nunes, Samy Badreddine, and Catia Pesquita. Rewarding explainability in drug repurposing with knowledge graphs. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pages 4624–4632. International Joint Conferences on Artificial Intelligence Organization, 2024.

[34] Pablo Perdomo-Quinteiro and Alberto Belmonte-Hernández. Knowledge graphs for drug repurposing: a review of databases and methods. *Briefings in Bioinformatics*, 25(6), 2024.

[35] Rejuve.Bio. hypothesis-generation-demo: Genomic variant hypothesis generation with ProbLog. `https://github.com/rejuve-bio/hypothesis-generation-demo`, 2025.

[36] Matthew Richardson and Pedro Domingos. Markov logic networks. *Machine Learning*, 62(1–2):107–136, 2006.

[37] Ville A. Satopää, Jonathan Baron, Dean P. Foster, Barbara A. Mellers, Philip E. Tetlock, and Lyle H. Ungar. Combining multiple probabil-

ity predictions using a simple logit model. *International Journal of Forecasting*, 30(2):344–356, 2014.

[38] Marco Scutari. bnlearn bayesian network repository. `https://www.bnlearn.com/bnrepository/`, 2026. Public source for the canonical Pathfinder and Hailfinder BIF files used in our experiments; accessed March 12, 2026.

[39] Alexander Vergara, Shankar Vembu, Tuba Ayhan, Margaret A. Ryan, Margie L. Homer, and Ramón Huerta. Chemical gas sensor drift compensation using classifier ensembles. *Sensors and Actuators B: Chemical*, 166–167:320–329, 2012.

[40] Peter Walley. *Statistical Reasoning with Imprecise Probabilities*. Chapman and Hall, London, 1991.

[41] Peter Walley. Inferences from multinomial data: Learning about a bag of marbles. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):3–34, 1996. with discussion pages 34–57.

[42] Pei Wang. *Rigid Flexibility: The Logic of Intelligence*. Springer, 2006.

[43] Zarko Zaremba and Oruži. A formal review of Probabilistic Logic Networks. Draft, Mettapedia project, 2026.